Secoinsa S.O. Multi-usuario 6.2

A practical guide to Secoinsa's Serie 20 computer operating system

Using the system • Administration • Programming • System reference

Independent guide and documentation - 27 September 2026

Eduardo Cruz - https://github.com/ArcadeHacker/

Contents

- Secoinsa S.O. Multi-usuario 6.2
- About this document
- Preface
- How to use this document

Part I — The system

- Chapter 1 — Meet the operating system
- Chapter 2 — Architecture at a glance
- Chapter 3 — Starting, connecting and leaving

Part II — Using the system

- Chapter 4 — Using the terminal
- Chapter 5 — Commands, filenames and disks
- Chapter 6 — Your first file
- Chapter 7 — Editing files and running programs
- Chapter 8 — Automating work with EXEC
- Chapter 9 — Accounts, messages and everyday habits

Part III — System administration

- Chapter 10 — Administering accounts
- Chapter 11 — Managing storage
- Chapter 12 — Devices, printing and partitions
- Chapter 13 — Backup and recovery
- Chapter 14 — Startup maintenance and ending work

Part IV — Programming

- Chapter 15 — Your first BASIC program
- Chapter 16 — BASIC values, decisions and loops
- Chapter 17 — Programming files
- Chapter 18 — Writing Z80 machine-code programs
- Chapter 19 — Using system services

Part V — System fundamentals

- Chapter 20 — Organization and memory
- Chapter 21 — Boot architecture
- Chapter 22 — Partitions, identity and protection
- Chapter 23 — Storage and device concepts

Part VI — Operating system internals

- Chapter 24 — Kernel entry and shared structures
- Chapter 25 — Scheduling, callbacks and synchronization
- Chapter 26 — Filesystem implementation
- Chapter 27 — Disk I/O and physical translation
- Chapter 28 — Terminal, serial, printer and tape drivers
- Chapter 29 — Deferred printing and messages
- Chapter 30 — Command processing, loading and termination

Part VII — Reference

- Chapter 31 — Command reference
- Chapter 32 — System service reference
- Chapter 33 — Diagnostics and messages
- Chapter 34 — Devices and system limits
- Chapter 35 — Command spelling and language reference

- Appendix A. Program, language and archive formats
- Appendix B. Memory, I/O, interrupt and routine maps
- Appendix C. Operating-system files and language components
- Appendix D. BASIC commands, statements and functions
- Appendix E. Binary structure atlas
- Glossary
- Index

- Emulation in MAME

About this document

This independent guide describes S.O. Multi-usuario Version 6.2 operating system for the Secoinsa Serie 20 computer.

This document exists because no surviving technical manual set for the operating system on this machine is known to its author at the time of publication. It is the product of reverse engineering a surviving Secoinsa Serie 20 computer and examining the behaviour and implementation of its operating system. The machine and its software supply the technical foundation; the brief company history in the Preface provides context. This is an independent reconstruction, not a reprint of a manufacturer manual. Further original documentation may yet come to light.

Secoinsa S.O. Multi-usuario may be the earliest known surviving copy of an operating system with a Spanish-language command environment. Its Spanish command names, messages and help text add to its rarity and historical importance. This possible chronological priority has not been established; earlier surviving systems may yet be identified.

Original product names, command names and messages retain their historical spelling. Their inclusion does not imply manufacturer authorship or endorsement of this edition. No rights in the operating-system software are granted by this document.

Some interfaces are incompletely specified. Such limitations are stated at the relevant operation. A syntax illustration explains the form of a command; it is not a promise that optional hardware is fitted or that an unknown interface has become usable. Examples that change files use practice names. Substitute your own device and account assignments only after checking them.

Preface

Secoinsa (Sociedad Española de Comunicaciones e Informática, S.A.) was founded in 1975 as part of Spain's effort to build a domestic computer industry. Its founding partners were the Instituto Nacional de Industria (INI), the state's industrial holding company, together with Telefónica and Fujitsu. The idea was to combine Spanish design and manufacturing with technology from abroad. Fujitsu gave the company international industrial ties and invested in its factory in Málaga.

Much of Secoinsa's technical background came from Telesincro, a Spanish electronics firm founded in 1963 that was already building business computers well before Secoinsa existed. Its Winner/Contafac billing machine was followed by the Factor Q and Factor R, and then in the early 1970s by the programmable Factor S. All of these were built for accounting, invoicing and everyday office work. Secoinsa negotiated majority control of Telesincro in 1975 and completed the purchase in 1976. Telesincro kept its own name and products within the group, bringing with it an established product range and long experience supplying business equipment.

Secoinsa's own range drew on both Telesincro and Fujitsu designs. One of its machines was the Secoinsa Serie 20 computer, a compact business system built around terminals introduced in the early 1980s. It was supplied with Secoinsa IS 9502 terminals, each with a keyboard and a green monochrome screen showing 24 lines of 80 characters. For the people using it, the terminal was effectively the computer: commands, programs and file contents all appeared there as text.

After a series of commercial struggles, including a failed move into personal-educational computers in 1984–85 with the Secoinsa-branded version of Fujitsu's FM-7. In 1986 the company agreed to merge with Fujitsu España, which took it over along with all its assets and obligations.

Very little of Secoinsa's work in computers seems to have survived. Its machines were built for offices, banks and public bodies, not for homes, and when they became obsolete they were usually scrapped rather than kept. Few examples of most models are known to survive, and the software and documentation is in an even worse state. Only a handful of brochures, technical descriptions and official records known to exist.

For a company at the centre of Spain's attempt to build its own computer industry, that is a very small record, and every recovered machine, manual or listing is worth documenting before it too disappears.

The historical value of this Secoinsa Serie 20 computer preservation lies in the complete working environment that includes its rare operating system. The Z80-based system described here combines several terminal users, banked memory, structured business files and an integrated BASIC environment. Explaining how those facilities work together preserves a part of Spanish computing history that a company chronology or hardware specification alone cannot convey.

This document teaches and describes the Secoinsa Serie 20 computer's operating system "S.O. Multi-usuario", its Spanish command names and compact file notation, and everyday sequences such as connect to the system, access files, do basic tasks, and session disconnection. Later chapters turn the same operations into command files and introduce BASIC programming. Technical chapters explain how these visible operations reach the executive and devices.

How to use this document

Start with Chapters 1–5 if this is your first session. Chapter 6 introduces file operations. Chapter 7 covers text editing and running compiled BASIC; Chapter 8 introduces EXEC command files from a first greeting through variables and loops. Chapter 9 covers accounts and messages. Work through the examples in order; later examples use files created earlier. Ask for an ordinary practice account and enough privilege to use EDITAR and BASIC before working through those examples.

Administrators should continue with Chapters 10–14. Programmers can move from the file-operation examples to Chapters 15–19: a first BASIC program, values and control flow, file I/O, Z80 machine code, then system services and BASIC extensions. Chapters 20–30 explain the system at increasing depth. Chapters 31–35 provide command, service, error, device and limit references, followed by command spelling and the separate editor, EXEC and query vocabularies. Emulation in MAME, after the Index, provides instructions for operating the system in MAME. The appendices collect program formats, hardware maps, system-file explanations, BASIC vocabulary and binary structures.

Entering examples

A code block headed by an instruction to *enter* contains exactly the text to type. Press Return after each command. Do not type a prompt unless the instructions explicitly say it is part of the input. The command processor displays >, EDITAR displays *, and BASIC displays -. Check the prompt before entering a command: BASIC's `RUN` and the command processor's `RUN` are different operations.

Each introductory command is presented separately. Read the paragraph after it before continuing. Longer program listings are complete examples to enter one numbered line at a time; the accompanying explanation accounts for every instruction.

Names such as `NOTE.TEXT:S`, `COPY.TEXT:S` and `HELLO.BASICOBJ:S` are practice names. Check that a name is unused before creating it. `S` is a logical disk association, not a directory. The examples require an accessible, writable S association; ask the administrator which association to use if yours differs.

Syntax notation

Notation	Meaning
`LISDIR`	Literal command name
`name.type:drive`	Replace these fields with your file specification
`[item]`	Optional item; do not type square brackets
`item...`	Repetition permitted by that command
`(VERIFI)`	Literal parentheses and option keyword; type them
`A` or `B` in prose	Choose the stated alternative; do not type the word “or”
`0x1234`	Hexadecimal number in the explanation
`08h`	Hexadecimal constant in assembly notation
`Ctrl-X`	Hold Control and press X
`Escape, Q`	Press Escape, release it, then press Q

Quotation marks in BASIC are literal. General CSI (command processor aka system prompt) quoting and command continuation are not assumed. Use full command names while learning. Chapter 5 explains command names and the limitations of abbreviations.

A warning before a maintenance example identifies a real change or destructive operation. Read its prerequisites before typing. Reference examples involving device A require an administrator to establish what A denotes. They are not instructions to format an arbitrary connected disk.

Levels of detail

User procedures describe what appears at the terminal. Programming sections describe the interface a program uses. Internals describe the implementation. Table notes identify probable interpretations, unknown fields and the limits of an interface. These distinctions prevent an incomplete contract from being mistaken for a general rule.

Part I — The system

Chapter 1 — Meet the operating system

S.O. Multi-usuario Version 6.2 is a multiuser operating system for the Z80-based Secoinsa Serie 20 computer. You work by typing commands at a terminal. The system replies with text, performs the requested operation and normally returns to a prompt for the next command.

The command processor is called CSI (command processor aka system prompt). It understands command names such as `AYUDA`, `LISDIR` and `COPIFI`. Most commands are programs loaded when requested. Learning these names is more useful than translating commands from another operating system: the punctuation, options and file rules belong to this system.

Your terminal, account and partition

Your account identifies your files and privileges. Your partition holds a running program, its memory, device assignments and connection state. Several partitions can operate at once. The executive switches among them so that one user can work while another user's program runs.

These terms describe different things. Changing account changes identity; stopping a partition stops an execution context. Logging out ends your connection. None of those actions means that the whole machine has shut down.

What you can do

The system provides file listing, text editing, copying, printing, account maintenance and storage administration. EXEC command files automate sequences of commands. BASIC supplies an editor, interpreter, compiler and runtime. Machine-code programs can call executive services for console input, files, devices and scheduling.

Files use a two-part name, such as `NOTE.TEXT`, followed when necessary by a disk association: `NOTE.TEXT:S`. There is no hierarchical directory path to learn. Ownership and disk associations determine which files a command can find.

What the version means

The operating system identifies itself as Version 6.2. CSI (command processor aka system prompt) and EXEC have their own 5.5 version numbers, and BASIC S20 has a 1.0 version number. A component number need not match the operating-system number. This document retains the Spanish names and messages you will encounter at the terminal, while explaining their operation in English.

Begin with startup in Chapter 3. You do not need to understand memory banking or executable formats to list a file. The next chapter provides a map of the system so that those details have a place when you reach them.

Chapter 2 — Architecture at a glance

The terminal, command environment and storage work together through the operating system. A command such as LISDIR begins as text typed at a terminal. CSI selects the command program, that program asks the operating system for file information, and the result appears on the terminal. This chapter identifies the components along that path; their memory addresses and internal interfaces come later.

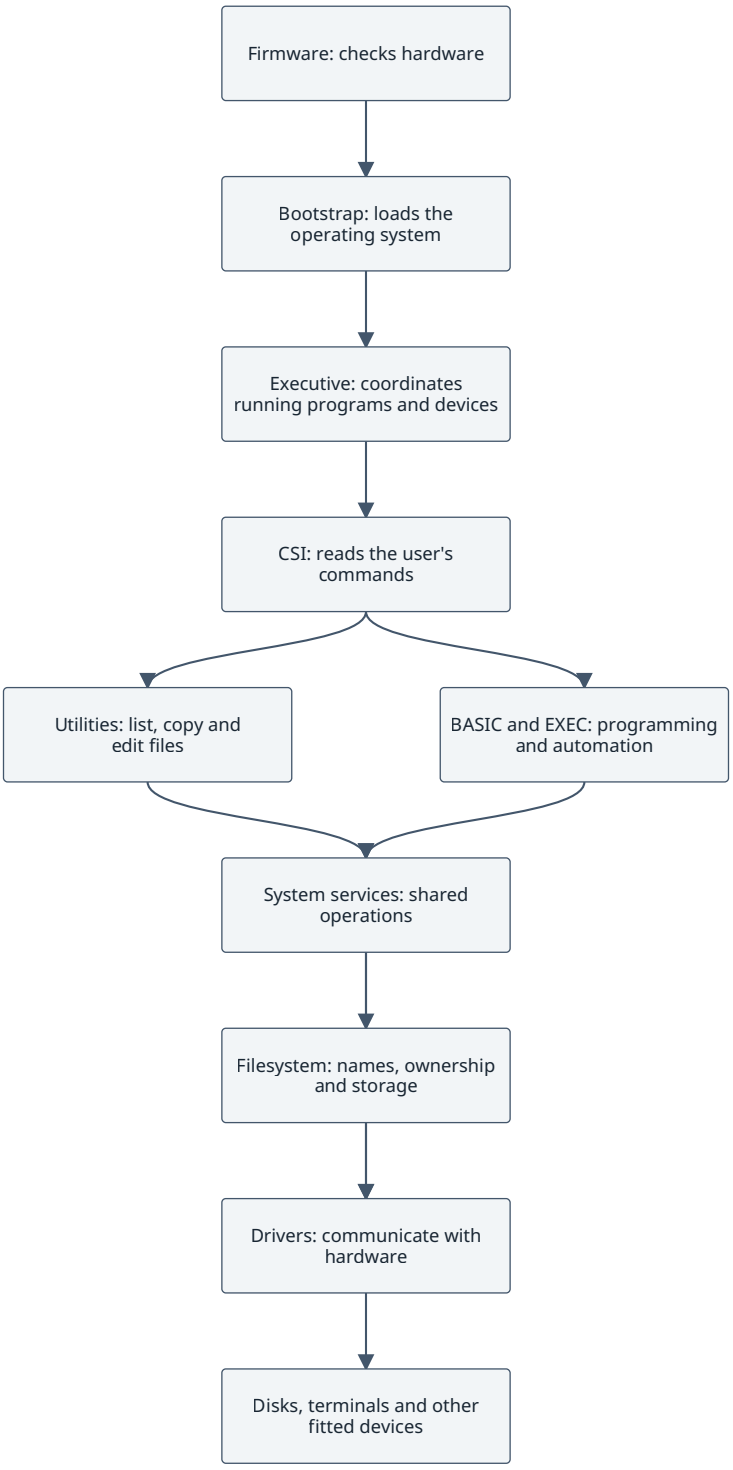


Figure 2.1. From startup to command execution. A device definition does not by itself establish that the corresponding hardware is fitted.

Starting the operating system

Firmware is the initial program held in the computer's read-only memory. It checks the hardware and reads a small disk program called the bootstrap. The bootstrap loads SISTEMA.NUCLEO, the operating system's executive. SISTEMA.STARTUP supplies further initialization code. It is an executable component, not a text file of commands for the operator to edit.

The executive remains responsible for coordinating execution and access to devices. Once initialization and account connection have completed, a user normally reaches CSI's command prompt. Chapter 3 explains that user-visible sequence; Chapter 21 describes the bootstrap and initialization in detail.

Accounts, partitions and shared work

An account identifies the user and the files that user can access. A partition provides a running environment, including memory and terminal assignments. The executive switches between partitions so that several users can make progress on the same computer.

Part of memory holds routines shared by the partitions. Other parts are selected as different programs run. This memory arrangement is called banking. It lets the system use more memory than the processor can address at one instant. You do not need to choose banks to list or edit a file; the memory arrangement is explained in Chapter 20.

Commands, languages and system services

CSI reads the command line and selects the requested program. Most utilities are separate files of type COMANDO. Thus EDITAR and COPIFI are programs that CSI starts, rather than editing and copying operations built into the prompt itself.

BASIC provides its own editor, interpreter, compiler and runtime for BASIC programs. EXEC reads command files that automate command-level work. Each has its own instructions and input context. The prompt tells you which component is currently accepting input; Chapter 4 teaches that distinction before you begin entering programs.

Programs request common operations through system services: displaying characters, opening files or obtaining information about the current account. The executive carries out those requests and returns the appropriate result. Chapter 19 introduces the programmer's calling interface; Chapter 24 explains its internal dispatch mechanism.

Files and devices

The filesystem connects a file's name to its ownership, organization and stored contents. Utilities use it to find the data they need. A name alone does not determine how those contents should be interpreted: ordinary text, BASIC source and compiled programs require different operations.

A logical device name gives a program a way to select a device without naming its hardware connections. For example, S selects the system disk association and IMPR1 selects a printer association. The administrator connects logical names to physical device definitions. A driver then supplies the hardware-specific operations. This separation lets a program use an assigned printer without knowing its controller ports.

Chapter 5 introduces the file notation seen by users. Chapter 11 and Chapter 12 explain storage and device administration; Chapter 23 develops the underlying storage and device model. Later chapters distinguish an operation accepted by software from data actually written or printed, which matters when diagnosing incomplete output.

Chapter 3 — Starting, connecting and leaving

Begin with the Secoinsa Serie 20 computer powered and configured by its administrator. During startup, ROM diagnostics precede `Carga del S.O.`—“loading the operating system”—and the system banner. Wait for the account prompt; do not type commands while loading is in progress.

Connect to your account

The system asks:

```
Nombre de cuenta a conectarse:
```

The administrative account used in this guide is SISTEMA. Type its name and press Return:

```
SISTEMA
```

SISTEMA is the real administrative account, not a practice account to create. For ordinary work, try to use an account with fewer privileges. Login masks password input. A successful connection normally reaches the CSI prompt, >. An account can instead start a command file automatically, so a menu after login can be normal.

Make your first enquiry

At >, enter:

```
AYUDA
```

AYUDA means “help.” With no topic it displays the command summary. If the display pauses, press Return to continue. You have asked a question of the system without changing a file.

Now ask about one command:

```
AYUDA LISDIR
```

This displays help for LISDIR, the directory-listing command. You will use that command in the following chapters. The distinction matters: `AYUDA LISDIR` describes LISDIR; `LISDIR` performs a listing.

AYUDA normally displays a matching topic. If lookup fails, it can invoke the named utility with AYUD as its argument. AYUDA BORRAR follows this fallback and reports `Falta tipo del fichero` instead of displaying help. Use the BORRAR reference for deletion instructions; do not assume that every topic request is a passive lookup.

Finally, check your identity:

```
ESTADO QUIEN
```

ESTADO reports a selected part of system state; QUIEN selects the current identity. Unlike AYUDA, ESTADO requires a subject. Entering ESTADO alone produces a syntax error.

Finish a session

Save your work and leave the application using its own completion command. At the CSI prompt, enter:

```
FIN
```

FIN disconnects the account and returns to the login path. It is a logout command. It does not promise to close another program's unfinished output or power down the whole machine.

Restart and interruption

Escape followed by Q requests interruption. The application's current handler determines when and how it responds; return to CSI need not be immediate. Prefer the application's own exit command and check which prompt returns. Forced interruption can leave incomplete output. Chapter 7 explains the pending-interruption behavior in EDITAR.

Escape followed by I asks `Desea IPL (S/N) -`. S confirms a restart; N declines the restart but terminates the interrupted command rather than resuming it. Neither response is a save operation. Restart affects other partitions as well. Chapter 14 discusses maintenance and the limits of the shutdown interface.

Part II — Using the system

Chapter 4 — Using the terminal

A terminal sends characters to the computer and displays characters returned by it. The current prompt tells you which program is reading those characters. Before typing, check whether you are at CSI (>), EDITAR (*) or BASIC (-).

The Secoinsa Serie 20 computer was paired with the Secoinsa IS 9502 computer terminal, with a separate keyboard and a green monochrome screen displaying 24 lines of 80 characters. The terminal sends keystrokes to the computer; CSI, the editor or the running program decides what those keystrokes mean. The keyboard and display do not form a separate computer running this operating system. The Preface gives the historical hardware context.

Enter and correct a line

At CSI, type the following, then press Return:

```
AYUDA
```

Return submits the line. The terminal connection uses carriage return, character 0x0D, for this purpose. If you mistype a character before submitting, Backspace or Delete removes the preceding character. Ctrl-X clears the whole input line so that you can start again. Escape followed by lowercase c also cancels the current CSI input line.

CSI normally folds lowercase letters to uppercase. Use uppercase letters and digits for practice filenames. Underscore is unsuitable here: the CSI input path can treat it as Delete. BASIC quoted strings have their own rules, so do not infer their behavior from CSI input editing.

Key	CSI operation
Return	Submit the current line; continue a paginated listing
Backspace or Ctrl-H	Erase the preceding character
Delete	Erase the preceding character
Ctrl-X	Discard the current input line
Escape, lowercase c	Cancel current input line

Ctrl-Z is not a CSI previous-command recall key. Physical BREAK has no sufficiently specified general operation in this guide. BASIC's BREAK command is a separate debugging facility.

Pause, resume and restore output

Escape followed by Q requests interruption through the application's current handler. It is not a universal immediate exit. EDITAR can retain the request until a later operation; use its completion command and check for the CSI prompt. See Chapter 7.

Escape followed by S pauses the requesting partition. Press the same pair again to resume. Do not type the rest of a command while paused: ordinary queued input can be discarded on resumption. Other partitions continue running.

Escape followed by O toggles ordinary output and keyboard echo. The program can keep running while its display is suppressed. If the terminal becomes silent after that control, repeat it to restore output. This differs from both a pause and an interrupt.

Recognize a terminal setup problem

If commands are accepted but cursor movement or screen layout is wrong, the system may be using the wrong terminal class, the component that translates screen controls for that terminal. If characters are garbled, the communication settings may differ. Report the symptom to the administrator rather than changing terminal settings while learning the command language.

If rapid pasted input loses characters, stop sending, clear the partial command and enter it again at a slower rate. The administrator should check flow control and the terminal's communication settings. A correct screen display does not by itself guarantee reliable sustained input.

The normal console dimensions are L79/P23. Leave them unchanged: reducing L can shorten the amount of input accepted on a command line, as well as affecting display behavior. Restore output with Escape–O if it was suppressed, and distinguish that condition from a communication fault.

Chapter 12 explains communication settings, terminal classes and their configuration. Chapter 28 explains how terminal controls reach the device.

Chapter 5 — Commands, filenames and disks

The command line tells CSI what to do and which object to use. Start by reading a concrete example from left to right:

```
LISDIR NOTE.TEXT:S
```

LISDIR is the operation. NOTE is the filename's first field. TEXT is its type field. The dot separates those fields. The suffix `:S` selects the system volume through its logical disk association, S. In these examples, read it as ‘on the system disk’: NOTE.TEXT:S means the file NOTE.TEXT on that volume. This command asks for the catalogue entry for one file; it does not display the file's contents.

S is a logical disk name, not the literal volume label SISTEMA. SISTEMA is also the administrative account name and the first field of many system filenames. A storage volume holds files; a partition in this operating system is a memory and execution environment, explained in Chapter 9. The `:S` suffix selects storage, not an execution partition.

Each filename field has at most eight characters. Use uncomplicated uppercase names such as NOTE.TEXT and HELLO.BASICOBJ while learning. A type is a name, not a guarantee of contents: naming a file BASICOBJ does not turn arbitrary text into a BASIC program.

List several files

Enter:

```
LISDIR *.*:S
```

The first asterisk matches names; the second matches types. This requests the files visible to your account on S. Ownership can hide another account's private files. An empty result therefore does not mean the entire volume is empty.

To narrow the selection, enter:

```
LISDIR SISTEMA.*:S
```

This selects the name SISTEMA with any type. The system uses that family for components such as SISTEMA.CSI and SISTEMA.AYUDA. Appendix C explains those components individually.

The question-mark wildcard can match a position in a listing pattern, but wildcard support is command-specific. Do not use a listing pattern in a copying or maintenance command unless that command accepts it.

Add an option

Enter:

```
LISDIR *.*:S (SORT)
```

Parentheses enclose options. SORT requests a sorted listing. These parentheses are literal characters, unlike the square brackets used to mark optional material in syntax descriptions.

Drive S is an association maintained by the system. A–G can designate other disks. They are not subdirectories. Omitting the drive can select a work unit that has not been mounted; spelling `:S` explicitly makes the destination explicit in these examples.

Read completion messages

A command may finish with `CR = n`, a time and `TE = ... minutos.`. CR is its completion result; TE is elapsed time. CR 0 reports command success. Read preceding messages too: a checker can finish successfully while reporting filesystem problems.

A syntax error usually calls for checking the command name, spaces, colon and parentheses before trying again. A file-not-found message calls for checking the name, disk association and current account. Chapter 33 separates command results from BASIC errors and system message numbers.

File organizations

Organization	What it is for
S	Sequential text or serialized records
SR	Sequential storage with read protection
D	Fixed-length records selected by number
I	Keyed records with an ordered index
K	Keyed records without the I-format ordered index
R	Relocatable machine-code programs
A	Programs placed at absolute addresses

You will normally choose a program appropriate to the file, rather than manipulate these letters directly. LISTAR displays sequential text; BASIC loads its own saved source; RUN executes compiled BASIC. Their internal formats are explained after the programming chapters.

Full names and abbreviations

This document uses full command names. With standard abbreviation enabled, most listed commands can also be shortened to their specified minimum prefix; intermediate prefixes are accepted. For example, ASOCIAR can be entered as ASO, ASOC, ASOCI, ASOCIA or ASOCIAR. They request the same association operation. ASIGNAR is a different command: its short forms include A and AS, which do not select ASOCIAR.

The minimum is defined separately for each command. It is not always three letters, and some command names must be written in full. There are two important exceptions to the intermediate-prefix rule: B selects BASIC but BA selects BACKUP, while BAS and BASI select BASIC; EXEC must always be written in full because EX and EXE can select the wrong command file. Use the full names until these distinctions are familiar.

Options have their own spelling rules. In ESTADO MEMORIA, ESTADO is the command and MEMORIA selects the information to display. MEM is a permitted abbreviation of that option even when command-name abbreviation is disabled. A full keyword is the system's accepted spelling, which may itself be a shortened Spanish word; do not extend it into an assumed longer word.

Chapter 35 lists the full names, permitted short forms and settings. It also explains exceptions caused by synonyms, filenames and option-name collisions.

Chapter 6 — Your first file

This chapter creates a small text file and takes it through the ordinary file operations. Use a practice account that may run EDITAR and write files on S. First make sure that NOTE.TEXT, COPY.TEXT and OTHER.TEXT are not names you already use.

1. Check the practice names

Enter:

```
LISDIR NOTE.TEXT:S
```

This asks whether NOTE.TEXT is visible on S. If it already exists, choose a different unused name and substitute it consistently below. A file you recognize as your own is not expendable merely because its name matches an example.

2. Start the editor

Enter:

```
EDITAR NOTE.TEXT:S
```

EDITAR opens a text-editing session for that name. Its prompt is *. You are now giving instructions to the editor rather than CSI.

Enter:

```
CASE U
```

This selects uppercase handling for the text you are about to enter.

Enter:

```
INPUT MY FIRST FILE
```

INPUT adds the words following it as a line of file text. INPUT itself is an editor instruction and is not part of that text.

Enter:

```
FILE
```

FILE saves the text and leaves EDITAR. Wait for the CSI > prompt.

3. Display what you saved

Enter:

```
LISTAR NOTE.TEXT:S
```

LISTAR displays the file. Its text includes `MY FIRST FILE`; headings depend on listing options. Compare this with LISDIR: LISTAR shows contents, whereas LISDIR shows the catalogue entry.

4. Copy the file

Check the proposed destination first:

```
LISDIR COPY.TEXT:S
```

If it already exists, choose another unused practice name. With the intended new name available, enter:

```
COPIFI NOTE.TEXT:S COPY.TEXT:S (VERIFI)
```

The first specification is the source. The second is the destination. COPIFI creates a separate copy; VERIFI requests verification. Do not reverse the two specifications.

Inspect the destination:

```
LISTAR COPY.TEXT:S
```

Its text should agree with NOTE.TEXT. The original is still available under its original name.

5. Rename the copy

Check the proposed new name:

```
LISDIR OTHER.TEXT:S
```

If it is unused, enter:

```
ATRIB COPY.TEXT:S OTHER.TEXT:S (NOCONSUL)
```

ATRIB changes the catalogue name. NOCONSUL suppresses the consultation prompt for this one-file example. The stored data is retained under OTHER.TEXT; this is not another copy.

Check the new name:

```
LISDIR OTHER.TEXT:S
```

You should now find the renamed entry. Use OTHER.TEXT for the next step, because COPY.TEXT is no longer its name.

6. Remove the practice copy

First inspect the exact name selected for deletion:

```
LISDIR OTHER.TEXT:S
```

Check that it is the practice copy you renamed. Keep NOTE.TEXT for later examples.

> Caution — immediate deletion. The following exact-name command deletes the selected file without a confirmation stop, even when CONSUL is supplied. Check the filename before entering it.

Delete the practice copy:

```
BORRAR OTHER.TEXT:S
```

BORRAR removes the entry immediately and releases its allocation. Deleted space can be reused; deletion is not a dependable undoable operation. The confirmation behavior for wildcard selection is different; see the command reference.

7. Check free space

Enter:

```
ESTADO DISCO
```

`Disponible` reports total free capacity. `Mayor zona` reports the largest contiguous free region. The distinction becomes important for direct files, which can need one continuous allocation. You have now completed the basic create–inspect–copy–rename–delete cycle while keeping the original practice text.

Chapter 7 — Editing files and running programs

A text file and a compiled program serve different purposes. EDITAR lets you enter and save ordinary text; LISTAR displays that text afterward. RUN starts a compiled BASIC program. This chapter develops the editing steps introduced in Chapter 6, then explains how to recognize and run a program that has already been compiled.

Prepare a text file

Use DRAFT.TEXT for this exercise. At the CSI prompt, check the name first:

```
LISDIR DRAFT.TEXT:S
```

If it already exists, choose an unused name and substitute it throughout this editing example. The example creates a new practice file; it must not replace text you want to keep.

Start the editor:

```
EDITAR DRAFT.TEXT:S
```

EDITAR opens a text-editing session. Its command prompt is *, rather than CSI's >. Until you leave the editor, the instructions you enter are editor instructions.

Enter two lines

At the editor prompt, enter:

```
CASE U
```

This selects uppercase handling for the text you enter. Add the first line:

```
INPUT FIRST LINE
```

INPUT puts the words following it into the file. The saved text is FIRST LINE; INPUT itself is not saved as part of the line.

Add another line:

```
INPUT SECOND LINE
```

The file now has two lines of text. These words are file contents, not commands to be executed.

Save and return to CSI

Enter:

```
FILE
```

Normally FILE reports `salvado y fin edicion`, saves the text and returns to CSI. Wait for the > prompt before entering another system command.

If Escape–Q was entered at the editor prompt, the interruption request can remain pending. The next FILE can report `salvado` and leave EDITAR active at its * prompt. On that path, enter FILE again to save and finish. This is a response to the remaining editor prompt, not a requirement to enter FILE twice for every save. Escape–Q is not a dependable substitute for the editor's completion command.

Once CSI has returned, inspect the saved file:

```
LISTAR DRAFT.TEXT:S
```

The contents should be FIRST LINE followed by SECOND LINE. LISTAR reads the text for display; it does not reopen an editing session. Check the saved contents before relying on the file in later work.

Protect existing text before editing

Before valuable edits, make an explicit copy using the COPIFI procedure in Chapter 6. EDITAR's COPIA option requests a backup and is the documented default; NOCOPIA suppresses it. The complete backup naming and recovery rules are unspecified, so the automatic backup is not a substitute for a copy whose name and contents you have checked.

The dependable text-creation workflow here uses CASE U, INPUT and FILE. The full movement, search, replacement and recovery command set is not sufficiently specified for a step-by-step editing course. Do not borrow commands from another editor. TEDITAR is a small-file editor; CEDIT edits structured records. Their interfaces are separate.

Recognize a program that RUN can execute

RUN executes compiled BASIC files, normally of type BASICCOM. The source from which they are built is handled by BASIC and saved as BASICOBJ. These are different stored representations. Renaming DRAFT.TEXT to give it a BASICCOM type would not turn its text into a program.

A COMANDO utility is normally invoked by its command name, as EDITAR and LISTAR were above. RUN is the entry point for compiled BASIC; it is not a general way to open every file.

The next example uses HELLO.BASICCOM:S, produced by the complete programming example in Chapter 15. Follow these steps when that compiled file is available. If you have not built it yet, continue with Chapter 8 and return to this example after completing that program.

Run a compiled program

At CSI, check that the compiled file exists:

```
LISDIR HELLO.BASICCOM:S
```

If it is absent, complete the source, save and compilation steps in Chapter 15. A source file alone is not sufficient for this RUN command.

With the compiled greeting available, enter:

```
RUN HELLO.BASICCOM:S
```

RUN loads the compiled BASIC program and executes it using the BASIC runtime. For the unchanged greeting from Chapter 15, the program displays HELLO WORLD and finishes. When CSI returns, you can enter another command.

Editing or saving BASIC source does not update an older BASICCOM automatically; compile the changed source before expecting RUN to execute the new version. To end an application, use its own exit operation. BASIC QUIT leaves the BASIC editor/interpreter, whereas BASIC END ends a running BASIC program. Forced interruption can leave open files incomplete, even when CSI returns to the screen.

The next chapter uses ordinary text files for a different purpose: storing instructions that automate command-level work.

For the editor’s complete list of recognized internal names and the limits of their argument descriptions, see the EDITAR reference in Chapter 35.

Chapter 8 — Automating work with EXEC

EXEC is the command-file interpreter. It reads a saved text file and carries out the instructions in that file, so you do not have to type the same sequence at the system prompt each time. A command file can contain ordinary system commands together with EXEC's own instructions for displaying text, using variables, making decisions and repeating work. Its usual file type is EXEC.

You create this text with EDITAR, using the editing steps from Chapter 7. You then leave the editor and ask EXEC to read the saved file. No BASIC compilation is involved: RUN executes compiled BASIC, while EXEC interprets command-file text. Begin with a file that displays a greeting and stops, before adding variables or loops.

Your first command file: HELLO WORLD

Use the name HELLO.EXEC. At CSI, check that it is available:

```
LISDIR HELLO.EXEC:S
```

If it already exists, choose another unused name and substitute it throughout the example. This command file is separate from DRAFT.TEXT, the plain text file created in Chapter 7.

Start the editor:

```
EDITAR HELLO.EXEC:S
```

The editor's * prompt means it is ready for editing instructions. Select uppercase handling:

```
CASE U
```

Now enter the line that will display the greeting:

```
INPUT &TYPE HELLO WORLD
```

INPUT belongs to EDITAR: it puts the rest of the line into the file. The saved instruction is `&TYPE HELLO WORLD`. TYPE belongs to EXEC and displays the following fields, here HELLO and WORLD. The ampersand introduces the EXEC instruction. Nothing is executed merely by entering this text in the editor.

Add the instruction that ends the command file:

```
INPUT &QUIT
```

This saves `&QUIT` as the second line. QUIT tells EXEC to finish the command file and return to its caller. It does not exit EDITAR; you are still composing text.

Save and leave the editor:

```
FILE
```

Wait for the CSI > prompt. If FILE leaves the editor active after a pending interruption, follow the save-and-exit guidance in Chapter 7.

Inspect the saved instructions

At CSI, enter:

```
LISTAR HELLO.EXEC:S
```

The file should contain these two lines:

```
&TYPE HELLO WORLD
&QUIT
```

This block shows the saved contents; do not type it as commands at CSI. INPUT, CASE U and FILE were instructions to the editor and must not appear in this command file. Check the contents before execution. A successful save means the text was written, not that the instructions are valid.

Execute the greeting

At CSI, enter:

```
EXEC HELLO
```

EXEC finds HELLO.EXEC through its file lookup and reads it. TYPE displays HELLO WORLD; QUIT ends the command file. When the CSI prompt returns, the sequence is complete. Use the full command name EXEC in these examples.

You can enter the same EXEC command again to repeat the greeting without reopening EDITAR. This is the purpose of a command file: the saved instructions can be reused.

The remaining sections introduce EXEC statements that belong inside command files. To put a statement into a file, use EDITAR's INPUT instruction as above; INPUT itself is not part of the saved statement. A complete program block shows the intended file contents. Examples described as commands entered at CSI, such as EXEC COUNT, are entered after saving and leaving the editor.

Display longer text

TYPE prints fields and substituted values. Ordinary fields have an eight-character limit; HELLO and WORLD each fit separately. For a longer line that should be printed literally, use this three-line construct in the command file:

```
&BEGTYPE
THIS IS A LONGER LINE OF TEXT
&END
```

BEGTYPE starts literal output; END closes it. The middle line is printed as text without variable substitution. The two delimiters are required parts of one construct. Retain a final &QUIT after the construct to end this greeting file.

Store a variable

Write:

```
&I = 1
```

This creates or changes variable I to the value 1. The ampersand marks EXEC language elements. A frame has sixteen ordinary variable cells; names and values occupy eight characters each.

Next write:

```
&TYPE VALUE &I
```

TYPE substitutes the value of I. The line therefore prints the label VALUE followed by 1.

To change it, write:

```
&I = &I + 1
```

The expression reads the old value, adds one and stores the result. Integer arithmetic includes truncating division: 7 / 2 gives 3. Addition wraps at the 16-bit boundary: 65535 + 1 gives 0.

Repeat a group

Create COUNT.EXEC with this complete body:

```
&CONTROL OFF
&I = 1
&WHILE &I LE 3
&TYPE COUNT &I
&I = &I + 1
```

```
&REPEAT
&QUIT 0
```

CONTROL OFF suppresses command echo. Assignment sets I to 1. WHILE checks whether I is less than or equal to 3. TYPE displays it; the assignment increments it; REPEAT returns to the condition. When I becomes 4, the condition fails and QUIT finishes with result 0.

At CSI, run it:

```
EXEC COUNT
```

The loop displays COUNT with values 1, 2 and 3, then returns to CSI.

Make a decision

The branch form is:

```
&IF &I EQ 4 &GOTO -DONE
```

IF tests equality; GOTO transfers control only if that condition succeeds. The target must exist in the file. A labelled ending can be written:

```
-DONE &QUIT 0
```

The label marks the destination, while QUIT is the statement executed there. Labels are not variables.

Pass arguments

For a script ARGTEST.EXEC whose body contains `&TYPE &1` and `&QUIT 0`, enter:

```
EXEC ARGTEST ALPHA
```

ALPHA becomes argument 1. `&2` selects the second argument, and `&INDEX` gives the count. EXEC stores at most sixteen argument fields. If IDX contains 2, `&&IDX` selects argument 2 indirectly.

The special operands `&*` and `&$` mean “any argument” and “every argument” in comparisons. For example:

```
&IF &* EQ ALPHA &GOTO -FOUND
```

This branches to a label named FOUND if at least one argument equals ALPHA. It does not expand into a string containing all arguments.

Work with short strings

Write:

```
&T = &SUB ABCDEFGH 3 5
```

SUB takes positions 3 through 5, inclusive, producing CDE. To join two fields, write:

```
&T = ABC | DEF
```

The vertical bar concatenates them into ABCDEF, subject to the ordinary value width.

Handle nested results

A script may call another using the full EXEC command. Ordinary variables belong to their own nesting level. After the child finishes, `&RETCODE` holds its result.

Register an error action before a child command:

```
&ERROR &GOTO -ERR
```

A qualifying nonzero result transfers to ERR. At that label, this statement propagates the result:

```
-ERR &QUIT &RETCODE
```

Without an error action, a nonzero child result need not stop the parent. Complete recursion and error-class rules remain unspecified.

EXEC keeps the state of nested command files in a work file for the current execution partition. The filename ends with the partition number: partition 1 uses SISTEMA.EXEC1, partition 2 uses SISTEMA.EXEC2, and so on. Even a script that only displays information can update this work file. Appendix C describes these files.

Optional implicit execution

At CSI enter:

```
PONER IMPEXEC ON
```

This permits an accessible script name to be entered without the EXEC prefix. To restore explicit-only operation, enter:

```
PONER IMPEXEC OFF
```

The full form remains the clearest form for instructions. Chapter 35 extends this introduction with queued program input, READ, SKIP, UNTIL, display controls and the complete directive index. It identifies the remaining interface limits separately from the examples.

Chapter 9 — Accounts, messages and everyday habits

An account gives your session an identity. A partition supplies the running context and terminal. Start sensitive work by checking both who you are and what you are allowed to do.

Check identity and privilege

Enter:

```
ESTADO QUIEN
```

QUIEN displays the current identity. Now enter:

```
ESTADO PRIVILEG
```

PRIVILEG displays the current privilege. A command can require a higher value than your account has; that is an authorization failure, not a spelling error.

Change account

To connect to the existing SISTEMA administrative account, when authorized, enter:

```
CONECTAR SISTEMA
```

CONECTAR asks for the account's password when required and changes the connection after successful authentication. When switching to an available account, you do not have to enter FIN first. An account number already connected in another partition cannot be connected a second time: the command reports `Esta cuenta esta siendo utilizada por la particion` followed by its partition number, with CR = 10. End the existing connection before trying that account again. This refusal precedes the password check. An incorrect password does not authorize the new identity. Check ESTADO QUIEN afterward if uncertain.

An account-named EXEC file can run automatically after connection. Its contents determine whether you reach a menu or the CSI prompt. This facility does not imply that every account has a startup file.

Send a short message

If the intended recipient is connected at partition 2, enter:

```
MENSAJE 2 HOLA
```

MENSAJE addresses the recipient and sends the short text HOLA. Use a known recipient; a partition number is not an account number.

For longer text, enter:

```
MENSAJE 2
```

The program prompts with ?. Enter one line at a time and finish with an empty line. Read the result message: CR 0 alone does not guarantee live delivery. When mailbox storage is offered, S accepts it. Address stored messages with the complete account name.

Read stored messages

Enter:

```
BUZON
```

BUZON displays messages for your account and removes the retrieved messages from the mailbox. Record anything you need before continuing; this is not a nondestructive repeated display.

To inhibit ordinary incoming utility messages, enter:

```
PONER MENSAJE OFF
```

To receive them again, enter:

```
PONER MENSAJE ON
```

Privileged broadcasts can still arrive while ordinary messages are inhibited.

A reliable daily sequence

Connect, check your account, inspect the file you plan to use and then start its application. Save before switching tasks. Use an explicit copy before a valuable edit. Read both command results and diagnostic text. Finish by closing the application and entering FIN at CSI.

For the basic file-work sequence, return to Chapter 6. The full alphabetical command reference is Chapter 31.

Part III — System administration

Chapter 10 — Administering accounts

An account identifies a user and determines the privilege used by commands. It also supplies the account number against which file ownership is checked. A partition is a separate matter: it provides memory and execution time. Creating an account does not create a partition, and stopping a partition does not remove its user's account.

SISTEMA is the administrative account used in the examples. Account maintenance requires privilege 5 and, at that privilege, the command requires the SISTEMA identity. Connect to SISTEMA as described in Chapter 9, then check the current privilege:

```
ESTADO PRIVILEG
```

The report must show the privilege required for maintenance. These are software checks; the machine does not provide a general protected-memory boundary between application and executive code.

List accounts and connected users

To see the registered accounts, enter:

```
CUENTA (PANT)
```

The listing contains columns headed `Cuenta`, `Pal.paso`, `Priv` and `Id`: account name, password, privilege and account number. This administrative listing can display stored passwords. Keep its screen or printed copy private. It is a list of account records, not a list of people currently using terminals.

To inspect connected users instead, enter:

```
ESTADO USUARIOS
```

Use that report before changing or removing an account. Arrange for the affected user to finish work and disconnect. Account names can be synonyms for the same number; deleting one name must not be mistaken for deleting every identity that shares its number.

Understand the maintenance dialogue

Start the interactive account editor:

```
CUENTA
```

The operation prompt is:

```
Modo (Nueva/Camb/Borrar/Fin)?
```

At this prompt the following letters select an operation. Each answer is followed by Return.

Answer	Meaning	What follows
N	Nueva: new account	Name, synonym, password and privilege
C	Camb: change an existing account	Name, password and privilege
B	Borrar: delete one account name	Name to remove
F	Fin: finish maintenance	Write the account list and return to CSI

Make one planned change at a time, finish with F, and inspect the account list again. F is the save-and-finish operation; leaving the machine or forcibly interrupting the program is not a substitute.

In the following examples, `<new-account>`, `<account-to-remove>` and `<new-password>` are placeholders. Type the chosen value without angle brackets. SISTEMA already exists: do not use it as the new-account or deletion example.

Add a user account

First use the account listing to choose an unused name. Use an uppercase name of no more than eight characters, beginning with a letter, and keep the remaining characters to letters and digits. Start CUENTA as above. At its operation prompt, enter:

```
N
```

At `Nombre cuenta`, enter the new name:

```
<new-account>
```

At `Cta.sinonima?`, press Return without typing anything. An empty answer requests a distinct account number. Supplying an existing name instead creates a synonym sharing that account's number; it is not the usual choice when adding a separate user.

At `Pal.paso?`, enter the password chosen for the account:

```
<new-password>
```

The maintenance dialogue echoes this input. Password storage is an eight-character field, and a comma is not accepted. Use no more than eight characters and avoid spaces, which this dialogue removes. An empty answer during creation leaves the new account without a password. Login masks input, but neither that masking nor a password makes the stored account file encrypted.

At `Privilegio?`, enter:

```
1
```

Privilege 1 permits ordinary file work but does not grant account maintenance. Some tools, including parts of the programming environment, require more privilege. Assign the privilege needed for the intended work rather than assuming every new account should match SISTEMA.

When the operation prompt returns, enter:

```
F
```

CUENTA writes the revised account list. Check it:

```
CUENTA (PANT)
```

Check the new name, privilege and assigned number. The program allocates a number; this example does not prescribe one. Test the new login from another terminal if available, keeping the administrative connection open. This is possible because the new account has a distinct account number; another name for an account already in use cannot be tested this way.

Change a password or privilege

Password changes are available through CUENTA's C operation. There is no separately established unprivileged password-change command. The procedure below changes SISTEMA's password; use another existing account name to maintain that account instead. Before changing the administrative password, ensure that a usable recovery arrangement is available. The new password cannot be tested while another partition remains connected under the same account number. An ordinary account must not be assumed able to restore administrative access; the backup and recovery limits are described in Chapter 13.

Enter:

```
CUENTA
```

At the operation prompt, enter:

```
C
```

At `Nombre cuenta`, enter:

```
SISTEMA
```

At `Pal.paso?`, supply the new password:

```
<new-password>
```

During a change, an empty answer keeps the existing password. It does not remove it. To remove a password deliberately, the special answer is a single hash character:

```
#
```

These are alternatives at the same prompt. Choose the new password, an empty answer to retain it, or # to clear it; do not type all three. Clearing the password permits connection without that password challenge.

CUENTA next asks `Privilegio?`, even if only the password was being changed. For SISTEMA's administrative privilege, enter:

```
5
```

When maintaining another account, enter its intended privilege. Do not copy 5 automatically. A blank privilege answer causes the prompt to repeat rather than preserving the previous value.

At the operation prompt, finish:

```
F
```

After F saves the change, end the existing SISTEMA connection at its terminal:

```
FIN
```

FIN disconnects that account. Then, at another available terminal, enter:

```
CONECTAR SISTEMA
```

Supply the new password when requested. If the command reports `Esta cuenta esta siendo utilizada por la particion 1` with CR = 10, partition 1 still holds that account number; another partition number can appear in the message. That refusal does not check the password. End the existing connection before retrying.

After the new login succeeds, check its privilege:

```
ESTADO PRIVILEG
```

For SISTEMA, the expected value is 5. Account aliases have separate name records, but names sharing an account number do not permit simultaneous connections. Changing one record must not be assumed to change every synonym's password.

Remove an account

Before removing an account, have its user close applications and enter FIN. Record its account number and decide what should happen to its files. Account deletion removes the named account record; it does not perform a file deletion or ownership transfer. Transfer required files before withdrawing the user's login.

> Caution — removes a login name. Do not delete SISTEMA as a practice operation. Check for synonyms and files owned by the account number before proceeding.

Enter:

```
CUENTA
```

At the operation prompt, enter:

```
B
```

At `Nombre cuenta`, supply the exact account selected for removal:

```
<account-to-remove>
```

The program removes the matching record from its working list and returns to the operation prompt. There is no additional per-account yes/no confirmation in this path. Finish and save:

```
F
```

Inspect the result:

```
CUENTA (PANT)
```

The removed name should be absent. Other names sharing its account number are separate records and may remain.

Do not substitute `CUENTA (ELIMINAR)` for this procedure. ELIMINAR removes the account system itself. `CUENTA (VACIAR)` clears account history; it also does not delete an individual account.

Ownership and sharing

The next operations require the current file owner's account. COMPART cannot be used by SISTEMA, and administrative privilege alone does not make CAMPRO select another owner's private files. Use a distinct non-SISTEMA account with privilege 3 or higher for these operations. Account creation and privilege maintenance are described above; do not assign privilege 5 merely to follow this example.

At CSI, connect to the chosen account, substituting its actual name:

```
CONECTAR <source-account>
```

The account number must not already be connected elsewhere. Supply its password when requested, then check the identity:

```
ESTADO QUIEN
```

Check that it has sufficient privilege:

```
ESTADO PRIVILEG
```

The following commands require at least 3. Create NOTE.TEXT under this account using the steps in Chapter 6. If that name already exists or resolves to a public file, choose an unused practice name and substitute it throughout this procedure. Do not reuse another account's file merely because it can be listed. Inspect the file and its ownership before proceeding:

```
LISDIR NOTE.TEXT:S
```

To display the accounts sharing this practice file, enter:

```
COMPART NOTE.TEXT:S (PANT)
```

PANT lists grants. With none present, an empty result and CR = 0 are expected. From SISTEMA, this operation instead reports `La cuenta sistema no puede compartir ficheros`, CR = 12. Ordinary COMPART enters a grant/revoke dialogue. Sharing creates an alias to the owner's file; it does not duplicate the allocation. The owner retains the data and its protection policy.

While still connected as the current owner, transfer the practice file to SISTEMA:

```
CAMPRO NOTE.TEXT:S (SISTEMA NOCONSUL)
```

This changes ownership to SISTEMA. NOCONSUL suppresses consultation. Running the same command from SISTEMA does not pull another account's private file into its ownership; it can report `No se ha cambiado ningun fichero`, CR = 12. Check the current owner when this occurs.

End the source account's session:

```
FIN
```

When no other partition is connected under SISTEMA's account number, connect to the recipient account:

```
CONECTAR SISTEMA
```

Supply its password if requested. Inspect the transferred entry:

```
LISDIR NOTE.TEXT:S
```

Read the practice text to check that its contents remain available:

```
LISTAR NOTE.TEXT:S
```

Transfer to SISTEMA does not guarantee loss of lookup access for the former owner: SISTEMA's account number 0 participates in public-file fallback. Use COMPART when the original owner is to retain ownership and another account needs shared access.

Protect the account service

SISTEMA.CUENTA stores account records and uses special ownership byte 0xFF. It is not an ordinary public text file to rename or rewrite casually. CONECTAR.COMANDO performs connection. If either becomes unreachable, CSI can appear without the normal authentication path. A visible command prompt alone is therefore not proof of a healthy account service.

CAMBIAR changes a command's minimum privilege field; it does not change the current account's privilege. Application checks can impose further restrictions. Chapter 31 explains that command and Appendix C describes the account components.

Chapter 11 — Managing storage

A disk association tells the system which volume a name such as S or A denotes. Before modifying storage, identify the association and inspect its capacity. Never infer the destination from a familiar letter alone.

Inspect capacity

Enter:

```
ASOCIAR
```

With no arguments, ASOCIAR lists the current associations. Find the disk letter you intend to use. Then enter:

```
ESTADO DISCO
```

Compare Disponible, the total free space, with Mayor zona, the largest continuous free region. A direct file can require one contiguous allocation. For example, 38 KiB split into separate runs of 1 KiB and 37 KiB cannot satisfy a 38 KiB contiguous request. A sequential file can use several runs.

ESTADO DISCO can correct a cached free-count field when it recomputes capacity. It is a status operation with a possible metadata update, rather than a byte-for-byte read-only inspection.

Associate and read a floppy

Distinguish the physical device name, the logical disk letter and the volume label. Each identifies a different part of the storage arrangement. FLOPPY1 and FLOPPY2 identify physical device definitions. The letters `A` through `G` designate logical disks that programs can access. The volume label is the name assigned to the medium, independent of its logical disk letter. Associating A with FLOPPY1 does not rename that floppy's volume, and A is not inherently a floppy drive.

The following procedure requires a working compatible floppy drive and an already initialized system-format floppy. FLOPPY1 and FLOPPY2 select different driver interfaces; use the definition matching the fitted controller. An association by itself does not prove that the drive is ready.

First display the associations:

```
ASOCIAR
```

Make sure A is available for this procedure. If an application is using its current association, close the application's files before changing it. Check the physical definitions:

```
LISTAR SISTEMA.NOMBDISP:S
```

For a fitted drive using FLOPPY1, associate it with A:

```
ASOCIAR A FLOPPY1
```

This selects the first floppy definition. Where the installed drive uses FLOPPY2 instead, substitute that physical name. Do not choose a definition merely because it appears in the file.

Insert the prepared floppy while the drive is idle and close its loading mechanism. Notify the operating system:

```
MONTAR A
```

MONTAR tells the system that the disk or floppy has changed. It refreshes the device's media state; it does not attach a filesystem beneath a directory path. It neither formats a blank floppy nor assigns its logical letter.

List the files visible to the current account:

```
LISDIR *.*:A
```

The suffix `:A` makes the device explicit. A missing file can reflect account ownership rather than an empty medium. To read the practice file when it is present, enter:

```
LISTAR NOTE.TEXT:A
```

This displays the file on the floppy. It does not read NOTE.TEXT on S.

Copy a file to and from a floppy

Use writable expendable media with enough free space and an unused destination name. To copy the closed practice text from S to A, enter:

```
COPIFI NOTE.TEXT:S NOTE.TEXT:A (VERIFI)
```

The first filename is the source; the second is the destination. VERIFI requests a readback check. Wait for completion, then inspect the result:

```
LISTAR NOTE.TEXT:A
```

To copy it back under a different, unused name, enter:

```
COPIFI NOTE.TEXT:A RETURN.TEXT:S (VERIFI)
```

The reversed drive suffixes reverse the direction. Using RETURN.TEXT avoids replacing the original practice file. Display the returned copy:

```
LISTAR RETURN.TEXT:S
```

Compare its text with the original. For a larger archive spanning removable media, use the ARCHIVAR and RECARGAR procedures in Chapter 13.

Change or release removable media

Before changing a floppy, finish transfers and close every file using it. Wait for drive activity to stop, replace the medium, then enter:

```
MONTAR A
```

List the new medium before using it. A `no prep.` diagnostic indicates that the device is not ready; check that the intended medium is inserted and available. A `prot. escritura` diagnostic indicates write protection. Restore the intended media or protection state before retrying; do not select Ignore to turn a failed write into an apparent success. Chapter 27 explains the distinct retry, ignore and abort choices. When several units have changed, the alternative form is:

```
MONTAR *
```

The asterisk asks the system to examine all units. It is not a wildcard filename.

To release the logical association after work has finished, enter:

```
ASOCIAR A
```

This removes A's association. It does not eject the floppy or close an application's files for it. No separate universal unmount-and-flush command is specified here; closing files and completing transfers come first. Do not remove active media or disassociate S while the operating system depends on it.

Format a floppy

Formatting prepares the physical tracks and sectors. INICIAR is the floppy utility; INICDISC is the disk utility. Both also offer operations that change labels, directory capacity or write protection. These are distinct requests, not synonyms for MONTAR.

> Caution — formatting destroys existing contents. Check A's association and the physical medium before issuing FORMAT or VACIA. Use an expendable floppy of the correct size, sidedness and density for the fitted drive. The minimum command privilege is 3.

Read the floppy utility's help before selecting a format:

```
AYUDA INICIAR
```

For a drive and medium supporting the utility's double-sided, double-density format, its documented format example can be written in full as:

```
INICIAR A (FORMAT CARA 2)
```

FORMAT requests physical formatting; CARA 2 selects two sides. In the FLOPPY2 path, this form uses 77 cylinders, two heads and 26 sector IDs per track. Both tracks of cylinder 0 use FM recording with 128-byte sectors; cylinders 1–76 use MFM with 256-byte sectors. These are the utility's software-format characteristics, not identification of a drive manufacturer, model or interchangeable PC format. The fitted drive and medium must support them.

The following is a syntax template, not a command to type literally:

```
INICIAR A (FORMAT CARA <sides> PISTA <tracks> SECTOR <sectors>)
```

The help lists these geometry options, but it does not establish that every PISTA or SECTOR value changes every physical formatting loop. The FLOPPY2 loop described above reaches cylinder 76. Do not use the template to assume support for arbitrary geometries or derive values from a hard disk or a modern floppy format. The options are:

Option	Operation
FORMAT	Physically format the floppy; required for a virgin medium
CARA n	Select the number of sides
PISTA n	Request tracks per side; physical-loop behavior depends on the controller path
SECTOR n	Request sectors per track; arbitrary physical-format values are not established
DIRE n	Request directory capacity for n files

Option	Operation
ETIQU	Change the volume label
VACIA	Erase the file contents described by the volume's directory
WP / NOPE	Set or clear software write protection
BOOT=d	Copy the bootstrap track from the specified disk

Do not add BOOT merely to make an ordinary data floppy. The source bootstrap and destination hardware must be compatible. The full initialization dialogue and its success messages are not specified here, so no sample completion output is supplied.

After the utility completes successfully, refresh the device:

```
MONTAR A
```

Check the format:

```
VERFORM A
```

Read the diagnostics as well as the completion result. This does not replace file-copy verification or certify compatibility with another drive. Then inspect its directory:

```
LISDIR *.*:A
```

Then use the preceding copy-and-readback procedure with a small practice file. A successful listing alone does not verify every track of the medium.

Initialize a hard-disk volume

INICDISC formats the selected disk volume and creates its filesystem structures. It operates on the disk named by a logical association such as A. It does not create an execution partition, and ASOCIAR does not reserve or divide physical disk space.

> Caution — formatting destroys the selected volume's contents and can change disk-mapping metadata. Back up any data to retain, stop applications using the destination, and verify its physical extent. Never apply the example to S or to a region overlapping the system volume. Minimum privilege is 3.

Select the destination

Begin by inspecting the current assignments:

```
ASOCIAR
```

A must designate the intended expendable hard-disk volume, not a floppy or another active device. If no suitable volume is configured, use the following additional-volume guidance before proceeding.

Optional: prepare an additional volume

An additional data volume can be made available when the disk configuration already provides a suitable unused extent and device definition. For a configuration in which DISCO2 names that intended extent, enter:

```
ASOCIAR A DISCO2
```

This connects logical letter A to the DISCO2 definition. It does not prove that storage has been reserved or that a usable filesystem exists there. Continue only after verifying that the target extent is available and does not overlap any volume to retain. DISCO2 is a conditional example, not a claim that an additional volume is supplied or configured on every system.

The documented formatting path for DISCO2 uses PISTA both for its cylinder count and its starting cylinder. With PISTA 397, its physical span is cylinders 397–793 inclusive. Changing PISTA can therefore move the beginning of the formatted region as well as change its size. Do not reduce it to shorten formatting without rechecking the entire disk layout. The number 397 is an example for a compatible arrangement, not a universal disk parameter.

The remaining steps initialize a filesystem on that configured extent. A general procedure for creating arbitrary new physical-volume descriptors or resizing populated volumes is not established. A name in SISTEMA.NOMBDISP alone is insufficient to perform those operations safely.

Format and review the dialogue

For an expendable destination whose configured geometry is five heads, 397 cylinders and 32 sectors per track, with 256-byte sectors, enter:

```
INICDISC A (FORMAT CARA 5 PISTA 397 SECTOR 32 DIRE 64)
```

FORMAT requests physical formatting. CARA gives the number of heads, PISTA the cylinder count with the placement qualification above, and SECTOR the sector count per track. DIRE requests directory capacity. The utility can adjust that request: this form reports 88 directory entries, not exactly 64. Review the displayed values before accepting them.

At `Usted quiere continuar (S/N)?`, enter the following only if the destruction warning names the intended destination:

```
S
```

The utility asks `Etiqueta del disco:`. Supply a new volume label; for this example:

```
PRACTICE
```

The label identifies the formatted volume. It does not rename association A or the physical device definition. The utility next displays directory capacity, interleave, track offset and geometry. At `Es correcta esta informacion (S/N)?`, accept only after checking the entire summary:

```
S
```

At the optional surface-analysis question, this elementary formatting example selects:

```
N
```

This skips the separate surface analysis; the result must not be described as a complete surface test. At `Desea reintentos en lectura y escritura (S/N)?`, enter:

```
S
```

This enables the requested read/write retries. At `Quiere incluir ahora alguna pista (S/N)?`, the example supplies no additional bad-track entries:

```
N
```

Do not reuse that answer to conceal known media defects. Bad-track handling requires accurate information for the actual medium. Wait for formatting to finish and inspect any diagnostics; a command result alone does not certify the data.

Mount and check the new filesystem

Notify the system of the initialized volume:

```
MONTAR A
```

MONTAR refreshes media state. Check the format:

```
VERFORM A
```

Read the reported errors before proceeding. Then list the catalogue:

```
LISDIR *.*:A
```

This lists entries visible to the current account. To check ordinary file use, start with the closed NOTE.TEXT practice file from Chapter 6 and unused destination names. Copy it to the new volume:

```
COPIFI NOTE.TEXT:S NOTE.TEXT:A (VERIFI)
```

VERIFI requests readback verification. Copy it back under a different name:

```
COPIFI NOTE.TEXT:A CHECK.TEXT:S (VERIFI)
```

This preserves the original source. Inspect the returned text:

```
LISTAR CHECK.TEXT:S
```

Compare it with NOTE.TEXT. These checks exercise mounting, catalogue access, allocation and file transfers; they do not establish bootstrap compatibility or replace a backup procedure.

IBM-compatible interchange floppies

IBM 3741 interchange is a separate format path. INICIBM writes the headers used by the interchange utilities; it is not a general synonym for INICIAR and does not by itself establish that a virgin floppy has been physically formatted correctly.

For an expendable, correctly prepared compatible floppy on A, enter:

```
INICIBM A
```

This changes the medium's headers. To inspect an IBM-format directory, use:

```
DIRIBM A
```

DIRIBM reports the volume and file records in that format. Use SECIBM to write sequential system data to an IBM 3741 floppy and IBMSEC for the reverse conversion. These utilities handle record-format and character-code conversion; ordinary COPIFI does not replace them.

Understand protection

ATRIB controls independent write, deletion and read restrictions. These bits affect particular access paths, not a universal “make everything safe” state.

For a practice sequential file owned by your account, enter:

```
ATRIB NOTE.TEXT:S (E NOCONSUL)
```

E sets write protection without changing the name. Attribute-only changes use one filename; the two-name form is for renaming. Inspect the result:

```
LISDIR NOTE.TEXT:S
```

For the sequential practice file, the format indication changes from S to SW while the text is retained. Write protection can block extension while still permitting replacement through deletion and recreation. B protects deletion; L is the internal read-protection bit; ATRIB accepts LE for this operation. The longer word LEER is rejected. Clearing it with NOL requires privilege 255.

Individual protection	Read existing contents	Replace by deleting and recreating	Append to existing contents
E, write	Allowed	Allowed through deletion/recreation	Refused
B, deletion	Allowed	Refused	Allowed
L, read	Refused	Allowed through deletion/recreation	Refused

The table describes the current owner's sequential-file access with each protection applied separately. Replacing a file can involve deleting the old entry and creating a new one, which is why write protection alone does not prevent every replacement. Shared access and combined attributes can impose additional restrictions. Chapter 17 maps these operations to BASIC opening modes. Clear the practice file's E bit when finished:

```
ATRIB NOTE.TEXT:S (NOE NOCONSUL)
```

NOE removes write protection. Inspect the file again:

```
LISDIR NOTE.TEXT:S
```

The format indication returns from SW to S. Neither attribute-only command renames the file.

Other disk maintenance

The hard-disk initialization procedure prepares a filesystem on a configured volume. Its geometry and placement requirements remain essential for later maintenance. Chapter 12 explains memory partitions; Chapter 13 explains tape initialization and backup.

ORGDIR reorganizes directory entries. ELIMINAR removes an entry without releasing its storage. UNOSEC rebuilds volume metadata. These are maintenance tools, not alternatives to ordinary copying or deletion. Make a usable backup before invoking them.

Do not append to an unfinished sequential file merely to make it usable again. Its incomplete end-of-file information can cause subsequent output to overwrite volume metadata. A correctly closed empty file is different from an unfinished file. Chapter 13 discusses recovery precautions; Chapter 17 gives the programmer's restriction and Chapter 26 explains the write path.

Chapter 12 — Devices, printing and partitions

Device names separate a program's purpose from the hardware serving it. IMPR1 means a logical printer; IMP names a physical printer definition. ASOCIAR connects the two. This chapter first configures terminal and printer devices, then turns to the memory partitions that use them. Disk and floppy associations are covered in Chapter 11.

Inspect associations and device definitions

First list the current associations:

```
ASOCIAR
```

This shows which logical names already select devices. Inspect their physical definitions separately:

```
LISTAR SISTEMA.NOMBDISP:S
```

This text lists names and definitions. A definition for optional hardware does not mean that hardware is fitted. Compare the list with the machine's actual configuration.

Configure a terminal

The terminal and its driver must agree on communication settings. The console can operate with a 19200-baud, seven-data-bit, no-parity, one-stop-bit connection. Baud specifies transmission speed; data width and parity specify the character framing. Use settings supported by the actual terminal and driver.

A terminal class translates the operating system's cursor and display operations into the sequences understood by that terminal. Class 1 uses ESC G coordinate sequences; class 2 uses an ANSI-like sequence. A class mismatch can leave typed commands working while cursor movement is wrong. The classes are executable translation components; Chapter 28 documents the actual sequences.

For a console on PAN1 that requires class 2, enter:

```
ASOCIAR CONSOLA PAN1 (C2)
```

C2 loads the class-2 display translation. The terminal must understand that control sequence family; a terminal accepting ordinary text is not necessarily compatible with cursor positioning and erasure. To select class 1 instead, enter:

```
ASOCIAR CONSOLA PAN1 (C1)
```

Choose the class appropriate to the terminal; these two commands are alternatives, not a required round trip. Repeated class changes can lower the partition's available-memory ceiling. The normal dimensions are L79/P23. L is used both by output formatting and as CSI's input budget; a very small L can submit a partial command before Return. Keep the normal values unless there is a specific reason to change them.

For serial input, configure the sending terminal to honor the computer's receive flow control where the connection supports it. The ALP driver requests a pause as its input ring fills and resumes reception after queued input drains. This does not remove the serial controller's smaller receive-buffer limit: pace long input streams if characters are lost. Exact modem-signal wiring depends on the fitted interface.

PAN9 and PAN10 are SS2 terminal device names, with IS9 and IS10 aliases. They do not mean partition numbers 9 and 10. A partition may use one of these consoles through ACTIVAR when the corresponding hardware is fitted; Chapter 28 describes their separate serial paths.

Print a text file

If the fitted printer uses the IMP definition and the stated serial settings, associate it:

```
ASOCIAR IMPR1 IMP (B4800 PE)
```

B4800 selects nominal 4800 baud and PE selects even parity. This form selects seven data bits and one stop bit; configure the printer for the same framing. Adding W8 would request a different, eight-bit width. Use the settings required by the actual printer. Now print the practice text:

```
LISTAR NOTE.TEXT:S (PRINT1 NOHEAD)
```

PRINT1 selects logical printer 1; NOHEAD suppresses the listing heading. Confirm that paper output actually appears. A successful command result does not certify physical printing.

IMP uses serial readiness signals. An offline or disconnected printer can produce `Impresora no preparada (R/Q):`; restore the connection and readiness before requesting R to retry, or choose Q to abandon the operation. CTS can hold transmission until the printer permits it. A successful return from a buffered output call is not proof that the complete page has arrived.

Keep IMPR1 associated if you will continue with deferred printing below.

Deferred printing

SPOOLER collects reports for later printing. Initialization can recover queue state and remove selected backing reports, so review existing work first. With the printer configured, enter:

```
SPOOLER INIC
```

This initializes the service. Inspect it:

```
SPOOLER ESTADO
```

To pause ordinary queue output, enter:

```
SPOOLER PARA
```

PARA does not override every explicit print/relaunch request. To inspect queued reports, enter:

```
SPOOLER LIST
```

Collection uses SISTEMA.SPOOLCOL and report files. Chapter 29 explains their relationship, and the command reference separates the remaining queue controls.

Release an unused printer association

The following cleanup is optional. Use it only after applications and queued printing no longer need IMPR1. Pausing a queue is not proof that its printer association is no longer needed; do not disassociate a printer still serving the spooler.

When the association is no longer in use, enter:

```
ASOCIAR IMPR1
```

This removes the logical association. It is different from ASOCIAR with no arguments, which lists associations, and it is not a spooler shutdown command.

The remaining sections manage partitions. Device configuration selects hardware; partition allocation supplies memory and execution state. Inspect those resources separately before making changes.

List partitions

To display the allocated memory partitions, enter:

```
ESTADO MEMORIA
```

This is the partition listing. Its heading includes `Puer`, `Modu`, `DIRBA`, `DIRMX`, `DIRAL`, `SP---`, `PC---`, `Tama|`, `Rutina` and `Usuario`. The partition number and memory-module number are separate columns. The addresses describe the partition's memory and execution state; `Rutina` and `Usuario` identify its routine and connected account when available. You do not need to interpret every address to find which partitions are allocated.

A stopped partition can remain in this report because PARAR retains its memory. An empty user field therefore does not by itself mean that the partition can be allocated again. The report can also contain resident components such as NUCLEO and SPOOLER; those rows are not additional user accounts.

For the available memory modules, enter:

```
ESTADO MODULO
```

A module is a memory resource from which a partition can be allocated. This report helps select the final argument of PONER MEMORIA; it is not a list of disk volumes.

For connected users, enter:

```
ESTADO USUARIOS
```

This shows the user connections and their connection dates and times. To list registered accounts, including those not connected, use CUENTA (PANT), as described in Chapter 10. Check these separate reports before allocating, stopping or removing a partition.

Create and activate a partition

A partition is an independently scheduled execution area in memory. It is not a region of a disk and has no filesystem to mount. PONER MEMORIA allocates it; ACTIVAR starts its terminal or command environment. Disk access is configured separately with ASOCIAR and MONTAR, as taught in Chapter 11.

Perform memory configuration from SISTEMA at privilege 5. Use the next partition number after those already allocated: for example, 2 when only partition 1 exists. The public numbering range is 1–16, but available memory and fitted terminals determine how many useful partitions can run.

Inspect available modules first:

```
ESTADO MODULO
```

The following configuration requires an available module 1 and a terminal on PAN2. It is an example for that configuration, not a command to repeat on an already occupied partition. Enter:

```
PONER MEMORIA 2 48000 1
```

This requests displayed partition 2 in module 1, using 48000 as the size parameter. The parameter is not an exact allocation guarantee. In the demonstrated configuration, the first allocation in that module occupies 0x4C8B–0xFFFF, a region of 45,941 bytes; a request of 40000 also receives that region. This result does not establish the allocation rule for every shared-module configuration. The final 1 is a module selection, not an account number.

Inspect the actual allocation:

```
ESTADO MEMORIA
```

Use the reported bounds and size rather than assuming that the requested byte count was assigned. Activate the terminal:

```
ACTIVAR 2 (PAN2)
```

The second terminal can now connect to an account. Allocation and activation are separate because a partition needs memory before it can run. Inspect the result:

```
ESTADO MEMORIA
```

The command reports partition memory. The accepted numbering range does not guarantee enough RAM for sixteen useful active users.

Stop a partition deliberately

At the target terminal, save and close the application. At its CSI prompt, enter:

```
FIN
```

FIN disconnects the account and leaves that terminal ready for another login. To stop the partition itself, return to the administrative terminal and enter:

```
PARAR 2
```

PARAR stops the partition while retaining its allocation. It can refuse a connected account. FORZAR can inject FIN into a target, but forced completion can leave a partial application record; use it only for recovery, not routine logout.

To use a stopped, still-allocated partition again, enter:

```
ACTIVAR 2 (PAN2)
```

This reactivates its terminal. It uses the existing memory allocation and does not create another account.

Remove an allocated partition

Stopping and removing are different operations. PARAR releases the active terminal but retains the partition's memory allocation. The removal form of PONER MEMORIA omits the size and module arguments.

The following continues the dedicated-module example above. Partition 2 must be the last allocated partition, must be stopped, and must not be the partition from which you issue the command. If you reactivated it in the preceding example, finish its session and stop it again before proceeding.

From the administrative partition, enter:

```
PONER MEMORIA 2
```

With no size argument, this requests removal of partition 2. Do not append `0` as a presumed equivalent: the size parser raises small explicit sizes to its minimum allocation. Check the result:

```
ESTADO MEMORIA
```

Then inspect the available modules:

```
ESTADO MODULO
```

The partition must no longer appear as allocated. Remove partitions in reverse allocation order; the command refuses removal of a non-final partition. An active target, the caller's own partition, or an incompatible neighboring partition can also prevent removal. The routine can rearrange context storage when partitions share a module. That shared-module case is outside this elementary procedure; use the dedicated-module arrangement shown here.

These commands alter the running configuration. They do not by themselves save it as the next startup configuration. GENSIST writes the system nucleus and is a separate, consequential maintenance operation. Removing a memory partition does not erase disk files or remove accounts.

Chapter 13 — Backup and recovery

A backup must be usable for the kind of recovery you need. File copies protect selected files; archive operations preserve selected files or accounts; full-medium copying serves a different purpose. The presence of a command does not guarantee that its required tape or floppy device is installed.

Make a file-level backup

For the practice text, choose an unused destination and enter:

```
COPIFI NOTE.TEXT:S SAFENOTE.TEXT:S (VERIFI)
```

COPIFI duplicates the file and requests verification. Inspect the copy before relying on it:

```
LISTAR SAFENOTE.TEXT:S
```

A copy on the same volume helps recover an accidental edit but does not protect against failure of that volume. For independent backup, select a separately configured destination device and verify the resulting files there.

Archive files to floppy media

ARCHIVAR saves selected files, an account's files, or a whole volume. RECARGAR reads media written by ARCHIVAR. These operations differ from copying an ordinary file with COPIFI. Prepare and associate the destination floppy as described in Chapter 11. The following examples give the documented command forms; successful physical operation requires the compatible drive and media.

With a writable archive floppy on A, save just the practice text:

```
ARCHIVAR NOTE.TEXT:S,A (PANT VERIFI CONSU)
```

NOTE.TEXT:S selects the source file; A selects the archive destination. PANT displays names, VERIFI requests readback after writing, and CONSU asks before saving each selected file. Read each request before answering. This writes destination media; do not use a floppy containing information that must be retained.

To select the connected account's files instead, the alternative command is:

```
ARCHIVAR S,A (CUENTA PANT VERIFI CONSU)
```

CUENTA limits selection to the current account. It does not mean “copy the account database.” This is a broader backup, not a required second step after the single-file example. ARCHIVAR can use a succession of media, but the exact media-change dialogue is not fully specified here; follow the utility's requests and retain the media in their written order.

For a trial restore of the practice file, connect to the intended receiving account and select a destination disk with no NOTE.TEXT file of that account. With the archive on A, enter:

```
RECARGAR NOTE.TEXT:A,S (CUENTA NOREEM PANT VERIFI CONSU)
```

RECARGAR selects that archived filename for the connected account, writes to S and refuses to replace an existing file because NOREEM is present. CONSU retains the per-file consultation. If NOTE.TEXT already exists, this is not a test that the archived copy can replace it; choose a separate receiving environment for a real restore test.

After a successful restore, inspect the file:

```
LISTAR NOTE.TEXT:S
```

Check the restored content. Readback while writing an archive and later restoring a usable file are different checks.

Prepare tape access

The tape commands use a logical device such as TAPE1. TAP is the physical tape-driver definition. A cartridge is sequential media, so the system does not present it as a disk directory that can be browsed with LISDIR.

The command language and initialization operations described below are available, but a complete physical tape backup-and-restore cycle is not established. The controller's physical recording details also remain incomplete. Treat the examples as operating instructions for a compatible, working installation, with that limitation; no promised transfer rate, media capacity or sample completion output is implied.

Begin by inspecting the associations:

```
ASOCIAR
```

If the fitted controller uses TAP and TAPE1 is available, enter:

```
ASOCIAR TAPE1 TAP
```

This loads and initializes the tape driver for that association. It queries the controller, so an absent or incompatible controller can leave the command waiting. Do not proceed to an archive command unless the association completes and the device is ready.

MONTAR is the disk/floppy media-change command. There is no established `MONTAR TAPE1` form. Cartridge loading, readiness and positioning belong to the tape utility and controller. Load the cartridge using the drive's physical controls while it is idle, and follow the utility's requests. No standalone universal tape mount, rewind or eject command is specified in this interface.

Initialize a cartridge

INITTAPE prepares a new cartridge or erases a previous tape volume. It uses TAPE1; its command line does not take a disk letter or a cartridge filename.

> Caution — destroys existing tape information. Check the cartridge before initialization. Never initialize the archive cartridge you intend to restore.

For ordinary initialization, enter:

```
INITTAPE
```

This begins tape initialization and writes a volume label. The utility has confirmation and label-entry steps; their complete dialogue is not reproduced here. Read them on the terminal before allowing the operation to proceed.

For a new cartridge requiring the utility's tensioning pass, use this alternative:

```
INITTAPE (TENSION)
```

TENSION traverses and rewinds the cartridge before initialization. It is not a harmless standalone rewind request: it remains part of INITTAPE and therefore belongs only in the initialization procedure. The volume label records a name of up to eight characters. Further archive-label details appear in Appendix A.

Save files or a volume to tape

With TAPE1 ready and an expendable initialized cartridge loaded, begin with one closed practice file:

```
ARCHIVAR NOTE.TEXT:S,TAPE1 (PANT VERIFI CONSU)
```

The source is NOTE.TEXT on S and the destination is tape. The options retain visible file selection, write verification and consultation. The operation writes an archive; LISTAR and ordinary character output to TAPE1 do not produce an equivalent tape archive.

To archive the connected account's files, use this broader selection instead:

```
ARCHIVAR S,TAPE1 (CUENTA PANT VERIFI CONSU)
```

CUENTA selects that account's files. To select every account on the source volume, the alternative is:

```
ARCHIVAR S,TAPE1 (VOLUM PANT VERIFI CONSU)
```

VOLUM broadens the selection to the volume. Arrange for applications to close files before a backup that must represent a consistent set of business records. These three commands are alternative selections, not a sequence to run on the same cartridge without considering the existing archive.

Restore an archive from tape

Use the cartridge written by ARCHIVAR, with TAPE1 associated to the compatible drive. Connect to the receiving account, choose the destination disk, and avoid existing filenames for the first restore trial. Do not run INITTAPE.

To restore the practice file to S, enter:

```
RECARGAR NOTE.TEXT:TAPE1,S (CUENTA NOREEM PANT VERIFI CONSU)
```

The tape appears before the comma because it is now the source. S is the destination. CUENTA selects the connected account, and NOREEM prevents replacement of an existing file. After a successful restoration, examine its contents:

```
LISTAR NOTE.TEXT:S
```

A full-volume restore has a much wider scope. Its documented form, still retaining nonreplacement and consultation, is:

```
RECARGAR TAPE1,S (VOLUM NOREEM PANT VERIFI CONSU)
```

Use this only when the destination volume and ownership requirements have been planned. VOLUM restores across accounts. REEMPLA permits overwriting existing files; COPREEM limits the operation to replacing existing files; NOREEM loads only absent files. These are alternatives. Omitting the filename can select whole-volume restoration by default, so keep the selection explicit.

RESCUE selects another archived account for restoration into the current account; see RECARGAR before using that form. Do not assume that restoring application files also restores every account or startup setting.

Whole-medium copying and tape completion

BACKUP performs binary copying between disks or between disk and tape. It is not ARCHIVAR's file-selection format. The explicit disk-to-tape form is:

```
BACKUP S TAPE1
```

This selects the entire source medium S for the tape destination. It requires compatible devices and expendable destination media. Do not run BACKUP without arguments while learning: its defaults are S as source and A as destination, which can overwrite the wrong disk. RECARGAR is documented for ARCHIVAR media; do not assume it restores BACKUP media merely because both use tape labels.

When tape work has completed and the drive is idle, the logical association can be removed:

```
ASOCIAR TAPE1
```

This releases the association. It is not a rewind, eject or wait-for-transfer-completion operation. Complete the utility and follow the drive's physical unloading procedure first. The precise controller recovery and full binary-tape restoration procedures remain unspecified.

Diagnose before repair

Symptom	Likely question to resolve	Next action
Disk full despite free space	Is the required contiguous run available?	Compare Disponible with Mayor zona
Exact file not found	Are account, drive and lookup visibility correct?	Check identity, association and wildcard listing
Directory full despite empty slots	Has a name-specific probe cycle filled?	Stop adding conflicting names; investigate the directory
OPEN waits	Does another partition hold exclusive access?	Ask the holder to CLOSE the channel
READ waits	Are overlapping sectors locked?	Identify the holder before interrupting it
Printer silent	Is the association valid and hardware ready?	Check device configuration and actual output
Prompt appears without login	Are account components reachable?	Check SISTEMA.CUENTA and CONECTAR.COMANDO

Run a filesystem check

After making a usable backup and arranging maintenance access, enter:

```
REPARAR S
```

This scans the volume. Read every diagnostic even if the final command result is 0. Checking can update a free-count field; it is not guaranteed to leave all metadata untouched.

The F option permits repair:

```
REPARAR S (F)
```

> Caution — repair can remove directory metadata. Use F only after evaluating the check report. It can reconcile the bitmap and remove rejected entries or alter sharing redirects. It does not reconstruct arbitrary lost application records.

Respond to disk errors

The common disk path retries before prompting. R retries again. M invokes the driver's recovery entry and retries. I ignores the failed sector. Q aborts through interruption handling. An ignored error can appear to the caller as success despite missing or invalid data, so I is not a repair command.

A failed write can leave a directory count that exceeds the useful records actually stored. A structurally consistent file can also contain a partial final BASIC record after interruption. Successful CLOSE, a clean allocation count and complete application data are different properties.

Deleted and unfinished files

BORRAR releases allocation; later writes may reuse it. REVIVIR attempts catalogue recovery, but complete collision and recovery behavior is unspecified. Do not continue ordinary writing on a volume from which deleted data is needed.

Unfinished sequential output can have a chain of sectors while its directory counts remain zero. Do not initialize, reorganize or append to such a file merely to make its metadata look normal. Establish which records are complete first. Chapter 26 describes the write order and Chapter 27 describes disk-error handling.

Chapter 14 — Startup maintenance and ending work

Startup rebuilds some state from system files and creates other state in memory. Account connection then adds identity and optional command-file execution. Understanding these stages helps distinguish a missing component from an ordinary application menu.

Startup responsibilities

The bootstrap loads SISTEMA.NUCLEO. The executive initializes hardware and memory, loads SISTEMA.STARTUP and binds CSI, EXECLANG, EXEC work storage and MENSERR. SISTEMA.STARTUP is executable code, not a text command procedure.

CSI starts the connection path when SISTEMA.CUENTA and CONECTAR.COMANDO are accessible. After login, CONECTAR searches for an account-named EXEC file on S and then A–G, considering private and public candidates within each drive. A public S profile can therefore be selected before a private A profile. EXEC work capacity must be available for the automatic procedure.

Cold IPL.EXEC is a separate fallback when login components are unavailable. A command file does not run at every boot merely because its name sounds like a startup procedure. Chapter 21 gives the detailed order.

Inspect before changing configuration

Enter:

```
ESTADO USUARIOS
```

This shows connected users. Arrange for them to finish before changing shared facilities. Inspect associations separately:

```
ASOCIAR
```

This tells you which logical names currently select each device. Changes to terminal classes, drivers and partitions can affect memory as well as hardware behavior.

GENSIST saves selected current settings into SISTEMA.NUCLEO:

```
GENSIST
```

> Caution — modifies the kernel file. Use this only when the current configuration is intentional and a usable backup exists. Complete persistence rules for every ASOCIAR/PONER combination are unspecified.

A status operation can update counters, EXEC can write a work frame, and BUZON removes retrieved messages. Plan maintenance according to these actual effects rather than treating every display-oriented command as read-only.

End work in order

Ask each user to save through the application, close its files and return to CSI. Each user then enters:

```
FIN
```

An authorized administrator can stop a logged-out partition with PARAR. This is session and partition management; it does not establish a global synchronization operation.

No complete whole-machine power-off procedure or universal shutdown command is specified here. Escape-I requests a restart and can discard pending output. Declining that restart also terminates the interrupted command. Neither is a substitute for application save and close. Do not promise users that a reset will complete their files.

Part IV — Programming

Chapter 15 — Your first BASIC program

BASIC lets you develop and run a program interactively from its source, then compile that source for later execution. In interactive use, you enter numbered lines at BASIC's prompt and ask the interpreter to run them. This makes it convenient to try an idea, correct a line and run the program again.

Compilation performs preparation ahead of execution and saves the result in a BASICCOM file. The compiler converts expressions and resolves source-line references into a form for the BASIC runtime, reducing work left to execution and allowing faster execution where that preparation matters. No fixed speed improvement is specified; the benefit depends on the program and its input/output. This is compilation before running, but it does not produce a statically linked, standalone Z80 program: the BASIC runtime remains necessary.

The following example takes the same greeting through both paths. You need permission to run BASIC, a writable S association and unused names HELLO.BASICOBJ and HELLO.BASICCOM.

1. Enter BASIC

At CSI enter:

```
BASIC
```

The dash prompt belongs to BASIC. Commands entered here are BASIC commands, not CSI commands.

2. Add a statement

Enter:

```
10 PRINT "HELLO WORLD"
```

10 is the line number. PRINT displays a value. The quotation marks make HELLO WORLD a literal string. Because the line begins with a number, BASIC stores it as program source rather than executing it immediately.

Enter the ending line:

```
20 END
```

END finishes program execution. It does not log you out or leave the BASIC editor.

3. Inspect the source

Enter:

```
LIST
```

LIST displays the stored source in line-number order. Check both lines. To replace a line, enter that line number with the corrected statement; do not add a second line with an unrelated number unless you intend a second operation.

4. Run it interactively

Enter:

```
RUN
```

The interpreter executes the source and displays:

```
HELLO WORLD
```

Control returns to BASIC after END. Running did not save the program.

5. Save the source

Enter:

```
SAVE HELLO:S
```

SAVE writes HELLO.BASICOBJ:S. Despite OBJ in the name, this is saved, tokenized BASIC source, not a Z80 linker object. Keep it for future edits.

Leave BASIC:

```
QUIT
```

QUIT returns to CSI. You can later re-enter BASIC and use LOAD HELLO:S to retrieve the saved source.

6. Compile

At CSI enter:

```
BASIC HELLO.BASICOBJ:S
```

With an explicit saved-source filename, BASIC compiles to HELLO.BASICCOM:S. Read the compiler's diagnostics and require zero compilation errors before continuing. Use the explicit BASICOBJ type; do not assume BASIC and RUN have identical default types.

7. Run the compiled program

At CSI enter:

```
RUN HELLO.BASICCOM:S
```

RUN loads the compiled representation and executes it with the BASIC runtime. It displays the same greeting. BASICCOM is runtime code, not a standalone native Z80 executable.

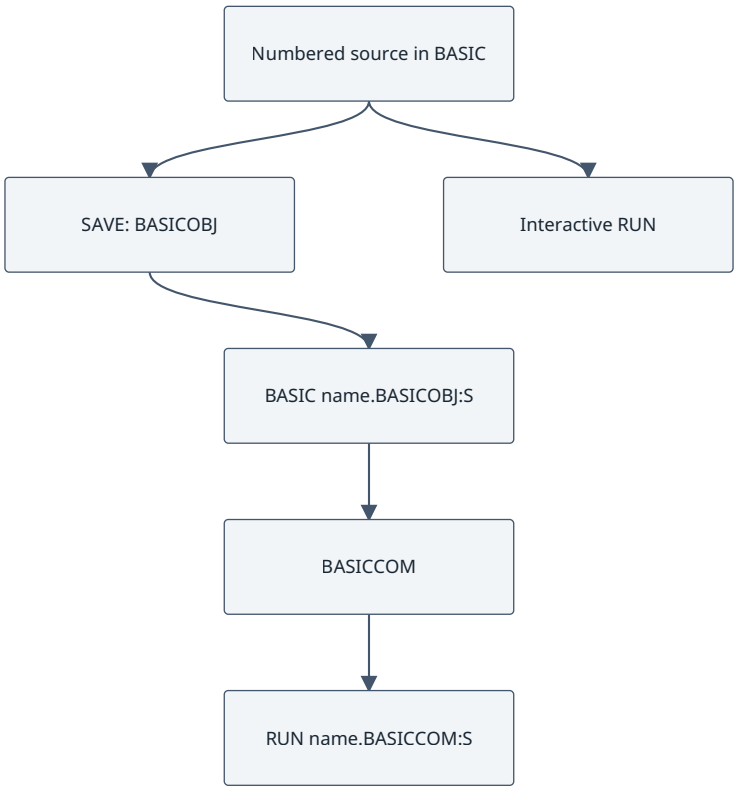


Figure 15.1. The interpreted and compiled development paths.

Change and rebuild

Enter BASIC again. At its prompt enter:

```
LOAD HELLO:S
```

LOAD retrieves the saved source. Change line 10 by re-entering it with new text, LIST to inspect it, RUN to try it, and SAVE to retain it. After QUIT, compile again to update BASICCOM. Editing source does not automatically update an older compiled file.

Compiler options

To display a compilation listing, use:

```
BASIC HELLO.BASICOBJ:S (LIST)
```

LIST requests a console listing. XREF requests cross-references; NOLIST suppresses the listing; IMPRn directs it to printer n; OBJ=drive selects output placement.

NOOBJ suppresses BASICCOM generation. It does not itself request a source listing, and compiler statistics can still appear. To list source without producing compiled output, enter:

```
BASIC HELLO.BASICOBJ:S (LIST NOOBJ)
```

LIST controls source display; NOOBJ controls compiled output. An older BASICCOM can remain from a previous compilation. Running that older file does not show that this invocation generated a new one.

To display compilation passes and source line numbers, enter:

```
BASIC HELLO.BASICOBJ:S (TRACE)
```

This produces a compilation trace, including PASO # 1 through PASO # 8. It is separate from tracing execution of a BASIC program. RUN with TRACE produces no visible runtime-address trace for the greeting and numeric examples described in Appendix D.

The supplied help advertises a compiler COMMA option, but this version rejects both COMMA and COMA with `Opcion no valida`, CR = 8. Use dot-decimal source in these examples. The advertised RUN COMA option also leaves the displayed value of PRINT 1.5 as 1.5 in the demonstrated numeric program; no decimal-display change is established by that example. PRINT 1,5 supplies two items to PRINT, not a decimal-comma literal. Appendix D gives individual examples and the limits of each option.

Chapter 16 — BASIC values, decisions and loops

Once a program can print a greeting, the next step is to store values and choose what to do with them. Enter immediate examples at the BASIC dash prompt. Enter complete numbered programs one line at a time; save previous work before NEW clears the editor.

Assign and display a value

Enter:

```
A=7
```

Assignment stores a value in A. To see it, enter:

```
PRINT A
```

The program uses the stored value rather than the letter A as text. A name ending in `$` holds a string; a name ending in `%` denotes an integer. Ordinary numeric variables use the real representation.

Enter:

```
PRINT 7/2
```

Integer operands give 3: division truncates toward zero. Now enter:

```
PRINT 7.0/2
```

The decimal operand selects real arithmetic and gives 3.5. Real values use packed decimal with thirteen significant digits, rather than IEEE binary floating point.

Store and join text

A string is a sequence of characters. Quotation marks enclose literal text, while a variable ending in `$` holds a string value. At the BASIC prompt, enter:

```
S$="UN"
```

This stores UN in S$. Display its value:

```
PRINT S$
```

PRINT displays the stored characters. To join that value to another string, enter:

```
PRINT S$&"DOS"
```

The ampersand joins the two strings, producing UNDOS without inserting a separator. Later in this chapter, the same operation places a special separator between fields.

Make a choice

Save any source you want to keep, then clear the editor:

```
NEW
```

This starts a fresh program. Enter the following numbered lines one at a time:

```
10 A=7
20 IF A=7 THEN PRINT "SEVEN" ELSE PRINT "OTHER"
30 END
```

Line 10 assigns A. IF compares it with 7. THEN introduces the true action; ELSE introduces the alternative. END finishes. Change line 10 to a different value and RUN again to see the other path.

Count with FOR and NEXT

Save the previous example if needed, then start a fresh program:

```
NEW
```

Enter this loop one numbered line at a time:

```
10 FOR I%=1 TO 3
20 PRINT I%
30 NEXT I%
40 END
```

FOR sets the starting value and limit. PRINT uses the current counter. NEXT advances it and repeats while the range applies. The output values are 1, 2 and 3; after this loop I% is 4. STEP can supply an increment, including -1 for a descending loop. A skipped range still assigns its initial value.

Reuse a subroutine

A subroutine performs a task and returns. Save the previous example, use NEW to start a fresh program, then enter:

```
10 GOSUB 100
20 PRINT "BACK"
30 END
100 PRINT "SUB"
110 RETURN
```

GOSUB remembers where to resume and transfers to line 100. RETURN resumes at line 20. END prevents execution from falling into the subroutine a second time. GOTO differs: it transfers without creating that return point.

Computed ON chooses among a one-based target list. For example, `ON 2 GOTO 100,200,300` selects line 200 in a program containing those targets. Selector zero produces ON Error Rango. An omitted middle entry, as in `ON 2 GOTO 100,,300`, continues after the statement.

Read embedded data

Start a fresh program after saving the preceding example. Enter:

```
10 DATA 11,2.5,"ABC",-4
20 READ A,B,S$,I%
30 PRINT A,B,S$,I%
40 RESTORE
50 READ A
60 END
```

DATA supplies literals in sequence. READ assigns them to variables. RESTORE returns the data cursor to the first DATA item; it does not undo other assignments. Reading past the available data raises No hay DATA.

Store an array

An array holds several values under one name; an index selects an individual element. Before declaring an array, select its lowest permitted index:

```
OPTION BASE 1
```

Then declare:

```
DIM ARR(2)
```

With BASE 1 the permitted indices are 1 and 2. Store an element:

```
ARR(1)=7
```

Read it:

```
PRINT ARR(1)
```

BASE 0 instead permits 0, 1 and 2 for DIM ARR(2). Set the base before declaration. Changing existing arrays and carrying this setting into another program through CHAIN have incompletely specified rules; they are outside this elementary example.

Extract a field from a string

A string can contain several fields separated by a special character. For example, two fields can hold UN and DOS. A delimiter marks the boundary; it is part of the stored string, not an instruction to print punctuation.

CHR$ constructs a one-character string from its numeric character code. This system's field delimiter has decimal code 222, or hexadecimal 0xDE. The following expression joins UN, that delimiter and DOS, then asks EXT$ for the second field. Enter:

```
PRINT EXT$("UN"&CHR$(222)&"DOS",2,0)
```

CHR$(222) supplies the separator and the ampersands join the pieces. EXT$ takes the complete string, field number 2 and subfield number 0. Zero in this last position requests the whole field, so the result is DOS. A literal caret is not the delimiter. A field can also be divided into subfields using code 0xDD; this example does not need that further division.

The next chapter applies these values and strings to files. BASIC can also call separately supplied extension modules, but that introduces another interface. Chapter 19 explains what those modules are before showing OPTION USR, and separately describes compiled-program privilege and serial restrictions. Appendix D provides the full statement and function reference.

Chapter 17 — Programming files

A program opens a file, transfers values, then closes it. This chapter first follows that cycle for a sequential file, where values are read in the order written. It then introduces direct access by record number, indexed access by key, and coordination between programs using the same files.

Use a writable S association and unused practice names LINES.DATOS, DIRECT.DATOS and INDEX.DATOS. You need permission to use BASIC and CREAR. Save any BASIC source you want to retain before leaving the interpreter. The file statements below are entered immediately at BASIC's dash prompt; CSI commands are identified separately.

Channels, records and opening modes

OPEN connects a file to a numbered channel. Later READ, WRITE and CLOSE statements use that number instead of repeating the filename. A BASIC channel is not a disk letter or a physical device number. These examples use channel 1 and close it before reusing it for another file.

A record is a group of stored values that a program transfers together. BASIC WRITE serializes the values with the information needed to read them back. Consequently a file written by BASIC is not necessarily ordinary text suitable for EDITAR, even when the value being written is a string.

The mode in OPEN states what access is requested. Its organization word states how records are selected. The examples below use these combinations:

Opening form	Purpose in this chapter
OUTPUT SEQUENTIAL	Create the new sequential practice file and write values in order
INPUT SEQUENTIAL	Read existing sequential values in order
OUTPUT SEQUENTIAL, EXTEND	Add output at the end of a correctly closed sequential file
UPDATE SEQUENTIAL	Replace existing sequential contents; this is destructive
UPDATE DIRECT	Read and write existing fixed-length records by number
UPDATE INDEXED	Read and write existing indexed records by key

The word UPDATE does not promise the same effect for every organization. In particular, sequential replacement must be kept separate from direct or indexed updates.

Create and write a sequential file

If you are already at the BASIC dash prompt, continue with OPEN below. Otherwise, at CSI enter the interpreter:

```
BASIC
```

At its dash prompt, open the unused practice filename:

```
OPEN #1: "LINES.DATOS:S", OUTPUT SEQUENTIAL
```

OPEN connects the file to channel 1. The hash introduces the channel number, the colon separates it from the file specification, and the comma introduces the opening mode. This example requires an unused name; it is not a way to preserve an existing file's contents.

Write one string value:

```
WRITE #1: "FIRST"
```

WRITE sends FIRST through channel 1. Finish the output:

```
CLOSE #1
```

CLOSE completes the required output handling and releases the channel. Do not substitute QUIT for CLOSE. You remain at the BASIC prompt, ready to reopen the file for reading.

Read the saved value

Open the existing file for sequential input:

```
OPEN #1: "LINES.DATOS:S", INPUT SEQUENTIAL
```

Read its next value into the string variable A$:

```
READ #1: A$
```

The channel selects the file; A$ receives its stored string. Display that value:

```
PRINT A$
```

The result is FIRST. READ changes the file position so that another READ asks for the next value rather than rereading the first one.

Recognize the end of a file

This practice file contains only the value just read. Request another:

```
READ #1: A$
```

There is no next value. Inspect the channel's end condition:

```
PRINT EOF(1)
```

EOF is the end-of-file query; its argument is the channel number. The result here is -1, indicating the end condition. Check this condition before treating a variable as newly read data. A missing record can set EOF without printing a separate diagnostic.

Close the input channel:

```
CLOSE #1
```

Append to a correctly closed file

The preceding output was closed normally. Reopen it to append:

```
OPEN #1: "LINES.DATOS:S", OUTPUT SEQUENTIAL, EXTEND
```

EXTEND is a trailing option. It positions output at the end of a valid closed sequential file. Add the next value:

```
WRITE #1: "TAIL"
```

Complete the output:

```
CLOSE #1
```

Do not READ before WRITE in this append workflow: the shared cursor can advance behind an EOF marker. Do not extend an unfinished file with a zero final-sector address. Extension trusts that field and can damage volume metadata if it is invalid. A correctly closed empty file has a nonzero final-sector address; it is not the same as unfinished output. Chapter 26 explains the storage details.

Check the appended contents

Open for input again:

```
OPEN #1: "LINES.DATOS:S", INPUT SEQUENTIAL
```

Read the first value:

```
READ #1: A$
```

Display it:

```
PRINT A$
```

It is still FIRST. Read the next value:

```
READ #1: A$
```

Display that value separately:

```
PRINT A$
```

This time the result is TAIL. Appending retained the original value and added another after it. Close the input:

```
CLOSE #1
```

Replace sequential contents deliberately

Replacement is a different operation from appending. The following optional exercise empties the practice file you have just read.

> Caution — UPDATE SEQUENTIAL replaces existing contents. OPEN can delete and recreate the old file before WRITE runs. Use only the expendable LINES.DATOS file from this example.

Open it in replacement mode:

```
OPEN #1: "LINES.DATOS:S", UPDATE SEQUENTIAL
```

Close it without writing:

```
CLOSE #1
```

The file is now logically empty; FIRST and TAIL have not been retained. Reopening it with INPUT SEQUENTIAL, attempting a READ and checking EOF(1) reaches the end condition immediately. Replacement begins at OPEN, not at the first WRITE, which is why selecting the correct mode matters before any data transfer.

Create a direct file

Direct access selects a fixed-length record by its number, rather than reading all preceding records. The file must first be created with the required record size and capacity. With all channels closed, return from BASIC to CSI:

```
QUIT
```

Using the unused practice name DIRECT.DATOS, enter:

```
CREAR DIRECT.DATOS:S (DIRECTO REGISTRO 300 NUMREG 3)
```

CREAR makes three fixed-length records of 300 bytes. DIRECTO selects direct organization, REGISTRO 300 specifies the bytes per record, and NUMREG 3 requests three records. Direct organization lets a program select a record by number. Enter BASIC again:

```
BASIC
```

At its dash prompt, open the file:

```
OPEN #1: "DIRECT.DATOS:S", UPDATE DIRECT
```

#1 selects the channel. UPDATE DIRECT permits reading and writing the existing direct-file records. Write record 2:

```
WRITE #1,2: "SECOND"
```

The number after the channel selects the record; the colon introduces its data. Read the same record:

```
READ #1,2: A$
```

READ places the stored string in A$. Display it:

```
PRINT A$
```

The result is SECOND. Finish the file operation:

```
CLOSE #1
```

CLOSE releases the open channel and completes its required output handling. Do not substitute QUIT for CLOSE. Record numbering starts at 1; reading past the configured range reaches the end condition, available through EOF(1).

Create and use an indexed file

An indexed file identifies each record by a key, such as an account code. The index also supports reading records in key order. This differs from selecting a direct record by its number.

After closing the direct file, return to CSI:

```
QUIT
```

Create the unused practice file INDEX.DATOS:

```
CREAR INDEX.DATOS:S (INDEXADO REGISTRO 32 NUMREG 16 LONCLA 4)
```

INDEXADO requests indexed organization, REGISTRO 32 selects record length, NUMREG requests capacity and LONCLA 4 selects a four-character key. The allocated slot capacity can exceed the requested count; a request for 16 produces 19 slots in this case.

Enter the interpreter:

```
BASIC
```

At the BASIC prompt, open the file:

```
OPEN #1: "INDEX.DATOS:S", UPDATE INDEXED
```

Write one keyed record:

```
WRITE #1,"AAAA": "FIRST"
```

The key identifies the record. Add another separately:

```
WRITE #1,"MMMM": "MIDDLE"
```

Read by key:

```
READ #1,"AAAA": A$
```

Display the retrieved value:

```
PRINT A$
```

The result is FIRST. READNEXT provides ordered traversal for indexed files. A missing key sets the EOF condition without necessarily printing a diagnostic. Close after use:

```
CLOSE #1
```

CLAVE creation and UPDATE KEYED provide keyed access without the I-format ordered index area. Do not assume their on-disk layouts are identical.

File protection and BASIC modes

The administrator can set independent read, write and deletion restrictions, as taught in Chapter 11. The table below connects those restrictions to the opening modes you have now used. It describes the current owner with each protection bit separately.

Individual protection	INPUT	UPDATE SEQUENTIAL replacement	EXTEND
E, write	Allowed	Allowed through deletion/recreation	Refused
B, deletion	Allowed	Refused	Allowed
L, read	Refused	Allowed through deletion/recreation	Refused

Sequential replacement can succeed despite write protection because its path deletes and recreates the entry. Combined attributes and shared access can impose further restrictions; do not infer their full behavior from a row that applies only one protection.

Coordinate shared access

An exclusive open prevents a competing opener from entering the same file until the holder closes it. Record locks instead coordinate access to parts of an open file. For the existing direct practice file, this form requests exclusive opening:

```
OPEN #1: "DIRECT.DATOS:S", UPDATE DIRECT, LOCK
```

A competing open can wait until the holder closes. UNLOCK is different:

```
UNLOCK #1
```

UNLOCK releases locked record spans, not the exclusive OPEN ownership. To finish the exclusive access, enter:

```
CLOSE #1
```

Record locks cover sectors, so two different 300-byte records can overlap a sector.

Ownership, sharing grants, open admission and record locks are separate rules. An owner can delete a file despite another partition holding an exclusive open. Avoid deleting active shared files.

Check the right completion condition

EOF, command result and physical write completion belong to different layers. A BASIC WRITE can require multiple kernel transfers, and interruption can leave a partial final BASIC record. Explicit CLOSE is essential, but ignored disk errors can still produce incomplete stored data. Handle reported errors and validate the records your application requires.

Chapter 18 — Writing Z80 machine-code programs

The Secoinsa Serie 20 computer uses a Z80 processor. A native machine-code program consists of instruction bytes executed directly by that processor. Assembly notation gives those bytes readable names, such as LD for loading a register and JP for a jump. This chapter explains a small greeting program, its exact bytes and the format the operating system's loader expects.

No native general-purpose assembler or linker command is specified here. The assembly listing explains the required machine code; it is not a complete installed edit–assemble–link procedure to type at CSI. Chapter 15 provides the complete BASIC development workflow. Here, the aim is to understand how to write the native instructions and represent them as a loadable program.

Registers, labels and operating-system requests

Registers hold values used by the processor. This example uses A for a command result and the DE register pair for the address of some text. A label names a position in the program so that an instruction can refer to it without spelling its numeric address.

To request an operating-system service, the program executes RST 08h followed by a byte selecting the operation. The executive reads that byte and carries out the request. Service 0x02 displays a string ending in a carriage return; service 0x00 ends the command using the result in A. Chapter 19 develops this calling convention with more examples.

Write the greeting

The program below prints HELLO Z80 and ends with command result 0. It has no open files, so it does not demonstrate completion of buffered file output. The names start and message are labels chosen for this listing.

```
        jp      start
start:  ld      de,message
        rst     08h
        db      02h
        xor     a
        rst     08h
        db      00h
message: db     "HELLO Z80",13,0
```

Understand each instruction

`JP start` transfers execution to the label start. It provides a conventional entry jump, although the file format does not require one.

`LD DE,message` places the address of the text in the DE register pair. LD copies a value; it does not print anything.

`RST 08h` enters the executive dispatcher. The following `DB 02h` emits the service selector byte. DB is an assembler data directive, not a processor instruction. Together these bytes request text output using DE.

`XOR A` clears A to zero. The following RST/DB pair selects service 0x00, which ends the command using that zero completion result.

The final DB directive emits the literal characters, a carriage return (13), and a zero byte. A label identifies the address of the following bytes; it does not allocate an extra byte.

Read the machine-code bytes

The complete program is 22 bytes. With its addresses initially expressed relative to zero, it is:

```
C3 03 00 11 0B 00 CF 02 AF CF 00 48 45 4C 4C 4F 20 5A 38 30 0D 00
```

The first three bytes, C3 03 00, encode the jump to start at offset 3. The next three, 11 0B 00, load DE with the message address at offset 11. The two CF bytes encode RST 08h; each is followed by its service selector. The character bytes and terminators follow the instructions.

The final service ends the command; it does not return to CSI through an ordinary application RET. Set the result explicitly rather than treating a processor return instruction as the operating-system exit interface.

Why addresses need relocation

The loader can place the program at different memory addresses. An address written as an offset from zero must therefore be adjusted to the chosen load address. This adjustment is relocation. An R-format executable supplies a list telling the loader which address words to adjust; ordinary data bytes and service numbers must not be changed.

In this example, the JP operand and the LD DE operand contain addresses. Their two-byte words begin at program offsets 0x0001 and 0x0004. Adding the load base to these words preserves the relationships between the instructions and their targets wherever the program is placed.

Package the program for the loader

The executable stores its program bytes in sectors, followed by the relocation information. Pad the 22-byte image to a 256-byte sector. A second sector begins:

```
02 01 00 04 00
```

The first byte says there are two relocation offsets. Each offset is a little-endian 16-bit value (LE16): its low byte is stored first. The offsets 0x0001 and 0x0004 identify the JP and LD DE address words. For each, the loader adds the selected load base modulo 65536. The directory records R format, two stored sectors, one allocated 1024-byte block and image length 0x0016.

For example, at a load base 0x5157, the relocated prefix becomes `C3 5A 51 11 62 51`; the message lies at 0x5162. The resulting program prints HELLO Z80 with CR 0. The base is not universal. Each partition reserves a context area for its state and workspaces before the program. Ordinary loading places code 0x04CC bytes after that context begins. Chapter 20 explains this memory arrangement.

An R image has no mandatory external header or JP magic. Many utilities begin with a jump and version text, but STARTUP begins with register saves. The directory supplies the length and format. BASICOBJ is not a linker object and BASICCOM is not this executable format.

Runtime responsibilities

Use each service's particular register and flag contract. The dispatcher preserves input registers while selecting a target, but the selected routine can modify them. Do not assume Unix-like arguments, a universal stack frame, uniform success in A, or automatic channel closure.

The partition's memory ceiling is the upper boundary available to commands and their stacks. Service 0x37 queries this mutable boundary, which CSI uses to establish the stack; 0x38 changes it. Drivers and classes can lower that ceiling. An SP or entry register value from one loader path is not a complete ABI for every application. Chapter 30 follows known loader placement and termination without inventing the unresolved initial-register contract.

Chapter 19 — Using system services

A system service performs an operating-system operation for a program: output a character, obtain the current privilege, open a file or wait for an event. The program selects the service and supplies its inputs; the service returns the documented result or changes execution as specified. This chapter begins with visible output before introducing file requests, partition state and asynchronous operations.

Native programs use the Z80 calling form introduced in Chapter 18: RST 08h followed by one byte selecting the service. Registers supply many of the arguments and receive results. BASIC extensions provide another route for selected operations and are introduced separately below. Neither an assembly fragment nor an extension call is a command to type at CSI.

Three elementary calls

The fragments below assume execution inside a valid partition with its normal stack and console. They are assembly notation, not CSI commands or complete standalone programs. Chapter 18 explains the instructions and complete bytes of a native greeting; use that context when reading these fragments.

To display one character, put its character code in C:

```
ld c,65
rst 08h
db 05h
```

LD prepares ASCII A, RST enters the dispatcher, and DB supplies the selector for console character output. This route applies ordinary output policy, so suppressed output can remain invisible.

To start a new line, use:

```
rst 08h
db 12h
```

The newline service needs no explicit character buffer. It is separate from printing a numeric byte value as text.

To obtain the current privilege, use:

```
rst 08h
db 5Ch
```

The returned A contains the privilege byte. The call does not print it or change identity. A program that wants a decimal display must separately format the value and send the resulting characters to the console.

Read a service's calling contract

For each service, identify the selector number, input registers, memory that the caller must prepare, result registers and error indication. A pointer is an address of data in memory; returning a pointer does not display the data. Registers that must survive a call need an explicit preservation guarantee.

The Z80 also returns condition flags, including carry and zero (Z). Their meaning depends on the service. For example, a queue read can distinguish an empty queue from a stored zero byte through carry, even though the data value alone is zero in both cases. Do not assume a universal success code in A or the same meaning for every flag.

The dispatcher reads the inline selector and ordinarily resumes after that byte when the service returns. The installed table covers 0x00–0x6E; no range check is established, so use only documented entries. Chapter 32 gives each contract. Its service names are descriptive labels, while the numbers and target addresses identify the actual interfaces. Chapter 24 explains dispatch and the distinction between resident and overlay implementations.

Console input and device output

An output request can pass through several layers. Console policy decides whether ordinary output is suppressed or copied to a printer. A terminal class translates cursor operations into terminal-specific sequences. A driver handles the device itself. Choose the layer whose behavior the program needs; a raw output call does not automatically perform the work of the higher layers.

Service 0x05 applies console policy, including Escape-O suppression and printer copying. Service 0x07 applies kernel cursor controls and terminal-class translation while bypassing that suppression. Service 0x40 with console association 9 reaches the raw driver and does not update tracked cursor coordinates. Three calls to 0x07 carrying `0x10`, column, row perform the documented coordinate operation.

A buffer is caller-provided memory in which input can be stored. The input budget limits the number of accepted characters, but space can also be needed for terminators. Line input 0x01 takes DE as the buffer address and B as the budget. Allocate room for terminators: an exhausted five-byte budget stores five characters plus CR; four characters followed by typed CR store that CR and an additional CR. A reports accepted count; DE and BC change. B=0 and arbitrary editing paths are not fully characterized.

The ALP family supplies serial-terminal drivers. DMA is direct memory access, used here for output transfers; DMA readiness and available input are different conditions. Device status is family-specific. ALP status returns A=0 in both empty and queued cases; Z tests input occupancy, carry tests DMA readiness. Its status call can flush pending output. Input 0x3F can return a real NUL with the same AF as an unassigned association, so establish the association independently. Event wait 0x6B can wake without a character; recheck the condition. It returns with interrupts enabled in the specified paths.

File channels and storage requests

The BASIC channel operations in Chapter 17 have lower-level service counterparts. A native caller must prepare the channel identity and provide a request describing the operation and its workspace. A workspace is memory used during the operation; its pointer is the address of that memory. These objects must exist before OPEN is called.

Initialize a channel name/class through 0x18, then pass a writable four-byte request to OPEN 0x20. It contains the channel number, opening-mode byte and a two-byte little-endian workspace pointer. OPEN modes and channel status are distinct fields; one must not be substituted for the other. OPEN can modify mode bits. Services 0x21–0x23, 0x28–0x2D and 0x5E/0x5F provide closure, record and counted-byte operations. Appendix E gives organization-specific channel tails; do not interpret an indexed tail as a sequential cursor.

Directory lookup 0x14 normalizes a writable drive/name/type request on every retry. A=0 with the established flags means found; A=1 can mean absent with a reusable slot. Carry-set failure may leave HL as a hash or last probe address, not an entry pointer. Classify the return before dereferencing it.

Allocation reserves disk blocks for a file. A contiguous allocation is one unbroken run of blocks, rather than the same total spread across several gaps. The allocator distinguishes exact contiguous allocation 0x1B from minimum-size allocation 0x63. Freeing 0x1C changes bits and counters without a demonstrated double-free or bounds guarantee. Direct sector access bypasses application-level record validation and retains the common retry/ignore policy.

Obtain the current partition's context

The executive keeps a context for each partition: a memory structure containing its identity, memory boundaries, channels and execution state. Code that needs that structure first obtains its address. Ordinary character-output calls do not require the caller to decode it.

Service 0x30 returns the current context address in IY, a Z80 index register:

```
rst 08h
db 30h
```

IY now provides a base for documented context fields; the call itself prints nothing. Field offsets are measured from that base. Chapter 20 explains the memory arrangement and Appendix E defines the fields. Do not assume every use of IY in every component denotes this structure.

Call a system-related extension from BASIC

A BASIC extension is a separately supplied machine-code module that the BASIC runtime calls on a program's behalf. Selecting a module and calling it are separate steps. The module has its own argument and result convention; it is not selected by a native system-service number.

For this example, enter BASIC at CSI if it is not already active:

```
BASIC
```

At the BASIC dash prompt, select the privilege getter:

```
OPTION USR "GETPRIV"
```

This selects GETPRIV.BASICUSR. Call its entry at offset 0 with numeric argument 0:

```
PRINT USR(0,0)
```

The first zero selects the entry at the beginning of this module; the second is the numeric argument required by this getter. The result is the current privilege, which PRINT displays. Other modules require their own interfaces; these arguments are not a general template for arbitrary extensions.

GETID similarly returns the account number. GETUSER uses an eight-byte string workspace to return the account name; its complete example is in Appendix C. Extension modules return through BASIC's wrapper rather than ending the CSI command. This differs from the native greeting in Chapter 18, whose service 0x00 terminates the command.

Set a compiled BASIC program's requirements

An application can require a minimum privilege or a matching installation serial before continuing. These are restrictions on execution, not ways to grant authority to its caller.

OPTION PRIV sets the minimum runtime privilege for a compiled BASIC program. OPTION SERIAL imposes a serial binding. The exact serial value is system-specific and must not be copied from an unrelated configuration. Their individual forms are given in OPTION PRIV and OPTION SERIAL. The checks belong to the BASIC runtime, not to the native service selector in the preceding assembly fragments.

Arrange a callback

A callback is a routine entered by the system when an event or requested delay occurs. Its code can run later than the request that registered it. The caller must therefore keep the callback code and its required storage valid, and must understand which partition supplies the execution context.

Timer service 0x35 uses six writable bytes followed by callback code, converts the supplied delay to eight-millisecond ticks, and invokes the callback in its owning partition. Normal A=0 operation is documented; nonzero-A ownership exceptions do not establish general cross-bank cancellation. Chapter 25 explains the node layout, timing and entry paths.

Coordinate access with locks

A lock records ownership so that another partition can wait rather than use the protected resource at the same time. A wakeup tells a waiter to check again; it does not automatically transfer ownership to that waiter.

Byte locks 0x54/0x55 use a one-byte owner tag. Release broadcasts to waiters, which must recheck their own lock. Repeated same-owner acquisition adds no recursion count. File-open admission and record spans are different structures with different cleanup rules; Chapter 17 explains their programmer-visible distinction.

Exchange bytes through a memory queue

A queue stores bytes until a consumer removes them. The queue services use a caller-provided header and payload area rather than allocating that storage themselves. Capacity, occupancy and read position are distinct header values.

Services 0x47/0x48 use this caller storage. Test carry for empty reads: successful zero data also sets Z. Do not append to a full queue; the original cached-full loop cannot notice an ordinary consumer. Code-derived capacities 1–128 avoid the demonstrated eight-bit indexing defects, but do not establish complete concurrent safety. Chapter 25 describes the algorithm and its limits; Appendix E gives the layout.

Distinguish banked copying from callback entry

Banking selects which memory is visible to the processor. Copying bytes through another partition's bank and executing a routine in that partition are different operations. These advanced interfaces require the memory and scheduler model in Chapter 20 and Chapter 22; their register-level contracts are in Chapter 32.

Services 0x68/0x69 copy through a scheduler record's bank, not an A-selected physical bank number. They advance HL/DE, return BC=0 and disable interrupts on the specified returns. A zero count is not an empty-copy convention. Callback 0x56 instead changes execution context and can clear stopped/wait state; it is not equivalent to a memory copy.

Interpret numeric-helper results

Arithmetic helpers use registers to supply operands and return results. Error handling matters as much as the ordinary result: an overflow result can be an intermediate value rather than a truncated answer.

Unsigned multiply 0x27 must be specified for carry. On overflow, HL can hold an early partial accumulator, not the low sixteen product bits. `0x9000 × 3` returns 0x9000 with carry, while reversed operands return 0x3000 with carry. Division 0x26 by zero returns HL=0, DE=the original dividend and carry, without a trap.

Numeric parsers can return carry clear without consuming any digit. They do not skip spaces or accept signs. Validate consumed and trailing text as well as carry. Overflow consumes the offending digit and returns an intermediate partial result.

Handle a disk-error callback

A disk-error callback lets a program respond during a failed disk operation. It runs within that operation's locking and register context, so the ordinary retry, ignore and abort meanings must be understood before installing one.

A disk-error callback registered by 0x4A runs while the disk lock is held. Returning A=0 consumes the failure; positive A supplies the normal error/retry policy; negative A aborts. Preserve the operation's working registers. Reentrant disk I/O inside that callback is not established. Clearing the cause and returning zero is not a retry.

Chapter 27 explains the retry path and its consequences for stored data. Use the exact service entry, data layout and failure qualifications together; the services do not supply one uniform return or cleanup convention.

Part V — System fundamentals

Chapter 20 — Organization and memory

The system divides its work between a common resident executive, banked partition memory, replaceable service overlays and ordinary files containing commands and drivers. Understanding these boundaries explains why a utility can disappear from memory after execution while its open-file state remains visible to the executive, and why an address alone does not always identify a byte of RAM.

Common and banked memory

The Z80 presents a 16-bit address space. Addresses 0x0000–0x3FFF are common to the partitions. The interval 0x4000–0xFFFF selects one of five 48-KiB banks in a 256-KiB configuration: 16 + 5 × 48 = 256 KiB. The low bank-selection bits of port 0xFD select the bank; bit 7 controls the ROM mapping at 0x8000. An emulator must preserve common storage while changing the selected upper region. A flat 64-KiB model cannot implement that configuration.

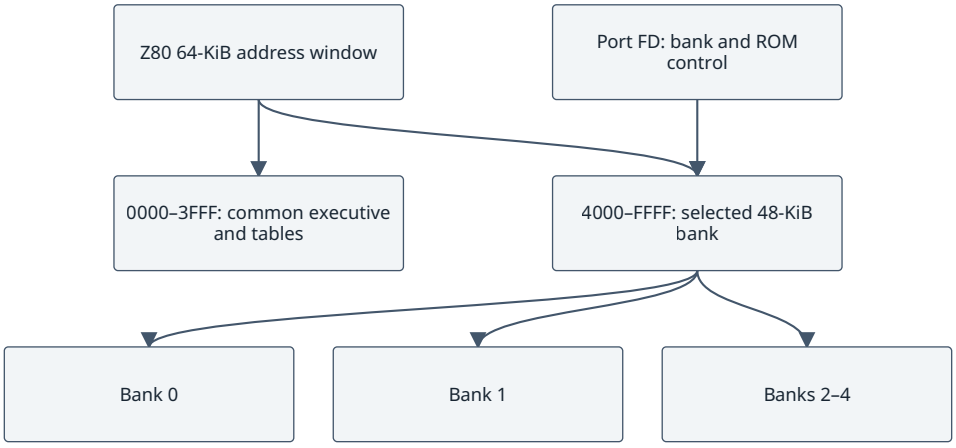


Figure 20.1. The CPU window and its backing storage. The diagram describes mapping, not simultaneous visibility of all banks.

Resident components and partition allocation

The NUCLEO body occupies 0x0000–0x39C1; the resident spooler follows at 0x39C2–0x4C8A. Their combined extent is 19,595 bytes. The first partition context begins at 0x4C8B and its program area at 0x5157. The boot file includes further initialization and overlay material; the native resident extent is not the same measurement as its 113 stored sectors.

A partition context reserves 0x4CC bytes before executable code. Its first 0xAC bytes hold identity, callback, console and command state. Seventeen 32-byte channels follow, then a 256-byte sector workspace and a 256-byte command buffer. Consequently the executable start is context + 0x4CC. A memory ceiling in the context bounds the upper end available to commands and their stacks. Drivers and terminal classes can be loaded downwards from that ceiling.

Loaded addresses are therefore configuration-dependent. Replacing a terminal class can change the ceiling; repeated C1 → C2 → C1 replacement was observed to reduce it from 0xFB64 to 0xF919 and then 0xF6CE. Treat a published load address as an address in a stated configuration, not a universal link address.

Overlays and ordinary commands

The RST 0x08 dispatcher and frequently used routines remain common. Less frequently used file and loader services are reached through stubs which select an overlay at 0x424A. Overlay 1 supplies indexed/keyed operations, overlay 2 supplies channel and sequential/direct operations, and overlay 3 supplies loading, termination, messages and initialization. The selected bytes at an overlay address must be known before disassembling that address.

CSI, the command processor, is an executable loaded into the current partition. Most named commands are separate `.COMANDO` files. BASIC has its own load file, overlays and runtime. EXEC is a separate interpreter with persistent level frames. These are cooperating components, not built-in syntax of one monolithic command processor.

The small common named-component table at 0x04BC provides exact eight-byte lookup. Its named spooler entry is `SPOOLER` → 0x39C2. It is not a directory, scheduler table or general symbol table. Chapter 24 describes its layout and lookup limitations.

See Appendix B for the canonical address and I/O maps, and Appendix E for context layouts.

Chapter 21 — Boot architecture

Startup transforms a physical sector stream into a banked multiuser environment. The physical bootstrap and the logical filesystem use different addressing rules. Keeping those stages separate is essential when rebuilding a disk or implementing the controller.

Firmware and the first loader

The firmware is 4,096 bytes mapped at 0x8000, with a reset-time alias at zero. Its initial jump bytes are `C3 03 80`, transferring control to 0x8003. Firmware reads physical cylinder 0, head 0, sectors 0–3 into memory beginning at 0x0100. Those four sectors form a 1,024-byte disk loader.

The loader relocates itself to 0xA000 and enters at 0xA023. An observed earlier SP is 0x3FFD; the relocated loader subsequently uses SP=0xA600 and IY=0xA700. These successive values belong to different stages and must not be presented as conflicting reset-stack specifications.

Physical sectors 4 and 5 contain volume descriptors. The loader uses the system-volume mapping to obtain the kernel. The loader obtains the kernel start from its catalogue entry. For example, a 113-sector kernel occupies 28,928 bytes (0x7100), first read at 0x0100. Its location is an allocation value, not a universal fixed sector. The following downward relocation copies 0x9000 bytes to zero, including memory beyond the bytes just read. That extra copied region must not be described as initialized kernel-file content.

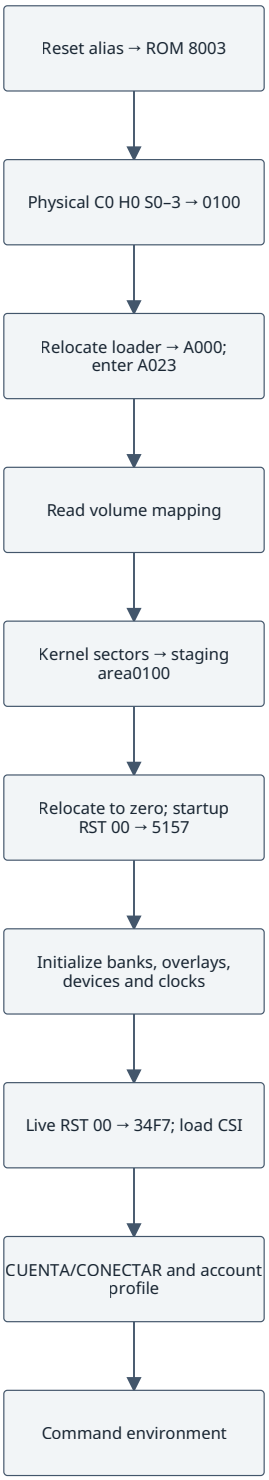


Figure 21.1. The documented disk boot path. Authentication occurs after CSI has been loaded.

Executive initialization

Before initialization, the newly relocated kernel's RST 0x00 vector jumps to 0x5157. Initialization changes the vector; at the observed login state it jumps to 0x34F7. SP is observed at 0xA5FE before initialization establishes its own stack at 0x5157. Initialization relocates overlay material into bank 4, installs common service pointers and constructs the available partition records. Service 0x0C illustrates the distinction between startup and running state: the disk table initially points to 0x5157; the live table points to 0x331B, a routine returning HL=0x5542. The meaning of that signature has not been established.

Service 0x4B with B=0xFF performs cold system-volume initialization. It locates NUCLEO, CSI, EXECLANG, EXEC1 and MENSERR, caches their positions and writes clock state into the kernel file. Startup therefore updates system state on disk before ordinary user work begins.

Account startup and profiles

CSI is loaded before account authentication. Startup uses account maintenance and connection components with a temporary owner value of 0xFF before restoring account 0 for CONECTAR. Missing account components can leave the environment at account 0, privilege 5, with an unresolved displayed name and then attempt IPL.EXEC or the prompt. This failure path does not provide a sound account-recovery procedure.

A connected account can have an EXEC profile named after it. The search visits S and then A–G, considering private and public candidates on each associated drive. Public S can therefore outrank a private A profile. The first matching nonsequential candidate stops the search instead of continuing to a later valid script; zero-capacity candidates are skipped. A profile must have the right format as well as the right spelling.

An explicit EXEC failure can mention `EXEC.COMANDO`, although the separate EXECLANG interpreter is involved. Diagnose the complete search and file format before concluding that a command executable has disappeared.

Restart is a new boot

The accepted Escape-I path reaches 0x2D01, writes 0x80 to port 0xFD and jumps into ROM at 0x8000. It reruns firmware diagnostics, physical bootstrap and cold initialization. Declining with N calls service 0x00 and ends the interrupted command; it does not resume it. Neither route supplies an implicit CLOSE for buffered application output. The operational consequences are described in Chapter 14.

Chapter 22 — Partitions, identity and protection

The partition is the system's scheduled execution environment. An account supplies identity and authority within that environment. These concepts are related during login but are not interchangeable: changing account does not allocate a new memory bank, and a lock owner is not an account number.

Three identifiers

The common scheduler uses a zero-based record index. Each three-byte record contains a context pointer and a bank/state byte. User commands display partition numbers using their own convention, commonly beginning at 1. The bank is the low nibble of the record's third byte. It can differ from the record index: record 1 can select bank 2.

The account number is context byte +0x1A. The account name occupies +0x12–+0x19, and privilege is +0x11. Lock structures instead store scheduler index + 1 so that zero can mean unowned. An engineer who decodes owner byte 2 as account 2 will misattribute synchronization state.

Scheduling and waiting

Multiple partitions are preempted by the timer; a computation containing no OS calls does not monopolize the processor. Programs can also yield through service 0x4F. Ordinary selection scans circularly for the first configured record whose high state nibble is zero. There is no specified per-record priority comparison in that path.

State bit	Meaning in the established paths
4	Waiting for a byte lock
5	Waiting for a device event
6	Disk-related wait state
7	Stopped partition

These are bits, not a set of mutually exclusive state numbers. Serial receive callbacks and cross-partition callback entry can clear the whole high nibble. A callback to a stopped, allocated partition can thus resume its dormant yield loop without restoring its console. Chapter 25 gives the exact selection, saved-frame and timer behavior.

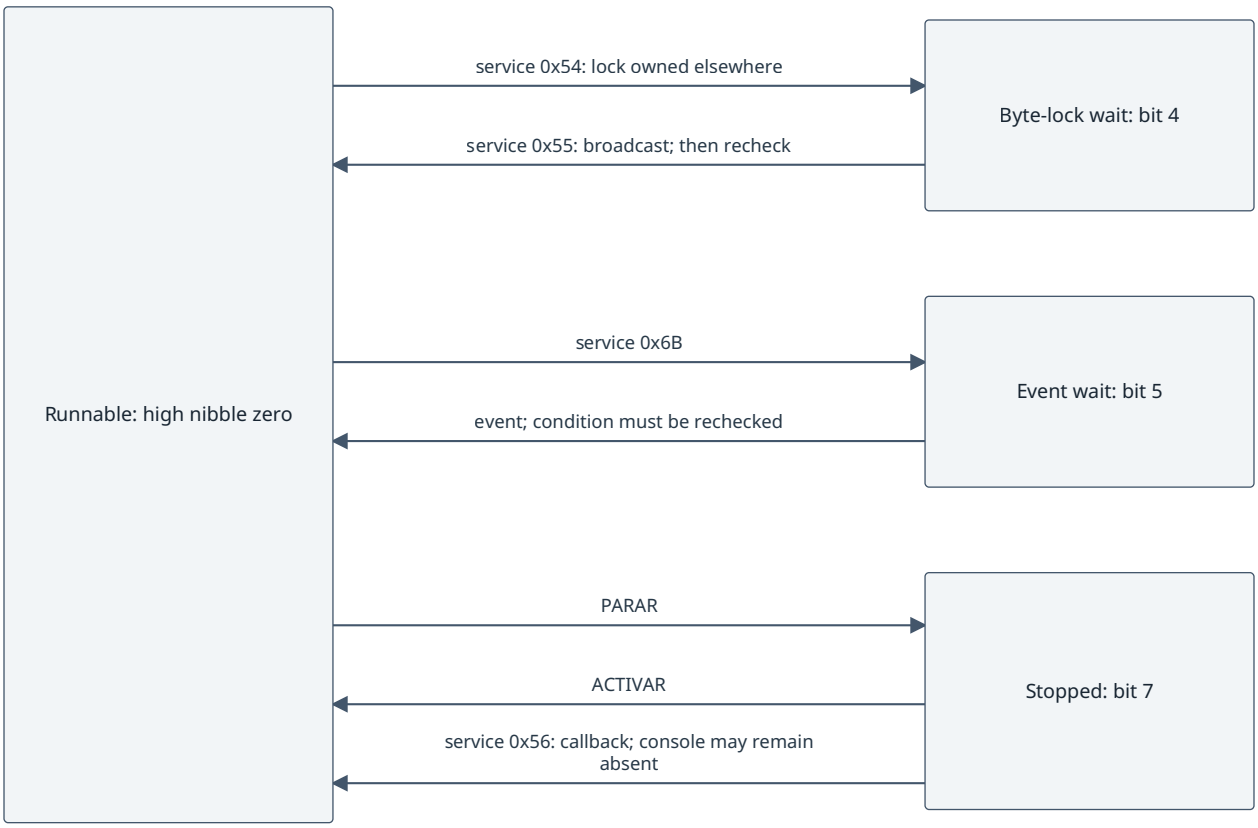


Figure 22.1. Selected documented transitions. The diagram does not prescribe a universal state machine for every combination of bits.

Protection boundaries

The system combines account ownership, public-file lookup, sharing aliases, file protection bits, command minimum privileges and software serial bindings. They protect different operations. The standard command prologue checks privilege and installation serial before the utility body; the utility may impose additional checks. BASIC implements corresponding runtime guards of its own.

File protection uses read, deletion and write bits in the directory format byte. The observed sequential replacement operation first deletes the old file. Consequently write protection alone can permit replacement while preventing append. Deletion protection alone prevents replacement but permits append. Read protection prevents INPUT and the established append path; it does not prevent same-owner replacement. Chapter 11 explains the administrative operations; Chapter 17 gives the BASIC-mode protection matrix and its limits for combined attributes.

An exclusive open controls admission to an open-file table. It does not reliably prevent native BORRAR from deleting the file: the deletion path calls the exclusive-admission helper but fails to test its carry before proceeding. The existing channel can survive while its blocks have been freed. This defect makes an accurate account of the behavior particularly important for compatible implementations.

No hardware-enforced kernel/user protection boundary has been established. Programs contain ordinary Z80 instructions and can call common routines; several low-level services perform no index or pointer validation. The historical design relies substantially on trusted programs and command-level discipline. This observation describes the specified architecture rather than a modern security certification.

Chapter 23 — Storage and device concepts

Files are catalogue entries interpreted according to a format byte. The catalogue identifies ownership and allocation, while an application's record format gives meaning to the bytes within that allocation. These layers explain why a structurally readable file can contain invalid BASIC records, or why a successful filesystem check can leave an application unusable.

Allocation, sectors and records

A logical sector is 256 bytes. The allocation bitmap manages blocks of four sectors, or 1,024 bytes. A file can therefore own more storage than its current payload consumes. Sequential files reserve the last two bytes of each sector for a link, leaving 254 payload bytes. Direct files instead expose fixed-length numbered records over full-sector storage; a 300-byte record necessarily crosses sector boundaries.

R and A executable formats use contiguous storage. An R file carries relocation records after its image; an A file carries an absolute load address in its directory entry. I files combine an ordered slot index with key/data/next-link slots. K files use hashed fixed slots without that prefix. None of those distinctions is encoded solely by the filename type.

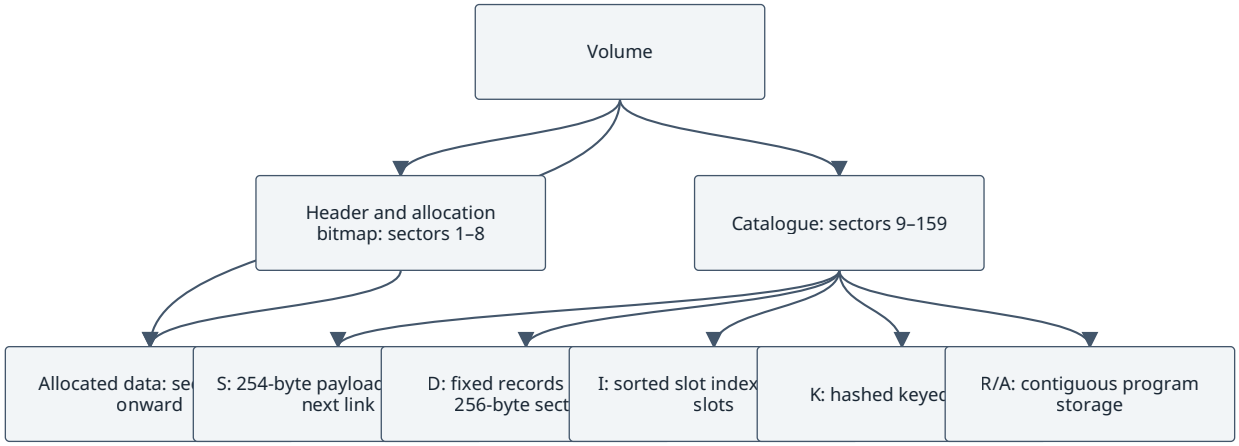


Figure 23.1. Filesystem layout for a volume with 151 directory sectors beginning at sector 9. These are configuration values, not universal defaults.

For the example geometry, the directory holds 151 sectors × 8 entries = 1,208 records. Its exact-name lookup is a probing algorithm rather than a scan of every record. A defect can exhaust a short repeating probe path while many catalogue entries remain unused. Wildcard listing scans all directory sectors and can expose entries that exact lookup misses. Chapter 26 distinguishes physical presence, catalogue reachability and application readability.

Logical and physical devices

Each partition has a 24-byte association map. An association names a use, such as S, console input or IMPR1; its byte selects a physical device descriptor. A descriptor in turn identifies a driver and device-specific parameters. These three levels let two partitions use the same logical name for different hardware without changing application code.

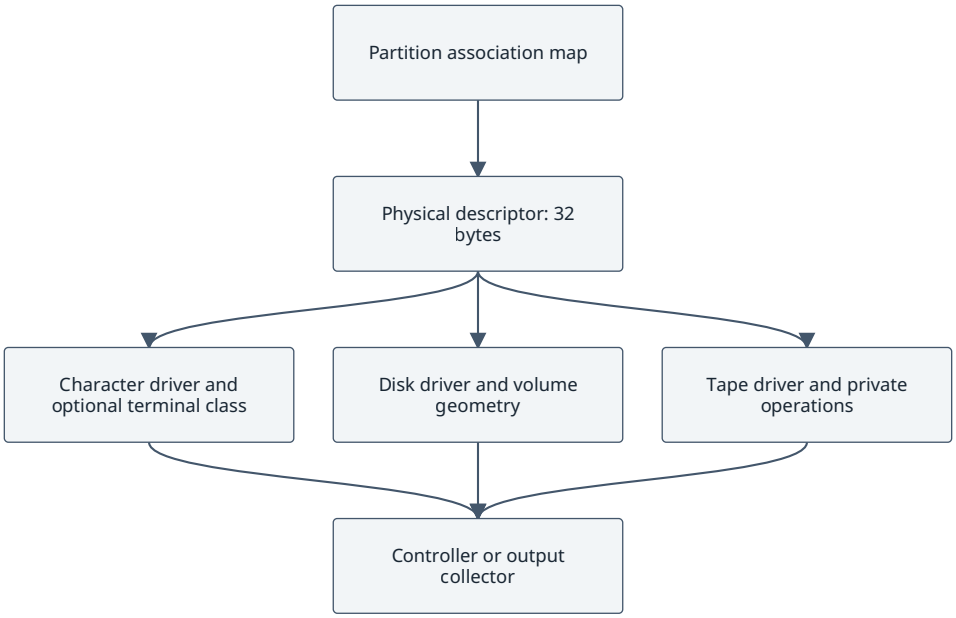


Figure 23.2. Device resolution. Descriptor fields form a family-dependent union; character and disk entry offsets do not imply identical functions.

Character drivers normally provide status, input, output, initialization and termination at offsets +0, +3, +6, +9 and +0x0C. The floppy drivers have four entries with different meanings: select/configure, recalibration request, read and write. Their byte at +0x0C is data, not a fifth jump. DIS10's public character stubs do not perform tape records; utilities call a private interface at driver +0x0F.

The terminal class translates the OS's logical controls into a terminal's output sequences. A raw device output service bypasses that translation and the executive's tracked cursor. The spooler supplies another layer by replacing printer output with a resident disk collector. Accepted output at any of these layers is not proof that a physical printer or serial peer received it.

Part VI — Operating system internals

Chapter 24 — Kernel entry and shared structures

Applications enter the executive through an inline-number restart convention. The common dispatcher is small; most complexity lies in the service-specific code and in the context selected when that code executes. This chapter connects the calling convention to the resident and overlay maps.

The RST 0x08 path

A call consists of opcode 0xCF followed immediately by a one-byte service number. RST pushes the address of that inline byte. The vector jumps to 0x08BD, where the dispatcher saves working HL, DE and AF, fetches the service number and advances the return address past it. A word table at 0x08E0–0x09BD selects one of 111 entries, numbered 0x00–0x6E.

There is no established bounds check for a larger number. The presence of a byte-sized selector does not establish 256 services. Symbolic names in Chapter 32 are descriptive editorial names unless an original name is explicitly identified.

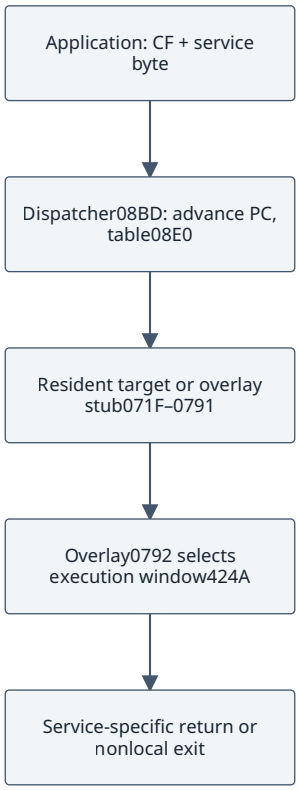


Figure 24.1. Service dispatch. A service that terminates or replaces a command does not follow the ordinary return arrow.

Stubs at 0x071F–0x0791 call 0x0792 and carry inline overlay-selection information. Overlay descriptors at 0x0713, 0x0717 and 0x071B are patched during startup. Thus the stored kernel code and initialized memory can legitimately show different contents in this area. Address tables in Appendix B identify their scope.

Context and channel storage

The current context pointer is the little-endian word at 0x0033. Service 0x30 returns it in IY. Service 0x50 instead returns IY=0, enabling access to common globals. Do not decode every IY-relative instruction as a partition field: driver code, BASIC and temporary wrappers use indexed registers for other structures.

Within a context, +0x00 is saved SP, +0x02 is the context base, +0x04 is the current memory ceiling and +0x06 is the allocated endpoint, with zero representing 0x10000. The sector workspace begins at +0x2CC and the command buffer at +0x3CC. The complete byte atlas in Appendix E keeps unknown fields explicitly unknown.

Service 0x18 initializes one of seventeen channel records by copying eighteen bytes of drive, filename, type and class, then clearing fourteen bytes. OPEN consumes a separate four-byte caller request containing channel, mode and a little-endian workspace pointer. The request is mutable: the executive and BASIC caller can both change its mode. An internal channel's status byte must not be substituted for that request byte.

Shared tables

The common scheduler rows begin at 0x048C. Record-lock rows begin at 0x1B15; file-open rows begin at 0x1DA8. Their owners all relate to execution state but their lengths and keys differ. The record-lock table's 65 initial rows include its terminating row, leaving 64 usable rows by code and layout. The file-open region contains 129 four-byte rows; a practical maximum supported by a saturation test has not been established.

The named-component table at 0x04BC has ten-byte rows: eight key bytes and a little-endian address. Service 0x6C compares all eight bytes exactly and stops when a row's first byte is zero or space. The initialized image has eight slots followed by a zero at 0x050C. The routine does not impose an independent count limit, store a bank or owner, or normalize case. Exact `SPOOLER` returns 0x39C2; a NUL-padded or lowercase spelling fails. In the established interface, success is AF=0x0044 and HL=address; absence is AF=0x0101 with the caller's HL preserved.

Interrupt entry is a separate interface

The reconstructed hardware uses IM 0 supplied restart opcodes. The timer enters through RST 0x10, the disk interrupt through RST 0x20 and the ALP serial path through RST 0x28. RST 0x18 is used by the tape-related path; RST 0x38 can invoke a per-context inline hook beginning at +0x39. These are not extra RST 0x08 service numbers.

A suspended scheduler frame contains ten register words and a continuation PC, but it is not a complete description of every interrupt's immediate stack. Appendix E documents this frame and Appendix B the observed vectors. Physical interrupt arbitration beyond the implemented and observed paths remains incomplete.

Chapter 25 — Scheduling, callbacks and synchronization

The scheduler switches a saved context, not merely a bank number. Its behavior also depends on shared wait bits and callback state. Those dependencies matter both to programs that coordinate work and to emulators that must reproduce a blocked application's next instruction.

Ordinary selection and quantum

Service 0x4F enters 0x0558. With one configured partition it preserves AF and returns after enabling interrupts. With several partitions it saves main and alternate registers, stores SP in context +0x00, switches to the common stack and scans three-byte rows. It selects the first eligible row after the scan origin, wrapping at the configured count. With none eligible, it restores the previous saved state, enables interrupts and HALTs before retrying.

The selected row updates current index 0x000C, current-row pointer 0x048A and context pointer 0x0033. Mapping proceeds through 0x21DE; the selected high state nibble is cleared. Restore at 0x062F–0x063E pops HL′, DE′, BC′, AF′, IY, IX, HL, DE, BC and AF, enables interrupts and returns through the saved continuation.

The timer subtracts 8 from the remaining quantum byte at 0x0554 and branches on sign or zero. This byte arithmetic has visible consequences. The measured `TIEMPO` values 10, 128, 135 and 136 produce 2, 16, 17 and 1 timer decrements respectively: approximately 16, 128, 136 and 8 milliseconds in the reconstructed timer configuration. A value of 136 does not mean a longer slice than 135. The 8-ms period follows the installed timer setup; arbitrary hardware clock variations remain separate.

Callback entry saves the remaining quantum in 0x0556 and sets hint 0x0555. If the saved amount is at least 4, the next special preparation backs the ordinary origin up one row and reuses that residual; otherwise it reloads the default at 0x000B. The origin at 0x0551/0x0553 can therefore differ from the currently executing partition. These globals are not per-partition priorities.

Callback context

Service 0x56 takes a target record index in A's low seven bits and a callback address in HL. Bit 7 suppresses the final interrupt-enable instruction. Cross-partition entry saves the sender, selects the target bank, restores the target's register frame and jumps to the callback with its continuation still on the stack. A callback RET resumes the target computation. The sender's seeded registers are not supplied as callback arguments.

Same-partition and single-partition branches take shorter paths. Invalid targets are not a safe error-return interface. The routine masks target state with 0x0F, including a stopped bit. A callback can therefore make an allocated PARAR context runnable while its console remains unassigned. Use ACTIVAR to restore the observed console lifecycle; do not equate callback injection with creating a new partition.

Timed callbacks use service 0x35. A six-byte writable node immediately precedes the callback code. Delay DE is converted to floor(DE/8) ticks; zero removes a matching node. The list stores delta times and partition ownership. Expiry unlinks and clears the six bytes before entering node +6 through service 0x56. A=0 is the established normal mode. A bit-0 exception bypasses ownership only for a numerically matching head and does not provide a documented general cross-bank cancellation operation.

Byte locks

A byte lock holds zero when free or current scheduler index +1 when owned. Service 0x54 acquires it, returns immediately for the same owner, or sets wait bit 4 and yields for another owner. There is no recursive acquisition count. One release is sufficient after two same-owner acquisitions.

Service 0x55 ignores another owner's lock. For the current owner it clears bit 4 in every configured row and then clears the byte. This is a broadcast wakeup: a partition waiting on an unrelated lock wakes, tests its own condition and may wait again. Neither release nor wakeup guarantees fairness or transfers ownership directly.

Generic command cleanup releases a fixed list of locks at 0x29A7, 0x2462, 0x21DD, 0x046C, 0x2108, 0x2109 and 0x0057, followed by the disk transaction. It is not established to release arbitrary application-defined lock bytes.

File admission and record spans

An open-file row keys on physical device and first logical sector; its fourth byte contains owner and an exclusive bit. Shared opens can coexist. An exclusive open conflicts with every same-key open, while a shared open conflicts with an existing exclusive one. Pending OPEN calls yield and retry without using byte-lock wait bit 4. Two shared channels of one owner create identical rows; CLOSE removes one row, and the last close or owner cleanup admits an exclusive waiter.

Record locks instead key on physical device, channel and an inclusive sector interval. With multiple partitions, direct UPDATE mode 0x58 becomes 0x5A, enabling record locking. A direct read optionally releases old channel spans, acquires its span and retains it after successful I/O. A write-back releases spans when bit 1 is set and bit 2 is clear. UNLOCK/service 0x53 releases record spans but leaves exclusive file admission intact.

The interval implementation is imperfect. With 300-byte records, retained read 1 can own sectors 0xDA48–0xDA49. Same-owner read 2 requests 0xDA49–0xDA4A but returns without widening that row. Another partition can then acquire record 3 at 0xDA4A–0xDA4B. Successful read 2 has not acquired all the coverage its caller might expect.

The lower disk-read path independently checks existing record spans. A direct request that skips acquisition can still wait for another owner's already-held sectors. This second check is why disabling the upper acquisition bit does not bypass all record contention.

Memory queues

Services 0x47 and 0x48 operate on caller storage containing capacity, occupancy, read index and payload. They must not be described as a reliable blocking queue abstraction. The append routine caches occupancy before its full test and does not reload it while waiting; a timer callback can remove a byte and the writer can still spin indefinitely. The loop does not yield.

The index arithmetic wraps at eight bits before all reductions. Capacities above 128 can select the wrong payload or overwrite header bytes; capacities 254 and 255 have demonstrated header corruption. Serialized capacities 1–128 avoid these particular arithmetic defects by implementation-defined bounds, but this is neither an original advertised limit nor a complete concurrency guarantee. Queue consumers must distinguish empty A=0/carry set from a successfully read zero byte with carry clear.

Chapter 26 — Filesystem implementation

A file operation passes through several independent decisions: name normalization, account and drive search, catalogue probing, format-specific channel setup, allocation and physical I/O. A failure or inconsistency at one layer does not establish failure at every other layer. This chapter gives the algorithms needed to reproduce that distinction.

Lookup and normalization

Service 0x14 receives a writable 17-byte request: drive followed by eight name bytes and eight type bytes. It folds lowercase, retains spaces, dollar signs, digits and letters, and substitutes `$` for other bytes. A leading character below `A`, except `$`, receives 0x11. A leading blank can therefore become `1`, then `B` on a later normalization pass.

Normalization repeats during account and drive retries. Account 0 searching an explicit blank name can look for `1` and fail; account 2 can first fail privately and then normalize again to find public `B`. This behavior is not an ownership transfer. CSI separately treats underscore as deletion when context +0x0A bit 6 is set, whereas quoted BASIC names retain underscore until filesystem normalization changes it to `$`.

The name hash sums eight big-endian words from the sixteen normalized name/type bytes, modulo 65,536. For an example volume, let N=151 directory sectors and F=9 be the first directory sector. The initial LBA is `hash % N + F`. Each sector contains eight entries. Zero terminates the search; 0xFF is a deleted candidate remembered for reuse. A matching private entry precedes the account-0 fallback unless that fallback is inhibited by context +0x0C bit 6.

The subsequent-sector rule is specified exactly: if L<N, add F; if L=N, use 1+F; otherwise use F. At initial L=9 it cycles through only seventeen sectors. Filling those seventeen sectors occupies only 136 of the nominal 1,208 slots. The bounded search can therefore exhaust its cycle while slots elsewhere remain available. OUTPUT allocates a block before detecting this condition and releases it on the established normal failure path.

Wildcard LISDIR scans all 151 sectors instead. It can list an intact later entry hidden from exact lookup by an earlier zero marker. OUTPUT can then create an ordinary same-name/type/owner duplicate at the earlier reusable location. Exact lookup selects the new entry; deleting it exposes the old one. REPARAR accepts these duplicates. Neither successful creation nor a clean checker result proves global name uniqueness.

Allocation and deletion

The bitmap begins at byte 0x20 of logical sector 1 and continues through sector 8. Block b=LBA//4 uses byte `0x100 + 0x20 + b//8` in the logical volume stream and mask `0x80 >> (b%8)`. Set bits represent allocation. The header and cached descriptor also track free blocks.

Service 0x1B requires an exact contiguous count. Service 0x63 accepts desired and minimum counts: it takes the first run meeting the desired length, or otherwise the largest qualifying smaller run, preferring the later run on a tie. Total free space is therefore insufficient to predict whether a contiguous allocation succeeds. A request for 38 blocks failed with 38 free but a largest run of 37.

Deletion marks the directory entry 0xFF before releasing its blocks. Sequential chains can be fragmented, so their runs are freed separately. No transactional rollback across all writes is established. The native deletion path's ignored exclusive-open conflict is described in Chapter 22; an open file is not protected merely by being open.

Sequential open, transfer and close

Sequential OUTPUT mode 0x88 replaces an existing file. BASIC UPDATE SEQUENTIAL also selects 0x88 unless EXTEND is specified. The implementation deletes the old entry, allocates a block, creates zero count/end metadata and initializes payload with 0x1A before returning. A crash after OPEN can therefore leave a valid allocation with an unfinished directory entry.

EXTEND starts with mode 0x84 and returns 0x8C. It loads the directory's final-sector word directly, restores counts and scans backwards through trailing 0x1A to determine the append position. Trailing 0x1A data are indistinguishable from padding and can be overwritten by later append. A properly closed empty file has a nonzero final sector. An unfinished entry with final sector zero is different: EXTEND can load logical sector zero, and CLOSE can overwrite it and zero sectors 1–3. This malformed-file case destroys reserved metadata on that malformed-file path.

Record input 0x22 copies through CR, but treats 0x1A as end of file without copying the marker or appending a terminator. A partial record before 0x1A remains in the destination. There is no caller destination-capacity argument in the specified disk loop. Counted input 0x5E instead copies raw payload bytes, including 0x1A, and follows links. B=0 requests 256 bytes. BASIC gives those returned bytes an additional field/type interpretation.

Counted reads eagerly follow a link at a sector boundary. Reading exactly 254 bytes from the final sector can copy all requested bytes and nevertheless return A=1. A larger request can copy only the available prefix and leave the remainder of the destination untouched. EOF latches in channel state; subsequent reads copy nothing. BC is not a universal copied-byte count.

Counted output 0x5F increments the channel's record count once per call and can leave a partial sector solely in the caller buffer. BASIC's application WRITE may span several such calls and is not atomic. A READ then WRITE under sequential UPDATE shares the cursor; reading a 0x1A marker can place subsequent output behind an embedded application end marker, producing counted but normally unreachable records.

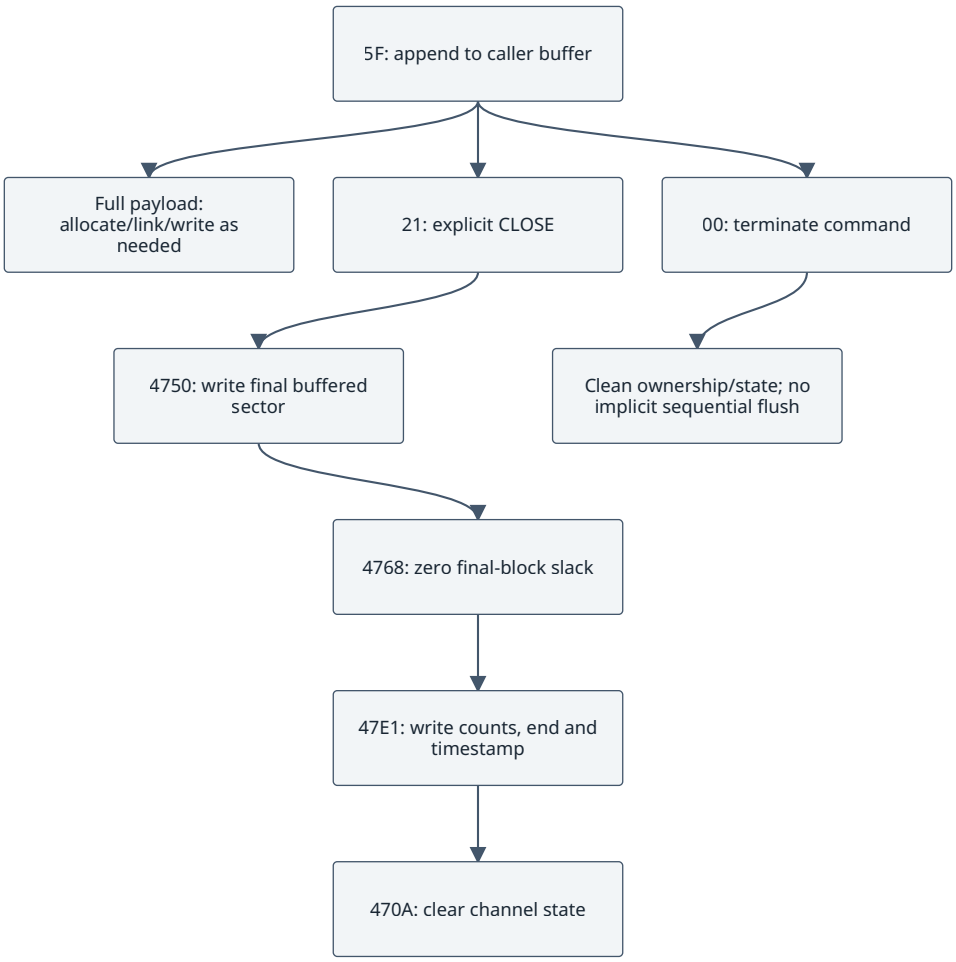


Figure 26.1. Sequential durability boundaries. Ignoring a failed disk write can make a later stage run without the preceding bytes reaching disk.

Direct and keyed files

Direct record n begins at `(n-1) × record_length`. Read and write can span sectors; write preserves surrounding bytes with read/modify/write operations. Record zero and out-of-range requests return A=0x80 in the code. Successful selected operations preserve BC, DE, HL, IX and IY; write protection can be accepted at OPEN and refused only at WRITE.

An indexed I file begins with N little-endian slot numbers, rounded up to a 1,024-byte boundary. Its first C entries identify active slots in key order. Each slot contains fixed key, fixed data and a two-byte next-slot link; remaining index entries are zero. The links follow the same ordered sequence. The index prefix holds ordered slot references, not a set of independent hash-chain heads.

A K file has fixed key/data slots without an index prefix or next word. Creation rounds the requested capacity using a prime-search rule: start with requested OR 3 and add 4 until prime. A request for 16 yields 19. Hashing applies `h=(10*h+byte)%N` through the key until NUL. Starting at h+1, collision steps

use a state initialized to -N and incremented by 2, adding its absolute value modulo N. Zero marks an unused slot; 0xFF is a reusable tombstone. Appendix E supplies byte layouts and equations for both formats.

Repair is an independent interpretation

REPARAR scans all 1,208 records, skipping zero, deleted and exactly 0x80 entries. Its supported format mask includes R, A, S, D, I and K, but its filename acceptance is stricter than kernel creation. It can reject and, with F, delete an interior-space name that BASIC successfully created and read. It does not validate application record counts or repair every semantic inconsistency.

The F path can delete bad entries, clear sharing redirects and reconcile the bitmap. Even a checking invocation may update the free counter. Checking directory allocation and validating application records answer different questions. A consistent bitmap does not certify complete application data.

Chapter 27 — Disk I/O and physical translation

The common disk path resolves a partition association, checks synchronization and protection, converts a logical request into driver coordinates, and applies error policy. The final controller transaction is only one part of the public service. A successful return can follow a deliberately ignored controller failure.

Address translation

For a disk configured with 830 cylinders, five heads and 32 sectors per track, using 256-byte sectors, capacity is 132,800 sectors or 33,996,800 bytes. Sector IDs on this hard-disk interface are zero-based. The linear sector index is `(cylinder*5 + head)*32 + sector`.

A volume mapping can run in a descending cylinder direction. Divide logical LBA by 32 to obtain track t and sector s, then divide t by five to obtain q and r. In a five-head volume descending from cylinder 396, the mapping is cylinder `396-q`, head `4-r`. Logical filesystem order need not match ascending physical cylinder order.

A descriptor for this example geometry begins with five heads and a little-endian 397-cylinder count. Forty-five source/replacement track pairs occupy offsets 0x03–0xB6. The runtime substitutes a matching track once and stops at zero or a larger source. Bootstrap traversal differs: it lacks the same zero-stop rule, so source zero can match. For the descending mapping, replacing logical track 169 with 1939 redirects C363/H0 to C9/H0. Complete behavior for the special 0xFD/0xFE descriptor signatures and 0x55/0x99 mapping variants is unspecified.

The descriptor's cylinder span is not itself the usable volume capacity. A filesystem declaring 1,939 logical tracks of 32 sectors has 62,048 sectors, corresponding to 15,512 four-sector allocation blocks. Use declared filesystem bounds for allocation.

Common request path

Single-sector services 0x0A and 0x0B take B=association, DE=LBA and HL=buffer; both force C=1. Multi-sector services 0x32 and 0x33 use C as count. Selected successful eight-sector calls preserve BC, DE, HL, IX and IY and return AF=0x0044. The single-sector forms preserve the specified DE, HL, IX and IY but return C=1 rather than the caller's original C.

The common path acquires the disk transaction lock and checks record spans. An invalidated descriptor can cause a preliminary header read before the requested transfer. Thus one requested sector can generate two physical reads without a retry. Service 0x09 invalidates cached descriptor bytes +0x14/+0x15; it is not a general flush operation.

For writes, descriptor +0x13 bit 7 rejects the request with driver error 2 before issuing a controller command. The WD path uses command 0x20 for read and 0x30 for write, transferring through port 0xB8 before examining status. The specified failure status at the relevant WD registers maps to error 7. The bytes returned after a physical fault are not guaranteed valid.

Track formatting and sector size

INICDISC issues WD command 0x50 to format a track. With 256-byte sectors, it sends a full 256-byte buffer through port 0xB8. The beginning of that buffer supplies pairs of bad-block flag and sector ID; for 32 sectors, 64 bytes hold the pairs and the remaining bytes complete the transfer buffer. Sector Count determines how many entries are used. The formatter builds the sector-ID permutation; its interleave selection should not be inferred from the logical order in which files are read.

The SDH register at 0xBE separates size bits 6:5, drive bits 4:3 and head bits 2:0. Size encoding 00 means 256 bytes, 01 means 512 bytes, and 11 means 128 bytes. Encoding 10 is not a defined sector size here. Thus setting bit 5 alone selects 512 bytes, not 256. The system's 256-byte requests use size bits 00 while setting head and drive independently.

A controller transfer can finish its normal data/completion sequence while reporting an error. The driver must examine status; completion is not a substitute for successful status. Chapter 11 covers the operator's formatting dialogue and the effect of PISTA on volume placement.

Retry, ignore and abort

The installed retry count is four. R retries; M calls the driver's +3 entry and retries; I consumes the failed sector; Q leaves the operation nonlocally. For the resident WD driver, +3 is RET, so M is not demonstrated to perform a controller reset.

An ignored failed read can return normal success with invalid buffer contents. There is no general guarantee that failed reads supply zeros. An ignored failed write returns normal success without writing a valid sector. Software must not interpret A=0 as independent evidence of media integrity after operator intervention.

A registered 0x4A callback runs before retry-count decrement while the disk lock is held. A=0 ignores, positive A substitutes an error code for ordinary policy, and negative A aborts. Removing write protection in the callback and returning zero skips that failed write rather than retrying it. CLOSE can then commit metadata for a record whose payload never reached disk.

Standalone Q normally produces CSI result 254 through an active interruption callback. During protected CLOSE, that callback may already be cleared; the Q handler returns A=0 and subsequent generic termination reports result 0 despite lost buffered data. Result codes describe control flow, not a transaction's durability.

Floppy interface

DIS1 and DIS2 use a distinct four-entry ABI. Private read/write inputs are A=sector count, B=head, C=zero-based sector, DE=track (E used), HL=buffer and IY=descriptor. The packet uses read opcode 0x06 or write 0x05, with 0x40 added for the established 256-byte mode. The first physical sector is C+1 for 256-byte sectors, or 2*C+1 for 128-byte sectors. End-of-track and count are correspondingly doubled for 128-byte translation.

DIS1 polls ports 0xCC–0xCE and can remain in initialization when the expected device does not respond. DIS2 uses 0xC0–0xC2 and can report not-ready error 1 without an available drive. The DIS2 transfer path writes the bank byte followed by buffer-address high and low bytes to port 0xC2, then polls bit 7 for completion. Controller result bytes come through 0xC1. The physical adapter wiring, bus timing and address carry across a 64-KiB boundary remain incompletely specified.

The FLOPPY2 format selected by INICIAR with CARA 2 has 77 cylinders and 26 sector IDs per track: cylinder 0 uses FM/128-byte sectors on both heads, and the remaining cylinders use MFM/256-byte sectors. The physical sector order is 1,14,2,15 through 13,26. These describe the software format, not a particular drive manufacturer or model. Chapter 11 explains formatting, mounting and readback checks. File transfers alone do not establish floppy bootstrap, IBM interchange or a complete removable-media archive workflow. Appendix B gives the packet and error mappings.

Chapter 28 — Terminal, serial, printer and tape drivers

The console's apparent simplicity conceals several layers: application policy, logical controls, terminal-class translation, device queues and controller interrupts. Drivers for other devices reuse some entry offsets but retain their own contracts. This chapter follows the software path from a character request to its driver and explains why accepted output is not proof of physical delivery.

Console policy and class translation

Service 0x05 applies context output suppression and printer-copy policy before service 0x07. Service 0x07 interprets logical cursor controls and calls terminal-class +3, with raw fallback when the class returns carry. Service 0x40 writes to an associated device directly and does not perform class translation or update the tracked cursor.

Escape-O toggles context +0x0A bit 1. It suppresses service 0x05 and echo generated by service 0x04, but direct 0x07 and 0x40 output continues. Escape-S runs a local pause loop with scheduler yields; other partitions continue. Resumption drains queued input, so a character typed while paused need not be delivered to the application. Escape-Q interrupts the established noncooperative loop and returns CSI result 254.

Class files have input entry +0 and output entry +3.CLASE1's control-pointer table begins at +0x8C; CLASE2's at +0xBE, each containing 32 little-endian pointers. Sequences terminate at 0xFF. Controls 0x00, 0x10, 0x15 and 0x19 receive kernel handling rather than an ordinary table emission. Control 0x10 consumes column and row bytes. CLASE1 encodes these as ESC G followed by coordinate+0x20; CLASE2 emits ESC `[` row+1 `;` column+1 `H`. Arithmetic wraps as bytes; it is not a clipping operation.

The descriptor's L and P values have caller-specific uses. CSI passes L directly as its line-input budget, and ordinary output tests the horizontal threshold before advancing. LF increments and caps the row at P. Default L79/P23 must therefore not be reinterpreted categorically as an 80-by-24 display size.

The transport is seven-bit in the reconstructed console setup. An emitted high-bit control can consequently appear differently on the terminal. Deferred form-feed state can consume the next echo while leaving the input character valid. Diagnose input bytes, echo and cursor tracking separately.

ALP serial drivers

DIS18–DIS21 implement PAN1–PAN4 on port pairs 0x32/0x33, 0x30/0x31, 0x36/0x37 and 0x34/0x35. DIS23–DIS26 supply the optional second group's pairs 0x3A/0x3B, 0x38/0x39, 0x3E/0x3F and 0x3C/0x3D. The 887-byte driver contains an 80-byte input ring and two 83-byte output buffers.

The ring at driver +0x240 stores capacity, count and read index followed by payload at +0x243. Full input is dropped. The established flow-control branches act when the previous count is 75 on receipt and 5 on consumption. The resulting occupancies are 76 and 4 respectively; confusing pre- and post-

update values misstates the driver's pause/resume thresholds. Ports 0x24–0x27 supply separate receive-ready outputs, while SCC WR5 controls RTS. Their exact physical combination remains incompletely traced. Software-ring backpressure and the SCC's smaller receive FIFO are separate limits; stopping the sender does not retract a character already in flight.

Output buffers begin at +0x293 and +0x2E6. Output flushes around Escape and after CR, FF, ETX or count above 82; status polling can also flush and wait for DMA. A returned character-output call may therefore describe buffering only. DMA readiness is held in fixed common bytes 0x02A6/0x02A7, not in a descriptor-local busy flag.

Service 0x67 registers a three-byte callback row at `0x23FF + 3*A`, for indices 0x02–0x20. The row contains owner index and callback pointer. PAN1 receive/DMA indices are 0x11/0x15, PAN2 0x12/0x16, and the later channels follow the installed map in Appendix B. Receive dispatch enters the registered owner through the cross-partition callback path and clears the high state nibble. DMA completion clears only wait bit 5.

A blocking input routine flushes output, disables interrupts, attempts a ring pop and calls 0x6B if empty. With one partition, 0x6B enables interrupts and HALTs; with several it marks event-wait bit 5 and schedules. Wakeup is not proof of queued input. DMA completion, consumed escape input and other callbacks can wake a reader which must test again.

SS2 serial drivers

DIS16 and DIS17 are 650-byte drivers for PAN9/IS9 and PAN10/IS10 at 0x7E/0x7F and 0x7C/0x7D. Their input rings hold 128 bytes; output is polled rather than ALP-buffered. Status carry derives from controller bit 2, while other flags reflect ring occupancy.

Initialization registers callbacks at indices 0x0B and 0x10. Termination waits on controller register 1 bit 0, resets the channel, unregisters the callback and enables interrupts. RST 0x10 dispatch reads port 0x60 masked with 0xE0. Source 0xE0 takes the SCC path through 0x2386, which reads RR3 at 0x7E: bits 3–5 select callback 0x0B, otherwise the selected path is callback 0x10. Source 0xC0 takes the timer path and is acknowledged by writing 0xA0 to port 0x60. Physical arbitration and complete source-mask wiring remain unspecified. The serial drivers require their own fitted controller and correctly framed terminal connection; they are separate from the ALP ports.

The driver temporarily replaces context callback +0x28 during status/output. In a controlled busy retry, it overwrote its saved pointer and returned with its own handler still installed. Ready-only output restored the previous handler. This is a driver lifetime defect rather than a different contract for service 0x39.

Printers

IP/DIS11 uses ports 0xA4/0xA5 and reports disconnection in the specified unmapped configuration, leaving its allocation removable by ASOCIAR. CDC/DIS12 can patch its ports to 0x50/0x51. It uppercases lowercase, substitutes ? for the observed unsupported special characters, complements bytes and writes them. IMP/DIS22 uses ports 0x78–0x7B with the SCN2651 register layout: data, status, alternating mode registers and command. The port assignment is supported by the driver and board component

identification; complete signal and clock wiring remains unspecified. Its status tests require transmit-ready, DCD and DSR, while CTS gates serial transmission. The normal initialization requests nominal 9600 baud with eight data bits, odd parity and one stop bit. The B4800 PE form selects seven data bits, even parity and one stop bit.

IMP maps 0x80 to 0x1F before output. It can also mistake an all-ones 0xFF status from an unavailable interface for readiness, so status acceptance alone does not prove delivery. An offline printer can trigger the native R/Q prompt. These are byte-transfer and readiness contracts; the physical printer model and its page-description behavior remain unspecified.

The common printer policy for associations 0x0C–0x0F polls readiness and manages controls, page state and delay. In the established default path, 0x1F and FF become CR, FF before the driver. An unassigned printer can silently return success. The printer input entries are RET stubs; their incidental A values are not input characters. These software returns do not certify that a page has been printed.

Tape

DIS10 is a 2, 980-byte driver. Its public status, input and output entries are XOR A/RET stubs; actual record operations use driver +0x0F with private selectors 0x80–0x8C. Utilities obtain that address through the physical descriptor. These selectors are not system calls beyond 0x6E.

Initialization registers callback index 0x03 and waits on ports 0xA0–0xA3. Initialization can wait indefinitely without the expected controller response. Complete read, write, rewind and archive-recovery procedures are unspecified.

Some private paths are especially uncertain: operation 0x8B has a branch that returns before restoring a pushed DE, operation 0x8C stores 0xDF where a reset interpretation might suggest zero, and 0x83 can return zero without output on a selected branch. These incomplete contracts require caution when implementing compatible behavior. The selector and 80-byte label tables in Appendix A are sufficient to identify the specified protocol without presenting it as a specified backup service.

Chapter 29 — Deferred printing and messages

Deferred printing and messaging both cross the boundary between an immediate request and later activity. They use different storage and execution mechanisms. The resident spooler intercepts printer output, live messages inject a callback into another partition, and the mailbox utilities maintain an ordinary sequential file.

Resident spooler

The named resident entry at 0x39C2 leads to administration code at 0x3FB2 and a selector table at 0x3FC4–0x3FE9. Selectors 0x00–0x12 have no specified independent bounds check. Collection enters at 0x39C8 and continues at 0x3B9C. Global word 0x0044 identifies the active spooler.

SPOOLER.COMANDO queries resident name SPOOLER through service 0x6C at file offset 0x0160. The missing-resident branch refers to `CARGAR SPOOLER`, stored at 0x0D47; a separate `VACIAR SPOOLER` string is at 0x0D1D. Both standalone executables are absent. Because the resident SPOOLER entry already exists, these missing companions do not prevent the documented initialization and paused collection. The references suggest a separate resident-component lifecycle, but do not establish its complete interface or make DESCARGAR a synonym for VACIAR.

Before initialization, SPOOLER LIST can report an empty queue with result 0 while ESTADO reports its uninitialized condition with result 12. After INIC, the established paused configuration replaces the printer driver's output-vector target with 0x39C8. The common printer path detects nonzero 0x0044 and bypasses its ordinary formatting, allowing the collector to retain the input stream.

A report can be collected while ordinary printing is paused. Form-feed and end-report controls affect its page count and completion state. Queued data describe collected output; they do not certify that a physical printer has produced it.

The sixteen collection slots at 0x39DD have a 16-byte stride. They hold flags, free-buffer count, buffer pointer, current sector, pages, blocks, queue number, queue selection, flags/copies and line state; the final byte remains unresolved. SPOOLCOL is a direct file of 64 sixteen-byte records in one block. A queue record stores an eight-byte report label, submitting partition index+1, three-byte timestamp, flags, queue and page count. The producer's equality wrap appears to use records 1–63; that effective capacity remains implementation-defined rather than saturated natively.

Report flags distinguish 0x80 open, 0x40 closed, 0xC0 printing and zero printed, with retention bit 0x20, copy count in bits 2–4 plus one, and printer in bits 0–1 plus one. The backing file's directory count is pages, not application writes. The collector writes its own metadata rather than relying on generic sequential CLOSE.

Report encoding and restart rules

Report data uses 0x06 followed by a byte to represent spaces; ordinary lengths are encoded as length+0x20. The exact decoder wraps, so malformed 0x21 and 0x20 counts yield 257 and 256 spaces. Byte 0x0D means CR/LF, 0x8D means bare CR, 0x1A ends the report and 0x1F skips the rest of a sector before following its chain. Other bytes, including 0xFF, are literal on the specified path. This is report-level encoding, unrelated to the directory read-protection bit 0x20.

On the implementation-defined INIC restart path, exactly 0x80 incomplete queue entries are deleted. Retained 0xA0 and multicopy 0x84 cases can survive; printing states 0xC0–0xFF become 0x40–0x7F. Failed report lookup clears flags. Backward cleanup of inactive tail entries clears the first label byte rather than every field. Invalid chains can encounter zero links; no visited-sector cycle guard is established. Native collection is documented, while the complete restart and physical-output lifecycle remains unresolved.

Live message transfer

Service 0x58 serializes a message with owner lock 0x21DD and a tagged transfer byte at 0x21DC. It enters callback 0x21BE in the recipient through service 0x56. The callback uses the recipient's console output policy. The sender's DE advances past NUL or CR; BC, HL, IX and IY are preserved in the selected calls.

Message bytes are masked to seven bits. NUL terminates, CR emits CR/LF and terminates, LF emits CR/LF and continues, and DEL is skipped. Control 0x10 consumes two coordinates, masks them to seven bits and changes 0x7F to 0x7E. A self or out-of-range target returns early, but still calls the lock-release routine; a caller-owned 0x21DD can therefore be released without delivery.

The service itself checks neither account nor message-OFF policy. MENSAJE imposes its own addressed-user rules, and the specified privilege-5 wildcard path bypasses OFF. Result 0 can mean no live delivery. Do not treat this interface as console input or a persistent message queue.

Persistent mailbox

MENSAJE deposits and BUZON retrieval use `SISTEMA.BUZON`, a sequential file owned under temporary account 0xFF. Each message starts with a 33-byte header: recipient eight bytes, sender eight, date eight, time eight and CR. Body lines end with CR. Byte 0x7F pads to a 254-byte payload boundary and ends the message; the final file sector uses 0x1A padding.

BUZON displays and removes messages for the exact account name, compacts surviving messages and updates blocks/end state through raw-sector operations. Its directory count can remain stale. A recipient prefix accepted during deposit need not match the exact name used on retrieval. The mailbox is service state and need not exist before stored messaging is used. Appendix E records the header separately from the sequential sector wrapper.

Chapter 30 — Command processing, loading and termination

CSI resolves a command into an executable or interpreter invocation. The loader then interprets the file's format and relocations, while the executable supplies its own policy and runtime behavior. A command result belongs to the final control path; it is not a substitute for inspecting file-close and device results.

Parsing and command selection

CSI uses terminal geometry to set the line-input budget. Service 0x01 can submit automatically when that budget is exhausted, so an accidentally small L setting can split a typed command before CR. Prepared command text supplied through service 0x36 bypasses this interactive budget. Service 0x36 accepts printable bytes, uppercases lowercase, terminates at CR and returns through cleanup into CSI; no length guard is apparent in the copy loop.

The original abbreviation table has 45 entries. It defines command spellings and minimum prefixes, not an inventory of installed programs. CLASIF and CARGAR have no matching executable or EXEC file. DESCARGAR is stored as nine letters, but lookup uses the eight-character field DESCARGA, for which neither file is installed. None of these entries supplies a built-in implementation. EXEC is a special case: EX and EXE select its path but transfer the wrong original token position to EXECLANG, permitting a different script to run. Spell EXEC in full.

CSI expands table abbreviations at file offsets 0x053A–0x0589. Its final file-not-found path at 0x06E1–0x06F2 restores the entered eight-byte name before displaying the error. Thus an error naming CAR.COMANDO after entering CAR does not mean expansion to CARGAR failed. The spelling shown in that error is not a complete record of the names searched.

Custom synonyms are considered before the standard abbreviation table. Within each searched disk association, CSI checks the original token's COMANDO file, then its EXEC file when IMPEXEC is enabled, before checking the expanded names. It then proceeds to the next association. This ordering means a real file can take precedence over the expected standard abbreviation. Chapter 35 gives the operator-visible settings and exact accepted prefixes.

Implicit EXEC execution depends on the relevant setting and file search. BASIC and EXEC language keywords are interpreted by their own components, not by CSI's command table. This separation explains why an editor command or BASIC statement cannot be pasted at the OS prompt and expected to have its language meaning.

R loading

An R file has no mandatory fixed header. Its load image begins at file offset zero and can contain both instructions and initialized data. The command-execution interface enters a command at its loaded base; a separately loaded overlay or driver follows its own entry convention. BASIC overlays begin with a component byte before their JP slot, while SORT overlays begin with a control/data area. Appendix C gives these layouts. Loading such a component is distinct from transferring control to its entry. The directory supplies image byte length, stored-sector count and start LBA. When present, relocation records begin after the image rounded up to a 256-byte sector boundary. Some small R modules have no relocation sector. Each relocation sector begins with a count from 1 to 127 followed by little-endian offsets. At each offset, the loader adds the load base to a little-endian word modulo 65,536.

CSI's load image, including code and initialized data, is 3,104 bytes (0x0C20), with relocation storage beginning at file offset 0x0D00. Its fifteen stored sectors include two relocation sectors with 250 fixups. At a load base of 0x5157, those fixups adjust the embedded addresses accordingly. A parser must distinguish image byte length, image-sector count and total stored sectors.

Service 0x1E loads and enters a command at context +0x4CC for R format; A format uses its absolute load address from directory +0x1E. Service 0x16 loads a named module without entering it. Service 0x62 constructs temporary metadata from a compact seven-byte descriptor for R submodules. These load interfaces share machinery but are not interchangeable calling conventions.

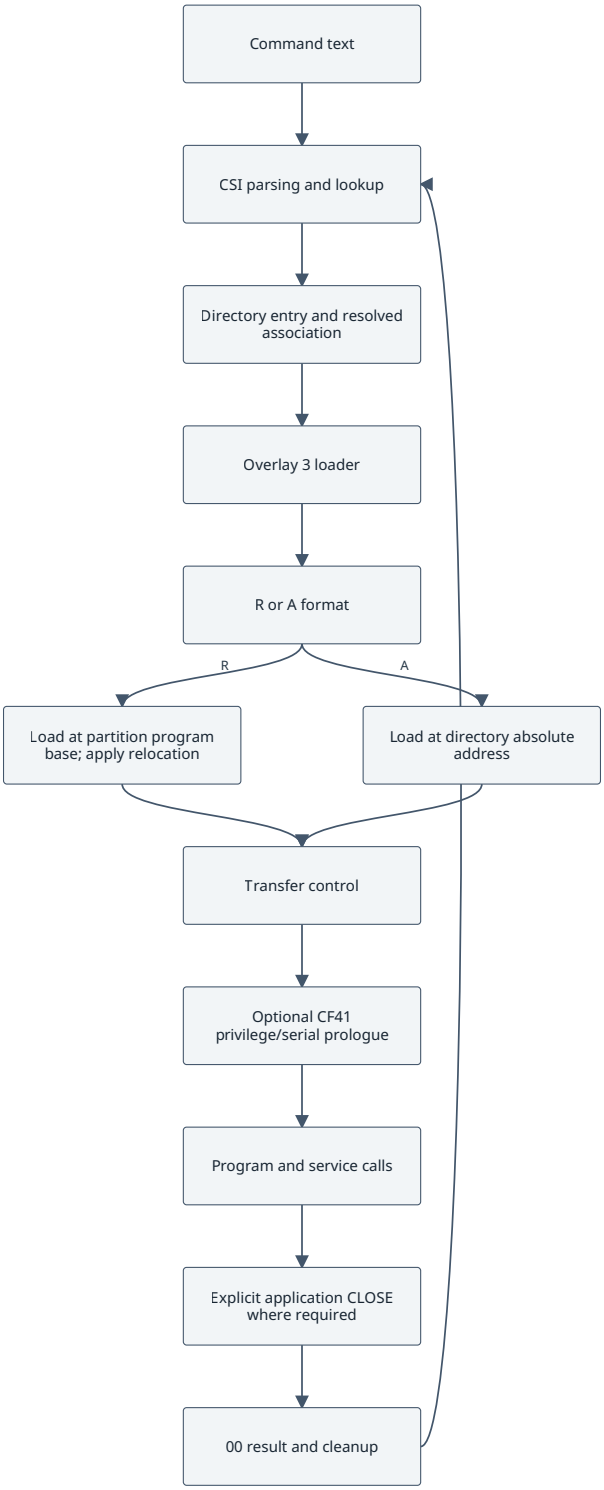


Figure 30.1. Command execution. The CLOSE step is an application responsibility, not an automatic loader or termination operation.

Command policy and runtime return

The optional standard prologue is `CF 41`. Four preceding bytes contain minimum privilege and a three-byte installation serial binding. A zero first serial byte bypasses the comparison. Failure displays message 19 or 20 and ends with result 252. The surveyed prologues use serial bytes `16 00 00` and privilege values 0–5.BASIC's OPTION PRIV/SERIAL guards use its own compiled representation and cleanup; their established failures return 253 instead.

The loader's full initial register and return-trampoline contract remains incomplete. The documented small Z80 example in Chapter 18 uses only established services and ends through 0x00; it does not assume a C runtime, argument vector, automatic heap or a RET-to-CSI convention. Small BASICUSR modules are different: their defined wrapper calls them and restores IX/IY around a normal RET.

Cleanup and continuation

Service 0x00 stores A at context +0x68, removes current-owner lock/open state, clears execution state and reloads CSI. It does not implicitly flush a partial sequential output buffer. An explicit CLOSE before termination is required for the established sequential durability path. A BASIC error handler may itself call CLOSE; this application cleanup must not be generalized to the executive.

Service 0x46 adds a continuation ceiling to command replacement. It sets the memory ceiling and SP from HL, marks context +0x38 bit 5 and uses the service-0x36 path. CSI later clears that bit and jumps to the ceiling. The code establishes the mechanism, but a complete general-purpose caller example and register contract remain unspecified.

FIN ends the account session and can unload/reload the console for login. PARAR stops a partition while retaining its allocated RAM. Accepted IPL restarts the machine. These operations have distinct scopes and none supplies evidence for a universal whole-system shutdown-and-flush command.

Part VII — Reference

Chapter 31 — Command reference

Look up a full command name here after learning the basic file operations. Each entry explains the purpose, syntax, options and one elementary example. Examples are individual operations, not a sequence to run from top to bottom. Practice files such as NOTE.TEXT come from Chapter 6; other filenames are explicit illustrations requiring compatible files to exist first.

A device letter in a maintenance example must be checked with ASOCIAR before use. Destructive examples explain syntax and are not instructions to alter your system disk. Some optional facilities have only a documented interface; their entries say where a complete operating procedure is unavailable. Names with no specified callable interface have an inspection or help example instead of an invented invocation.

Full command names are used throughout the examples. Chapter 35 gives their permitted abbreviations, and its option tables distinguish full keywords from their shorter accepted forms.

Privilege is the standard minimum check where known. A command can make additional checks. See Chapter 33 for exact messages; a completion code is not the same as a system message number.

Command	Purpose
ACTIVAR	Activate a partition or background command
ARCHDIR	List an archive floppy
ARCHIVAR	Archive files, accounts or volumes
ASIGNAR	Assign channels
ASOCIAR	Associate logical and physical devices
ATRIB	Rename or protect files
AYUDA	Display command help
BACKUP	Copy a complete disk or tape medium
BASIC	Edit, interpret or compile BASIC
BORRAR	Delete files and release storage
BSC	Configure 2780/3780 communications
BUSCAR	Test file existence
BUZON	Read and remove stored messages
CAMBIAR	Change a command's minimum privilege
CAMCU	Change an existing account number
CAMPRO	Transfer private-file ownership

Command	Purpose
CARGAR	Executable absent; retained CSI table entry
CEDIT	Edit direct, indexed or keyed records
CIFRA	Calculate a file check value
CLASIF	Executable absent; not a SORT alias
CLAVIN	Installed CLAVE.CUENTA helper; startup returns CR 254
COMPART	Grant, revoke or list shared access
CONECTAR	Connect to another account
CONSULTA	Enter the Spanish query language
COPFICH	Copy keyed records between files
COPIFI	Copy files
CORLUZ	Recount indexed or keyed records
CREAR	Create structured files
CUENTA	Maintain user accounts
DESCARGAR	Executable absent; lookup name DESCARGA
DIRIBM	List an IBM-3741 floppy
DUPLICAR	Duplicate floppies through disk storage
EDITAR	Edit sequential text
ELIMINAR	Remove a damaged catalogue entry
ENVIAR	Send a file synchronously
ENVIARA	Send a file asynchronously
ESPIAR	Display another user's console activity
ESTADO	Inspect system settings and state
EXEC	Execute a command file
FIN	Disconnect the account
FISDEF	Display physical defective tracks
FORZAR	Inject a command into another partition
GENSIST	Save current system configuration
HASH	Inspect key distribution
IBMSEC	Import IBM-3741 records

Command	Purpose
INICDISC	Initialize or relabel a disk
INICIAR	Initialize a floppy
INICIBM	Write IBM-compatible floppy headers
INITTAPE	Initialize or erase a cartridge
LCOMPART	Unavailable: help topic without an executable
LISDIR	List catalogue entries
LISTAR	Display or print a file
LOGDEF	Display logical defective tracks
MENSAJE	Send live or stored messages
MODIMPR	Change printer dimensions
MONTAR	Notify the system of changed media
O8PRECOM	Disk-precompensation and migration utility
ORGDIR	Reorganize the catalogue
PANTALLA	Design data-entry screens
PARAR	Stop a partition
PONER	Change system or session settings
RECARGAR	Restore archived files/accounts/volumes
RECIBIR	Receive synchronous files
RECIBIRA	Receive asynchronous files
REPARAR	Check and reconcile filesystem metadata
REVIVIR	Recover a deleted ASCII catalogue entry
RUN	Run compiled BASIC
SECIBM	Export sequential records to IBM-3741
SECTOR	List a file's occupied sectors
SERIAL	Edit an executable serial binding
SORT	Sort records by keys
SPOOLER	Manage deferred printing
TECLAS	Emit terminal programming sequences
TEDITAR	Edit small files

Command	Purpose
TERMINAL	Use a console as a remote terminal
TITULO	Print large lettering
UNOSEC	Reconstruct logical sector 1
VERFORM	Check media formatting
VOLCAR	Dump file or logical-sector bytes

ACTIVAR — activate a partition or background command

Activate a partition or background command.

Syntax

```
ACTIVAR partition (PANn)
```

Options. Input/output consoles and ASOCIAR device options; command help also describes a command in the first free partition.

Minimum privilege. 2.

Example

```
ACTIVAR 2 (PAN2)
```

After allocating partition 2 as described in Chapter 12, this activates it on terminal PAN2. Allocation must precede activation.

Use parentheses around PANn. Displayed partition numbers and memory-module numbers are different.

ARCHDIR — list an archive floppy

List an archive floppy.

Syntax

```
ARCHDIR drive [(options)]
```

Options. IMP prints; VOL n starts at the selected archive volume (default 1).

Minimum privilege. 1.

Example

```
ARCHDIR A
```

With archive media in the device associated with A, this lists the archive contents. An archive catalogue is different from an ordinary LISDIR listing.

ARCHIVAR — archive files, accounts or volumes

Archive files, accounts or volumes.

Syntax

```
ARCHIVAR [name.type:]source,destination [(options)]
```

Options. Two dates bound selection; CUENTA selects current account, VOLUM all accounts; PANT/NOPANT, VERIFI/NOVERIFI and CONSU/NOCONSU control listing, verification and prompting.

Minimum privilege. 2.

> **Caution — Writes archive destination media.**

Example

```
ARCHIVAR NOTE.TEXT:S,A (VERIFI CONSU)
```

This syntax example selects NOTE.TEXT on S for an archive on A, requests verification and retains consultation. It requires a compatible archive device and expendable destination media; a complete physical-device procedure is unspecified.

ASIGNAR — assign channels

Assign channels.

Syntax

```
ASIGNAR [channel device-or-file] [DESASI]
```

Options. Channels 1–16; CONSOLA or name.type:drive; DESASI removes assignments.

Minimum privilege. 0.

Example

```
ASIGNAR 1 CONSOLA
```

This assigns application channel 1 to CONSOLA. It does not change which physical terminal CONSOLA denotes; ASOCIAR handles that relationship.

ASOCIAR — associate logical and physical devices

Associate logical and physical devices.

Syntax

```
ASOCIAR [logical [physical [(options)]]]
```

Options. L line length; P page length; B baud; C class; LF/FF delays; PE/PO parity; ENAB protocol; PP page parity; O blank lines; ALF automatic LF; H head load; R retries; STP seek; STS latency; PUBLIC public device.

Minimum privilege. 0.

Example

```
ASOCIAR
```

With no arguments the command lists current logical-to-physical associations. Inspect this list before changing a disk or printer destination.

ATRIB — rename or protect files

Rename or protect files.

Syntax

```
ATRIB old [new] [(options)]
```

Options. PANT/NOPANT; CONSUL/NOCONSUL; E/NOE write protection; B/NOB deletion protection; LE read protection (L is also accepted; LEER is not).

Minimum privilege. 1.

Example

```
ATRIB COPY.TEXT:S OTHER.TEXT:S (NOCONSUL)
```

This renames the practice copy to OTHER.TEXT and suppresses consultation. Check that the destination is unused. Allocation is retained; no second data copy is created.

E, B and read protection apply to different access paths. Sequential replacement can bypass E by deleting and recreating the entry; see Chapter 11.

To change attributes without renaming, omit the second filename. For a practice file owned by the current account:

```
ATRIB NOTE.TEXT:S (E NOCONSUL)
```

E enables write protection; the sequential practice file's format indication changes from S to SW. To clear that protection:

```
ATRIB NOTE.TEXT:S (NOE NOCONSUL)
```

NOE returns the indication to S. Inspect each result with LISDIR. Repeating the same filename as both arguments is not the attribute-only form and produces Error sintactico, CR = 8.

AYUDA — display command help

Display command help.

Syntax

```
AYUDA [command]
```

Options. No topic gives the summary; a matching topic normally gives its command help. Failed lookup can invoke the named utility with AYUD.

Minimum privilege. 0.

Example

```
AYUDA LISDIR
```

This displays the help topic for LISDIR. It describes the command rather than performing the directory listing.

If lookup fails, AYUDA can invoke the named utility with AYUD. AYUDA BORRAR takes this path and reports Falta tipo del fichero, CR = 8; it does not display the intended topic. Use the BORRAR entry for deletion instructions. The working LISDIR example does not establish that every topic request is passive.

BACKUP — copy a complete disk or tape medium

Copy a complete disk or tape medium.

Syntax

```
BACKUP [source destination]
```

Options. Original defaults are S and A.

Minimum privilege. 3.

> **Caution — Replaces destination contents.**

Example

```
BACKUP S A
```

This syntax example copies the complete source medium S to destination A. It is appropriate only when A is a correctly configured compatible backup destination whose contents may be replaced. Do not rely on omitted-device defaults.

BASIC — edit, interpret or compile basic

Edit, interpret or compile BASIC.

Syntax

```
BASIC [name.type:drive] [(options)]
```

Options. LIST/NOLIST listing; IMPRn printer; XREF references; NOOBJ suppress BASICCOM generation; NOPANT suppress console; OBJ=drive output location; TRACE compilation passes and line numbers. COMMA is advertised in help but rejected by this version.

Minimum privilege. 2.

Example

```
BASIC HELLO.BASICOBJ:S
```

This compiles saved BASIC source to HELLO.BASICCOM:S. First create and save the source as taught in Chapter 15. BASIC without a filename enters the editor/interpreter.

NOOBJ controls compiled output independently of LIST. To display source without generating BASICCOM:

```
BASIC HELLO.BASICOBJ:S (LIST NOOBJ)
```

NOOBJ alone can display compilation statistics without listing source. An older BASICCOM may remain; it must not be mistaken for output from this invocation.

COMMA and the alternative spelling COMA both produce Opcion no valida, CR = 8, as compiler options in this version. Use dot-decimal source; a working compiler decimal-comma option has not been established. Appendix D retains the advertised form with this restriction.

BORRAR — delete files and release storage

Delete files and release storage.

Syntax

```
BORRAR name.type[:drive] [(options)]
```

Options. PANT/NOPANT names; CONSUL/NOCONSUL consultation for wildcard selection. Exact-name deletion is immediate even with CONSUL.

Minimum privilege. 1.

> Caution — immediate deletion. Exact-name selection has no confirmation stop.

Example

Inspect the exact filename with LISDIR before proceeding. This example deletes immediately, even if CONSUL is supplied.

```
BORRAR OTHER.TEXT:S
```

This deletes the named practice file immediately and releases its storage. There is no confirmation stop for this exact-name form, even with CONSUL. Inspect the exact filename with LISDIR before entering the deletion command.

CONSUL can request confirmation on the wildcard-selection path: N declines a selected file and S permits deletion. That behavior does not apply to the exact-name example above. Do not introduce a wildcard merely to obtain a confirmation prompt; it broadens the selection.

BSC — configure 2780/3780 communications

Configure 2780/3780 communications.

Syntax

```
BSC [parameter-file] [AYUDA] [(options)]
```

Options. COMU1 selects the logical channel; MODIF edits or creates saved protocol parameters.

Minimum privilege. 1.

Example

```
AYUDA BSC
```

This displays the central BSC help topic without establishing a communications connection. BSC AYUDA requests the utility's own options display, but requires COMU1 association even for that request. Without it, the command reports COMU1 no esta asociado, CR = 16. Protocol operation also requires a compatible driver and peer; the DIS27 dependency is not a guaranteed installed component.

BUSCAR — test file existence

Test file existence.

Syntax

```
BUSCAR [name.type[:drive]] [(option)]
```

Options. PANT displays (default); NOPANT suppresses; PUBLIC includes public files.

Minimum privilege. 0.

Example

```
BUSCAR NOTE.TEXT:S (PANT)
```

This searches for the practice file and displays the result. A negative result can reflect the current account or association, not global absence from all storage.

BUZON — read and remove stored messages

Read and remove stored messages.

Syntax

```
BUZON
```

Options. No established options.

Minimum privilege. 0.

Example

```
BUZON
```

This retrieves messages for the connected account. Retrieval removes the selected stored messages, so record anything needed before finishing.

CAMBIAR — change a command's minimum privilege

Change a command's minimum privilege.

Syntax

```
CAMBIAR name.type:drive (PRIV n)
```

Options. PRIV gives the new threshold.

Minimum privilege. 3.

> **Caution — Changes executable policy.**

Example

```
CAMBIAR DEMO.COMANDO:S (PRIV 4)
```

This maintenance illustration raises the selected practice executable's standard minimum privilege to 4. Use only a disposable copy of a command, not an essential system utility. It does not raise your own privilege.

CAMCU — change an existing account number

Change an existing account number.

Syntax

```
CAMCU
```

Options. Interactive interface.

Minimum privilege. 5.

Example

```
CAMCU
```

This enters interactive account-number maintenance. Account 0/SISTEMA and its synonyms are excluded; numbers above 254 are excluded. The complete dialogue and effects on every owned file are unspecified, so no renumbering sequence is prescribed.

CAMPRO — transfer private-file ownership

Transfer private-file ownership.

Syntax

```
CAMPRO name.type[:drive] (account [options])
```

Options. PANT/NOPANT and CONSUL/NOCONSUL.

Minimum privilege. 3.

> **Caution — Transfers file ownership.**

Example

Before this example, connect as the non-SISTEMA owner of NOTE.TEXT and check that its privilege is at least 3. The destination SISTEMA account is not the account from which to select the private source file.

```
CAMPRO NOTE.TEXT:S (SISTEMA NOCONSUL)
```

Run this while connected as the current non-SISTEMA owner of NOTE.TEXT, with privilege 3 or higher. It transfers the file to SISTEMA. Administrative identity alone does not select another owner's private files: from the wrong account the command can report No se ha cambiado ningun fichero, CR = 12. Inspect the recipient's entry and contents after transfer. Account 0 participates in public fallback, so transfer to SISTEMA does not guarantee that the former owner loses lookup access.

CARGAR — command-table entry; executable absent

CSI recognizes CARGAR and expands its minimum abbreviation CAR, but the required executable is not installed. Both CARGAR alone and CARGAR SPOOLER fail with a file-not-found error. The table entry supplies a spelling, not an implementation.

SPOOLER.COMANDO contains `CARGAR SPOOLER` in the path used when the resident spooler cannot be found. This indicates an intended resident-component loading role; it does not establish the input file type, allocation rules or a general loading interface. CARGAR must not be treated as a documented linker.

Syntax and options. No usable interface has been established.

Minimum privilege. Unknown.

Example: consult available help

```
AYUDA
```

AYUDA displays available help topics. This is a help example, not an invocation of the missing CARGAR utility. For the installed printing facility, see SPOOLER. Its resident component can already be present even though CARGAR is absent.

CEDIT — edit direct, indexed or keyed records

Edit direct, indexed or keyed records.

Syntax

```
CEDIT
```

Options. Interactive record editor described by command help.

Minimum privilege. 2.

Example

```
CEDIT
```

This enters the structured-record editor. It can modify records in direct, indexed or keyed files. Its complete selection/save/exit dialogue is unspecified; do not apply EDITAR commands to it.

CIFRA — calculate a file check value

Calculate a file check value.

Syntax

```
CIFRA name.type[:drive] [(option)]
```

Options. P console (default); I printer.

Minimum privilege. 4.

Example

```
CIFRA NOTE.TEXT:S (P)
```

This asks for the check value of the sequential practice file on the console. Record the result if comparing repeated copies. The algorithm is not specified as a modern hash or CRC.

CLASIF — command-table entry; executable absent

CSI recognizes CLASIF and expands its minimum abbreviation CL, but no corresponding executable or EXEC file is installed. Invoking either spelling fails; CSI does not redirect it to SORT. Whether CLASIF was an earlier name, an optional utility or an obsolete table entry is unknown.

Syntax and options. No usable interface has been established. SORT's options cannot be assigned to CLASIF.

Minimum privilege. Unknown.

Example: consult the installed sorting utility

```
AYUDA SORT
```

This displays help for SORT, whose executable and two overlays are installed. Continue with the SORT reference for its own syntax and examples. CLASIF is not an alias for SORT.

The CLASIFICAR and ORDENAR words, and references to a CLASIF file, belong to a separate context within CONSULTA. They do not make CLASIF an available CSI command.

CLAVIN — installed CLAVE.CUENTA helper; startup failure

CLAVIN.COMANDO is installed and loads, but invocations both without arguments and with a test argument return `CR = 254` during startup, including under SISTEMA at privilege 5. This failure occurs before the apparent main operation; neither form establishes a usable interface.

The executable identifies itself as `CLAVIN ver 1.0 01JUN83`. Its code contains separate paths apparently intended to write an argument to CLAVE.CUENTA and to read and display a value when no argument is supplied. Together with CLAVOUT.BASICUSR, it appears to exchange a counted string or value through that file. The value's application meaning and the intended argument format remain unknown.

Syntax and options. No working operational syntax is established; no central help topic is available.

Minimum privilege. 5, as specified by the executable's prologue. Meeting this requirement does not prevent the startup failure.

Example: inspect the installed executable

```
LISDIR CLAVIN.COMANDO:S
```

This lists the utility's catalogue entry on the system disk. It establishes that the file is present; it does not establish that the utility works.

Invocation and failure

```
CLAVIN
```

The executable displays its version heading and returns `CR = 254`. This documents the failing invocation, not a successful value-reading operation. The same result with an argument does not demonstrate a working write operation.

CLAVE.CUENTA is not present in the documented installation. Creating an empty file does not correct the startup code and relocation inconsistencies. No reliable repair is established; Appendix C describes the executable and its BASIC companion.

CLAVE.CUENTA is not the login-password database. Account credentials belong to SISTEMA.CUENTA; use CUENTA for the documented account and password operations.

COMPART — grant, revoke or list shared access

Grant, revoke or list shared access.

Syntax

```
COMPART name.type[:drive] [(PANT)]
```

Options. PANT displays sharing accounts; the ordinary dialogue grants or revokes.

Minimum privilege. 3.

Example

Before this example, connect as a non-SISTEMA account with privilege 3 or higher and create the practice file under that identity.

```
COMPART NOTE.TEXT:S (PANT)
```

Connect as the file's non-SISTEMA owner with privilege 3 or higher before using this example. It lists accounts granted access; with no grants, an empty result and CR = 0 are expected. SISTEMA is excluded and receives La cuenta sistema no puede compartir ficheros, CR = 12. Omit PANT only when intending the separate grant/revoke dialogue. Grants retain the original owner.

CONECTAR — connect to another account

Connect to another account.

Syntax

```
CONECTAR account
```

Options. Password prompting and account profile execution depend on the account.

Minimum privilege. 0.

Example

```
CONECTAR SISTEMA
```

This requests connection to SISTEMA when its account number is not in use by another partition. Enter its password when requested. A duplicate connection is refused before the password check, with Esta cuenta esta siendo utilizada por la particion followed by the partition number and CR = 10. End the existing connection before retrying; an alias with the same account number does not avoid this restriction. A successful connection can run that account's EXEC profile.

CONSULTA — enter the spanish query language

Enter the Spanish query language.

Syntax

```
CONSULTA [statement-file]
```

Options. The optional sequential file supplies a query; otherwise input is interactive.

Minimum privilege. 1.

Example

```
CONSULTA
```

This starts the interactive Spanish query facility. Its full query grammar is unspecified; entering SQL statements is not a supported substitute.

Language reference. See Chapter 35 for the vocabulary of this component.

COPFICH — copy keyed records between files

Copy keyed records between files.

Syntax

```
COPFICH source.name:drive destination.name:drive
```

Options. No wildcard specifications in the command help.

Minimum privilege. 1.

Example

```
COPFICH INDEX.DATOS:S TARGET.DATOS:S
```

This syntax example copies records between compatible indexed or keyed files. Prepare the destination with the required organization first. Wildcards are not accepted by the documented interface; collision behavior is incompletely specified.

COPIFI — copy files

Copy files.

Syntax

```
COPIFI source destination [(options)]
```

Options. PANT/NOPANT; SISFECH/ANTFECH timestamps; DESD n, DSTRING text, NUM n, HSTRING text selection; TRUNC/NOTRUNC; PACK/UNPACK; LMINUSC/MAYUSC; NOREEMP/REEMPLA; A@ADIR append; MONTA; VERIFI/NOVERIFI; CONSU/NOCONSU; PUBLIC; TABLA conversion.

Minimum privilege. 1.

Example

```
COPIFI NOTE.TEXT:S COPY.TEXT:S (VERIFI)
```

This copies from NOTE.TEXT to a new COPY.TEXT and requests verification. The source specification comes first. Inspect the destination afterward.

CORLUZ — recount indexed or keyed records

Recount indexed or keyed records.

Syntax

```
CORLUZ name.type[:drive]
```

Options. No established options.

Minimum privilege. 0.

> **Caution — Updates directory metadata.**

Example

```
CORLUZ INDEX.DATOS:S
```

This recounts a keyed or indexed file and updates its directory count. Make a backup first: it is a metadata correction, not just a listing.

CREAR — create structured files

Create structured files.

Syntax

```
CREAR name.type:drive (options)
```

Options. DIRECTO, INDEXADO or CLAVE; NUMREG count; REGISTRO length; LONCLA key length; VACIAR empties an existing file.

Minimum privilege. 1.

Example

```
CREAR DIRECT.DATOS:S (DIRECTO REGISTRO 32 NUMREG 10)
```

This creates a new direct file with ten records of 32 bytes. DIRECTO selects organization, REGISTRO supplies record length and NUMREG supplies the requested count.

VACIAR empties an existing file and destroys its records. I and K capacities and allocation formulas differ.

CUENTA — maintain user accounts

Maintain user accounts.

Syntax

```
CUENTA [(option)]
```

Options. PANT; IMPRIMIRn; VACIAR history; ELIMINAR account system; COPIAR=drive; HISTORI.

Minimum privilege. 5.

Example

```
CUENTA (PANT)
```

This displays registered accounts, including the password column. Keep the listing private. CUENTA without options enters maintenance: N adds an account, C changes its password and privilege, B removes one named account, and F writes the revised list and finishes. At privilege 5, use the SISTEMA account. Chapter 10 gives separate step-by-step procedures for creation, password changes and removal.

ELIMINAR removes the account system; VACIAR clears account history. Neither is an ordinary single-account deletion command.

DESCARGAR — command-table entry; executable absent

DESCARGAR is the original nine-letter name in CSI's abbreviation table. The command lookup field holds only eight characters, so the expanded lookup name is DESCARGA. Neither a matching COMANDO executable nor an EXEC file is installed, and invocation fails.

The minimum abbreviation is DESCAR; DESCARG and DESCARGA are also accepted prefixes. Recognition of these spellings does not supply the missing program. An error naming DESCARGA.COMANDO after entering DESCARGAR reflects the lookup field's length.

Syntax, purpose and options. Unknown. In particular, the name does not establish an unloading operation or make DESCARGAR equivalent to VACIAR.

Minimum privilege. Unknown.

Example: consult available help

```
AYUDA
```

This displays the available help summary. It is not an example of unloading a component. SPOOLER.COMANDO separately contains `VACIAR SPOOLER`, but the standalone VACIAR executable is also absent. The VACIAR options of CUENTA, CREAR and INICIAR belong to those individual commands and remain separate from these unavailable programs.

DIRIBM — list an ibm-3741 floppy

List an IBM-3741 floppy.

Syntax

```
DIRIBM drive
```

Options. No established options.

Minimum privilege. 0.

Example

```
DIRIBM A
```

With a suitable IBM-3741 floppy in A, this lists its volume and file information. Track/sector fields use PPSSS notation; this is not a listing of the native filesystem.

DUPLICAR — duplicate floppies through disk storage

Duplicate floppies through disk storage.

Syntax

```
DUPLICAR
```

Options. Requires a prepared SISTEMA.DUPLICAR direct file.

Minimum privilege. Unknown standard minimum.

> **Caution — Writes destination floppy media.**

Example

```
DUPLICAR
```

This enters floppy duplication using disk workspace. It requires a prepared SISTEMA.DUPLICAR direct file; the documented workspace example is 4,000 records of 256 bytes. The full media-exchange dialogue is unspecified.

EDITAR — edit sequential text

Edit sequential text.

Syntax

```
EDITAR name.type[:drive] [(COPIA or NOCOPIA)]
```

Options. COPIA creates a backup by default; NOCOPIA suppresses it.

Minimum privilege. 2.

Example

```
EDITAR NOTE.TEXT:S
```

This opens the sequential-text editor for the named file. At its star prompt use the file-creation steps in Chapter 6; FILE saves and exits.

Language reference. See Chapter 35 for the vocabulary of this component.

ELIMINAR — remove a damaged catalogue entry

Remove a damaged catalogue entry.

Syntax

```
ELIMINAR name.type:drive
```

Options. No established options.

Minimum privilege. 3.

> **Caution — Removes the entry without freeing storage.**

Example

```
ELIMINAR BROKEN.TEXT:S
```

This maintenance illustration removes a damaged entry named BROKEN.TEXT without freeing its storage. It is not ordinary deletion. Use only when the entry and its allocated blocks have been examined and a backup exists.

ENVIAR — send a file synchronously

Send a file synchronously.

Syntax

```
ENVIAR file [(options)]
```

Options. NOEOT suppresses final EOT; COMUn selects channel, default COMU1.

Minimum privilege. 1.

Example

```
ENVIAR NOTE.TEXT:S (COMU1)
```

This selects the file for synchronous transmission through COMU1. It requires a configured communications link and compatible receiving endpoint; it is not a local copy.

ENVIARA — send a file asynchronously

Send a file asynchronously.

Syntax

```
ENVIARA file [(options)]
```

Options. Command help identifies an asynchronous ENVIAR counterpart; exact option equivalence is not independently documented.

Minimum privilege. 1.

Example

```
ENVIARA NOTE.TEXT:S
```

This selects the file for asynchronous transmission. Exact equivalence to ENVIAR's option set is unspecified, so this example uses no additional options.

ESPIAR — display another user's console activity

Display another user's console activity.

Syntax

```
ESPIAR account-or-partition
```

Options. No established options.

Minimum privilege. 5.

Example

```
ESPIAR 2
```

This requests observation of partition 2's console activity. Use only for authorized administration. The full termination and policy behavior is unspecified.

ESTADO — inspect system settings and state

Inspect system settings and state.

Syntax

```
ESTADO option
```

Options. ALL, QUIEN, HORA, FECHA, SISTEMA, FORFECHA, PRIVILEG, RELOJ, DIA, MENSAJE, TIEMPO, USUARIOS, MEMORIA, MODULO, DISCO, ERRNUM, ERRTEXT, INCOM, ABREV, IMPEXEC, HISTOR, RTCAVAIL, SISFECHA, INIREL, QET, ALEMAN, ESCAPE, SUBIR, BAJAR, IZQU, DERECHA, RUBOUT, CANCEL, SINONIMO, VERSION. Each is an information query; the accepted shorter forms are listed in Chapter 35.

Minimum privilege. 0.

Example

```
ESTADO DISCO
```

This reports disk state, including total free space and largest contiguous free region. It may update a cached free count. ESTADO requires a subject; the bare command is a syntax error.

List partitions

```
ESTADO MEMORIA
```

This reports allocated memory partitions, including stopped allocations. See the partition-listing procedure.

List memory modules

```
ESTADO MODULO
```

This shows usable memory modules, which supply partition storage.

List connected users

```
ESTADO USUARIOS
```

This reports current user connections. CUENTA (PANT) instead lists registered accounts.

EXEC — execute a command file

Execute a command file.

Syntax

```
EXEC name [arguments]
```

Options. Language directives and substitution are documented in Chapter 8.

Minimum privilege. Unknown standard minimum.

Example

```
EXEC HELLO
```

This runs the HELLO.EXEC greeting command file created in Chapter 8. Use the complete EXEC name to avoid the abbreviation argument-transfer defect.

Ordinary fields are eight characters; arguments are limited to sixteen fields. Use BEGTYPE/END for literal long text.

Language reference. See Chapter 35 for the vocabulary of this component.

FIN — disconnect the account

Disconnect the account.

Syntax

```
FIN
```

Options. No established options.

Minimum privilege. 0.

Example

```
FIN
```

After saving and closing application files, enter this at CSI to disconnect the account. It does not power off the machine or promise a global flush.

FISDEF — display physical defective tracks

Display physical defective tracks.

Syntax

```
FISDEF:unit-number
```

Options. Help's colon form is retained pending native verification.

Minimum privilege. Unknown standard minimum.

Example

```
FISDEF:3
```

This is the documented colon-form illustration for physical unit number 3. Confirm the unit numbering before use. It requests physical defective-track information; the colon form is not a general device syntax rule.

FORZAR — inject a command into another partition

Inject a command into another partition.

Syntax

```
FORZAR partition command
```

Options. No established options.

Minimum privilege. 5.

> **Caution — Can interrupt unsaved work.**

Example

```
FORZAR 2 FIN
```

This forces the FIN command into partition 2. It can terminate work before a complete application record is saved. Reserve it for recovery when normal logout is unavailable.

GENSIST — save current system configuration

Save current system configuration.

Syntax

```
GENSIST
```

Options. Help names ASOCIAR and selected PONER settings.

Minimum privilege. 5.

> **Caution — Writes SISTEMA.NUCLEO.**

Example

```
GENSIST
```

This saves selected current association and system settings into SISTEMA.NUCLEO. Back up first and use an intentional configuration; complete persistence rules are unspecified.

HASH — inspect key distribution

Inspect key distribution.

Syntax

```
HASH [I]
```

Options. I prints statistics; dialogue asks capacity, file or manual keys.

Minimum privilege. Unknown standard minimum.

Example

```
HASH
```

This enters the dialogue for examining key distribution and placement. It asks for capacity and file or manual-key input. Its name does not identify a cryptographic file checksum; use CIFRA for the separate file-check-value operation.

IBMSEC — import ibm-3741 records

Import IBM-3741 records.

Syntax

```
IBMSEC destination.name:drive source-drive [(options)]
```

Options. Optional source name; REEM replaces destination.

Minimum privilege. 3.

Example

```
IBMSEC IMPORT.TEXT:S A
```

This syntax example imports records from an IBM-3741 floppy on A into IMPORT.TEXT on S, converting EBCDIC to ASCII. Prepare compatible media and choose an unused destination.

INICDISC — initialize or relabel a disk

Initialize or relabel a disk.

Syntax

```
INICDISC drive (options)
```

Options. ETIQU requests a label change; VACIAR erases files; DIRE requests directory entries; WP/NOWP changes software write protection; FORMAT requests physical formatting; CARA specifies heads; PISTA specifies cylinders/tracks per head and can affect volume placement; SECTOR specifies sectors per track; BOOT=drive supplies a bootstrap source; INCR selects the format increment/interleave parameter; its complete hardware-specific effect is unspecified.

Minimum privilege. 3.

> **Caution — Initialization or formatting can destroy contents.**

Example

```
INICDISC A (FORMAT CARA 5 PISTA 397 SECTOR 32 DIRE 64)
```

This formats an expendable, correctly configured volume on A using five heads, 397 cylinders and 32 sectors per track, and requests directory capacity. It is not a universal geometry. For DISCO2, PISTA also sets the starting cylinder: 397 gives cylinders 397–793. DIRE 64 can be adjusted to 88 entries. The Chapter 11 procedure explains association, geometry checks, each dialogue response, mounting and file verification.

INICIAR — initialize a floppy

Initialize a floppy.

Syntax

```
INICIAR drive (options)
```

Options. ETIQU, VACIA, DIRE, WP/NOPE, FORMAT, CARA, PISTA, SECTOR, BOOT=drive.

Minimum privilege. 3.

> **Caution — Initialization or formatting can destroy contents.**

Example

```
INICIAR A (FORMAT CARA 2)
```

This requests the utility's double-sided, double-density floppy format using its remaining geometry defaults. A must denote the intended expendable floppy, and the drive and medium must support that format. It destroys existing contents. Chapter 11 explains the geometry options and subsequent access checks.

INICIBM — write ibm-compatible floppy headers

Write IBM-compatible floppy headers.

Syntax

```
INICIBM drive
```

Options. No established options.

Minimum privilege. 3.

> **Caution — Changes media headers.**

Example

```
INICIBM A
```

This writes IBM-compatible floppy headers on A for subsequent compatible operations. Existing information can be altered; the example requires expendable media.

INITTAPE — initialize or erase a cartridge

Initialize or erase a cartridge.

Syntax

```
INITTAPE [(TENSION)]
```

Options. TENSION traverses and rewinds the medium.

Minimum privilege. 1.

> **Caution — Destroys existing tape information.**

Example

```
INITTAPE (TENSION)
```

This initializes the selected tape facility and requests tensioning, which traverses and rewinds the cartridge. It destroys existing tape information and requires a compatible controller.

LCOMPART — unavailable sharing/account utility

Availability. There is no matching COMANDO executable or standard abbreviation-table entry. The help topic describes inspecting or reconstructing sharing and accounts; it does not establish an available operating procedure.

Syntax

```
LCOMPART drive [(options)]
```

Options. BORRA removes sharing; IMP prints; CUENTA creates missing accounts at privilege 0; PRIVn selects 1–5.

Minimum privilege. Unknown standard minimum.

Example

```
LCOMPART S (IMP)
```

This historical syntax illustration describes a printed sharing report on S. It cannot be run with the available commands: the matching utility is absent, so it is not a recovery procedure. BORRA/CUENTA forms would additionally modify sharing or account state.

LISDIR — list catalogue entries

List catalogue entries.

Syntax

```
LISDIR [name.type][:drive] [(options)]
```

Options. PRINTn; EXEC/FICH selection files; AP append; FN/FT/FD/DIA/ALOC/FORMAT/LETIQUETA detail; SORT/NOSORT; OCUENTA; HCAB/NOHCAB; PUBLIC; date lower bound; * all detail.

Minimum privilege. 0.

Example

```
LISDIR *.*:S
```

This lists visible catalogue entries on S. Each asterisk selects all values in its field. It does not display file contents or guarantee visibility of other accounts' private files.

OCUENTA is the privileged all-account selection. NOHCAB can clear OCUENTA, so do not combine them when a complete all-account listing is required.

LISTAR — display or print a file

Display or print a file.

Syntax

```
LISTAR name.type[:drive] [(options)]
```

Options. PRINTn; first/last records; HEAD/NOHEAD; FORMAT; INSER margin; TRUNC length; PAG first page; TITULO heading.

Minimum privilege. 0.

Example

```
LISTAR NOTE.TEXT:S
```

This displays the practice text file. Use LISDIR instead when you need its catalogue metadata rather than its contents.

LOGDEF — display logical defective tracks

Display logical defective tracks.

Syntax

```
LOGDEF unit-number
```

Options. No established options.

Minimum privilege. Unknown standard minimum.

Example

```
LOGDEF 3
```

This illustrates the logical defective-track report for unit 3. Confirm the unit identity first. Logical replacement-track information is different from FISDEF's physical report.

MENSAJE — send live or stored messages

Send live or stored messages.

Syntax

```
MENSAJE account-or-partition-or-* [text]
```

Options. Omitted text enters multiline mode.

Minimum privilege. 0.

Example

```
MENSAJE 2 HOLA
```

This sends the short text HOLA to partition 2. Omit the text to enter a multiline dialogue. Read the diagnostic because a zero completion result does not guarantee live delivery.

MODIMPR — change printer dimensions

Change printer dimensions.

Syntax

```
MODIMPR columns lines
```

Options. Help bounds both dimensions strictly between 0 and 255.

Minimum privilege. 1.

Example

```
MODIMPR 80 60
```

This requests printer dimensions of 80 columns and 60 lines. Both must be greater than zero and less than 255. The interface is documented but complete driver-specific effects are unspecified.

MONTAR — notify the system of changed media

Notify the system of changed media.

Syntax

```
MONTAR drive-or-*
```

Options. * requests all units.

Minimum privilege. 0.

Example

```
MONTAR A
```

After media replacement in the associated device, this refreshes the system's view of A. It is not a request to create a directory mount point.

O8PRECOM — disk-precompensation and migration utility

O8PRECOM changes disk-controller precompensation state. CAMBVERS.EXEC invokes it during a 6.1/6.1A-to-6.2 migration. It is not a language precompiler. Its controller path writes disk data; complete parameters and recovery behavior remain unspecified. Inspection example only.

Syntax

```
No documented operational syntax
```

Options. No central help topic.

Minimum privilege. Unknown standard minimum.

Example

```
LISDIR O8PRECOM.COMANDO:S
```

This inspects the utility's catalogue entry without starting its controller-write path. The historical migration role does not supply a safe general maintenance recipe; no execution or compiler workflow is prescribed.

Interface limit. The missing operating contract is not supplied by the filename alone.

ORGDIR — reorganize the catalogue

Reorganize the catalogue.

Syntax

```
ORGDIR drive [(RELAN)]
```

Options. RELAN resumes after an interrupted reorganization according to help.

Minimum privilege. 5.

> **Caution — Rewrites directory entries.**

Example

```
ORGDIR A
```

This maintenance illustration reorganizes the directory on A. It rewrites entries and can discard deleted entries and unfinished sequential entries. Back up and review damaged files before using it.

PANTALLA — design data-entry screens

Design data-entry screens.

Syntax

```
PANTALLA [screen-name]
```

Options. Embedded ;I include, ;M modify, ;B delete field, ;S save, ;F finish, ;G/;R field save/retrieve, ;D/;H move, ;P grid, ;C centre, ;N number, ;L cursor, ;A help.

Minimum privilege. 1.

Example

```
PANTALLA AYUDA
```

This requests the screen designer's help branch. The editor creates .PANT output, but its full save/replace behavior is unspecified. Do not assume files with other screen-related types are interchangeable.

Editor controls include ;I include, ;M modify, ;B delete field, ;S save, ;F finish, ;G/;R save/retrieve field, ;D/;H move, ;P grid, ;C centre, ;N number, ;L cursor and ;A help. Uppercase and lowercase forms differ; these are individual controls within the designer, not CSI commands. Lowercase ;i/;j, ;s/;t and ;p/;q select inverse-video, underline and blink controls. Chapter 35 gives the literal forms, window-movement syntax and remaining limits.

PARAR — stop a partition

Stop a partition.

Syntax

```
PARAR partition
```

Options. No established options.

Minimum privilege. 5.

Example

```
PARAR 2
```

After the user in partition 2 has saved work and disconnected, this stops that partition while retaining its allocated memory. It is not whole-system shutdown.

PONER — change system or session settings

Change system or session settings.

Syntax

```
PONER option value
```

Options. FECHA; ABREV, INCOM, IMPEXEC, MENSAJE, VERSION, ERRNUM, ERRTEXT, HISTORIA, SISFECHA ON/OFF; FORFECHA 1/2/3; ESCAPE, SUBIR, BAJAR, IZQ, DER, CANCEL control definitions.

Minimum privilege. 1.

Example

```
PONER IMPEXEC ON
```

This enables invocation of accessible EXEC files by bare script name. PONER IMPEXEC OFF disables that convenience; full EXEC remains usable.

Partition forms

Memory configuration requires privilege 5. To allocate displayed partition 2 from available module 1, when it is the next unallocated partition, enter:

```
PONER MEMORIA 2 48000 1
```

This requests partition 2 in module 1 using 48000 as the size parameter; it does not guarantee that exact byte count. Inspect ESTADO MODULO before allocation and ESTADO MEMORIA afterward. In the demonstrated configuration, the first allocation occupies 0x4C8B–0xFFFF, or 45,941 bytes. Activation is a separate operation. To remove partition 2 after stopping it, when it is the last allocated partition and is not the caller, use the distinct form:

```
PONER MEMORIA 2
```

Omitting the size selects removal; appending a zero size is not equivalent. Chapter 12 teaches listing, allocation, activation and removal, including the restrictions on shared modules. The general forms are `PONER MEMORIA partition bytes module` for allocation and `PONER MEMORIA partition` for removal.

The timing form requires privilege 5 and is explained in Chapter 25. TIEMPO arithmetic has a discontinuity at 135/136.

RECARGAR — restore archived files/accounts/volumes

Restore archived files/accounts/volumes.

Syntax

```
RECARGAR [name.type:]source,destination [(options)]
```

Options. CUENTA; RESCUE account; REEMPLA/NOREEM/COPREEM; CONSU/NOCONSU; PANT/NOPANT; VERIFI/NOVERIFI; VOLUM.

Minimum privilege. 3.

> **Caution — Restoration can replace files unless restricted.**

Example

```
RECARGAR NOTE.TEXT:A,S (NOREEM CONSU)
```

This syntax illustration selects NOTE.TEXT from archive source A for restoration to S, refusing replacement and retaining consultation. It requires a compatible archive; the complete media procedure is device-dependent.

RECIBIR — receive synchronous files

Receive synchronous files.

Syntax

```
RECIBIR [name[type]]drive [(COMUn)]
```

Options. COMU1 default.

Minimum privilege. 1.

Example

```
RECIBIR S (COMU1)
```

This uses the documented drive-only form to receive through COMU1 onto S. Optional name/type punctuation and replacement behavior are incompletely specified. Use only a prepared receiving environment with an agreed peer protocol.

RECIBIRA — receive asynchronous files

Receive asynchronous files.

Syntax

```
RECIBIRA [name[type]]drive [(COMUn)]
```

Options. Help identifies the asynchronous RECIBIR counterpart; exact option equivalence remains untested.

Minimum privilege. 1.

Example

```
RECIBIRA S (COMU1)
```

This is the analogous documented asynchronous reception form. Configure the link and receiving destination first; complete option equivalence and replacement behavior are unspecified.

REPARAR — check and reconcile filesystem metadata

Check and reconcile filesystem metadata.

Syntax

```
REPARAR drive [(F)]
```

Options. F updates allocation and can remove rejected entries or redirects.

Minimum privilege. 3.

Example

```
REPARAR S
```

This checks filesystem metadata on S. It can report errors while returning command result 0; read the entire report. The F option authorizes additional repairs and possible entry removal.

Even checking can update the free counter. F can remove rejected entries or sharing redirects; it cannot reconstruct arbitrary application data.

REVIVIR — recover a deleted ascii catalogue entry

Recover a deleted ASCII catalogue entry.

Syntax

```
REVIVIR name.type:drive (start-sector)
```

Options. Optional numeric search start described by help.

Minimum privilege. 3.

> **Caution — Writes recovery metadata.**

Example

```
REVIVIR LOST.TEXT:S (160)
```

This syntax illustration starts the recovery search at sector 160 for the named deleted ASCII entry. The number is illustrative, not a prescribed location. Recovery writes metadata; success and collision handling are not guaranteed.

RUN — run compiled basic

Run compiled BASIC.

Syntax

```
RUN name[.type[:drive]] [(option)]
```

Options. Default type BASICCOM. Help advertises TRACE for runtime addresses and COMA for decimal comma; the demonstrated greeting and numeric programs do not show those effects. See the qualifications below.

Minimum privilege. 0.

Example

```
RUN HELLO.BASICCOM:S
```

This executes the compiled BASIC program from Chapter 15. The BASIC runtime is required; the file is not a native Z80 command.

A successful RUN result does not prove that an optional word was recognized. RUN with TRACE produces no visible runtime-address trace for the greeting and numeric examples, and COMA leaves PRINT 1.5 displayed as 1.5 in the numeric example. An arbitrary option also completes with the same numeric output. These observations do not establish input-conversion behavior or exclude other internal effects. See runtime TRACE and runtime COMA.

BASIC with TRACE traces compilation, a separate operation described in Chapter 15. Do not substitute it for a runtime debugger.

SECIBM — export sequential records to ibm-3741

Export sequential records to IBM-3741.

Syntax

```
SECIBM source.name:drive destination-drive [(options)]
```

Options. Optional output name and record length, maximum 128 by help; default longest source record.

Minimum privilege. 3.

> **Caution — Writes destination media.**

Example

```
SECIBM NOTE.TEXT:S A
```

This syntax example exports sequential records from S to an IBM-compatible floppy on A, converting ASCII to EBCDIC. The medium must have suitable headers; overlong records can be truncated.

SECTOR — list a file's occupied sectors

List a file's occupied sectors.

Syntax

```
SECTOR name.type[:drive]
```

Options. No established options.

Minimum privilege. 3.

Example

```
SECTOR NOTE.TEXT:S
```

This lists the file's occupied logical sectors. It describes allocation, not the file's text. Logical sector numbers require device translation before becoming controller addresses.

SERIAL — edit an executable serial binding

Edit an executable serial binding.

Syntax

```
SERIAL name.type:drive (options)
```

Options. SERIAL nnnnn, PANT, NOPANT in embedded help.

Minimum privilege. 5.

> **Caution — Changes executable policy.**

Example

```
SERIAL DEMO.COMANDO:S (SERIAL 22 PANT)
```

This syntax illustration binds a disposable executable copy to serial 22 and requests display. Replace 22 only with an intended binding value. This changes executable policy; it does not configure a serial port.

SORT — sort records by keys

Sort records by keys.

Syntax

```
SORT [input [output [(options)]]]
```

Options. FICH parameter file; CLAVE position length with A/D and N/U/I; REGSAL, CLAVESAL, NUMSAL, DIRSAL; NOCLAVE; NOSORT; MONTAR/MONTARE/MONTARS/MONTARW; WORK drive; PANT; ESTADIS.

Minimum privilege. 1.

Example

```
SORT INPUT.TEXT:S SORTED.TEXT:S (CLAVE 1 4 A U REGSAL)
```

This documented-form illustration sorts by a four-character uppercase key beginning at position 1, ascending, and writes sorted records. Prepare a compatible input and unused output; complete record-format and option interactions are unspecified.

With only one filename, SORT treats it as a parameter file. Two filenames identify input and output.

SPOOLER — manage deferred printing

Manage deferred printing.

Syntax

```
SPOOLER option
```

Options. INIC, SAL, ACTIV, ESTADO, PARA, CAMBIA, ELIM, LIST, FINAL, ALINE, SALVAR, FORM code, IMPR n, RELANZA n page, COPIAS n, RETE/NORETE, COLA A–Z.

Minimum privilege. 0.

Example

```
SPOOLER ESTADO
```

This reports the deferred-printing service state. If it is not initialized, diagnose that state before submitting output; Chapter 12 gives the basic initialization sequence.

PARA pauses ordinary output but explicit print/relaunch requests can take precedence. Physical printer completion must be checked separately.

The command first checks for the resident SPOOLER component. The documented system already provides it, so ordinary initialization does not require CARGAR. If that resident component is absent, the fallback refers to the unavailable `CARGAR SPOOLER` command. A separate `VACIAR SPOOLER` reference also lacks its standalone executable. These references do not provide a usable procedure for loading or unloading the resident component; see Chapter 29.

TECLAS — emit terminal programming sequences

Emit terminal programming sequences.

Syntax

```
TECLAS
```

Options. No argument parser established.

Minimum privilege. Unknown standard minimum.

Example

```
TECLAS
```

This invokes the terminal-programming sequence emitter. The compatible terminal family and exact effects are unknown, so treat this as identification of the invocation, not a recommended setup step.

TEDITAR — edit small files

Edit small files.

Syntax

```
TEDITAR name.type[:drive]
```

Options. No established options.

Minimum privilege. 1.

Example

```
TEDITAR NOTE.TEXT:S
```

This opens the small-file editor for the practice text. Its complete editing and exit dialogue is unspecified; use Chapter 6's EDITAR workflow when you need a dependable creation procedure.

TERMINAL — use a console as a remote terminal

Use a console as a remote terminal.

Syntax

```
TERMINAL [(options)]
```

Options. COMUn (default COMU1); IBER or HAZELTIN remote-system mode.

Minimum privilege. 1.

Example

```
TERMINAL (COMU1)
```

This selects COMU1 for using the console as a remote terminal. Establish the compatible peer and termination method before entering; a complete escape/exit procedure is unspecified.

TITULO — print large lettering

Print large lettering.

Syntax

```
TITULO channel title1 [title2 ... title12]
```

Options. P console; In printer n; up to twelve titles, each up to eight characters by help.

Minimum privilege. Unknown standard minimum.

Example

```
TITULO P HELLO
```

This requests large lettering for HELLO on the console selected by P. Each title is limited to eight characters in the documented interface. The utility references MAESTRO.TITULO, whose probable character-pattern role and storage details are described in Appendix C.

UNOSEC — reconstruct logical sector 1

Reconstruct logical sector 1.

Syntax

```
UNOSEC drive [(options)]
```

Options. C=heads (default 4), P=tracks (243), S=sectors (42), E=directory entries; otherwise infer directory boundary.

Minimum privilege. 5.

> **Caution — Overwrites volume metadata.**

Example

```
UNOSEC A (C=5 P=397 S=32 E=1208)
```

This is a parameter-reading illustration, not a repair recipe: it specifies heads, tracks, sectors and directory entries. Every value must come from the target volume's actual format. The command overwrites volume metadata.

VERFORM — check media formatting

Check media formatting.

Syntax

```
VERFORM drive
```

Options. No established options.

Minimum privilege. 3.

Example

```
VERFORM A
```

This checks the format of the volume associated with A. Use it after initialization and MONTAR, then verify an ordinary file copy with COPIFI (VERIFI), as taught in Chapter 11. Read diagnostics as well as the completion result. Its complete side-effect contract is not specified; do not substitute it for a backup or assume it is an assured read-only diagnostic.

VOLCAR — dump file or logical-sector bytes

Dump file or logical-sector bytes. Interface unspecified; inspection example only.

Syntax

```
Complete syntax not established
```

Options. Code has file and inclusive first/last-sector paths; P-leading console and I-leading printer options.

Minimum privilege. 2.

Example

```
LISDIR VOLCAR.COMANDO:S
```

This inspects the utility entry. VOLCAR displays hexadecimal and character bytes, but its complete invocation grammar is unspecified and some output paths write files or operate tape. No guessed dump command is offered.

Interface limit. The missing operating contract is not supplied by the filename alone.

Chapter 32 — System service reference

All 111 RST 0x08 entries appear below in numeric order. Names describe functions; they are not official symbolic identifiers. Each entry specifies its function, stored and initialized targets, inputs, outputs, known register effects and restrictions. An omitted register guarantee is unknown; it must not be inferred from another service. Error flags are service-specific.

The common entry convention is `RST 08H` followed by `DB number`; the return address skips that byte. The instruction spelling is illustrative Z80 notation, not proof of an installed assembler. For example, service 0x02 is encoded `CF 02`. Chapter 18 provides an executable byte-exact example using 0x02 and 0x00; Chapter 17 supplies documented BASIC callers of file services. Each entry includes a calling-sequence example. Labels such as request, buffer and callback denote caller-prepared storage or code; they are not built-in symbols. Allocate and initialize them according to the input contract and Appendix E before making the call. Fragments demonstrate argument placement and selection, not a standalone assembled program. Maintenance calls retain their full preconditions and risks.

Data structures are defined in Appendix E; executable and runtime formats in Appendix A. Chapter 24 explains dispatch and overlays, Chapter 25 synchronization and Chapter 27 error policy. The table below is the canonical target index.

Number	Descriptive function	Stored target	Live target
0x00	End command / restore CSI	0x075B	0x075B
0x01	Console line input	0x1139	0x1139
0x02	Display string	0x1290	0x1290
0x03	Poll console input	0x134E	0x134E
0x04	Read console character	0x1360	0x1360
0x05	Console character output	0x1461	0x1461
0x06	Read console character with direct echo	0x144E	0x144E
0x07	Terminal-class character output	0x149B	0x149B
0x08	Printer-association character output	0x1698	0x1698
0x09	Invalidate cached disk descriptor fields	0x31E3	0x31E3
0x0A	Read logical sector	0x09BF	0x09BF
0x0B	Write logical sector	0x09C5	0x09C5
0x0C	Return initialized kernel signature	0x5157	0x331B
0x0D	Create/write directory entry	0x27CA	0x27CA
0x0E	Parse unsigned hexadecimal	0x1899	0x1899

Number	Descriptive function	Stored target	Live target
0x0F	Parse unsigned decimal	0x18CC	0x18CC
0x10	Format byte as hexadecimal	0x1916	0x1916
0x11	Format unsigned word as decimal	0x192C	0x192C
0x12	Console newline	0x1317	0x1317
0x13	Calibrated busy delay	0x2C83	0x2C83
0x14	Find directory entry	0x2463	0x2463
0x15	Get associated device descriptor	0x3145	0x3145
0x16	Load named executable without entering it	0x0765	0x0765
0x17	Printer-association string output	0x1673	0x1673
0x18	Initialize channel name/class	0x31CA	0x31CA
0x19	Convert drive character to association index	0x327A	0x327A
0x1A	Convert disk association to character	0x325C	0x325C
0x1B	Allocate exact contiguous block count	0x2847	0x2847
0x1C	Free contiguous block range	0x29BB	0x29BB
0x1D	Delete named file	0x262B	0x262B
0x1E	Load and execute command	0x0760	0x0760
0x1F	Rename named file	0x2741	0x2741
0x20	Open channel	0x0733	0x0733
0x21	Close channel	0x0738	0x0738
0x22	Read record from channel	0x3865	0x3865
0x23	Write record to channel	0x38D6	0x38D6
0x24	Format current date	0x2B34	0x2B34
0x25	Format current time	0x2BC7	0x2BC7
0x26	Unsigned division	0x1973	0x1973
0x27	Unsigned multiplication	0x1998	0x1998
0x28	Read numbered direct record	0x0751	0x0751
0x29	Write numbered direct record	0x0756	0x0756
0x2A	Parse decimal or H-suffixed hexadecimal	0x18FA	0x18FA
0x2B	Read indexed/keyed record	0x071F	0x071F

Number	Descriptive function	Stored target	Live target
0x2C	Read next indexed/keyed record	0x0724	0x0724
0x2D	Write indexed/keyed record	0x0729	0x0729
0x2E	Pack current date/time	0x2BE8	0x2BE8
0x2F	Resolve volume label or one-character drive	0x31F9	0x31F9
0x30	Get current partition context	0x310E	0x310E
0x31	Pagination pause	0x32D5	0x32D5
0x32	Read multiple logical sectors	0x09C1	0x09C1
0x33	Write multiple logical sectors	0x09C7	0x09C7
0x34	Display string with selected mode flags suppressed	0x1326	0x1326
0x35	Schedule, replace or cancel timed callback	0x2EA4	0x2EA4
0x36	Replace command input and return to CSI	0x0779	0x0779
0x37	Get partition memory ceiling	0x3172	0x3172
0x38	Set partition memory ceiling	0x317F	0x317F
0x39	Set context word 28 callback	0x318C	0x318C
0x3A	Resolve association	0x31AB	0x31AB
0x3B	Get associated terminal geometry fields	0x32A0	0x32A0
0x3C	Delete indexed/keyed record	0x072E	0x072E
0x3D	Device initialization dispatch	0x16AE	0x16AE
0x3E	Device status dispatch	0x169F	0x169F
0x3F	Device input dispatch	0x16A4	0x16A4
0x40	Device output dispatch	0x16A9	0x16A9
0x41	Standard command privilege/serial prologue	0x0774	0x0774
0x42	Copy volume label	0x324E	0x324E
0x43	Set physical device driver entry	0x170D	0x170D
0x44	Terminate/unload associated driver	0x16B3	0x16B3
0x45	Poll and consume console-event flag	0x35C2	0x35C2
0x46	Replace command input with continuation ceiling	0x077E	0x077E
0x47	Read byte from memory queue	0x3320	0x3320
0x48	Append byte to memory queue	0x3356	0x3356

Number	Descriptive function	Stored target	Live target
0x49	Change console-mode bits	0x32B1	0x32B1
0x4A	Set disk-error callback	0x3199	0x3199
0x4B	Initialize or change system volume	0x078D	0x078D
0x4C	Scaled busy delay	0x2C90	0x2C90
0x4D	Get channel-table entry	0x312B	0x312B
0x4E	Dispatch terminal control command	0x3619	0x3619
0x4F	Yield / schedule	0x0558	0x0558
0x50	Get global base	0x31A6	0x31A6
0x51	Read global byte 003B	0x315A	0x315A
0x52	Get current scheduler index	0x3113	0x3113
0x53	Release channel record locks	0x19C1	0x19C1
0x54	Acquire byte lock	0x208A	0x208A
0x55	Release byte lock	0x20AC	0x20AC
0x56	Enter callback in selected partition	0x063F	0x063F
0x57	Get association-map base	0x3121	0x3121
0x58	Send text to another partition console	0x2141	0x2141
0x59	Acquire fixed global lock	0x20C9	0x20C9
0x5A	Release fixed global lock	0x20D2	0x20D2
0x5B	Get sector workspace	0x3117	0x3117
0x5C	Get privilege byte	0x315E	0x315E
0x5D	Compare memory	0x3559	0x3559
0x5E	Read counted sequential bytes	0x0747	0x0747
0x5F	Write counted sequential bytes	0x074C	0x074C
0x60	Display numbered system message	0x0783	0x0783
0x61	Display numbered message and end command	0x0788	0x0788
0x62	Load R submodule from compact descriptor	0x076F	0x076F
0x63	Allocate a block run with minimum size	0x2849	0x2849
0x64	Look up an abbreviable keyword table	0x3564	0x3564
0x65	Get account number	0x3168	0x3168

Number	Descriptive function	Stored target	Live target
0x66	Filter device input for escape controls	0x35D8	0x35D8
0x67	Register indexed callback	0x21E4	0x21E4
0x68	Copy from selected record bank	0x1FAC	0x1FAC
0x69	Copy to selected record bank	0x1FF0	0x1FF0
0x6A	Format supplied BCD date	0x2B4B	0x2B4B
0x6B	Wait for device event or interrupt	0x06EA	0x06EA
0x6C	Find named resident component	0x20DB	0x20DB
0x6D	Write current global time through RTC callback	0x2C61	0x2C61
0x6E	Write current global date through RTC callback	0x2C6B	0x2C6B

0x00 — End command / restore CSI

Entry and input. A=command result.

Output, registers and errors. Does not return; stores result at context+68 and reloads CSI.

Structures, restrictions and notes. Releases owner locks/open rows and fixed executive byte locks, but does not CLOSE buffered sequential output. Result zero is not a durability guarantee. See Chapters 26 and 30.

Calling example

```
xor a
rst 08h
db 00h
```

The zero in A requests successful command completion. This call does not return; CLOSE any output channels first.

0x01 — Console line input

Entry and input. DE=destination; B=input budget.

Output, registers and errors. A=accepted count; HL/IY restored; BC/DE change.

Structures, restrictions and notes. Budget exhaustion submits without typed CR. Typed CR is stored and followed by another CR; budget exit appends one. CSI underscore deletion depends on context+0A bit 6. B=0 and every editing combination are unspecified.

Calling example

```
ld de,buffer
ld b,20
rst 08h
db 01h
```

The caller supplies a buffer larger than the 20-character budget to allow termination bytes. The returned count is in A; do not expect DE or BC to remain unchanged.

0x02 — Display string

Entry and input. DE=string.

Output, registers and errors. DE advances; BC/HL/IY saved.

Structures, restrictions and notes. NUL ends; CR emits newline and ends; LF emits newline and continues. Tab/control behavior follows console context. No caller length is supplied.

Calling example

```
ld de,message
rst 08h
db 02h
```

Define message as the bytes for HELLO followed by CR and zero, as in Chapter 18. DE identifies the first byte; the call displays the string.

0x03 — Poll console input

Entry and input. No explicit argument; current console association 8.

Output, registers and errors. Driver/input-state-dependent status; BC/IY preserved on physical status path.

Structures, restrictions and notes. Injected-input state can return NZ directly. Do not interpret carry as a universal input error; see 3E.

Calling example

```
rst 08h
db 03h
```

The fragment selects poll console input.

0x04 — Read console character

Entry and input. Current console input.

Output, registers and errors. A=translated character; other complete register effects are not uniformly established.

Structures, restrictions and notes. May block, consume injected input, translate controls and echo through 05. Escape-O suppresses echo without preventing character receipt. CSI/BASIC case and underscore modes differ.

Calling example

```
rst 08h
db 04h
```

The fragment selects read console character.

0x05 — Console character output

Entry and input. C=character.

Output, registers and errors. Selected calls preserve BC/DE/HL/IX/IY; AF is not fixed.

Structures, restrictions and notes. Applies suppression and printer-copy policy before 07. Context+0A bit 1 suppresses output. A returned call does not prove physical transmission.

Calling example

```
ld c,65
rst 08h
db 05h
```

C contains character code 65, the letter A. This path applies ordinary console policy, including output suppression.

0x06 — Read console character with direct echo

Entry and input. Current console input.

Output, registers and errors. A=character; BC/IY preserved by wrapper.

Structures, restrictions and notes. Uses translated input and echoes values at least 20h through 07, bypassing 05 suppression policy for that echo.

Calling example

```
rst 08h
db 06h
```

The fragment selects read console character with direct echo.

0x07 — Terminal-class character output

Entry and input. C=logical terminal character/control.

Output, registers and errors. Selected calls preserve BC/DE/HL/IX/IY; AF changes.

Structures, restrictions and notes. Class translation, cursor tracking and control 10 coordinate consumption apply. Direct 07 bypasses Escape-O. See Chapter 28 for the output layers.

Calling example

```
ld c,65
rst 08h
db 07h
```

C contains the letter A. This lower output layer applies terminal-class translation and differs from policy-controlled service 0x05.

0x08 — Printer-association character output

Entry and input. C=printer character; association 0C.

Output, registers and errors. Selected calls preserve BC/DE/HL/IX/IY; AF varies.

Structures, restrictions and notes. Unassigned printer can return 0044 silently. Active spooler collects instead of printing. Common printer control expansion is driver/path dependent.

Calling example

```
ld c,65
rst 08h
db 08h
```

The fragment selects printer-association character output.

0x09 — Invalidate cached disk descriptor fields

Entry and input. B=disk association 0.. 7.

Output, registers and errors. BC/DE/HL preserved.

Structures, restrictions and notes. Sets descriptor+14/+15 toFF. Other B values do not mutate. Refresh can add a preliminary header read; this is not a general flush.

Calling example

```
ld b,0
rst 08h
db 09h
```

The fragment selects invalidate cached disk descriptor fields.

0x0A — Read logical sector

Entry and input. B=drive association; DE=logical sector; HL=256-byte destination.

Output, registers and errors. Selected normal and ignored-error returns AF0044; C=1; DE/HL/IX/IY preserved.

Structures, restrictions and notes. Checks existing record spans. Ignore can return invalid data as success; Q exits nonlocally. Invalid associations can abort before I/O. See Chapter 27.

Calling example

```
ld b,0
ld de,1
ld hl,buffer
rst 08h
db 0Ah
```

Association 0 selects S. The request reads logical sector 1 into a 256-byte buffer. It does not print that buffer; use the header layout in Appendix E to interpret it.

0x0B — Write logical sector

Entry and input. B=drive; DE=logical sector; HL=256-byte source.

Output, registers and errors. Selected returns AF0044; C=1; DE/HL/IX/IY and source preserved.

Structures, restrictions and notes. Descriptor+13 bit 7 rejects before hardware. Ignoring consumes an unwritten sector as success. Callback clearing protection does not retry an ignored write.

Calling example

```
ld b,0
ld de,target_sector
ld hl,buffer
rst 08h
db 0Bh
```

target_sector must identify storage deliberately allocated for this operation. This writes 256 bytes and can destroy metadata if pointed at a header or directory sector.

0x0C — Return initialized kernel signature

Entry and input. Initialized kernel only.

Output, registers and errors. Live routine returns HL=5542h.

Structures, restrictions and notes. The disk entry initially targets initialization 5157; live target 331B differs. Signature meaning and complete register preservation are unknown.

Calling example

```
rst 08h
db 0Ch
```

The fragment selects return initialized kernel signature.

0x0D — Create/write directory entry

Entry and input. B=drive; DE=32-byte candidate entry.

Output, registers and errors. Selected creations AF0044; BC/DE/HL/IX/IY and caller bytes preserved.

Structures, restrictions and notes. Normalizes through lookup with public fallback inhibited. Existing/error branch returns AFF/carry by code. Does not establish allocation atomicity or global uniqueness; hidden entries can permit duplicates.

Calling example

```
ld b,0
ld de,candidate
rst 08h
db 0Dh
```

The fragment selects create/write directory entry. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x0E — Parse unsigned hexadecimal

Entry and input. DE=uppercase unsigned hexadecimal text.

Output, registers and errors. HL=value/partial overflow result; DE advances; BC restored.

Structures, restrictions and notes. Digits 0–9/A–F, optional H; no sign/space/lowercase. No-digit input can succeed. Overflow consumes its digit and can return a partially shifted HL. Validate syntax and carry independently.

Calling example

```
ld de,hex_text
rst 08h
db 0Eh
```

For an elementary input, hex_text can contain ASCII 2A followed by NUL. Check both carry and the advanced DE pointer; success without digit consumption is possible on other inputs.

0x0F — Parse unsigned decimal

Entry and input. DE=unsigned decimal text.

Output, registers and errors. HL=value/partial overflow result; DE advances; BC restored.

Structures, restrictions and notes. Optional D suffix, no sign/space. Empty or D-only input can succeed. Overflow at multiply/add stages returns an intermediate result, not necessarily modulo 65536.

Calling example

```
ld de,decimal_text
rst 08h
db 0Fh
```

decimal_text can contain ASCII 123 followed by NUL. Validate trailing text as well as the numeric return.

0x10 — Format byte as hexadecimal

Entry and input. B=byte; DE=destination.

Output, registers and errors. Two uppercase hexadecimal digits; DE+=2; BC/HL unchanged by target; AF changes.

Structures, restrictions and notes. No terminator. Supply two writable bytes.

Calling example

```
ld b,42
ld de,buffer
rst 08h
db 10h
```

42 decimal is the byte to format. The caller supplies writable output storage; the call formats rather than printing it.

0x11 — Format unsigned word as decimal

Entry and input. HL=unsigned word; DE=destination.

Output, registers and errors. Writes decimal digits; DE advances; IX restored; AF/alternateAF/BC/HL not generally preserved.

Structures, restrictions and notes. No leading zeros except single 0 for zero; no terminator. Supply space for five digits.

Calling example

```
ld hl,123
ld de,buffer
rst 08h
db 11h
```

123 is the unsigned value to format. buffer must have room for the produced decimal characters and any required termination convention.

0x12 — Console newline

Entry and input. No explicit arguments.

Output, registers and errors. Wrapper preserves AF/BC.

Structures, restrictions and notes. Sends LF then CR through 05. Suppression and printer-copy policy still apply; printer-string newline has another order.

Calling example

```
rst 08h
db 12h
```

This requests a console newline using the current console. It requires no separate character buffer.

0x13 — Calibrated busy delay

Entry and input. A=iteration count.

Output, registers and errors. BC preserved; other complete effects unknown.

Structures, restrictions and notes. Calibrated nested busy loop using 0046. A=0 performs 256 iterations. Absolute time unit is unresolved; not a scheduler sleep.

Calling example

```
ld a,1
rst 08h
db 13h
```

The fragment selects calibrated busy delay.

0x14 — Find directory entry

Entry and input. DE=writable drive+name 8+type 8; HL=256-byte directory buffer.

Output, registers and errors. Found AF0044, DE=LBA, HL=entry; absent reusable slot AF0100; B=resolved drive. IX/IY preserved in specified matrix; BC changes.

Structures, restrictions and notes. Carry-set returns do not identify a usable slot. Request can be normalized repeatedly. Public fallback, FF drive scan, early 00, deleted-slot memory and short probe cycles are specified in Chapter 26.

Calling example

```
ld de,filename
ld hl,buffer
rst 08h
db 14h
```

filename must be writable and contain drive plus two eight-byte fields. The 256-byte buffer receives directory data. Test the result before treating HL as an entry pointer.

0x15 — Get associated device descriptor

Entry and input. B=association.

Output, registers and errors. A/C=physical index; HL=0069h+32*index; DE preserved; carry on unresolved association.

Structures, restrictions and notes. Descriptor fields are a device-family union. Tape utilities add 0F to its driver pointer for a separate private interface.

Calling example

```
ld b,9
rst 08h
db 15h
```

The fragment selects get associated device descriptor.

0x16 — Load named executable without entering it

Entry and input. DE=drive+name/type; HL=requested load address in established callers.

Output, registers and errors. Loads A/R without entering; carry marks errors. Selected BASICUSR loads return AF0100 and preserve HL.

Structures, restrictions and notes. Full error/register matrix is incomplete. Do not require A=0 on every successful loader call. See Chapter 18 and Appendix A.

Calling example

```
ld de,filename
ld hl,load_address
rst 08h
db 16h
```

The fragment selects load named executable without entering it. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x17 — Printer-association string output

Entry and input. DE=printer string.

Output, registers and errors. DE advances; BC preserved; selected calls also preserve HL/IX/IY.

Structures, restrictions and notes. NUL ends, CR emits CR/LF and ends, LF emits CR/LF and continues. Unassigned printer can succeed without output.

Calling example

```
ld de,message
rst 08h
db 17h
```

The fragment selects printer-association string output.

0x18 — Initialize channel name/class

Entry and input. B=channel 0.. 16; DE=18 bytes: drive, name 8, type 8, class.

Output, registers and errors. Copies 18 bytes and clears 14; BC/DE preserved; HL=one past channel. Invalid channel carry.

Structures, restrictions and notes. Does not perform filename normalization. Selected BASIC calls returnAF0040. Internal entry begins context+AC+32*B.

Calling example

```
ld b,1
ld de,channel_name
rst 08h
db 18h
```

channel_name contains drive, padded name, padded type and class, totaling 18 bytes. This initializes channel 1; it does not open the file until service 0x20 is called.

0x19 — Convert drive character to association index

Entry and input. B=uppercase drive character.

Output, registers and errors. A=association; carry on invalid/unassigned.

Structures, restrictions and notes. S→0; A–G→1–7 after validation; *→FF. Does not mount media.

Calling example

```
ld b,53h
rst 08h
db 19h
```

0x53 is ASCII S. The operation converts that drive character to its disk association index; it does not mount a device.

0x1A — Convert disk association to character

Entry and input. B=disk association.

Output, registers and errors. A=S or A–G; invalid/unassigned gives * and carry; BC/IY preserved.

Structures, restrictions and notes. Inverse display conversion after association resolution.

Calling example

```
ld b,0
rst 08h
db 1Ah
```

The fragment selects convert disk association to character.

0x1B — Allocate exact contiguous block count

Entry and input. B=drive; DE=nonzero exact block count.

Output, registers and errors. Success A0/Z/carry 0, HL=first LBA, DE=allocated blocks; failure AFF/NZ/carry 0; association failure carry.

Structures, restrictions and notes. Requires contiguous 1-KiB blocks. C can change; IX/IY preserved in selected tests. Zero and interrupted allocation are not documented.

Calling example

```
ld b,0
ld de,1
rst 08h
db 1Bh
```

This requests one contiguous allocation block on S. Keep the returned start address and release only storage you own; allocation changes filesystem state.

0x1C — Free contiguous block range

Entry and input. B=drive; HL=first LBA; DE=block count.

Output, registers and errors. Success A0/Z/carry 0, HL=block-aligned LBA, DE=count.

Structures, restrictions and notes. Clears bitmap and updates free counts under 29A7 lock. No double-free/bounds guarantee. Write-protected header may returnAFF. Normal failed-OPEN rollback is documented.

Calling example

```
ld b,0
ld hl,allocated_start
ld de,1
```

```
rst 08h
db 1Ch
```

allocated_start must be the start of a one-block run owned by the caller. A guessed address or repeated free can corrupt allocation accounting.

0x1D — Delete named file

Entry and input. DE=drive+name/type.

Output, registers and errors. Missing file can return A0/Z; protected/wrong-owner returnsAFF/carry; DE preserved on ordinary paths.

Structures, restrictions and notes. Marks entryFF before freeing. Deletes S chains by runs. Ignores an exclusive-admission carry in the measured multi-partition path; an exclusive open is not deletion protection.

Calling example

```
ld de,filename
rst 08h
db 1Dh
```

The fragment selects delete named file. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x1E — Load and execute command

Entry and input. B=resolved drive; DE=directory entry; HL=CSI argument context.

Output, registers and errors. Loads and transfers control; full initial-register/trampoline contract incomplete.

Structures, restrictions and notes. R enters partition+4CC; A uses directory absolute load address. EX/EXE have documented wrong-token argument behavior. End through 00 in the documented native example.

Calling example

```
ld b,0
ld de,entry
ld hl,arguments
rst 08h
db 1Eh
```

The fragment selects load and execute command. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x1F — Rename named file

Entry and input. DE=old descriptor; HL=writable new descriptor.

Output, registers and errors. A08 destination exists, A04 source absent, A0A protection/owner/device refusal; full flags incomplete.

Structures, restrictions and notes. Copies old drive to new descriptor, marks old entryFF, creates renamed metadata. Does not move data; crash atomicity unknown.

Calling example

```
ld de,old_name
ld hl,new_name
rst 08h
db 1Fh
```

The fragment selects rename named file. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x20 — Open channel

Entry and input. DE=four-byte request: channel, mode, workspaceLE16; channel initialized by 18.

Output, registers and errors. Selected success A0/Z; BC/HL can change; request mode mutable.

Structures, restrictions and notes. Sequential 88 replaces, 84→8C extends, 90 inputs; direct 58→5A and 59→5B with multiple partitions. Already-open A1; absent 40h, full-probe 20h, protection 0Ah in selected paths. Chapters 17/26 specify failures and buffer obligations.

Calling example

```
ld de,request
rst 08h
db 20h
```

request must be a writable four-byte channel/mode/workspace structure, after channel-name initialization. Select the mode intentionally: sequential replacement can delete existing contents during OPEN.

0x21 — Close channel

Entry and input. DE=request as 20.

Output, registers and errors. Selected success A0/Z; BC/HL clobbered; clears channel state.

Structures, restrictions and notes. Flushes sequential tail, slack and metadata in order; clears mode bit 3 on established append paths. Ignored writes can still return success. Releases direct record spans and one file-open row. See Chapter 26.

Calling example

```
ld de,request
rst 08h
db 21h
```

Use the request belonging to the open channel. CLOSE completes buffered output handling; it is not interchangeable with generic command termination.

0x22 — Read record from channel

Entry and input. DE=request; HL=destination.

Output, registers and errors. Disk success A0; marker/link EOF A1, possibly after partial copy; DE/HL/IX/IY preserved in selected calls; BC not byte count.

Structures, restrictions and notes. Copies CR record, stops before 1A, no automatic NUL. No visible destination bound. Latches EOF; repeat copies nothing. Device classes dispatch differently.

Calling example

```
ld de,request
ld hl,buffer
rst 08h
db 22h
```

The fragment selects read record from channel.

0x23 — Write record to channel

Entry and input. DE=request; HL=record source.

Output, registers and errors. Complete source-length/result/register contract remains incomplete.

Structures, restrictions and notes. Validates open/write state and dispatches disk or device class. Prefer documented counted-output or BASIC examples rather than inventing a terminated-string contract.

Calling example

```
ld de,request
ld hl,record
rst 08h
db 23h
```

The fragment selects write record to channel.

0x24 — Format current date

Entry and input. DE=destination.

Output, registers and errors. Formats current date; DE advances; BC/HL preserved after refresh.

Structures, restrictions and notes. Uses BCD state 0013–0015 and 6A; no NUL. Hardware refresh does not independently refresh year.

Calling example

```
ld de,buffer
rst 08h
db 24h
```

The fragment selects format current date.

0x25 — Format current time

Entry and input. DE=destination.

Output, registers and errors. Writes exactly eight bytes HH: MM: SS; DE advances 8; BC/HL saved around formatting.

Structures, restrictions and notes. Refreshes time/date through 2C4D/2C57 from BCD globals 001B–001D; no NUL.

Calling example

```
ld de,buffer
rst 08h
db 25h
```

The fragment selects format current time.

0x26 — Unsigned division

Entry and input. HL=dividend; DE=divisor.

Output, registers and errors. Unsigned quotient in HL, remainder in DE; BC restored; A=high quotient byte, so Z tests that byte, not the whole quotient; carry on zero divisor.

Structures, restrictions and notes. Zero divisor returns HL0 and original dividend in DE without a trap. Full flags case-specific; seeded BC/IX/IY preserved in native arithmetic controls.

Calling example

```
ld hl,7
ld de,2
rst 08h
db 26h
```

The inputs represent 7 divided by 2. Inspect the quotient/remainder contract above; zero divisors require explicit error handling.

0x27 — Unsigned multiplication

Entry and input. HL and DE=unsigned factors.

Output, registers and errors. HL=product if no overflow; BC/DE restored; carry marks overflow. A/Z reflect working multiplier state, not product zero.

Structures, restrictions and notes. Overflow may return an early partial accumulator. 9000h×3 returns 9000h/carry; reversed factors return 3000h/carry. Do not assume low 16 product on failure.

Calling example

```
ld hl,3
ld de,4
rst 08h
db 27h
```

The factors are 3 and 4. Test carry before using HL as a valid product; an overflow result is not generally the low word of the mathematical product.

0x28 — Read numbered direct record

Entry and input. DE=request; BC=one-based direct record; HL=record-length destination.

Output, registers and errors. Success A0/Z/carry 0; out of range A80; selected BC/DE/HL/IX/IY preserved.

Structures, restrictions and notes. Optional inclusive-sector locks remain held after read. Retained same-owner overlap can omit new coverage. Failed range leaves destination unchanged in selected control.

Calling example

```
ld de,request
ld bc,2
ld hl,buffer
rst 08h
db 28h
```

The direct file must already be open. BC selects one-based record 2; buffer must accommodate the configured record length.

0x29 — Write numbered direct record

Entry and input. DE=request; BC=one-based direct record; HL=record-length source.

Output, registers and errors. Selected success A0/Z/carry 0 and BC/DE/HL/IX/IY preserved; closedAFF, protected 0A, range 80 by code/selected controls.

Structures, restrictions and notes. Read/modify/write preserves surrounding sector bytes. Release policy follows request bits. No new explicit span acquisition in measured write-back path.

Calling example

```
ld de,request
ld bc,2
ld hl,record
rst 08h
db 29h
```

The direct file must already be open for writing. BC selects record 2 and record supplies its data; protection may be rejected at WRITE rather than OPEN.

0x2A — Parse decimal or H-suffixed hexadecimal

Entry and input. DE=numeric text.

Output, registers and errors. HL parsed value, DE advanced; shares 0E/0F preservation/errors.

Structures, restrictions and notes. Looks ahead through hex characters and selects hex only for uppercase H. FF alone succeeds as decimal zero without consuming; FFH→255; 12AB consumes 12; 12DH is hex012D.

Calling example

```
ld de,numeric_text
rst 08h
db 2Ah
```

The fragment selects parse decimal or h-suffixed hexadecimal.

0x2B — Read indexed/keyed record

Entry and input. DE=request; BC=fixed key buffer; HL=data destination.

Output, registers and errors. A0 success; A1/NZ/carry 0 missing with unchanged destination on the stated path; BC/DE/HL/IX/IY preserved.

Structures, restrictions and notes. Can NUL-pad writable key and mutate channel/workspace. I/K layouts differ; see Appendix E.

Calling example

```
ld de,request
ld bc,key
ld hl,buffer
rst 08h
db 2Bh
```

The fragment selects read indexed/keyed record.

0x2C — Read next indexed/keyed record

Entry and input. DE=request; BC=writable key destination; HL=data destination.

Output, registers and errors. Returns next key/data; selected A0 and BC/DE/HL preserved.

Structures, restrictions and notes. I order follows sorted slot links; K scan differs. Full EOF/error/register mapping incomplete.

Calling example

```
ld de,request
ld bc,key
ld hl,buffer
rst 08h
db 2Ch
```

The fragment selects read next indexed/keyed record.

0x2D — Write indexed/keyed record

Entry and input. DE=request; BC=key buffer; HL=fixed record source.

Output, registers and errors. Selected success A0/Z; BC/DE/HL/IX/IY preserved.

Structures, restrictions and notes. Creates/replaces records and updates index, links/count and workspace. Zero first key byte returns immediately by code. Capacity/error paths remain partial.

Calling example

```
ld de,request
ld bc,key
ld hl,record
rst 08h
db 2Dh
```

The fragment selects write indexed/keyed record.

0x2E — Pack current date/time

Entry and input. DE=three-byte destination.

Output, registers and errors. Writes packed current date/time.

Structures, restrictions and notes. Layout is Appendix E's three-byte timestamp. Complete register and malformed-time behavior remains incomplete.

Calling example

```
ld de,packed_date
rst 08h
db 2Eh
```

The fragment selects pack current date/time.

0x2F — Resolve volume label or one-character drive

Entry and input. DE=eight-byte padded label.

Output, registers and errors. Label success A=index/Z/carry 0; missing A1/NZ/carry 1; DE/HL/IY preserved on scan; B replaced before BC save.

Structures, restrictions and notes. If byte 1 is space, byte 0 is interpreted as drive character. Otherwise scans associations 0.. 7 and refreshes invalid descriptors.

Calling example

```
ld de,label
rst 08h
db 2Fh
```

The fragment selects resolve volume label or one-character drive.

0x30 — Get current partition context

Entry and input. No explicit input.

Output, registers and errors. IY=word 0033; other target registers unchanged.

Structures, restrictions and notes. Returns current context, not common-global base. GETUSER uses its name field.

Calling example

```
rst 08h
db 30h
```

IY receives the current partition context. The fields are laid out in Appendix E; they are not a C structure supplied by an external runtime.

0x31 — Pagination pause

Entry and input. Current terminal/context.

Output, registers and errors. Wrapper preserves AF/BC/IY.

Structures, restrictions and notes. Pagination marker and input wait; accepts space, CR, LF or Ctrl-X. Context flags can suppress pause.

Calling example

```
rst 08h
db 31h
```

The fragment selects pagination pause.

0x32 — Read multiple logical sectors

Entry and input. B=drive; C=sector count; DE=start LBA; HL=destination.

Output, registers and errors. Selected C8 calls returnAF0044 and preserve BC/DE/HL/IX/IY.

Structures, restrictions and notes. Shares 0A error policy. C=0, partial failures and atomicity remain unresolved. EXEC usesC2 for 512-byte frames.

Calling example

```
ld b,0
ld c,2
ld de,1
ld hl,buffer
rst 08h
db 32h
```

This reads two sectors beginning at logical sector 1 on S into at least 512 bytes of writable storage. Physical requests may include a preliminary descriptor refresh.

0x33 — Write multiple logical sectors

Entry and input. B=drive; C=count; DE=start LBA; HL=source.

Output, registers and errors. Selected C8 calls returnAF0044 and preserve BC/DE/HL/IX/IY/source.

Structures, restrictions and notes. Shares 0B policy; writes may include unchanged sectors. Count 0/ partial failure atomicity unknown.

Calling example

```
ld b,0
ld c,2
ld de,target_sector
ld hl,buffer
rst 08h
db 33h
```

This writes two sectors from at least 512 bytes of caller data. target_sector must be valid, deliberately selected writable storage; the fragment is not a safe generic disk test.

0x34 — Display string with selected mode flags suppressed

Entry and input. DE=string as 02.

Output, registers and errors. Restores context+0A/+0B and IY.

Structures, restrictions and notes. Temporarily clears 0A bits 1/2 and 0B bit 7 before ordinary string display. Other string effects follow 02.

Calling example

```
ld de,message
rst 08h
db 34h
```

The fragment selects display string with selected mode flags suppressed.

0x35 — Schedule, replace or cancel timed callback

Entry and input. HL=six writable bytes followed by callback; DE=delay; A0 normal.

Output, registers and errors. Selected calls preserve AF/BC/DE/HL/IX/IY; return with interrupts enabled.

Structures, restrictions and notes. DE/8 delta ticks; DE0 cancels; expiry clears node and invokes+6 in owner context. A bit 0 head exception is not a general cross-bank cancel. See Chapter 25.

Calling example

```
xor a
ld hl,timer_node
ld de,80
rst 08h
db 35h
```

timer_node consists of six writable bytes immediately followed by callback code. Delay 80 is converted to ten eight-millisecond ticks. The callback executes in its owning context.

0x36 — Replace command input and return to CSI

Entry and input. DE=CR-terminated replacement command.

Output, registers and errors. Does not return; cleanup then CSI with result 0.

Structures, restrictions and notes. Copies printable 20–7E, folds lowercase, drops others and sets context+38 bit 1. No visible length guard. Bypasses interactive input budget.

Calling example

```
ld de,command
rst 08h
db 36h
```

The fragment selects replace command input and return to csi. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x37 — Get partition memory ceiling

Entry and input. No explicit input.

Output, registers and errors. HL=context+04 memory ceiling; IY preserved.

Structures, restrictions and notes. The ceiling changes as drivers/classes occupy upper memory.

Calling example

```
rst 08h
db 37h
```

HL receives the current memory ceiling. Loaded drivers and classes can lower it; do not replace it with a fixed maximum address.

0x38 — Set partition memory ceiling

Entry and input. HL=new memory ceiling.

Output, registers and errors. Stores context+04; IY preserved.

Structures, restrictions and notes. No general allocation/validation contract. Changing this pointer alone does not safely allocate memory.

Calling example

```
ld hl,new_ceiling
rst 08h
db 38h
```

new_ceiling must fit the partition's valid memory arrangement and retained code. An arbitrary larger value does not allocate more memory.

0x39 — Set context word 28 callback

Entry and input. HL=callback or zero.

Output, registers and errors. Stores context+28; IY preserved.

Structures, restrictions and notes. Interruption/control callback used by CSI and drivers. SS2's restoration defect belongs to its caller, not this store routine.

Calling example

```
ld hl,callback
rst 08h
db 39h
```

The fragment selects set context word28 callback. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x3A — Resolve association

Entry and input. B=association 0.. 23.

Output, registers and errors. A=physical index; valid carry 0/Z0; invalid/unassigned AFF/Z1/carry 1; BC/IY preserved.

Structures, restrictions and notes. Map starts context+7D. Logical association, physical descriptor and partition index are different values.

Calling example

```
ld b,9
rst 08h
db 3Ah
```

The fragment selects resolve association.

0x3B — Get associated terminal geometry fields

Entry and input. B=association, normally 8/9.

Output, registers and errors. B = byte at physical-device descriptor +0x02; C = byte at descriptor +0x03; A = byte at descriptor +0x04 (terminal-class number). HL/DE preserved; unresolved association: BC=0x4F17/carry.

Structures, restrictions and notes. L is CSI input budget and an output threshold, P is row cap. Do not assert generic 80×24 from 79/23.

Calling example

```
ld b,9
rst 08h
db 3Bh
```

The fragment selects get associated terminal geometry fields.

The offsets above address fields in the physical-device descriptor, not bytes in the terminal-class executable. The descriptor layout is given in Appendix E.

0x3C — Delete indexed/keyed record

Entry and input. DE=request; BC=key.

Output, registers and errors. Deletes found I/K record; absent returns success by code; full register guarantees incomplete.

Structures, restrictions and notes. Marks tombstoneFF, adjusts ordered references/count for I. Not a file deletion.

Calling example

```
ld de,request
ld bc,key
rst 08h
db 3Ch
```

The fragment selects delete indexed/keyed record. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x3D — Device initialization dispatch

Entry and input. B=associated character device.

Output, registers and errors. Driver+09 result, not a uniform success status; selected BC/DE/HL/IX/IY preserved.

Structures, restrictions and notes. Can patch code, register callbacks or exit nonlocally. For floppy drivers+09 means write; do not call it as initialization.

Calling example

```
ld b,device_association
rst 08h
db 3Dh
```

The fragment selects device initialization dispatch. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x3E — Device status dispatch

Entry and input. B=association.

Output, registers and errors. Driver+00 result; selected serial calls preserve BC/DE/HL/IX/IY.

Structures, restrictions and notes. ALP Z describes input occupancy, carry DMA readiness; polling can flush output. SS2/printer/tape differ. Unassigned can return 0044, not a distinct error.

Calling example

```
ld b,8
rst 08h
db 3Eh
```

The fragment selects device status dispatch.

0x3F — Device input dispatch

Entry and input. B=association.

Output, registers and errors. Character driver's+03; selected serial A=byte/carry 0 with BC/DE/HL/IX/IY preserved.

Structures, restrictions and notes. May block/recheck after events. Unassigned returns zero in specified case; printerRET values incidental; tape stub performs no record read; floppy+03 means recalibration request.

Calling example

```
ld b,8
rst 08h
db 3Fh
```

The fragment selects device input dispatch.

0x40 — Device output dispatch

Entry and input. B=association; C=character for serial/printer.

Output, registers and errors. Driver/path-dependent AF; selected BC/DE/HL/IX/IY preserved.

Structures, restrictions and notes. Raw console bypasses class/cursor. Printer policy and spool hook apply by association. Buffered serial return is not transmission completion. Not the disk driver's register ABI.

Calling example

```
ld b,9
ld c,65
```

```
rst 08h
db 40h
```

The fragment selects device output dispatch.

0x41 — Standard command privilege/serial prologue

Entry and input. Bytes preceding CF41: minimum privilege atP-4; serial atP-3.. P-1.

Output, registers and errors. Pass continues; failure displays 19/20 and terminatesCR252.

Structures, restrictions and notes. First serial byte 0 bypasses binding. Context+0C bit 5 can request version display. Extra utility checks remain separate.

Calling example

```
db 1,0,0,0
entry: rst 08h
db 41h
```

Execution must enter at the RST, not at the preceding data. This example requires privilege 1 and uses an all-zero serial field. The prologue is a software guard; it does not elevate privilege.

0x42 — Copy volume label

Entry and input. B=drive; DE=eight writable bytes.

Output, registers and errors. Copies label; DE+=8; BC/HL preserved.

Structures, restrictions and notes. Target lacks explicit unresolved-association carry branch. Validate association before relying on copied bytes.

Calling example

```
ld b,0
ld de,label
rst 08h
db 42h
```

The fragment selects copy volume label.

0x43 — Set physical device driver entry

Entry and input. B=physical index 8.. 31; HL=driver entry.

Output, registers and errors. Stores descriptor word; DE preserved; HL ends descriptor+1.

Structures, restrictions and notes. Out-of-range B returns without store. This is physical descriptor installation, not logical association.

Calling example

```
ld b,physical_index
ld hl,driver_entry
rst 08h
db 43h
```

The fragment selects set physical device driver entry. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x44 — Terminate/unload associated driver

Entry and input. B=character-device association.

Output, registers and errors. Driver+0C and conditional unload; AF not uniform.

Structures, restrictions and notes. Clears class fields; matching ceiling branch reclaims loaded code. Does not CLOSE application files. DIS1/2 have no fifth entry; disk removal uses another path.

Calling example

```
ld b,device_association
rst 08h
db 44h
```

The fragment selects terminate/unload associated driver. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x45 — Poll and consume console-event flag

Entry and input. Current console.

Output, registers and errors. A1/NZ if context+0B bit 5 was set, elseA0/Z; clears bit; IY preserved.

Structures, restrictions and notes. Calls 03 first. Exact event interpretation outside observed cancellation paths remains incomplete.

Calling example

```
rst 08h
db 45h
```

The fragment selects poll and consume console-event flag.

0x46 — Replace command input with continuation ceiling

Entry and input. DE=CR command; HL=continuation ceiling.

Output, registers and errors. Does not ordinarily return; sets ceiling/SP and context+38 bit 5, then 36.

Structures, restrictions and notes. CSI later clears bit 5 and jumps to ceiling. Complete safe caller stack/register recipe unspecified.

Calling example

```
ld de,command
ld hl,continuation
rst 08h
db 46h
```

The fragment selects replace command input with continuation ceiling. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x47 — Read byte from memory queue

Entry and input. HL=queue header/payload.

Output, registers and errors. EmptyA0/carry 1 unchanged; dataA/carry 0; selected BC/DE/HL/IX/IY preserved.

Structures, restrictions and notes. Nonempty path enables interrupts; draining retains previous read index. Eight-bit arithmetic can reach header at capacities 254/255. See Chapter 25.

Calling example

```
ld hl,queue
rst 08h
db 47h
```

queue must have the header and storage described in Appendix E. Test carry for empty; a returned zero byte can be valid data.

0x48 — Append byte to memory queue

Entry and input. HL=queue; A=byte.

Output, registers and errors. Ordinary selected calls preserve A/BC/DE/HL/IX/IY; flags change; enables interrupts.

Structures, restrictions and notes. Full loop caches occupancy and can spin even after a callback removes data. Capacities above 128 have demonstrated indexing defects. No safe full error return.

Calling example

```
ld hl,queue
ld a,65
rst 08h
db 48h
```

The queue must have free capacity before appending character code 65. A full queue can enter a loop that does not notice ordinary consumer progress.

0x49 — Change console-mode bits

Entry and input. B=set mask; C=clear mask.

Output, registers and errors. A=new context+0A; HL/IY restored; B/C masked in place.

Structures, restrictions and notes. Only bits 2–7 apply; preserves bits 0/1. Uses DI/EI. MENSAJE OFF is instead context+0C bit 7.

Calling example

```
ld b,set_mask
ld c,clear_mask
rst 08h
db 49h
```

The fragment selects change console-mode bits. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x4A — Set disk-error callback

Entry and input. HL=disk-error callback or zero.

Output, registers and errors. Selected registration preserves AF/BC/DE/HL/IX/IY.

Structures, restrictions and notes. Callback runs locked before retry decrement: A0 ignore, positive replace error, negative abort. Preserve working registers. Reentrant disk calls unspecified.

Calling example

```
ld hl,error_callback
rst 08h
db 4Ah
```

The fragment selects set disk-error callback. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x4B — Initialize or change system volume

Entry and input. B=FF cold initialization, otherwise new system association.

Output, registers and errors. Enters initialization/CSI-loading path; not a returning query.

Structures, restrictions and notes. Locates system components and writes kernel clock state. See Chapter 21.

Calling example

```
ld b,new_system_association
rst 08h
db 4Bh
```

The fragment selects initialize or change system volume. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x4C — Scaled busy delay

Entry and input. A low 6=count, high 2=scale 1/10/100/1000.

Output, registers and errors. Low 6 zero returns immediately; BC restored; A0/Z on ordinary return.

Structures, restrictions and notes. Scaled busy delay based on 13. Absolute unit unresolved.

Calling example

```
ld a,1
rst 08h
db 4Ch
```

The fragment selects scaled busy delay.

0x4D — Get channel-table entry

Entry and input. B=channel 0.. 16.

Output, registers and errors. HL=context+AC+32*B; invalid carry.

Structures, restrictions and notes. Returns an internal 32-byte channel, not the caller's four-byte request.

Calling example

```
ld b,1
rst 08h
db 4Dh
```

The fragment selects get channel-table entry.

0x4E — Dispatch terminal control command

Entry and input. A=control letter.

Output, registers and errors. Path-dependent; can change stack/control flow or not return.

Structures, restrictions and notes. Normalizes case; recognizes Q, D, O, P, I, W, A, B, S,],^, C. Established Q/S/O/I/C behavior is in Chapters 4/14/28; other combinations remain partial.

Calling example

```
ld a,control_letter
rst 08h
db 4Eh
```

The fragment selects dispatch terminal control command. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x4F — Yield / schedule

Entry and input. Current partition.

Output, registers and errors. Saves/restores register frame and selects runnable context; interrupt-enable state changes.

Structures, restrictions and notes. Circular first eligible, callback residual preference, byte-quantum anomalies. Single-partition path enables interrupts. See Chapter 25.

Calling example

```
rst 08h
db 4Fh
```

The fragment selects yield / schedule.

0x50 — Get global base

Entry and input. No explicit input.

Output, registers and errors. IY=0.

Structures, restrictions and notes. Common-global base, used by BASIC serial policy. Does not imply writable fields are safe to change.

Calling example

```
rst 08h
db 50h
```

The fragment selects get global base.

0x51 — Read global byte 003B

Entry and input. No explicit input.

Output, registers and errors. A=byte 003B.

Structures, restrictions and notes. A serial-related consumer exists, but a complete independent semantic name is not established. Other register effects follow the short target only.

Calling example

```
rst 08h
db 51h
```

The fragment selects read global byte003b.

0x52 — Get current scheduler index

Entry and input. No explicit input.

Output, registers and errors. A=current zero-based scheduler index 000C.

Structures, restrictions and notes. Not account number, bank number or displayed partition number.

Calling example

```
rst 08h
db 52h
```

The fragment selects get current scheduler index.

0x53 — Release channel record locks

Entry and input. DE=request with channel 0.. 16 atbyte 0.

Output, registers and errors. ValidA0/Z; selected BC/DE/HL/IX/IY preserved; invalidA04/NZ by code.

Structures, restrictions and notes. Releases current-owner/channel record spans. Does not release file-open exclusive admission. Single-partition path skips table operation.

Calling example

```
ld de,request
rst 08h
db 53h
```

The fragment selects release channel record locks.

0x54 — Acquire byte lock

Entry and input. HL=one-byte lock.

Output, registers and errors. Selected immediate/delayed calls preserve AF/BC/DE/HL/IX/IY.

Structures, restrictions and notes. Zero→owner=index+1; same owner returns without recursion count; other owner setswaitbit 4/yields/rechecks. IFF preservation/fairness/timeouts unknown.

Calling example

```
ld hl,lock_byte
rst 08h
db 54h
```

lock_byte must be a shared byte initialized to zero before use. Acquisition records ownership and can yield while waiting. Repeated acquisition by the owner adds no recursion count.

0x55 — Release byte lock

Entry and input. HL=one-byte lock.

Output, registers and errors. Selected calls preserve AF/BC/DE/HL/IX/IY.

Structures, restrictions and notes. Another owner is ignored, without an error flag. Own release clearsbit 4 in all rows thenlock; wakeup is broadcast and requires recheck.

Calling example

```
ld hl,lock_byte
rst 08h
db 55h
```

Release only according to the lock ownership protocol. Woken partitions recheck their own conditions; wakeup does not itself grant a lock.

0x56 — Enter callback in selected partition

Entry and input. A low 7=target index, bit 7=suppressEI; HL=callback in target mapping.

Output, registers and errors. Restores target frame and jumps to callback; callbackRET resumes target.

Structures, restrictions and notes. Masks high state nibble, including STOP. Sender arguments are not restored as target registers. Invalid/nested targets have no general safe contract. See Chapter 25.

Calling example

```
ld a,target_index
ld hl,callback
rst 08h
db 56h
```

The fragment selects enter callback in selected partition. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x57 — Get association-map base

Entry and input. No explicit input.

Output, registers and errors. IY=context+7D; DE preserved.

Structures, restrictions and notes. Points to 24 association bytes. Unassigned isFF.

Calling example

```
rst 08h
db 57h
```

The fragment selects get association-map base.

0x58 — Send text to another partition console

Entry and input. A=target index; DE=sender string.

Output, registers and errors. Selected BC/HL/IX/IY preserved; DE advances for send, unchanged for skip; A is not a generic status.

Structures, restrictions and notes. Seven-bit text protocol via recipient callback and 21DD lock. Self/range skips can release an already caller-owned lock; OFF policy is not checked here. See Chapter 29.

Calling example

```
ld a,target_index
ld de,message
rst 08h
db 58h
```

The fragment selects send text to another partition console.

0x59 — Acquire fixed global lock

Entry and input. No explicit input.

Output, registers and errors. Acquire lock 0006 through 54; restores callerHL.

Structures, restrictions and notes. Same ownership/wait semantics and limitations as 54.

Calling example

```
rst 08h
db 59h
```

The fragment selects acquire fixed global lock.

0x5A — Release fixed global lock

Entry and input. No explicit input.

Output, registers and errors. Release lock 0006 through 55; restores callerHL.

Structures, restrictions and notes. Same broadcast/nonowner semantics and limitations as 55.

Calling example

```
rst 08h
db 5Ah
```

The fragment selects release fixed global lock.

0x5B — Get sector workspace

Entry and input. No explicit input.

Output, registers and errors. HL=context+2CC; DE preserved.

Structures, restrictions and notes. Returns 256-byte sector workspace. It can be used internally by other services; avoid unsupported lifetime assumptions.

Calling example

```
rst 08h
db 5Bh
```

The fragment selects get sector workspace.

0x5C — Get privilege byte

Entry and input. No explicit input.

Output, registers and errors. A=context+11 privilege; IY preserved; selected BC/DE/HL/IX/IY preserved.

Structures, restrictions and notes. GETPRIV zero-extends intoHL. Getter does not elevate authority.

Calling example

```
rst 08h
db 5Ch
```

The result in A is the current privilege. Reading it does not change the account or grant a higher privilege.

0x5D — Compare memory

Entry and input. BC=count; DE/HL=memory ranges.

Output, registers and errors. Compares until difference/exhaustion; pointers/count advance.

Structures, restrictions and notes. Initial BC0 is not established as a safe empty comparison. Full flag/register contract outside comparison use remains incomplete.

Calling example

```
ld bc,4
ld de,first_bytes
ld hl,second_bytes
rst 08h
db 5Dh
```

The fragment selects compare memory.

0x5E — Read counted sequential bytes

Entry and input. DE=request; HL=destination; B=count, 0=256.

Output, registers and errors. A0 ordinary, A1 EOF possibly after copying; selected DE/HL/IX/IY preserved, BC changes.

Structures, restrictions and notes. Raw 254-byte payload stream, eager next-sector fetch, no terminator. Exact final-boundary read can copy all bytes and returnEOF. LatchedEOF prevents later copying; see Chapter 26.

Calling example

```
ld de,request
ld hl,buffer
ld b,4
rst 08h
db 5Eh
```

The open sequential request supplies four bytes to buffer. A boundary return can indicate EOF even after all requested bytes were copied; inspect the documented status.

0x5F — Write counted sequential bytes

Entry and input. DE=request; HL=source; B=count.

Output, registers and errors. SelectedA0/Z/carry 0; DE/HL/IX/IY restored; BC can change.

Structures, restrictions and notes. One channel-count increment per call. Partial buffer not durable untilCLOSE; B0's 256-loop behavior implementation-defined. Trailing 1A can later be trimmed byEXTEND.BASIC may change request toA8/AC first.

Calling example

```
ld de,request
ld hl,record
ld b,4
rst 08h
db 5Fh
```

The open output request accepts four bytes from record. They can remain buffered until explicit CLOSE; this call is not a persistence guarantee.

0x60 — Display numbered system message

Entry and input. HL=one-based message record; B=system drive; DE=eight-byte substitution fields.

Output, registers and errors. BC/DE/HL/IY saved.

Structures, restrictions and notes. Reads 64-byte records atsector 25h+(HL-1)//4, offset 64*((HL-1)%4). Global 000E bits 6/7 affect display. Invalid number bounds not checked here.

Calling example

```
ld hl,1
ld b,0
ld de,substitutions
rst 08h
db 60h
```

Record 1 is the syntax-error message. substitutions points to the caller's fixed-width substitution area, even when the chosen text needs no substitutions.

0x61 — Display numbered message and end command

Entry and input. As 60, plus A=command result.

Output, registers and errors. Displays message then terminates through 00; no return.

Structures, restrictions and notes. Message number and command result are separate numbers.

Calling example

```
ld hl,1
ld b,0
```

```
ld de,substitutions
ld a,8
rst 08h
db 61h
```

This displays message 1 and terminates with result 8. It does not return to the instruction after the selector.

0x62 — Load R submodule from compact descriptor

Entry and input. A=format; B=drive; DE=baseLBA; HL=seven-byte module descriptor.

Output, registers and errors. Loads R submodule through constructed metadata; complete return/error matrix incomplete.

Structures, restrictions and notes. Descriptor: loadLE16, imagebytesLE16, storedsectorsbyte, relativeLBALE16. Used by BASIC; do not invent general object-linker behavior.

Calling example

```
ld a,module_format
ld b,0
ld de,base_sector
ld hl,module_descriptor
rst 08h
db 62h
```

The fragment selects load r submodule from compact descriptor. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x63 — Allocate a block run with minimum size

Entry and input. B=drive; DE=desired blocks; HL=minimum, both nonzero.

Output, registers and errors. SuccessA0/Z/carry 0, HL=firstLBA, DE=count; no runAFF/NZ/carry 0; unresolved associationcarry.

Structures, restrictions and notes. First desired-size run, else largest qualifying run with later ties. BC changes; selectedIX/IY preserved. Shares locked bitmap updater with 1B/1C.

Calling example

```
ld b,0
ld de,4
ld hl,1
rst 08h
db 63h
```

The desired run is four blocks and the minimum acceptable run is one. The allocator can return a smaller qualifying run; use its returned count, not the requested count.

0x64 — Look up an abbreviable keyword table

Entry and input. DE=token; HL=keyword table.

Output, registers and errors. MatchA0/Z, HL=result; absentA1/NZ; DE restored; BC clobbered.

Structures, restrictions and notes. Record: minimum-prefixbyte, keyword with finalbit 7, resultLE16; zero minimum ends table. A–Z/0–9/$ accepted; caller folds case. First qualifying match wins.

Calling example

```
ld de,token
ld hl,keywords
rst 08h
db 64h
```

The fragment selects look up an abbreviable keyword table.

0x65 — Get account number

Entry and input. No explicit input.

Output, registers and errors. A=context+1A account number; IY preserved.

Structures, restrictions and notes. GETID zero-extends intoHL. Public 0, temporaryFF and partition lock ownership are distinct concepts.

Calling example

```
rst 08h
db 65h
```

A receives the account number. This is different from service 0x52, which returns the scheduler index.

0x66 — Filter device input for escape controls

Entry and input. A=input character; IY=device descriptor.

Output, registers and errors. Enabled path restores BC/HL; callback can change other state; carry means consumed in established paths.

Structures, restrictions and notes. Descriptor+05 bit 7 enables, bit 6 tracks escape; optional+16 callback. Calls 4E after escape/case handling. Not a universal raw character filter.

Calling example

```
ld a,input_byte
ld iy,device_descriptor
rst 08h
db 66h
```

The fragment selects filter device input for escape controls. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x67 — Register indexed callback

Entry and input. A=index 02.. 20h; DE=callback.

Output, registers and errors. Stores owner and pointer and pointer at 23FF+3*A; selected AF/BC/DE/HL/IX/IY preserved.

Structures, restrictions and notes. Invalid range returns without store by code. Does not itself DI; enabled-caller atomicity unknown. DMA owner rows need not invoke their stored callback.

Calling example

```
ld a,callback_index
ld de,callback
rst 08h
db 67h
```

The fragment selects register indexed callback. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x68 — Copy from selected record bank

Entry and input. A=record index; HL=source there; DE=destination in caller; BC=positive count.

Output, registers and errors. BC0; HL/DE advance; A/IX/IY preserved; returnsDI.

Structures, restrictions and notes. Cross-bank preservesAF, uses common stack; same-bankLDIR changesF and overlaps propagate forward. No pointer/index/zero-count validation. Current identity pointers unchanged.

Calling example

```
ld a,target_index
ld hl,source_address
ld de,destination
ld bc,16
rst 08h
db 68h
```

target_index selects a scheduler record, not a physical bank number. Sixteen bytes are copied into the caller's destination. Account for the disabled-interrupt return.

0x69 — Copy to selected record bank

Entry and input. A=record index; HL=caller source; DE=destination there; BC=positive count.

Output, registers and errors. As 68.

Structures, restrictions and notes. Direction reverses 68. Record index is not bank. BC0 could iterate 65536; this and arbitrary overlap with control structures are not supported recipes.

Calling example

```
ld a,target_index
ld hl,source
ld de,destination_address
ld bc,16
rst 08h
db 69h
```

The source belongs to the caller and the destination to the selected record's mapping. The count is sixteen; zero is not an empty-copy convention.

0x6A — Format supplied BCD date

Entry and input. C=BCD month, H=BCD day, L=BCD year; DE=output.

Output, registers and errors. DE advances 8 normally or 9 with bothformatbits; BC/HL/IY preserved.

Structures, restrictions and notes. Context+0C bit 4 choosesMM/DD/YY, defaultDD-MM-YY; bit 2 aloneDDMMMYY plus space. Spanish months. No NUL; invalidBCD/month unchecked.

Calling example

```
ld c,01h
ld h,01h
ld l,90h
ld de,buffer
```

```
rst 08h
db 6Ah
```

The supplied BCD date is 01/01/90. buffer must be writable and large enough for the selected date format; this formats data rather than changing the clock.

0x6B — Wait for device event or interrupt

Entry and input. Current device/event condition managed by caller.

Output, registers and errors. SelectedAF/BC/DE/HL/IX/IY preserved; returns with interrupts enabled.

Structures, restrictions and notes. Single partitionEI/HALT; multiple setwaitbit 5 and schedule. Wakeup may carry no input. Always recheck condition.

Calling example

```
rst 08h
db 6Bh
```

Wait only inside a loop that rechecks the relevant condition. A wakeup need not mean a character is available.

0x6C — Find named resident component

Entry and input. DE=eight readable exact key bytes.

Output, registers and errors. FoundAF0044/HL=address; absentAF0101/HL unchanged; BC/DE/IX/IY preserved on the stated paths.

Structures, restrictions and notes. Ten-byte rows from 04BC, stopfirst 00/space; no numeric bound. No normalization, size, bank or owner field. Exact SPOOLER-space resolves 39C2.

Calling example

```
ld de,component_key
rst 08h
db 6Ch
```

component_key must contain exactly eight readable bytes, for example SPOOLER followed by one space. Check success before using HL as the returned address.

0x6D — Write current global time through RTC callback

Entry and input. Current global BCD time 001B–001D; callback 0062 installed.

Output, registers and errors. Wrapper preserves AF/BC/DE/HL.

Structures, restrictions and notes. WRITES RTC through 2E88 to 84/83/82 and enables interrupts. It is not a read-only clock query.

Calling example

```
rst 08h
db 6Dh
```

The fragment selects write current global time through rtc callback. This is an internal or configuration operation, not an ordinary application setup shortcut.

0x6E — Write current global date through RTC callback

Entry and input. Current global month/day 0013/0014; callback 0064 installed.

Output, registers and errors. Wrapper preserves AF/BC/DE/HL.

Structures, restrictions and notes. WRITES RTC through 2E98 to 87/86; does not write year 0015. Callback configuration can differ on another machine.

Calling example

```
rst 08h
db 6Eh
```

The fragment selects write current global date through rtc callback. This is an internal or configuration operation, not an ordinary application setup shortcut.

Chapter 33 — Diagnostics and messages

A message record number, a bracketed number inside its text, a BASIC runtime error and a CSI command result are separate namespaces. Record 30 itself contains the bracketed number 29. Do not renumber it to make the table regular. Some records are prompts or successful status messages rather than errors.

SISTEMA.MENSERR contains 200 fixed 64-byte records: 141 have nonblank text, four contain only CR, and 55 are zero-filled. The table preserves all 145 nonzero records; omitted records are zero-filled. `{1}` and similar fields are substitutions supplied by callers. A literal vertical bar is preserved as text. Control bytes are shown as `\xNN`. No trailing byte has been reinterpreted as an English punctuation mark.

The subsystem column is an editorial aid based on the text and identified callers. For records with an unspecified caller, the text gives a meaning without promising a universal corrective action. Chapter 13 gives the diagnostic workflow.

Diagnose common workflow failures

The following table relates displayed diagnostics to specific operations. Its CR column gives the CSI command result where observed; it is not the message-record number in the catalogue below. Some commands display message numbers only when configured to do so.

Displayed text	Operation and result	Corrective action
Esta cuenta esta siendo utilizada por la particion 1	CONECTAR; CR = 10; partition number varies	End the existing connection under that account number before retrying. The password has not yet been checked. See password changes.
No se ha cambiado ningun fichero	CAMPRO; CR = 12	Check selection and connect as the current file owner with sufficient privilege. SISTEMA does not automatically select another owner's private files.
La cuenta sistema no puede compartir ficheros	COMPART; CR = 12	Use a non-SISTEMA file-owning account with privilege 3 or higher.
Error sintactico	ATRIB attribute-only example; CR = 8	Supply one filename for an attribute change; reserve the two-name form for renaming.
Falta tipo del fichero	AYUDA BORRAR fallback; CR = 8	Use the BORRAR reference. AYUDA has invoked the utility with AYUD instead of displaying its topic.
COMU1 no esta asociado	BSC AYUDA; CR = 16	Use AYUDA BSC for central help. The utility itself requires compatible COMU1 setup.
Opcion no valida	BASIC with COMMA or COMA; CR = 8	These compiler options are rejected in this version. Use dot-decimal source.
FALTAN PARENTESIS	Four-argument DEL$ example; BASIC correction editing	Use the demonstrated three-argument form in Appendix D. Do not assume that a CSI prompt follows this BASIC diagnostic.

Displayed text	Operation and result	Corrective action
NECESITA EXPRESION STRING	NBR with numeric argument; BASIC correction editing	Supply a string expression as in NBR. The returned value's meaning remains unspecified.
[27] Fich. Cerrado	LINE(1), PAGE(1) or POS(1) with no channel open	The bare example lacks an open-channel context. Acceptance of its syntax does not establish a useful query result.

Stored message catalogue

Record	Exact stored text (escaped controls)	Meaning	Subsystem	Cause or corrective action
1	`[1] Error sintactico\x0D`	Syntax error	Command/files	Check required arguments; bare ESTADO is an observed cause.
2	`[2] Falta nombre del fichero\x0D`	Missing filename	Command/files	Supply the required filename; see the command syntax.
3	`[3] Falta tipo del fichero\x0D`	Missing file type	Command/files	Include the type after a dot, such as NOTE.TEXT.
4	`[4] Falta codigo de unidad\x0D`	Missing drive code	Command/files	Include the drive after a colon and check ASOCIAR.
5	`[5] Disco no asociado\x0D`	Disk not associated	Command/files	Inspect ASOCIAR and the requested drive.
6	`[6] Disco no montado\x0D`	Disk not mounted/ recognized as the expected medium	Command/files	Check media identity and association; MONTAR is the notification interface.
7	`[7] Opcion no valida\x0D`	Invalid option	Command/files	Check the option spelling and the command-specific option list.
8	`[8] Fichero no encontrado\x0D`	File not found	Command/files	Check spelling, drive association and current account before creating a replacement.
9	`[9] Disco lleno\x0D`	Disk full or no adequate allocation run	Command/files	Compare both total free blocks and largest contiguous run with ESTADO DISCO.
10	`[10] Directorio lleno\x0D`	Directory full according to the lookup path	Command/files	Preserve the catalogue; a short probe cycle can exhaust lookup before physical capacity.
11	`[11] Nombre de fichero no valido\x0D`	Invalid filename	Command/files	Check both name and explicit type; BASIC compilation needs the documented full form.
12	`[12] Codigo de unidad no valido\x0D`	Invalid drive code	Command/files	Use an assigned disk letter accepted by the command.
13	`[13] Fichero "{1}.{2}" no encontrado\x0D`	Named file not found	Command/files	Check both fields of the named file and the selected drive.

Record	Exact stored text (escaped controls)	Meaning	Subsystem	Cause or corrective action
14	`[14] "{1}" no es reconocible\x0D`	Token not recognized	Command/files	Check the displayed token against the command vocabulary.
15	`[15] "{1}" es un numero no valido\x0D`	Invalid numeric value	Command/files	Check the numeric argument against the permitted range and notation.
16	`[16] Esperando "{1}", encontrado "{2}"\x0D`	Expected one token but found another	Command/files	Supply the expected token in the documented position.
17	`[17] Esperando "{1}", encontrado fin de linea\x0D`	Expected token but reached end of line	Command/files	Complete the command with the expected argument.
18	`[18] Fichero protegido\x0D`	File protected	Command/files	Check owner and the protection relevant to this operation.
19	`[19] Privilegio insuficiente\x0D`	Insufficient privilege	Command/files	Check ESTADO PRIVILEG and the command/runtime guard; do not alter policy merely to hide the diagnostic.
20	`[20] Numero de serie incorrecto\x0D`	Incorrect installation serial number	Command/files	Check the executable binding against installation identity; this is not a serial-port fault.
21	`[21] Esperando fin de linea, encontrado "{1}"\x0D`	Expected end of line but found another token	Command/files	Caller-specific; no universal corrective operation is specified.
22	`[22] Falta parametro necesario\x0D`	Required parameter missing	Command/files	Caller-specific; no universal corrective operation is specified.
23	`[23] Debe ser "ON" u "OFF"\x0D`	Value must be ON or OFF	Command/files	Caller-specific; no universal corrective operation is specified.
24	`[24] Formato incorrecto, deberia ser "{1}"\x0D`	Wrong format; required format is substituted	Command/files	Caller-specific; no universal corrective operation is specified.
25	`[25] No se puede desasociar la CONSOLA\x0D`	Console cannot be disassociated	Device/media	Use bare ASOCIAR to list; ASOCIAR CONSOLA attempts an impermissible removal.
26	`[26] Registro no valido en "{1}.{2}"\x0D`	Invalid record in named file	Device/media	Caller-specific; no universal corrective operation is specified.
27	`[27] No se puede asociar a un dispositivo de solo salida\x0D`	Cannot associate an output-only device for this use	Device/media	Caller-specific; no universal corrective operation is specified.
28	`[28] No se puede asociar a un dispositivo de solo entrada\x0D`	Cannot associate an input-only device for this use	Device/media	Caller-specific; no universal corrective operation is specified.
29	`[29] Solo se puede asociar a un dispositivo de acceso aleatorio\x0D`	Random-access device required	Device/media	Caller-specific; no universal corrective operation is specified.
30	`[29] No se puede asociar a un dispositivo de acceso aleatorio\x0D`	Random-access device is not permitted for this use	Device/media	Caller-specific; no universal corrective operation is specified.

Record	Exact stored text (escaped controls)	Meaning	Subsystem	Cause or corrective action
31	`[31] No se puede desasociar el disco del sistema\x0D`	System disk cannot be disassociated	Device/ media	Caller-specific; no universal corrective operation is specified.
32	`[32] Dispositivo logico no definido\x0D`	Logical device undefined	Device/ media	Caller-specific; no universal corrective operation is specified.
33	`[33] Dispositivo fisico no definido\x0D`	Physical device undefined	Device/ media	Caller-specific; no universal corrective operation is specified.
34	`[34] Debe ser organizacion secuencial\x0D`	Sequential organization required	Program/ runtime	Caller-specific; no universal corrective operation is specified.
35	`[35] El fichero ya existe\x0D`	File already exists	Command/files	Caller-specific; no universal corrective operation is specified.
36	`[36] Falta el codigo de organizacion\x0D`	Organization code missing	Command/files	Caller-specific; no universal corrective operation is specified.
37	`[37] Falta opcion: "{1}"\x0D`	Required option missing	Command/files	Caller-specific; no universal corrective operation is specified.
38	`[38] {1} no esta asociado\x0D`	Named device not associated	Device/ media	Caller-specific; no universal corrective operation is specified.
39	`[39] El dispositivo no esta asociado\x0D`	Device not associated	Device/ media	Caller-specific; no universal corrective operation is specified.
40	`[40] El canal del fichero no esta asignado\x0D`	File channel not assigned	Program/ runtime	Caller-specific; no universal corrective operation is specified.
41	`[41] Ningun fichero encontrado\x0D`	No matching file found	Command/files	Check account, protection and exact versus wildcard lookup before declaring bytes absent.
42	`[42] {1} fichero(s), {2} bloque(s)\x0D`	File/block count summary	Command/files	Caller-specific; no universal corrective operation is specified.
43	`[43] Debe haber al menos dos discos asociados\x0D`	At least two disks must be associated	Device/ media	Caller-specific; no universal corrective operation is specified.
44	`[44] Las unidades fuente y destino no pueden ser las mismas\x0D`	Source and destination cannot be the same unit	Device/ media	Caller-specific; no universal corrective operation is specified.
45	`[45] Ningun fichero borrado\x0D`	No file deleted	Command/files	Caller-specific; no universal corrective operation is specified.
46	`[46] {1} fichero(s) borrado(s), {2} bloque(s) recuperado(s)\x0D`	Deleted-file and specified-block summary	Command/files	Caller-specific; no universal corrective operation is specified.
47	`[47] Fuente y destino no tienen la misma capacidad\x0D`	Source and destination capacities differ	Device/ media	Caller-specific; no universal corrective operation is specified.

Record	Exact stored text (escaped controls)	Meaning	Subsystem	Cause or corrective action
48	`[48] Receptor desconectado \x0D`	Receiver disconnected	Device/media	Caller-specific; no universal corrective operation is specified.
49	`[49] Receptor timed out\x0D`	Receiver timed out	Device/media	Caller-specific; no universal corrective operation is specified.
50	`[50] Debe ser fichero privado \x0D`	Private file required	Command/files	Caller-specific; no universal corrective operation is specified.
51	`[51] Modulo de carga con referencias externas no resueltas\x0D`	Load module has unresolved external references	Program/runtime	Caller-specific; no universal corrective operation is specified.
52	`[52] No se pueden mezclar modulos ABS con REL o COM\x0D`	ABS modules cannot be mixed with REL or COM	Program/runtime	Caller-specific; no universal corrective operation is specified.
53	`[53] No puede tener varios modulos ABS\x0D`	More than one ABS module is not permitted	Program/runtime	Caller-specific; no universal corrective operation is specified.
54	`[54] Overflow en tabla de simbolos\x0D`	Symbol-table overflow	Program/runtime	Caller-specific; no universal corrective operation is specified.
55	`[55] Fin de linea no esperado \x0D`	Unexpected end of line	Program/runtime	Caller-specific; no universal corrective operation is specified.
56	`[56] El programa no tiene PABs!\x0D`	Program has no PABs; exact structure meaning unresolved	Program/runtime	Caller-specific; no universal corrective operation is specified.
57	`[57] Solo los ficheros privados pueden ser compartidos\x0D`	Only private files can be shared	Account/session	Caller-specific; no universal corrective operation is specified.
58	`[58] Falta el fichero SISTEMA.CUENTA\x0D`	SISTEMA.CUENTA is missing	Account/session	Restore account-file reachability and identity from a preserved copy.
59	`[59] La cuenta sistema no puede compartir ficheros\x0D`	System account cannot share files through this interface	Account/session	Caller-specific; no universal corrective operation is specified.
60	`[60] Palabra de paso incorrecta\x0D`	Incorrect password	Account/session	Re-enter the correct account password; wrong-password rejection is documented.
61	`[61] Cuenta no encontrada\x0D`	Account not found	Account/session	Caller-specific; no universal corrective operation is specified.
62	`[62] El fichero "{1}.{2}:{3}" ya existe en la cuenta "{4}"\x0D`	Named file already exists in the specified account	Account/session	Caller-specific; no universal corrective operation is specified.
63	`[63] El spooler no esta inicializado\x0D`	Spooler not initialized	Spooler	The inactive ESTADO path returns result 12; do not infer that the resident code is absent.

Record	Exact stored text (escaped controls)	Meaning	Subsystem	Cause or corrective action
64	`[64] El spooler esta imprimiendo el fichero #{1}\x0D`	Spooler is printing the numbered report	Spooler	Caller-specific; no universal corrective operation is specified.
65	`[65] El spooler esta esperando trabajo\x0D`	Spooler waiting for work	Spooler	Caller-specific; no universal corrective operation is specified.
66	`[66] Spooler imprimiendo fichero #{1}, impresora desconectada!\x0D`	Spooler printing but printer disconnected	Spooler	Caller-specific; no universal corrective operation is specified.
67	`[67] Spooler suspendido \x0D`	Spooler suspended	Spooler	Caller-specific; no universal corrective operation is specified.
68	`[68] No hay suficiente memoria\x0D`	Insufficient memory	Partition/ message	Inspect the current ceiling and loaded drivers/classes before changing allocation.
69	`[69] El dispositivo esta asociado a la particion {1}\x0D`	Device associated with another partition	Partition/ message	Caller-specific; no universal corrective operation is specified.
70	`[70] Numero de particion no valido \x0D`	Invalid partition number	Partition/ message	Caller-specific; no universal corrective operation is specified.
71	`[71] La particion no esta ubicada\x0D`	Partition not allocated	Partition/ message	Caller-specific; no universal corrective operation is specified.
72	`[72] La particion aun esta activa\x0D`	Partition still active	Partition/ message	Caller-specific; no universal corrective operation is specified.
73	`[73] La particion no esta activa\x0D`	Partition not active	Partition/ message	Caller-specific; no universal corrective operation is specified.
74	`[74] No se puede enviar un mensaje a su propia particion\x0D`	Cannot send a message to one's own partition	Partition/ message	Choose another partition; self delivery is refused by the utility.
75	`[75] El usuario no esta conectado \x0D`	User not connected	Partition/ message	Verify the recipient session; stored delivery is a separate path.
76	`[76] El usuario no esta recibiendo mensajes\x0D`	User not accepting messages	Partition/ message	Addressed-user message inhibition can cause refusal; wildcard/API policy differs.

Record	Exact stored text (escaped controls)	Meaning	Subsystem	Cause or corrective action
77	`[77] Debe ser modo unico usuario o volumen de disco privado \x0D`	Single-user mode or a private disk volume required	Partition/ message	Caller-specific; no universal corrective operation is specified.
78	`[78] Debe ser modo unico usuario\x0D`	Single-user mode required	Partition/ message	Caller-specific; no universal corrective operation is specified.
79	`[79] El spooler debe ser parado\x0D`	Spooler must be stopped	Spooler	Caller-specific; no universal corrective operation is specified.
80	`[80] No se ha cambiado ningun fichero\x0D`	No file changed	Command/files	Caller-specific; no universal corrective operation is specified.
81	`[81] El fichero "{1}.{2}: {3}" pertenece ahora a la cuenta "{4}"\x0D`	File ownership transferred to named account	Command/files	Caller-specific; no universal corrective operation is specified.
82	`[82] Particion demasiado pequena para activar\x0D`	Partition too small to activate	Partition/ message	Caller-specific; no universal corrective operation is specified.
83	`[83] Particion demasiado grande\x0D`	Partition too large	Partition/ message	Caller-specific; no universal corrective operation is specified.
84	`[84] El numero de particion debe ser mayor que el actual\x0D`	Partition number must exceed the current number	Partition/ message	Caller-specific; no universal corrective operation is specified.
85	`[85] Solo se puede modificar la particion mas alta\x0D`	Only the highest partition can be modified	Partition/ message	Caller-specific; no universal corrective operation is specified.
86	`[86] La particion anterior todavia esta activa\x0D`	Previous partition still active	Partition/ message	Caller-specific; no universal corrective operation is specified.
87	`[87] El modulo {1} no esta cargado \x0D`	Named module not loaded	Partition/ message	Caller-specific; no universal corrective operation is specified.
88	`[88] El modulo {1} ya esta cargado\x0D`	Named module already loaded	Partition/ message	Caller-specific; no universal corrective operation is specified.
89	`[89] Demasiados modulos re-entrantes\x0D`	Too many reentrant modules	Partition/ message	Caller-specific; no universal corrective operation is specified.

Record	Exact stored text (escaped controls)	Meaning	Subsystem	Cause or corrective action
90	`[90] "{1}.{2}:{3}" salvado y fin edicion\x0D`	File saved and editing ended	Editor status	Caller-specific; no universal corrective operation is specified.
91	`[91] "{1}.{2}:{3}" salvado\x0D`	File saved	Editor status	Caller-specific; no universal corrective operation is specified.
92	`[92] ** La particion ha sido parada ! ** \x07\x0D`	Partition has been stopped; includes bell	Partition/ message	Caller-specific; no universal corrective operation is specified.
93	`[93] Esta cuenta esta siendo utilizada por la particion {1}\x0D`	Account in use by another partition	Account /session	Caller-specific; no universal corrective operation is specified.
94	`[94] No se puede ESPIAR a si mismo !\x0D`	Cannot observe one's own console with ESPIAR	Partition/ message	Caller-specific; no universal corrective operation is specified.
95	`[95] No puede ejecutar FORZAR en su particion ! \x0D`	Cannot FORZAR one's own partition	Partition/ message	Caller-specific; no universal corrective operation is specified.
96	`[96] Todas las particiones estan activas\x0D`	All partitions active	Partition/ message	Caller-specific; no universal corrective operation is specified.
97	`[97] Tiene mensajes esperando. Para recibirlos ejecute BUZON\x0D`	Stored messages waiting; BUZON retrieves them	Partition/ message	BUZON retrieves and removes stored messages; preserve the file for non-consuming inspection.
98	`[98] Debe ser OASIS multi-usuario\x0D`	Multiuser OASIS required according to original text	Partition/ message	Caller-specific; no universal corrective operation is specified.
99	`\x0D`	CR-only record; no diagnostic text	Program/ runtime	Caller-specific; no universal corrective operation is specified.
100	`\x0D`	CR-only record; no diagnostic text	Program/ runtime	Caller-specific; no universal corrective operation is specified.
101	`\x0D`	CR-only record; no diagnostic text	Program/ runtime	Caller-specific; no universal corrective operation is specified.
102	`\x0D`	CR-only record; no diagnostic text	Program/ runtime	Caller-specific; no universal corrective operation is specified.
103	`[103] El spooler esta planificando el siguiente trabajo\x0D`	Spooler selecting its next job	Spooler	Caller-specific; no universal corrective operation is specified.

Record	Exact stored text (escaped controls)	Meaning	Subsystem	Cause or corrective action
104	`[104] Form "{1} {2} {3} {4} {5} {6} {7} {8}" montado en spooler\x0D`	Form mounted in spooler	Spooler	Caller-specific; no universal corrective operation is specified.
105	`[105] El spooler esta parado\x0D`	Spooler stopped	Spooler	Caller-specific; no universal corrective operation is specified.
106	`[106] El fichero de spool se esta imprimiendo actualmente\x0D`	Spool file currently printing	Spooler	Caller-specific; no universal corrective operation is specified.
107	`[107] El fichero de spool esta abierto o no existe\x0D`	Spool file open or absent	Spooler	Caller-specific; no universal corrective operation is specified.
108	`[108] Parametros spool: Cola={1}, Copias={2}, {3}\x0D`	Queue/copies/retention parameter summary	Spooler	Caller-specific; no universal corrective operation is specified.
109	`[109] Conectado a las {1}, del {2}\x0D`	Connection time/date	Account/session	Caller-specific; no universal corrective operation is specified.
110	`[110] Desconectado a las {1}, el dia {2}. Conectado {3} minutos\x0D`	Disconnection time/date and session duration	Account/session	Caller-specific; no universal corrective operation is specified.
111	`[111] Esperando parada del spooler\x0D`	Waiting for spooler to stop	Spooler	Caller-specific; no universal corrective operation is specified.
112	`[112] Codigo de cola y formulario del spooler no coinciden\x0D`	Queue and form codes do not match	Spooler	Caller-specific; no universal corrective operation is specified.
113	`[113] No se puede ejecutar FORZAR sobre el usuario\x0D`	FORZAR not permitted for that user	Partition/message	Caller-specific; no universal corrective operation is specified.
114	`[114] Nombre de cuenta a conectarse: |\x0D`	Account-name login prompt	Account/session	Caller-specific; no universal corrective operation is specified.
115	`[115] Palabra de paso? |\x0D`	Password prompt	Account/session	Caller-specific; no universal corrective operation is specified.
116	`[116] Overflow en la tabla {1}{2}-no. excesivo de atributos\x0D`	Attribute-table overflow	Application/query	Caller-specific; no universal corrective operation is specified.
117	`|\x0D`	Literal vertical-bar formatting record	Application/query	Caller-specific; no universal corrective operation is specified.
118	`[118] Imposible cargar el overlay requerido\x0D`	Required overlay cannot be loaded	Application/query	Check required overlay file, format and runtime memory; exact trigger coverage is incomplete.
119	`[119] Imposible abrir/cerrar el fichero despues del overlay1\x0D`	File open/close failed after overlay1	Application/query	Caller-specific; no universal corrective operation is specified.

Record	Exact stored text (escaped controls)	Meaning	Subsystem	Cause or corrective action
120	`[120] Imposible cargar el ACB {1}.{2}\x0D`	Named ACB cannot be loaded; complete ACB contract unresolved	Application/ query	Caller-specific; no universal corrective operation is specified.
121	`[121] {1}.{2} tipo de fichero no procesable por ACCESO\x0D`	File type cannot be processed by ACCESO	Application/ query	Caller-specific; no universal corrective operation is specified.
122	`[122] Imposible abrir el fichero {1}.{2}\x0D`	Cannot open the named file	Application/ query	Caller-specific; no universal corrective operation is specified.
123	`[123] La frase {1}.{2} es erronea\x0D`	Query/statement file is erroneous	Application/ query	Caller-specific; no universal corrective operation is specified.
124	`[124] {1}.{2} no encontrado en {3}.{4}\x0D`	Named item not found within the specified file	Application/ query	Caller-specific; no universal corrective operation is specified.
125	`[125] Tipo de fichero invalido en {1}.{2}\x0D`	Invalid file type	Application/ query	Caller-specific; no universal corrective operation is specified.
126	`[126] Todas las entradas al vocabulario deben ser textos\x0D`	Vocabulary entries must be text	Application/ query	Caller-specific; no universal corrective operation is specified.
127	`[127] Error en un operador de relacion\x0D`	Relational-operator error	Application/ query	Caller-specific; no universal corrective operation is specified.
128	`[128] Imposible leer el siguiente registro de "{1}.{2}"\x0D`	Cannot read the next record	Application/ query	Caller-specific; no universal corrective operation is specified.
129	`[129] Error de sintaxis en una correlacion\x0D`	Correlation syntax error	Application/ query	Caller-specific; no universal corrective operation is specified.
130	`[130] Correlacion "&STK" erronea\x0D`	Invalid &STK correlation	Application/ query	Caller-specific; no universal corrective operation is specified.
131	`[131] Operador de correlacion indefinido\x0D`	Undefined correlation operator	Application/ query	Caller-specific; no universal corrective operation is specified.
132	`[132] Correlacion de relacion indefinida\x0D`	Undefined relation correlation	Application/ query	Caller-specific; no universal corrective operation is specified.
133	`[133] Una correlacion no puede referenciar a ese dato\x0D`	Correlation cannot reference this datum	Application/ query	Caller-specific; no universal corrective operation is specified.
134	`[134] Falta el delimitador de fin de literal\x0D`	Literal closing delimiter missing	Application/ query	Caller-specific; no universal corrective operation is specified.

Record	Exact stored text (escaped controls)	Meaning	Subsystem	Cause or corrective action
135	`[135] El fichero destino de la translacion debe ser I o D\x0D`	Translation destination must have I or D organization	Application/ query	Caller-specific; no universal corrective operation is specified.
136	`[136] Elemento referenciado del stack fuera de limites\x0D`	Referenced stack element outside bounds	Application/ query	Caller-specific; no universal corrective operation is specified.
137	`[137] Codigo de retorno de la translacion erroneo\x0D`	Invalid translation return code	Application/ query	Caller-specific; no universal corrective operation is specified.
138	`[138] Error en el tipo de fichero de la correlacion\x0D`	Wrong file type in correlation	Application/ query	Caller-specific; no universal corrective operation is specified.
139	`[139] Error de lectura en la translacion\x0D`	Translation read error	Application/ query	Caller-specific; no universal corrective operation is specified.
140	`[140] Imposible grabar en "{1}.{2}"\x0D`	Cannot write the named file	Application/ query	Caller-specific; no universal corrective operation is specified.
141	`[141] \x0D`	Number-only record; no explanatory text	Application/ query	Stored record contains only its bracketed number and spacing.
142	`[142] No existe tal banco\x0D`	Requested bank does not exist	Partition/ message	Caller-specific; no universal corrective operation is specified.
143	`[143] Parametros numericos fuera de rango\x0D`	Numeric parameters outside range	Command/files	Caller-specific; no universal corrective operation is specified.
144	`[144] El spooler no puede ser descargado !!\x0D`	Spooler cannot be unloaded	Spooler	Caller-specific; no universal corrective operation is specified.
145	`[145] No se puede archivar sobre el disco del sistema !!\x0D`	Cannot archive onto the system disk	Device/ media	Select a separate archive destination; do not select the system disk as the archive destination.

Command results and BASIC errors

Value or text	Established meaning and action
CSI CR0	Normal reported completion in selected paths; does not prove flush, delivery or physical I/O
CSI CR8	Several syntax/device/file refusals; use accompanying text
CSI CR12	Several search/storage/spool refusals; not a unique kernel error
CSI CR252	Standard command privilege/serial prologue denial

Value or text	Established meaning and action
CSI CR253	BASIC runtime error path, including policy or computed-branch failure
CSI CR254	Active interruption/disk-abort paths; also CLAVIN startup failure. The code alone does not identify the cause. Protected CLOSE can instead report 0
BASIC `[30] Fich. no Esta`	Missing or unreachable file, including early 00-hidden entries
BASIC `[32] Directori.Llen`	Directory lookup exhaustion; other catalogue slots can remain unused
BASIC error 33	Established OPEN/WRITE protection refusal; distinguish operation and owner
BASIC `[40] Programa no esta en posicion 4CED`	Observed unsuccessful RUN; verify BASICCOM and runtime path rather than treating 4CED as a fixed universal address
BASIC `[41] Format Fich Invalid`	Invalid record type; ignored I/O/latched EOF can leave stale destination bytes that BASIC interprets as a type
Computed ON error 0x14	Selector 0 on the specified branch path; ordinary result 253

Native checker diagnostics

Diagnostic text or parameterized form	Meaning	Action
`***** Fin de fichero incorrecto`	Traversed final S sector disagrees with directory end	Back up the chain and entry; F does not guarantee reconstruction of the final sector
`La longitud del fichero no coincide con el directorio`	Rounded chain extent differs from declared allocation	Keep both interpretations and prior metadata
`Sector <n> ocupado pero no pertenece a ningun fichero`	The numbered sector is allocated but has no owner in the checker scan	Preserve candidate data before F frees the bits

The checker does not necessarily report a stale sequential count, duplicate identity or application-level EOF gap. Its silence is not a complete filesystem certification.

Disk-driver status

The specified driver vocabulary includes 01 not ready (`no prep.`), 02 write-protected, 03 uninitialized, 04 transfer error, 07 sector/ID failure, 08 track failure and 09 head failure. Exact meanings vary with driver translation and not every physical failure has a complete defined contract. R retries; M invokes the driver's recalibration/reset slot and retries; I ignores; Q aborts nonlocally. The installed WD M slot is RET. Ignoring does not reconstruct data. See Chapter 27 for callback policy and the protected-CLOSE result-zero exception.

Chapter 34 — Devices and system limits

Logical names designate uses within a partition. Physical names select installed definitions, which may describe optional hardware not fitted to the machine. The presence of a name is therefore weaker evidence than a successful device operation.

Logical association index

Index (hex)	Name or role
00	S, system disk
01–07	A–G, other disks
08	Console input
09	Console output
0A–0B	Remaining slots not fully classified here
0C–0F	IMPR1–IMPR4
10	COMU1 in the established caller
11–13	Further communication slots; full usage not independently exercised
14	TAPE1 in DIS10 initialization
15–17	Remaining slots; complete active-device mapping unknown

The association byte is zero-based physical descriptor index, withFF unassigned. The displayed definition number is index+1. Thus installed DISCO1/device 3 resolves to descriptor index 2, at 0x0069+2*32=0x00A9. ASOCIAR manipulates logical-to-physical assignment; ASIGNAR assigns application channels. A filename uses `name.type:drive`, with S and A–G interpreted through the current partition.

Physical names

Name	Definition number	Driver/class and availability
FLOPPY1, FLOPPY2	1,2	DIS1/2 with separate controller paths; Chapter 11 covers access and the FLOPPY2 format; fitted mechanism and bootstrap compatibility require matching hardware
DISCO1–DISCO6	3–8	Disk definitions; S association is configuration-dependent
TAP	10	DIS10; controller/physical tape operation unresolved

Name	Definition number	Driver/class and availability
IP, CDC	11,12	Printer drivers; printer-driver definitions
PAN9, PAN10; IS9, IS10	16,17	DIS16/17 on the SS2 SCC; 128-byte input rings and polled output; hardware and matching terminal framing required
PAN1–PAN4	18–21	ALP consoles; four established serial ports
IMP	22	DIS22 serial printer interface using the SCN2651 register layout and modem readiness; physical printer model unspecified
PAN5–PAN8; IS5–IS8	23–26	Optional second ALP; optional hardware
COM1–COM4	27–30	Definitions; corresponding complete operational drivers not established

IS2/IS3/IS4 share the corresponding PAN2/PAN3/PAN4 driver files. Their names do not imply that every default mode or simultaneous alias configuration has been specified. Appendix B gives exact driver sizes, entry offsets, ports and callback indices.

Canonical limits

Quantity	Established value	Scope
CPU address space	65,536 bytes	Z80 visible window
Implemented RAM	256 KiB:16 common +5×48 banked	Example five-bank configuration
Example physical disk	830×5×32×256 =33,996,800 bytes	Example geometry; verify fitted device
Logical volume sectors	62,048	Example volume geometry
Allocation blocks	15,512 of 1,024 bytes	Example volume geometry
Sector/payload	256 physical/logical;254 S payload	Established formats
Catalogue slots	1,208 for 151 directory sectors	151×8; probe defect can lower reachable capacity
Filename/type	Eight bytes each	Space-padded; normalization restrictions apply
Timestamp years	1977–1992	Four-bit year offset; not full hardware RTC range
Associations	24 per partition	Context layout
Application channel indices	0–16	17 entries; user ASIGNAR help exposes 1–16
Scheduler rows	16	Table and public numbering; sixteen active partitions not proven
Account name	Eight characters in CUENTA	Use an initial letter followed by letters/digits for ordinary new names

Quantity	Established value	Scope
Account password	Eight-character field in CUENTA	Input removes spaces; comma rejected; Chapter 10 explains blank and # answers
Account numbers	CUENTA allocates new distinct numbers through 250; CAMCU help permits 0–254; FF is special	Different command ranges; maximum simultaneous accounts/sessions not established
Privilege values	0–5 ordinary documented/specified range	Additional internal 6/FF checks do not establish user levels
Default terminal fields	L79/P23	Input budget and cursor thresholds, not generic display dimensions
EXEC arguments	16×8 bytes	17th supplied field excluded in specified case
EXEC variables	16 name 8/value 8 cells per frame	Layout; all exhaustion behavior not established
EXEC frame	512 bytes	Installed work files offer four frames each
Record-lock table	64 usable +terminator	Code/layout; full-table behavior not induced
File-open table	129 physical 4-byte rows including needed terminator storage	Effective saturation behavior unknown
I/K record field	R=(packed&511)+1	Representational 1–512; creation/runtime limits may be narrower
I/K key field	K=(packed>>9)+1	Representational 1–128; complete extreme-value behavior unknown
BASIC decimal	13 significant decimal digits in specified representation	Arithmetic rounding/range not fully established
Maximum program size	Unknown as one constant	Depends on partition, ceiling, drivers/classes and runtime
General simultaneous open-file maximum	Unknown	Channel count and shared table capacity are distinct
Spool queue	64 stored records; producer appears to wrap 1–63	Code-derived effective capacity, native saturation untested
Safe queue arithmetic range	1–128 under valid serialized state	Derived avoidance of specific defects, not an advertised OS maximum

Capacity fields must not be promoted to guaranteed usable maxima without checking sentinel rows, reserved records and algorithms. Conversely, a small example does not establish the system's maximum.

Chapter 35 — Command spelling and language reference

The system has several command languages. CSI selects a system command; each utility then interprets its own arguments. EDITAR, EXEC, CONSULTA and BASIC each recognize a separate vocabulary. This chapter supplies spelling and lookup details after the working introductions in Chapters 5, 7 and 8. BASIC's language reference remains in Appendix D.

Full command names and permitted abbreviations

Use the full names in the Command column when following procedures. Most commands accept every prefix between their listed minimum and full name. The Accepted spellings column lists the exact alternatives so that exceptions are visible. Recognition depends on abbreviation being enabled and the standard table being selected. A recognized name does not create an executable that is absent.

BASIC is an exception: B selects BASIC, BA selects BACKUP, and BAS through BASIC select BASIC. EXEC has a separate argument-transfer limitation: EX and EXE are recognized but can run a command file named EX or EXE instead of the requested file. Always enter EXEC in full. DESCARGAR is a nine-character table name constrained by an eight-character command field; it has no available matching executable.

Command	Accepted standard spellings	Notes
ACTIVAR	ACTIV, ACTIVA, ACTIVAR	Standard command.
ARCHIVAR	AR, ARC, ARCH, ARCHI, ARCHIV, ARCHIVA, ARCHIVAR	Standard command.
ASIGNAR	A, AS, ASI, ASIG, ASIGN, ASIGNA, ASIGNAR	Standard command.
ASOCIAR	ASO, ASOC, ASOCI, ASOCIA, ASOCIAR	Standard command.
ATRIB	ATR, ATRI, ATRIB	Standard command.
AYUDA	AY, AYU, AYUD, AYUDA	Standard command.
BACKUP	BA, BAC, BACK, BACKU, BACKUP	Standard command.
BASIC	B, BAS, BASI, BASIC	BA selects BACKUP.
BORRAR	BO, BOR, BORR, BORRA, BORRAR	Standard command.
BUSCAR	BUSC, BUSCA, BUSCAR	Standard command.
BUZON	BUZ, BUZO, BUZON	Standard command.
CAMBIAR	CAM, CAMB, CAMBI, CAMBIA, CAMBIAR	Standard command.
CAMPRO	CAMPRO	Standard command.

Command	Accepted standard spellings	Notes
CARGAR	`CAR`, `CARG`, `CARGA`, `CARGAR`	Executable absent; resident-loading reference in SPOOLER.
CLASIF	`CL`, `CLA`, `CLAS`, `CLASI`, `CLASIF`	Executable absent; does not redirect to SORT.
COMPART	`COMPART`	Standard command.
CONECTAR	`CONEC`, `CONECT`, `CONECTA`, `CONECTAR`	Standard command.
COPIFI	`C`, `CO`, `COP`, `COPI`, `COPIF`, `COPIFI`	Standard command.
CREAR	`CRE`, `CREA`, `CREAR`	Standard command.
CUENTA	`CUEN`, `CUENT`, `CUENTA`	Standard command.
DESCARGAR	`DESCAR`, `DESCARG`, `DESCARGA`	Executable and EXEC file absent; lookup name DESCARGA.
EDITAR	`ED`, `EDI`, `EDIT`, `EDITA`, `EDITAR`	Standard command.
ELIMINAR	`ELIM`, `ELIMI`, `ELIMIN`, `ELIMINA`, `ELIMINAR`	Standard command.
ESTADO	`EST`, `ESTA`, `ESTAD`, `ESTADO`	Standard command.
EXEC	`EX`, `EXE`, `EXEC`	Use EXEC in full; short forms can select the wrong file.
FIN	`FIN`	Standard command.
FORZAR	`FORZ`, `FORZA`, `FORZAR`	Standard command.
GENSIST	`GENS`, `GENSI`, `GENSIS`, `GENSIST`	Standard command.
IBMSEC	`IBMSEC`	Standard command.
INICIAR	`INI`, `INIC`, `INICI`, `INICIA`, `INICIAR`	Standard command.
LISDIR	`LISD`, `LISDI`, `LISDIR`	Standard command.
LISTAR	`LIST`, `LISTA`, `LISTAR`	Standard command.
MENSAJE	`MENS`, `MENSA`, `MENSAJ`, `MENSAJE`	Standard command.
MONTAR	`MON`, `MONT`, `MONTA`, `MONTAR`	Standard command.
PARAR	`PARAR`	Standard command.
PONER	`PON`, `PONE`, `PONER`	Standard command.
RECARGAR	`RECAR`, `RECARG`, `RECARGA`, `RECARGAR`	Standard command.
REPARAR	`REP`, `REPA`, `REPAR`, `REPARA`, `REPARAR`	Standard command.
REVIVIR	`REV`, `REVI`, `REVIV`, `REVIVI`, `REVIVIR`	Standard command.
RUN	`R`, `RU`, `RUN`	Standard command.
SECIBM	`SECIBM`	Standard command.
SECTOR	`SEC`, `SECT`, `SECTO`, `SECTOR`	Standard command.

Command	Accepted standard spellings	Notes
SPOOLER	SP, SPO, SPOO, SPOOL, SPOOLE, SPOOLER	Standard command.
TEDITAR	TED, TEDI, TEDIT, TEDITA, TEDITAR	Standard command.
VOLCAR	VOLC, VOLCA, VOLCAR	Standard command.

The following executable names have no standard abbreviation. They are already the full names of their commands, even where their spelling is compact:

ARCHDIR, BSC, CAMCU, CEDIT, CIFRA, CLAVIN, CONSULTA, COPFICH, CORLUZ, DIRIBM, DUPLICAR, ENVIAR, ENVIARA, ESPIAR, FISDEF, HASH, INICDISC, INICIBM, INITTAPE, LOGDEF, MODIMPR, O8PRECOM, ORGDIR, PANTALLA, RECIBIR, RECIBIRA, SERIAL, SORT, TECLAS, TERMINAL, TITULO, UNOSEC, VERFORM.

LCOMPART has a help topic but no matching executable or standard table entry. CARGAR, CLASIF and DESCARGAR retain table entries but have no available executable or EXEC file; DESCARGAR is looked up as DESCARGA. CLASIF does not redirect to SORT. Conversely, CLAVIN is installed but returns CR 254 during startup in tested invocations. Their Chapter 31 entries distinguish recognized names, installed files and usable commands.

Enable or disable abbreviation

At CSI, enter:

```
PONER ABREV ON
```

This enables command-prefix expansion. To require complete command names, enter:

```
PONER ABREV OFF
```

Exact filenames and exact configured synonyms still work. These settings do not disable a utility's option abbreviations.

To select the built-in abbreviation table, enter:

```
PONER SINONIM (S)
```

To disable that table, enter:

```
PONER SINONIM (N)
```

Disabling the table does not rename command files. To display its entries, enter:

```
ESTADO SINONIMO
```

This lists the standard names and their advertised minima. There is a limitation when a user synonym file is selected: CSI reads type SINONIMO, but this ESTADO program looks for SINONIM. The listing can therefore end with file-not-found/CR 12 even when the custom synonyms work. Keep the working definition file's SINONIMO type.

User synonyms and filename precedence

A synonym supplies another name for a command. For example, a definition file named SHORT.SINONIMO can contain this line:

```
ESTADO SHOW 2
```

This is file content, not a CSI command: ESTADO is the real name, SHOW the synonym, and 2 the minimum synonym length. Create the file using the text-editing procedure in Chapter 7. Then select it at CSI:

```
PONER SINONIM SHORT
```

With abbreviation enabled, SHOW, SHO and SH select ESTADO unless an actual file takes precedence. With abbreviation disabled, the exact synonym SHOW still works. To cancel the custom definitions, enter:

```
PONER SINONIM (C)
```

Custom synonyms are considered before standard expansion. Within each searched disk association, CSI tries the original token's COMANDO file, then its EXEC file if implicit execution is enabled, before trying the expanded names. Only then does it continue to another association. Thus an actual ASO.COMANDO or enabled ASO.EXEC can take precedence over expansion to ASOCIAR. The full EXEC command follows its own earlier interpreter path. See Chapter 30 for loading.

Utility options: full keywords and shorter forms

The tables below record accepted spellings, not complete operating procedures. Use each command's Chapter 31 entry for purpose, arguments, precautions and examples. Recognition of an option does not establish that the requested hardware operation can complete. Full keywords are shown first; accepted abbreviations are alternatives rather than corrected names.

Most tabulated options accept the intermediate prefixes shown. Selection follows each utility's table order, so overlapping prefixes can select different operations. For example, COPIFI PA selects PANT; use PACK to request packing. SPOOLER SAL selects a different operation from SALVAR, although S and SA select SALVAR. These exceptions are recorded explicitly below.

A full keyword means the literal word recognized by that program. ESTADO uses HISTOR and INIREL, while PONER uses HISTORIA and INIRELOJ. They are separate interfaces. COPIFI's lowercase selector is LMINUSC; MINUSC and MINUSCU are not aliases. COPIFI's append keyword is literally A@ADIR, with an at sign. Its short form A has the same append function.

Options with character-based parsing

Some utilities inspect individual letters instead of using the tables below. Do not expand these forms into invented words.

Command	Form	Interpretation and restrictions
ASOCIAR	L79, P23, C2, B9600	Width, page, class and speed forms have attached numbers. L 79 and LINEA79 are not substitutes for L79.
ASOCIAR	LFnn, FFnn	Device-specific delay fields; separate from width and page dimensions.
ASOCIAR	W8	Accepted word-size form; W7 is rejected.
ASOCIAR	PE, PO, PP, ALF, PUBLIC	Parity, page/line handling and public association flags; see the command's device-specific options.
ASOCIAR	ENAB1–ENAB4, O, H, R, STP, STS	Individually parsed device settings; arguments and effects depend on the device.
ASOCIAR	SY, SD, WA	Recognized branches whose complete operator contracts remain unspecified.
ATRIB	LE	Read protection. L is also accepted; LEER is not. NOL requires privilege 255 and is not an unrestricted inverse operation.
ATRIB	E/NOE, B/NOB	Write- and deletion-protection controls.
BORRAR	PANT/NOPANT, CONSUL/NOCONSUL	Display and consultation controls. Exact-name deletion can proceed without a consultation prompt.
LISTAR	PA followed by page number	Page selection; other P-leading forms select printer output, with a final digit selecting printer 1–4.
LISDIR	FN, FT, FD, FO, FI, AP, NOS, NOH	Individual field, output and listing selectors; see LISDIR rather than applying CSI's abbreviation rules.

Use balanced option parentheses in examples. The tokenizer also accepts a carriage return as the end of options, which explains command files that omit the closing parenthesis. This does not make spaces optional inside numeric arguments: ASOCIAR uses L79, whereas CREAR uses REGISTRO 128.

PONER settings

A setting follows PONER and takes its own arguments. For example:

```
PONER IMPEXEC ON
```

This enables implicit EXEC-file lookup. It does not change the spelling rules inside an EXEC file.

Full setting	Accepted spellings
HORA	HO, HOR, HORA
PRIVLEV	PRIV, PRIVL, PRIVLE, PRIVLEV
ERRNUM	ERRN, ERRNU, ERRNUM
ERRTEXT	ERRT, ERRTE, ERRTEX, ERRTEXT
FECHA	FE, FEC, FECH, FECHA
ABREV	AB, ABR, ABRE, ABREV
MENSAJE	MEN, MENS, MENSA, MENSAJ, MENSAJE
TIEMPO	TI, TIE, TIEM, TIEMP, TIEMPO
MEMORIA	MEM, MEMO, MEMOR, MEMORI, MEMORIA
MODULO	MO, MOD, MODU, MODUL, MODULO
VERSION	VER, VERS, VERSI, VERSIO, VERSION
HISTORIA	H, HI, HIS, HIST, HISTO, HISTOR, HISTORI, HISTORIA
DIA	DIA
SINONIM	SIN, SINO, SINON, SINONI, SINONIM
INCOM	INC, INCO, INCOM
IMPEXEC	IMP, IMPE, IMPEX, IMPEXE, IMPEXEC
ESCAPE	ESC, ESCA, ESCAP, ESCAPE
SUBIR	SU, SUB, SUBI, SUBIR
BAJAR	BA, BAJ, BAJA, BAJAR
IZQ	IZ, IZQ
DER	DE, DER
RUBOUT	RU, RUB, RUBO, RUBOU, RUBOUT
CANCEL	CA, CAN, CANC, CANCE, CANCEL
RTCAVAIL	RTC, RTCA, RTCAV, RTCAVA, RTCAVAI, RTCAVAIL
SISFECHA	SIS, SISF, SISFE, SISFEC, SISFECH, SISFECHA
INIRELOJ	INI, INIR, INIRE, INIREL, INIRELO, INIRELOJ
QET	QET
FORFECHA	FORFE, FORFEC, FORFECH, FORFECHA

Full setting	Accepted spellings
ALEMAN	ALE, ALEM, ALEMA, ALEMAN

CREAR options

REGISTRO, NUMREG and LONCLA take their numeric argument in the following field. DIRECTO, INDEXADO and CLAVE select file organization; VACIAR changes existing contents and requires care. The full creation examples are in Chapter 17.

Full option	Accepted spellings
DIRECTO	D, DI, DIR, DIRE, DIREC, DIRECT, DIRECTO
INDEXADO	I, IN, IND, INDE, INDEX, INDEXA, INDEXAD, INDEXADO
CLAVE	C, CL, CLA, CLAV, CLAVE
REGISTRO	R, RE, REG, REGI, REGIS, REGIST, REGISTR, REGISTRO
LONCLA	LONC, LONCL, LONCLA
NUMREG	N, NU, NUM, NUMR, NUMRE, NUMREG
VACIAR	VA, VAC, VACI, VACIA, VACIAR

ARCHIVAR keywords

These keywords belong to ARCHIVAR. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
PANT	P, PA, PAN, PANT	—
NOPANT	NOP, NOPA, NOPAN, NOPANT	—
VERIFI	V, VE, VER, VERI, VERIF, VERIFI	—
NOVERIFI	NOV, NOVE, NOVER, NOVERI, NOVERIF, NOVERIFI	—
CUENTA	C, CU, CUE, CUEN, CUENT, CUENTA	—
CONSU	CO, CON, CONS, CONSU	C selects CUENTA
NOCONSU	NOC, NOCO, NOCON, NOCONS, NOCONSU	—
INF	IN, INF	—
NOINF	NOI, NOIN, NOINF	—
VOLUM	VOL, VOLU, VOLUM	—

BSC keywords

These keywords belong to BSC. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
COM1	COM1	—
COM2	COM2	—
COM3	COM3	—
COM4	COM4	—
COMU1	COMU1	—
COMU2	COMU2	—
COMU3	COMU3	—
COMU4	COMU4	Exact spelling is accepted despite the stored prefix threshold.
TEST	TEST	—
IMP	IMP	—
IMPR	IMP, IMPR	—
IMP1	IMP, IMP1	—
IMPR1	IMP, IMPR, IMPR1	—
IMP2	IMP2	—
IMPR2	IMPR2	—
IMP3	IMP3	—
IMPR3	IMPR3	—
IMP4	IMP4	—
IMPR4	IMPR4	—
PANT	PANT	—
MODIF	M, MO, MOD, MODI, MODIF	—

COPIFI keywords

These keywords belong to COPIFI. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
PANT	P, PA, PAN, PANT	—
NOPANT	NOP, NOPA, NOPAN, NOPANT	—
SISFECH	SISF, SISFE, SISFEC, SISFECH	—
ANTFECH	ANT, ANTF, ANTFE, ANTFEC, ANTFECH	—
DESD	DE, DES, DESD	—
NUM	NU, NUM	—
DSTRING	DST, DSTR, DSTRI, DSTRIN, DSTRING	—
HSTRING	HST, HSTR, HSTRI, HSTRIN, HSTRING	—
TRUNC	TR, TRU, TRUN, TRUNC	—
NOTRUNC	NOTR, NOTRU, NOTRUN, NOTRUNC	—
NOREEMP	NORE, NOREE, NOREEM, NOREEMP	—
REEMPLA	REE, REEM, REEMP, REEMPL, REEMPLA	—
A@ADIR	A, A@, A@A, A@AD, A@ADI, A@ADIR	—
PACK	PAC, PACK	PA selects PANT
UNPACK	UNP, UNPA, UNPAC, UNPACK	—
MAYUSC	MA, MAY, MAYU, MAYUS, MAYUSC	—
LMINUSC	LMI, LMIN, LMINU, LMINUS, LMINUSC	—
MONTA	M, MO, MON, MONT, MONTA	—
VERIFI	V, VE, VER, VERI, VERIF, VERIFI	—
NOVERIFI	NOV, NOVE, NOVER, NOVERI, NOVERIF, NOVERIFI	—
TABLA	TABL, TABLA	—
CONSU	CONS, CONSU	—
NOCONSU	NOC, NOCO, NOCON, NOCONS, NOCONSU	—
PUBLIC	PUB, PUBL, PUBLI, PUBLIC	—
FILL	FI, FIL, FILL	—
SPECS	SP, SPE, SPEC, SPECS	—

DIRIBM keywords

These keywords belong to DIRIBM. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
IMPRESO	I, IM, IMP, IMPR, IMPRE, IMPRES, IMPRESO	—
IMPRESO	I, IM, IMP, IMPR, IMPRE, IMPRES, IMPRESO	—
IMP	I, IM, IMP	—
IMP1	I, IM, IMP, IMP1	—
IMPRESO2	IMPRESO2	IMPRE selects IMPRESO; IMPRES selects IMPRESO; IMPRESO selects IMPRESO
IMP2	IMP2	—
IMPRESO3	IMPRESO3	IMPRE selects IMPRESO; IMPRES selects IMPRESO; IMPRESO selects IMPRESO
IMP3	IMP3	—
IMPRESO4	IMPRESO4	IMPRE selects IMPRESO; IMPRES selects IMPRESO; IMPRESO selects IMPRESO
IMP4	IMP4	—
PANT	P, PA, PAN, PANT	—

ENVIAR keywords

These keywords belong to ENVIAR. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
NOEOT	NOE, NOEO, NOEOT	—
COM1	COM1	—
COMU1	COMU1	—
COM2	COM2	—
COMU2	COMU2	—
COM3	COM3	—
COMU3	COMU3	—

Full keyword	Accepted spellings	Notes
COM4	COM4	—
COMU4	COMU4	—
CONSULTA	C, CO, CON, CONS, CONSU, CONSUL, CONSULT, CONSULTA	—
MENSAJE	ME, MEN, MENS, MENSA, MENSAJ, MENSAJE	—
NOCONSUL	NOC, NOCO, NOCON, NOCONS, NOCONSU, NOCONSUL	—
PUBLICO	PUB, PUBL, PUBLI, PUBLIC, PUBLICO	—

ENVIARA keywords

These keywords belong to ENVIARA. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
NOEOT	NOE, NOEO, NOEOT	—
COM1	COM1	—
COMU1	COMU1	—
COM2	COM2	—
COMU2	COMU2	—
COM3	COM3	—
COMU3	COMU3	—
COM4	COM4	—
COMU4	COMU4	—
CONSULTA	C, CO, CON, CONS, CONSU, CONSUL, CONSULT, CONSULTA	—
MENSAJE	ME, MEN, MENS, MENSA, MENSAJ, MENSAJE	—
NOCONSUL	NOC, NOCO, NOCON, NOCONS, NOCONSU, NOCONSUL	—
PUBLICO	PUB, PUBL, PUBLI, PUBLIC, PUBLICO	—

ESTADO keywords

These keywords belong to ESTADO. FRELOJ and ERRTEST are not accepted keywords; use the listed queries. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
ALL	ALL	—
QUIEN	QU, QUI, QUIE, QUIEN	—
HORA	HO, HOR, HORA	—
FECHA	FE, FEC, FECH, FECHA	—
SISTEMA	SIS, SIST, SISTE, SISTEM, SISTEMA	—
FORFECHA	FORFE, FORFEC, FORFECH, FORFECHA	—
PRIVILEG	PRIV, PRIVI, PRIVIL, PRIVILE, PRIVILEG	—
RELOJ	R, RE, REL, RELO, RELOJ	—
DIA	DIA	—
MENSAJE	MEN, MENS, MENSA, MENSAJ, MENSAJE	—
TIEMPO	TI, TIE, TIEM, TIEMP, TIEMPO	—
USUARIOS	U, US, USU, USUA, USUAR, USUARI, USUARIO, USUARIOS	—
MEMORIA	MEM, MEMO, MEMOR, MEMORI, MEMORIA	—
MODULO	MO, MOD, MODU, MODUL, MODULO	—
DISCO	DI, DIS, DISC, DISCO	—
ERRNUM	ERRN, ERRNU, ERRNUM	—
ERRTEXT	ERRT, ERRTE, ERRTEX, ERRTEXT	—
INCOM	INC, INCO, INCOM	—
ABREV	AB, ABR, ABRE, ABREV	—
IMPEXEC	IMP, IMPE, IMPEX, IMPEXE, IMPEXEC	—
HISTOR	HI, HIS, HIST, HISTO, HISTOR	—
RTCAVAIL	RTC, RTCA, RTCAV, RTCAVA, RTCAVAI, RTCAVAIL	—
SISFECHA	SISF, SISFE, SISFEC, SISFECH, SISFECHA	SIS selects SISTEMA
INIREL	INI, INIR, INIRE, INIREL	—
QET	QET	—
ALEMAN	ALE, ALEM, ALEMA, ALEMAN	—
ESCAPE	ESC, ESCA, ESCAP, ESCAPE	—
SUBIR	SU, SUB, SUBI, SUBIR	—
BAJAR	BA, BAJ, BAJA, BAJAR	—

Full keyword	Accepted spellings	Notes
IZQU	IZ, IZQ, IZQU	—
DERECHA	DE, DER, DERE, DEREC, DERECH, DERECHA	—
RUBOUT	RUB, RUBO, RUBOU, RUBOUT	—
CANCEL	CAN, CANC, CANCE, CANCEL	—
SINONIMO	SIN, SINO, SINON, SINONI, SINONIM, SINONIMO	—
VERSION	VER, VERS, VERSI, VERSIO, VERSION	—

INICDISC keywords

These keywords belong to INICDISC. INCR is the accepted increment/interleave selector; INTERL is not an alias. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
ETIQU	E, ET, ETI, ETIQ, ETIQU	—
VACIAR	V, VA, VAC, VACI, VACIA, VACIAR	—
DIRE	D, DI, DIR, DIRE	—
WP	W, WP	—
NOWP	NOW, NOWP	—
FORMAT	F, FO, FOR, FORM, FORMA, FORMAT	—
INCR	I, IN, INC, INCR	—
TIEM	TI, TIE, TIEM	—
CARA	C, CA, CAR, CARA	—
PISTA	P, PI, PIS, PIST, PISTA	—
CILINDRO	CI, CIL, CILI, CILIN, CILIND, CILINDR, CILINDRO	—
SECTOR	S, SE, SEC, SECT, SECTO, SECTOR	—
HSECT	H, HS, HSE, HSEC, HSECT	—
REPUES	RE, REP, REPU, REPUE, REPUES	—
DEVTYPE	DEV, DEVT, DEVTY, DEVTYP, DEVTYPE	—

INICIAR keywords

These keywords belong to INICIAR. NOPE clears software write protection; NOWP belongs to INICDISC. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
ETIQU	ET, ETI, ETIQ, ETIQU	—
VACIA	V, VA, VAC, VACI, VACIA	—
DIRE	D, DI, DIR, DIRE	—
WP	W, WP	—
NOPE	NOP, NOPE	—
FORMAT	F, FO, FOR, FORM, FORMA, FORMAT	—
INCR	I, IN, INC, INCR	—
TIEM	TI, TIE, TIEM	—
CARA	C, CA, CAR, CARA	—
PISTA	PI, PIS, PIST, PISTA	—
CILINDRO	CI, CIL, CILI, CILIN, CILIND, CILINDR, CILINDRO	—
SECTOR	S, SE, SEC, SECT, SECTO, SECTOR	—
HSECT	H, HS, HSE, HSEC, HSECT	—
SPARE	SP, SPA, SPAR, SPARE	—

PONER keywords

These keywords belong to PONER. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
NUL	NUL	—
SOH	SOH	—
STX	STX	—
ETX	ETX	—
EOT	EOT	—
ENQ	ENQ	—

Full keyword	Accepted spellings	Notes
ACK	ACK	—
BEL	BEL	—
BS	BS	Exact spelling is accepted despite the stored prefix threshold.
HT	HT	Exact spelling is accepted despite the stored prefix threshold.
LF	LF	Exact spelling is accepted despite the stored prefix threshold.
VT	VT	Exact spelling is accepted despite the stored prefix threshold.
FF	FF	Exact spelling is accepted despite the stored prefix threshold.
CR	CR	Exact spelling is accepted despite the stored prefix threshold.
SO	SO	Exact spelling is accepted despite the stored prefix threshold.
SI	SI	Exact spelling is accepted despite the stored prefix threshold.
DLE	DLE	—
DC1	DC1	—
DC2	DC2	—
DC3	DC3	—
DC4	DC4	—
NAK	NAK	—
SYN	SYN	—
ETB	ETB	—
CAN	CAN	—
EM	EM	Exact spelling is accepted despite the stored prefix threshold.
SUB	SUB	—
ESC	ESC	—
FS	FS	Exact spelling is accepted despite the stored prefix threshold.
GS	GS	Exact spelling is accepted despite the stored prefix threshold.
RS	RS	Exact spelling is accepted despite the stored prefix threshold.
US	US	Exact spelling is accepted despite the stored prefix threshold.
DEL	DEL	—

RECARGAR keywords

These keywords belong to RECARGAR. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
CUENTA	C, CU, CUE, CUEN, CUENT, CUENTA	—
RESCUE	RESC, RESCU, RESCUE	—
REEMPLA	R, RE, REE, REEM, REEMP, REEMPL, REEMPLA	—
NOREEM	N, NO, NOR, NORE, NOREE, NOREEM	—
COPREEM	CO, COP, COPR, COPRE, COPREE, COPREEM	C selects CUENTA
CONSU	CON, CONS, CONSU	C selects CUENTA; CO selects COPREEM
NOCONSU	NOC, NOCO, NOCON, NOCONS, NOCONSU	—
PANT	P, PA, PAN, PANT	—
NOPANT	NOP, NOPA, NOPAN, NOPANT	—
INF	IN, INF	—
NOINF	NOI, NOIN, NOINF	—
VERIFI	V, VE, VER, VERI, VERIF, VERIFI	—
NOVERIFI	NOV, NOVE, NOVER, NOVERI, NOVERIF, NOVERIFI	—
VOLUM	VO, VOL, VOLU, VOLUM	—

SERIAL keywords

These keywords belong to SERIAL. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
SERIAL	S, SE, SER, SERI, SERIA, SERIAL	—
V	No accepted prefix established.	—
S	No accepted prefix established.	—
M	No accepted prefix established.	—
N	No accepted prefix established.	—
E	No accepted prefix established.	—
PANT	P, PA, PAN, PANT	—

Full keyword	Accepted spellings	Notes
NOPANT	NOP, NOPA, NOPAN, NOPANT	—

SPOOLER keywords

These keywords belong to SPOOLER. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
INIC	I, IN, INI, INIC	—
FINAL	F, FI, FIN, FINA, FINAL	—
LIST	L, LI, LIS, LIST	—
FORM	FO, FOR, FORM	F selects FINAL
COLA	C, CO, COL, COLA	—
COPIAS	COP, COPI, COPIA, COPIAS	CO selects COLA
COPI	COP, COPI	CO selects COLA
ESTADO	E, ES, EST, ESTA, ESTAD, ESTADO	—
RETE	R, RE, RET, RETE	—
NORETE	NOR, NORE, NORET, NORETE	—
PARA	PAR, PARA	—
ACTIV	ACT, ACTI, ACTIV	—
SAL	SAL	—
MUINIT2	MUINIT2	—
MUINIT3	MUINIT3	—
CAMBIA	CA, CAM, CAMB, CAMBI, CAMBIA	—
ELIM	ELIM	—
SALVAR	S, SA, SALV, SALVA, SALVAR	SAL selects SAL
BACKSPAC	B, BA, BAC, BACK, BACKS, BACKSP, BACKSPA, BACKSPAC	—
IMPR	IM, IMP, IMPR	I selects INIC
ALINE	AL, ALI, ALIN, ALINE	—
NOMB	N, NO, NOM, NOMB	—
RELANZA	REL, RELA, RELAN, RELANZ, RELANZA	R selects RETE; RE selects RETE

SPOOLER keywords

These keywords belong to SPOOLER. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
RETE	RE, RET, RETE	—
NORETE	NOR, NORE, NORET, NORETE	—

TERMINAL keywords

These keywords belong to TERMINAL. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
AUTOLF	AU, AUT, AUTO, AUTOL, AUTOLF	—
RUB	RUB	—
CTL	CTL	—
NULL	NUL, NULL	—
ESC	ESC	—
BEEHIVE	B, BE, BEE, BEEH, BEEHI, BEEHIV, BEEHIVE	—
ADDS	AD, ADD, ADDS	—
SOROC	S, SO, SOR, SORO, SOROC	—
ADM3A	ADM, ADM3, ADM3A	—
HAZELTIN	HAZ, HAZE, HAZEL, HAZELT, HAZELTI, HAZELTIN	—
COM1	COM1	—
COM2	COM2	—
COM3	COM3	—
COM4	COM4	—
COMU1	COMU1	—
COMU2	COMU2	—
COMU3	COMU3	—
COMU4	COMU4	—

Full keyword	Accepted spellings	Notes
HALF	HA, HAL, HALF	—
VDM	V, VD, VDM	—
RDRON	RDRON	—
RDROFF	RDROF, RDROFF	—
PCHON	PCHON	—
PCHOFF	PCHOF, PCHOFF	—
MASTER	M, MA, MAS, MAST, MASTE, MASTER	—
GO	G, GO	—

TERMINAL keywords

These keywords belong to TERMINAL. Different rows can share an operation; the spellings column includes aliases that select that same operation.

Full keyword	Accepted spellings	Notes
CONSOLE	CON, CONSOLE	—
READER	READER	—
PUNCH	PUNCH	—
PRINTER1	PRINT, PRINTE, PRINTER, PRINTER1	—
PRINTER2	PRINTER2	—
PRINTER3	PRINTER3	—
PRINTER4	PRINTER4	—
COMU1	COM, COMU, COMU1	—
COMU2	COMU2	—
COMU3	COMU3	—
COMU4	COMU4	—
TAPE1	TAP, TAPE, TAPE1	—
TAPE2	TAPE2	—
TAPE3	TAPE3	—
TAPE4	TAPE4	—
PRINTER	PRINT, PRINTE, PRINTER	—

Full keyword	Accepted spellings	Notes
COMU	COM, COMU	—
TAPE	TAP, TAPE	—
CON	CON	—
RDR	RDR	—
PCH	PCH	—
PRT1	PRT1	—
PRT2	PRT2	—
PRT3	PRT3	—
PRT4	PRT4	—
COM1	COM, COM1	—
COM2	COM2	—
COM3	COM3	—
COM4	COM4	—
TAP1	TAP, TAP1	—
TAP2	TAP2	—
TAP3	TAP3	—
TAP4	TAP4	—

EXEC instruction reference

EXEC directives belong inside an EXEC text file. The ampersand distinguishes them from ordinary commands that the file asks CSI to execute. Chapter 8 teaches file creation, invocation, variables, comparisons and loops. The following index accounts for all nineteen dispatcher names; END is an additional block delimiter rather than a dispatcher entry.

Directive	Operation or interface limit
&TYPE	Display fields and substituted values; Chapter 8.
&BEGTYPE	Display literal lines through END; Chapter 8.
&SPACE	Output line spacing; see the example below.
&CONTINUE	Continue reading the next command-file line; not BASIC execution resumption.
&QUIT	Finish this EXEC level, optionally returning a result; Chapter 8.

Directive	Operation or interface limit
`&STACK`	Queue a line for program input.
`&BEGSTACK`	Queue literal lines through END.
`&SKIP`	Move the command-file reading position by a line count.
`&GOTO`	Branch to a command-file label; Chapter 8.
`&CONTROL`	Select echo/control and queued-input handling.
`&IF`	Test a condition before executing the rest of the instruction; Chapter 8.
`&CRT`	Send screen-control output or position the cursor.
`&ESC`	Control selected Escape handling; complete mode contract remains unspecified.
`&READ`	Read an input line into named EXEC variables.
`&WAIT`	Request the executive pagination pause; see the example below.
`&WHILE`	Repeat while a condition holds, paired with REPEAT; Chapter 8.
`&UNTIL`	Repeat until a condition holds, paired with REPEAT.
`&REPEAT`	Return to the controlling WHILE or UNTIL statement.
`&ERROR`	Register error handling for later command results; Chapter 8.

SPACE: separate output lines

This syntax illustration belongs in a command file:

```
&SPACE 2
```

SPACE issues two line-spacing operations through the executive. With no count it issues one; zero is also treated as one. The count is processed through an eight-bit value, so it should not be treated as an unrestricted page-layout quantity. SPACE is separate from BASIC's SPACE$ string function.

CONTINUE: proceed to the next line

For example, a labelled destination may contain:

```
-READY &CONTINUE
```

When execution reaches READY, CONTINUE goes to the next command-file line. It neither restarts a terminated program nor performs BASIC's CONTINUE operation.

READ: obtain an answer

This form occurs in interactive command procedures:

```
&READ &ANSWER
```

READ obtains input and assigns a field to ANSWER. Keep beginner answers to a single word within EXEC's eight-character value width. Multiple-variable and empty-input handling need more care than this elementary form; a READ without a variable follows a different command-processing path and is not the same operation.

STACK: provide input to the next program

A command procedure can queue a response instead of waiting for an operator to type it. For example, a procedure that is about to enter BASIC can queue its exit command:

```
&STACK FIFO QUIT
```

The queued text is QUIT. FIFO means first in, first out: older queued lines are consumed first. The optional LIFO form instead places newer input ahead of older input. Omitting the order selector uses FIFO. Queuing does not itself execute QUIT; the consuming program must interpret it. BASIC recognizes QUIT, while another program could interpret that same line differently.

To enter BASIC after queuing that response, the next line of the file is:

```
BASIC
```

This illustrates the input mechanism. Use it only after saving any BASIC work. A queued response can answer a destructive confirmation as readily as an editing prompt; do not reuse input queues without knowing the receiving program's dialogue.

BEGSTACK: provide several literal input lines

This syntax illustration supplies BASIC statements as queued input:

```
&BEGSTACK FIFO
PRINT 1
QUIT
&END
```

BEGSTACK reads literal lines until END and queues them in the selected order. These lines are input for a later program, not EXEC instructions, and the block does not substitute EXEC variables. A following BASIC command would consume the PRINT and QUIT lines. This differs from repeated STACK instructions, where substitutions can construct each response.

CONTROL: echo and queued-input policy

Command procedures commonly begin:

```
&CONTROL OFF NOSTACK
```

OFF suppresses command echo. NOSTACK selects a queued-input control mode; it does not prohibit later STACK instructions. It sets bit 3 of the partition's execution-control byte, while STACK clears that bit. The complete downstream effect of this flag remains unspecified. Existing procedures combine this mode with queued responses for the utilities they invoke. STACK selects the opposite control mode:

```
&CONTROL STACK
```

These CONTROL words are separate from FIFO/LIFO, which choose queue insertion order. Complete interactions with interruptions, nested interpreters and abnormal program termination remain unspecified; use an existing procedure's deliberate sequence rather than changing modes by analogy with another command language.

SKIP: bypass lines

The following instruction appears in source-insertion procedures:

```
&SKIP 2
```

It advances past two subsequent command-file lines. The count addresses physical file lines, so changing comments or inserted text can alter the destination. A labelled GOTO is clearer for new procedures when the destination should survive such editing. Negative-count handling exists but is not an elementary backward-loop interface.

UNTIL: repeat until a condition is true

This syntax illustration uses the same condition and REPEAT machinery as Chapter 8:

```
&A = 1
&UNTIL &A > 3
&TYPE &A
&A = &A + 1
&REPEAT
&QUIT
```

UNTIL tests the condition at the beginning of the group. The group continues while A is not greater than 3; REPEAT returns to that test. It therefore differs from languages in which an UNTIL test always occurs after the first iteration. Source-insertion procedures use this form to collect a fixed number of arguments.

CRT, ESC and WAIT

CRT CLEAR is used before interactive procedure displays:

```
&CRT CLEAR
```

It requests the terminal clear operation. The numeric CRT form takes horizontal and vertical positions; the interpreter subtracts one from each before emitting the terminal-positioning control sequence. For example:

```
&CRT 47 14
```

This requests position 47 on line 14 on a compatible terminal. Device handling remains significant; it is not a portable screen-graphics API.

Procedures also use this Escape-control form around operations with their own diagnostics:

```
&ESC O OFF
```

The matching form is:

```
&ESC O ON
```

These select and restore an O-mode control flag. The complete meaning of that flag and all ESC modes remains unspecified; these two lines are forms to recognize when reading procedures, not a recommendation to suppress errors or interruption generally. WAIT invokes the executive's pagination-pause service. Its elementary form is:

```
&WAIT
```

It calls service 0x31 and resumes command-file reading after that service returns. No duration is parsed; this is not a timed sleep instruction. The exact pause and cancellation behavior follows the active console's pagination policy; see service 0x31.

Special operands

Special operands supply values or form part of expressions; they are not commands to enter at CSI.

Operand	Meaning or interface limit
&RETCODE	Result of the preceding command or child procedure; see Chapter 8.
&INDEX	Number of positional arguments, up to sixteen; see Chapter 8.
&NULL	Empty, space-filled value; used in argument and variable comparisons.
&PAGE, &LINE	Values obtained through console geometry service 0x3B; not a document page counter. Complete operator interpretation remains unspecified.

Operand	Meaning or interface limit
&TYP, &LEN, &LIT	Recognized expression names; full argument, result and quoting contracts remain unspecified.
&CAT	String-construction operand used by vocabulary procedures; full general quoting and concatenation rules remain unspecified.
&SUB	Inclusive start/end substring selection; see Chapter 8.

For example, this procedure fragment checks whether ARG has no value:

```
&IF &ARG EQ &NULL &GOTO -MISSING
```

MISSING must be a label in the same procedure. Do not substitute the literal word NULL for &NULL. For general string joining, use the documented vertical-bar expression from Chapter 8 until the complete CAT contract is available.

EDITAR internal commands

EDITAR edits sequential text and displays its own asterisk prompt. Its command names are independent of CSI abbreviations and BASIC's editor commands. Begin with the working CASE U, INPUT and FILE procedure in Chapter 7.

The table below preserves all forty-seven internal names. A stored minimum is a parser threshold, not a guarantee that every prefix selects that row: earlier matching rows can take precedence. Full names avoid that ambiguity. A name and handler identify an available parser branch, but do not by themselves establish its arguments, default range or modifying effects. Where those details are unspecified, no invented edit command is supplied.

Full editor command	Stored minimum length	Operation or limitation
AGAIN	1	Recognized editor command; complete argument and effect contract unspecified.
TABSET	2	Recognized editor command; complete argument and effect contract unspecified.
HELP	1	Recognized editor command; complete argument and effect contract unspecified.
NEXT	1	Shares the DOWN handler; complete movement arguments unspecified.
UP	1	Recognized editor command; complete argument and effect contract unspecified.
BOTTOM	1	Recognized editor command; complete argument and effect contract unspecified.
TYPE	1	Shares the LIST handler; complete range/output syntax unspecified.
FILE	4	Write and leave the editor; see Chapter 7 for pending-interruption behavior.
SAVE	4	Recognized editor command; complete argument and effect contract unspecified.

Full editor command	Stored minimum length	Operation or limitation
LOCATE	1	Source-insertion procedures use LOCATE followed by a marker string. Full matching rules unspecified.
ZONE	2	Recognized editor command; complete argument and effect contract unspecified.
UNNUMBER	2	Recognized editor command; complete argument and effect contract unspecified.
HTAB	2	Recognized editor command; complete argument and effect contract unspecified.
X	1	Recognized editor command; complete argument and effect contract unspecified.
Y	1	Recognized editor command; complete argument and effect contract unspecified.
Z	1	Recognized editor command; complete argument and effect contract unspecified.
VERIFY	1	Recognized editor command; complete argument and effect contract unspecified.
WRAPMODE	1	Recognized editor command; complete argument and effect contract unspecified.
FIND	1	Recognized editor command; complete argument and effect contract unspecified.
RENUMBER	3	Source-insertion procedures use RENUMBER 10 10 with line mode active. Full range and collision rules unspecified.
PAGE	1	Recognized editor command; complete argument and effect contract unspecified.
TOP	2	Used to position source-insertion operations; subsequent editing mode matters.
QUIT	4	Exit path; do not assume it saves changes. Use the Chapter 7 FILE procedure.
?	1	Recognized editor command; complete argument and effect contract unspecified.
INPUT	1	INPUT text inserts the following text; Chapter 7.
REPLACE	1	Recognized editor command; complete argument and effect contract unspecified.
DELETE	2	Used with a numeric argument by source-insertion procedures; default range and recovery are unspecified.
MODIFY	2	Recognized editor command; complete argument and effect contract unspecified.
CASE	2	CASE U preserves uppercase input; Chapter 7.
COLUMN	2	Recognized editor command; complete argument and effect contract unspecified.
COMBINE	3	Recognized editor command; complete argument and effect contract unspecified.
CSI	3	Recognized editor command; complete argument and effect contract unspecified.
TRUNCATE	2	Recognized editor command; complete argument and effect contract unspecified.
NAME	2	Recognized editor command; complete argument and effect contract unspecified.
LINEMODE	3	ON/OFF forms occur in source-insertion procedures; effects on numbering and position require care.

Full editor command	Stored minimum length	Operation or limitation
PROMPT	3	Recognized editor command; complete argument and effect contract unspecified.
CHANGE	1	Recognized editor command; complete argument and effect contract unspecified.
DUP	2	Recognized editor command; complete argument and effect contract unspecified.
LIST	2	Shares the TYPE handler; complete range/output syntax unspecified.
DOWN	1	Shares the NEXT handler; complete movement arguments unspecified.
GET	3	Source-insertion procedures use GET name BASIC S to import sequential source. Full insertion-position and error rules unspecified.
PUTD	4	Recognized editor command; complete argument and effect contract unspecified.
NUMBER	2	Recognized editor command; complete argument and effect contract unspecified.
PUT	3	Source-insertion procedures use PUT name BASIC S followed by a numeric limit. Complete limit semantics unspecified.
ERROR	2	Recognized editor command; complete argument and effect contract unspecified.
SKIP	2	Recognized editor command; complete argument and effect contract unspecified.
SPLIT	2	Recognized editor command; complete argument and effect contract unspecified.

For a safe first insertion at the asterisk prompt, enter:

```
INPUT A SHORT LINE
```

This inserts A SHORT LINE into the current text. It does not execute that text. Save through FILE as described in Chapter 7. Advanced source-insertion procedures deliberately change LINEMODE, position, import text and renumber lines; those sequences are not interchangeable with BASIC editor commands bearing the same names. CEDIT and TEDITAR are separate editors whose complete internal dialogues remain unspecified.

CONSULTA vocabulary and dictionary dependencies

CONSULTA interprets Spanish query text against file and attribute definitions. CONTROL.VOC supplies vocabulary maintained by the VOC and GENVOC procedures. CONSULTA uses a work file whose name ends with the execution partition number, such as CONSULTA.TRAB1 for partition 1. See Appendix C for those support tools and Chapter 31 for entry requirements.

The following statement is the dictionary-listing form used by VOC. It belongs to CONSULTA, not CSI. It requires a correctly initialized CONTROL.VOC and compatible work storage:

```
LISTAR CONTROL VOC VIS NOMBRE ATRIBUTO CAMPO LONGITUD FORMATO ROTULO SUP.ID CABECERA 'Diccionario
del fichero CONTROL' @FECHA
```

LISTAR selects listing; CONTROL VOC names the dictionary file. VIS selects terminal output. The following attribute names choose the displayed fields. SUP.ID and CABECERA control presentation; the quoted text supplies a heading, and @FECHA requests date substitution. These attribute names come from the dictionary and are not universal reserved words for every application file. The complete width, pagination and query-error contract is unspecified. The wrapper example below avoids typing a long query at the system prompt.

At CSI, this procedure form requests the same dictionary listing:

```
EXEC VOC LIST
```

VOC queues its query and enters CONSULTA. It requires the dictionary to exist; it is not an instruction to create one automatically. To print rather than display, the wrapper's IMPR function uses a configured printer.

Reserved vocabulary

The table groups words with identical parser-result bytes. This shows where the parser gives several spellings the same token; it does not guarantee that whole statements can be rearranged freely. Punctuation is significant: POR.DESC, NO.COMP, @PAGINA and comparison symbols are literal forms. SON occurs twice in the vocabulary, with different result pairs.

Words	Parser result bytes	Interpretation boundary
`LISTAR`, `BUSCAR`	0x00, 0x00	Same parser result; surrounding statement grammar still applies.
`CLASIFICAR`, `ORDENAR`, `ORGANIZAR`	0x00, 0x02	Same parser result; surrounding statement grammar still applies.
`SELECC`	0x00, 0x01	Reserved token; use in its query context.
`CSELECC`	0x00, 0x03	Reserved token; use in its query context.
`DESDE`	0x0F, 0x00	Reserved token; use in its query context.
`TIENEN`, `CON`	0x14, 0x00	Same parser result; surrounding statement grammar still applies.
`IMP`	0x1E, 0x00	Reserved token; use in its query context.
`VIS`	0x1E, 0x01	Reserved token; use in its query context.

Words	Parser result bytes	Interpretation boundary
`NO.COMP`	0x21, 0x00	Reserved token; use in its query context.
`COMPLETO`	0x22, 0x00	Reserved token; use in its query context.
`TOTAL`	0x32, 0x00	Reserved token; use in its query context.
`POR`	0x46, 0x41	Reserved token; use in its query context.
`POR.DESC`	0x46, 0x44	Reserved token; use in its query context.
`SALTAR.EN`, `SALTO`, `SUBTOTAL`, `ROMPER`	0x5A, 0x00	Same parser result; surrounding statement grammar still applies.
`CABECERA`	0x6E, 0x00	Reserved token; use in its query context.
`PIE.DER`	0x6F, 0x00	Reserved token; use in its query context.
`PIE.IZQ`	0x70, 0x00	Reserved token; use in its query context.
`@PAGINA`, `@FECHA`, `@HORA`	0x18, 0x00	Same parser result; surrounding statement grammar still applies.
`SUP.ID`	0x82, 0x00	Reserved token; use in its query context.
`ID`	0x87, 0x00	Reserved token; use in its query context.
`SIN`	0x15, 0x00	Reserved token; use in its query context.
`NO`, `NINGUN`, `NINGUNO`	0x13, 0x00	Same parser result; surrounding statement grammar still applies.
`=`, `IGUAL`, `COMO`, `EN`, `SIMILAR`, `SEMEJANTE`	0x17, 0x01	Same parser result; surrounding statement grammar still applies.
`>`, `MAYOR`, `DESPUES`, `SUPERIOR`, `MAS`, `POSTERIOR`	0x17, 0x02	Same parser result; surrounding statement grammar still applies.
`<`, `MENOR`, `ANTERIOR`, `MENOS`, `INFERIOR`	0x17, 0x03	Same parser result; surrounding statement grammar still applies.
`#`, `<>`, `NI`, `DISTINTO`, `DESIGUAL`	0x17, 0x04	Same parser result; surrounding statement grammar still applies.
`MENOR.IGUAL`, `<=`	0x17, 0x05	Same parser result; surrounding statement grammar still applies.

Words	Parser result bytes	Interpretation boundary
`MAYOR.IGUAL`, `>=`	0x17, 0x06	Same parser result; surrounding statement grammar still applies.
`CONTENIENDO`, `QUE.CONTENGA`, `INCLUYENDO`, `INCLUYE`	0x17, 0x07	Same parser result; surrounding statement grammar still applies.
`Y`	0x19, 0x01	Reserved token; use in its query context.
`O`	0x19, 0x00	Reserved token; use in its query context.
`UN`, `UNA`, `SON`, `CUALQUIER`, `CADA`, `TODOS`, `FICHERO`, `PARA`, `CUYO`, `REGISTROS`, `DE`, `LOS`, `ES`, `DEL`, `A`, `QUE`, `EL`, `DONDE`, `TAMBIEN`, `CUYOS`, `SON`, `AQUELLOS`, `CUYA`, `POR.FAVOR`	0x0A, 0x00	Same parser result; surrounding statement grammar still applies.

The vocabulary includes operations, selection words, comparison synonyms, output controls, substitutions and connecting words. It is not a list of independent system commands. In particular, CONSULTA's CLASIFICAR does not establish a CSI command called CLASIF. A complete grammar for arbitrary queries is not yet available; the dictionary-listing form above should not be extended using SQL conventions.

PANTALLA: case-sensitive screen controls

PANTALLA designs data-entry screens. Its uppercase semicolon controls are listed in Chapter 31. The following lowercase controls are additionally advertised for label appearance:

Literal controls	Label attribute
;i / ;j	Inverse video
;s / ;t	Underlining
;p / ;q	Blinking

For example, the literal control is:

```
;i
```

It belongs to the inverse-video pair inside PANTALLA. The complete set/clear direction, persistence and behavior on each terminal are unspecified; do not assume it is equivalent to uppercase ;I, which inserts a field. Likewise lowercase ;s is distinct from uppercase ;S, which saves the design.

Window movement is advertised in this form:

```
;<arrow>nn<return>
```

After the semicolon, press an arrow key, enter the number of positions and press Return. The angle-bracket expressions denote keys, not literal text. Exact arrow encoding and movement limits depend on the terminal and remain incompletely specified.

Appendix A. Program, language and archive formats

This appendix distinguishes executable relocation, tokenized BASIC source, compiled BASIC instructions, runtime values and EXEC work frames. Similar token bytes can mean different things in these namespaces. There is no established general assembler-object/library specification merely because legacy messages mention ABS, REL, COM, symbols or PABs.

Native executable and submodule formats

An R load image starts at file offset 0 and includes code and initialized data. The command-execution path enters a command at its loaded base, but loading an R submodule does not by itself specify where to begin execution. BASIC overlays have a component byte followed by a JP slot at offset 0x0001; SORT overlays place their entry slots after a 0xF0-byte control area. The caller must follow the component-specific interface described in Appendix C. Directory+0x1E supplies image length, +0x11 total stored sectors and +0x15 firstLBA. When present, relocation records start at `ceil(image_bytes/256)*256`; a module requiring no fixups can have no relocation sector. Each relocation sector has a one-byte count 1–127 followed by that manyLE16 image offsets. Add the load base to theLE16 word at each offset, modulo 65536. Validate that both bytes lie in the image before applying a fixup in a loader or format reader.

A images load at directory+0x1E. The complete general A-image runtime contract is less well specified than R loading. An optional `CF41` prologue is preceded by minimum privilege and three serial bytes, as described in Chapter 30; it is not a mandatory file header.

The service 0x62 compact submodule descriptor is seven bytes: load addressLE16 at+0, image lengthLE16 at+2, stored-sector byte at+4 and relativeLBALE16 at+5. Its caller supplies the baseLBA.BASICUSR modules can return through an defined BASIC wrapper; this does not establish a general linker/library toolchain.

BASICOBJ source records

Record offset	Size	Meaning
00	1	Total record length N, including this byte and terminator
01	2	Source line number, big-endian
03	1	Number of spaces between line number and statement in specified source
04	N−5	Tokenized statement body
N−1	1	CR (0Dh)

A zero record-length byte ends the source stream; 0x1A follows as sequential padding. A record length must include its final CR. A malformed length or missing terminator is a source-format error; do not guess subsequent line boundaries.

Bytes	Tested interpretation
20 followed by remaining text	REM statement; comment spelling/spaces preserved
29	END statement
3A	PRINT statement
05 hi lo	Integer literal, big-endian word; sign depends on source context
06 length text...	String literal; one-byte length followed by exact characters
1C before numeric literal	Unary minus in specified source
03 followed by 8 bytes	Packed decimal numeric literal, also used above 32767

BASICCOM header and expressions

Offset	Size	Meaning
00	1	Required zero checked by runtime; not the R-executable magic
01	1	Array-base policy:00 sets 0,01 sets 1;FF retains current policy
02	2 BE	Program-relative linked-definition head;zero means no list
04	2 BE	Program-relative DATA head copied toIX+4B/+4C;zero when no DATA
06	variable	Main compiled statement stream begins

Low opcode	Following bytes	Role
03	8	Packed decimal literal
04	2 BE	Integer literal/value word
06	1 length + that many bytes	String literal
07	Encoded name through its first high-bit-set byte	Variable reference
08	none	Unary minus
0F/10/11	none	NOT/AND/OR
12/13/14	none	Greater/less/equal comparisons
18/19/1A/1B/1C	none	Power/multiply/divide/add/subtract
1D	none	One-dimensional array access after variable/index operands
1F	none	String concatenation
30	none	ABS function marker within an expression
21	none	Native-function apply with one argument in specified form
26	none	User-function apply with one argument in specified form

Expression opcode bit 7 marks the end of the expression. It is distinct from the high bit terminating an encoded variable name. For example, compiled 2+3*4 is `04 00 02 04 00 03 04 00 04 19 9B`: operands precede multiply and add, with the final add carrying the end bit. Compiled PRINT constants can therefore use 84 for integer, 83 for decimal and 86 for string without introducing three new underlying literal types.

Tested encoded names include A=C1, AB=41 C2, I%=89, S$=F3, FNT=46 4E D4 and X=D8. A selected ARR reference reduces to40 80; a complete general name-encoding algorithm has not been established. Do not infer it from those examples alone.

Branches and mutable loop state

Compiled form	Fields after opcode	Runtime meaning
2E IF	condition expression; false-target BE16	Zero selects the target; nonzero skips its two bytes.
2D GOTO	target BE16	Transfer to target; also emitted to skip ELSE.
2C GOSUB	target BE16	Save continuation and transfer.
40 RETURN	none	Remove the specified three-byte continuation record and resume it.
20 explicit RETURN	target BE16	Code-established target override; complete target-override behavior unspecified.
2A range FOR	NEXT offset BE16; encoded name; initial, limit and step expressions	Assign initial value, initialize NEXT state, test range and enter/skip body.
53 expression-list FOR	NEXT offset BE16; encoded name; expressions ending with 0Dh	Evaluate one expression for each body iteration.

Offset from NEXT	Integer 34	Real 63	List 64
+01	BE16 encoded-name offset in FOR	same	same
+03	BE16 loop-body offset	same	same
+05	BE16 signed limit	eight-byte packed-decimal limit	BE16 next-expression cursor
+07 / +0D	BE16 signed step at+07	eight-byte step at+0D	none
Total record bytes	9	21	7

Position	Size	Meaning
+00	1	36h, computed ON opcode
+01	variable	Selector expression, ending with its expression-opcode bit 7
after expression	1	2Dh GOTO or 2Ch GOSUB in documented forms
next	1	Unsigned target count N
next	2×N	Big-endian target offsets relative to BASICCOM byte zero, including its six-byte header
after table	—	Next statement and computed-GOSUB continuation

Compiled GOSUB pushes a three-byte continuation `FF` plus a big-endian program-relative offset; the live continuation-top pointer atIX-2 is little-endian. Interpreted computed calls instead use a seven-byte source continuation, whose final four context bytes have only selected meanings. NEXT state is mutable in RAM and begins zero in the compiled file's corresponding fields.

ON's selector is one-based. A missing target is stored 0000 and falls through; computed GOSUB returns after the complete table. Selector 0 takes the observed error 0x14 path and result 253. ON ERROR uses opcode 55 plus a BE16 target; its live pointer is atIX-0x0C/-0x0B. The complete grammar of all error/branch combinations remains incomplete.

Eight-byte decimal and data records

Offset	Size	Meaning
00	1 byte	Exponent biased by 80h;00 is the arithmetic zero convention
01 bit 7	1 bit	Sign:0 positive,1 negative
01 bits 6–4	3 bits	Zero in specified normalized output; runtime result normalization clears them
01 bits 3–0	1 decimal digit	Leading significant digit
02–07	6 bytes	Remaining twelve digits, high nibble then low nibble in each byte

Let E=byte 0-0x80 and C be the 13-digit decimal coefficient from the low nibble ofbyte 1 and the twelve following BCD digits. A nonzero value is `sign*C*10^(E-13)`. Exponent byte zero is the arithmetic zero convention. Examples: 45.5=`82 04 55 00 00 00 00 00`; -0.5=`80 85 00 00 00 00 00 00`. Bits 6–4 ofbyte 1 are zero in normalized specified output; arbitrary malformed-input behavior is not established.

BASIC WRITE fields use 03 plus decimal payload, 04 plus signed BE16 integer, and 06 plus one-byte string length and characters. Byte 00 terminates a record; 0x1A denotes EOF at a field/type boundary. Source literals instead use 05 for the specified small integer and a separate unary-minus token where observed. These distinctions prevent a source decoder from misreading data records.

DATA chains use the compiled header+4 head. READ opcode 54 carries 07-encoded names; RESTORE opcode 3F supplies a BE16 resolved target. DATA literals do not use the expression-end high bit. Runtime IX offsets and scalar structures follow.

Relative to IX	Size	Code-supported role
−34h/−33h	2 LE	Compiled program base
−02h/−01h	2 LE	Current BASIC continuation-record top; compiled GOSUB records are FF+BE16 relative offset, while interpreted 230 records occupy seven bytes
−30h/−2Fh	2 LE	Current expression/temporary stack pointer
−2Ch/−2Bh	2 LE	Variable/definition linked-list head
+1Dh	1	Array-base byte 0/1, also compared with compiled header+1
+4Bh/+4Ch	2 LE	Program-relative DATA cursor initialized from header+4

Flags	Payload after flags
03	Eight-byte packed decimal
04	BE16 signed integer
06	BE16 string-descriptor pointer
0B	BE16 upper index; consecutive eight-byte decimal elements

Offset	Bytes	Observed meaning
+00	1	Continuation-token offset within the calling source record;14h at its final CR in these calls
+01	2	Calling source line, big endian
+03	1	Copy of pre-call context IX+1Eh; broader meaning unknown here
+04	1	Copy of IX+1Fh
+05	1	Copy of IX+20h
+06	1	Copy of IX+3Ah

EXEC level frame

Each level uses 512 bytes in a direct work file. Arguments occupy 0x000–0x07F as sixteen eight-byte values. Variables occupy 0x080–0x17F as sixteen name 8/value 8 pairs. Bytes 0x180–0x1F1 remain unclassified. Byte 0x1F2 saves context+0x38 with bit 0 cleared; 0x1F3–0x1F5 save context+0x2A–0x2C; 0x1F6–0x1FF save context+0x2D–0x36. Within the latter, 0x1F6–0x1FD contain the script name and 0x1FE is an opaque word.

Level N begins at byte 512*(N-1), or direct record 4*N-3 for 128-byte records. Original EXEC1–EXEC4 files each contain sixteen 128-byte records, sufficient for four 512-byte frames per file. This is an installation allocation, not a universally documented maximum nesting depth. CONECTAR derives capacity from twice the low block-count byte; malformed/extreme cases remain unknown.

Tape label templates

The following 80-byte structures are specified from original utility/driver templates and code. The complete physical media stream is unspecified. Preserve unclassified bytes, embedded CRs and literal producer names rather than imposing another system's label standard.

VOL1

Offset	Size	Interpretation
00	4	Literal`VOL1`
04	8	Name copied by INITTAPE0137 onward, up to 8 bytes, stopping atCR or space; untouched initial suffix is spaces
0C	25	Spaces in original template; further meaning unknown
25	14	Owner text copied toINITTAPE0358
33	28	Spaces in original template
4F	1	Literal ASCII1; semantic version interpretation unspecified

First file label

Offset	Size	Interpretation
00	3	HDR, EOV or EOF prefix
03	1	Literal1
04	17	File identifier comparison span
15	6	Unclassified template bytes; includesCR in some originals
1B	4	First incrementing label counter; apparently section
1F	4	Second incrementing label counter; apparently sequence
23	6	Unclassified template bytes
29	6	YYMMDD derived from current date bytes
2F	7	Unclassified template bytes
36	6	Written decimal block count
3C	13	Producer text begins here; template containsOASIS orOASIS-ARCHIVE
49	7	Spaces in observed templates

Template variations

Source / offset	EmbeddedCR offsets	Producer bytes
DIS10+0AC5	13	OASIS
BACKUP+0FA5	13,15	OASIS
ARCHIVAR+1493	15	OASIS-ARCHIVE
RECARGAR+1197	15	OASIS

Second file label

Offset	Size	Observed content	Interpretation
00	4	HDR2, with prefix replaced for trailers	—
04	1	F	Tentative fixed-record indicator
05	5	04096	numeric span; exact field role Unknown
0A	5	04096	numeric span; exact field role Unknown
0F	65	Spaces	template

The strings OASIS and OASIS-ARCHIVE support lineage identification; they do not prove compatibility with another release's tape utilities. Archive payload ordering, controller handshake, retries and complete restoration remain unresolved. The label widths 04096 identify numeric template spans, not independently documented universal record-size guarantees.

Appendix B. Memory, I/O, interrupt and routine maps

Addresses here refer to the initialized common image unless a bank, overlay or relative driver offset is specified. Dynamic driver addresses must be obtained from descriptors. Low port numbers identify decoded device groups; Z80 I/O instructions generate 16-bit addresses and the complete hardware upper-address decode remains unspecified. Account for the full address presented by the instruction.

CPU and resident memory

CPU range	Mapping	Notes
0000–0FFF at reset	Boot ROM alias	—
0000–3FFF after alias removal	Common RAM	—
4000–FFFF	48-KiB selected bank, FD low nibble	banks 0–4
8000–8FFF with FD bit 7 set	Boot ROM read overlay	—

Range	Native description
0000–39C1	NUCLEO, 14786 bytes in ESTADO MEMORIA
39C2–4C8A	SPOOLER, 4809 bytes in ESTADO MEMORIA
0000–4C8A	NUCLEO region in ESTADO MODULO, 19595 bytes
4C8B–FFFF	Modules 00–03, each 45941 bytes
50C3–E3EB	Module 04, 37673 bytes

Address	Observation
0008	JP to RST service dispatcher 08BD
0033 word	Current partition context pointer, 4C8B in primary session
046B byte	CPU bank bookkeeping used by mapper 21DE
048A word	Current scheduler-record pointer
048C–04BB	16 three-byte scheduler records; first flags byte 048E; PONER enforces public numbers 1–16
0551/0553	Ordinary scan-origin pointer/index, distinct from current execution during callbacks
0554–0557	Remaining quantum, callback hint, saved quantum and temporary scan-origin flag
0513 onward	Four-byte module bounds candidates
0713,0717,071B	Kernel overlay storage descriptors
08E0–09BD	111-entry syscall target table
424A	Shared execution window for kernel overlays
5157	Loaded CSI in primary partition after initialization
FC67	Prior observed PAN1 driver load address; dynamic, not a universal constant

Shared globals

Address	Meaning
0x0003	Weekday
0x0006	Fixed byte lock used by 59/5A
0x0007	Installed retry count 4
0x000B	Default quantum byte
0x000C / 0x000D	Current record index / configured count
0x000E	Global display/configuration flags
0x0013–0x0015	Month/day/year BCD
0x0016	Timer-list head pointer
0x001B–0x001D	Hour/minute/second BCD
0x003B–0x003D	Installation serial bytes
0x0044	Active spooler pointer
0x0046	Busy-delay calibration
0x004A / 0x004C	Time/date-read callback pointers, installed 2E4C/2E6E
0x0062 / 0x0064	Time/date-write callback pointers, installed 2E88/2E98
0x0069	Physical descriptor array,32-byte stride
0x02A6 / 0x02A7	ALP DMA-ready bitmaps
0x0485	Timer-head owner
0x0551 / 0x0553	Ordinary scheduler origin pointer/index
0x0554–0x0557	Residual quantum, callback hint, saved quantum, temporary origin flag
0x2078	Saved stack pointer for cross-bank copying
0x208A	Common copy-stack top; also executable lock-acquire entry in ordinary mapping/use
0x21DC / 0x21DD	Message transfer byte / owner lock
0x23FF+3*i	Callback registry row formula

Physical device and port map

Low port	R/W	Hardware / bit meaning	Notes
00–0F	R/W	First ALP 8237: address/count 00–07, status/command 08, request 09, mask 0A, mode 0B, pointer 0C, reset 0D	—
10–1F	R/W	Second ALP 8237, same register offsets	—
20–23	W	DMA bank latch for each terminal channel pair; low nibble selects bank	function; other bits Unknown
24–27	W	Per-terminal receive flow control, bit 0=ready	Probable RTS wiring
28–2B	W	Clear additional ALP high-nibble interrupt sources	Unknown, unmapped
2C	R	ALP summary: bit 0 first SCC, bit 1 second SCC, bit 2 DMA0 TC latch, bit 3 DMA1 TC latch	low nibble
2E/2F	W	Clear DMA0/DMA1 terminal-count latch	—
30/31	R/W	SCC0 B control/data, PAN2	—
32/33	R/W	SCC0 A control/data, PAN1	—
34/35	R/W	SCC1 B control/data, PAN4	—
36/37	R/W	SCC1 A control/data, PAN3	—
60	R/W	SS2 source code: E0 serial, C0 timer; A0 write acknowledges timer	Original dispatch paths; physical priority and complete mask wiring unspecified
61	R	Configuration readback: bit 0 language, bit 1 skip ASI test, bit 2 skip HDD test, bit 3 enable diagnostics	firmware meanings; physical DIP/shift logic Unknown
61	W	Configuration/shift latch control	Unknown, unmapped
74–77	R/W	8254 counters 0/1/2/control	registers; 1 MHz clock Probable
78–7B	R/W	IMP serial interface: 78 data, 79 status, 7A alternating modes, 7B command	SCN2651 layout; complete clock/modem wiring unspecified
7C–7F	R/W	SS2 SCC: B control/data 7C/7D for PAN10; A control/data 7E/7F for PAN9	DIS16/17; RR3 source dispatch through 7E
80–9F	R/W	MM58167 RTC counters/status/control	Probable; interrupt route complete physical behavior unspecified

Low port	R/W	Hardware / bit meaning	Notes
B8	R/W	WD1001 data FIFO	256 bytes per sector
B9	R/W	WD error/precompensation	device-compatible
BA	R/W	WD sector count	—
BB	R/W	WD sector ID,0..31	—
BC/BD	R/W	WD cylinder low/high	—
BE	R/W	WD SDH: bits 6:5 sector size, 4:3 drive, 2:0 head	Size 00=256 bytes, 01=512, 11=128; 10 unspecified
BF	R/W	WD status/command: busy 80, DRQ08, error 01; ready 50 after transfer; restore 13/16, read 20, write 30, format 50, seek 70	Completion must be followed by status/error inspection
C0/C1	R/W	uPD765A main status/data for DIS2	Command and result bytes use C1
C2	R/W	DIS2 transfer adapter: writes bank, address high, address low; read bit 7 polled for completion	Software sequence established; full physical wiring and boundary carry unspecified
FD	W	Low nibble bank, bit 7 ROM overlay at 8000; first write releases reset alias at 0000	observed behavior; unused bits Unknown
FE	W	Active-high interrupt mask	Probable, execution validated

RST 0x00 changes during startup: the newly relocated kernel jumps through it to 0x5157 before initialization; at the observed login state its target is 0x34F7. These are successive boot phases, not interchangeable live entry addresses. See Chapter 21.

For WD FORMAT 0x50, port 0xB8 receives a sector-sized table buffer containing flag/sector-ID pairs. With 32 sectors and 256-byte size, 64 bytes describe the entries and the host still supplies the full 256-byte buffer. The count is at 0xBA. Normal completion signaling can accompany an error; inspect 0xBF and the error register at 0xB9. The original OS's Ignore policy can subsequently return success without valid data, as explained in Chapter 27.

Interrupt and DMA map

Path	Supplied opcode/vector	Established use
SS2 level 2	D7 / RST10 /0x0010	Serial source E0 and timer source C0 selected through port 60
WD level 4	E7 / RST20 /0x0020	Disk
ALP level 5	EF / RST28 /0x0028	SCC receive and DMA completion
Tape-related	RST18 /0x0018	Registry 03 route; physical delivery unspecified
Context hook	RST38 /0x0038	Context+0x39 inline hook

Port 0xFE supplies an active-high mask in the specified interrupt model. Highest-active-level arbitration remains provisional. The timer uses 8254 counter 1=8000, mode 2; a 1-MHz source gives an 8-ms tick. A 4-MHz CPU configuration is used for the timing values here; the complete physical clock routing is unspecified.

DMA0 channels 0/1 serve PAN1 TX/RX and 2/3 PAN2; DMA1 channels 0/1 serve PAN3 and 2/3 PAN4. The installed receive path uses SCC interrupts and CPU reads, not RX DMA. TX uses DTR/REQ; channel-pair bank latches select DMA memory independently of portFD. The serial source 2.4576 MHz supports the observed baud configuration; SS2 source routing remains inferred.

External driver catalogue

Original file	Loaded bytes	Matching installed names	JP target offsets at 00/03/06/09/0C
SISTEMA.DIS1	605	FLOPPY1	000D / 0045 / 004A / 005B / not JP
SISTEMA.DIS2	575	FLOPPY2	000D / 0044 / 0049 / 005A / not JP
SISTEMA.DIS10	2980	TAP	0029 / 0029 / 0029 / 002B / 005B
SISTEMA.DIS11	237	IP	000F / 002A / 002B / 004D / 0061
SISTEMA.DIS12	438	CDC	000F / 00B6 / 00B7 / 0151 / 018B
SISTEMA.DIS16	650	PAN9, IS9	000F / 004C / 005D / 0093 / 0105
SISTEMA.DIS17	650	PAN10, IS10	000F / 004C / 005D / 0093 / 0105
SISTEMA.DIS18	887	PAN1	000F / 0029 / 003D / 006B / 0118
SISTEMA.DIS19	887	PAN2, IS2	000F / 0029 / 003D / 006B / 0118
SISTEMA.DIS20	887	PAN3, IS3	000F / 0029 / 003D / 006B / 0118
SISTEMA.DIS21	887	PAN4, IS4	000F / 0029 / 003D / 006B / 0118
SISTEMA.DIS22	314	IMP	000F / 0070 / 0071 / 008D / 00F9
SISTEMA.DIS23	887	PAN5, IS5	000F / 0029 / 003D / 006B / 0118
SISTEMA.DIS24	887	PAN6, IS6	000F / 0029 / 003D / 006B / 0118
SISTEMA.DIS25	887	PAN7, IS7	000F / 0029 / 003D / 006B / 0118
SISTEMA.DIS26	887	PAN8, IS8	000F / 0029 / 003D / 006B / 0118

ALP template parameters

All entries below are hexadecimal except the DIS file number.

DIS number	Status/data ports	Fixed common DMA byte / bit	Receive 67 index	DMA-owner 67 index
18	32/33	02A6 / 0	11	15
19	30/31	02A6 / 2	12	16
20	36/37	02A6 / 4	13	17
21	34/35	02A6 / 6	14	18
23	3A/3B	02A7 / 0	19	1D
24	38/39	02A7 / 2	1A	1E
25	3E/3F	02A7 / 4	1B	1F
26	3C/3D	02A7 / 6	1C	20

Internal routine index

These names are descriptive annotations. They are not original exported symbols or a stable public CALL ABI. Use the RST08 service where one exists; internal addresses depend on this exact image and overlay.

Callback and control paths

Address range	Function
2EA4–3061	Schedule/replace/cancel timed callback
3062–30EE	Timer expiry and quantum update
3564–35C1	Abbreviable keyword-table lookup
35D8–3618	Device input escape filtering
3619–3864	Terminal control dispatch

Additional callback entry points

Address range	Function
3435–346F	Save/disable per-partition record callback
3470–3497	Restore callback and deliver pending interruption
3671–3679,36C0–36C4	Mark Q interruption pending when callback is zero
overlay 3:448C–4545	Generic command cleanup, channel-state reset and lock release

Console layers

Address range	Function
1461–149A	Console policy wrapper 05
149B–161A	Console stream/cursor/control state 07
161B–1634	Class output+3 and raw fallback
1728–1757,1861–1873	Raw device 40 association 9 path
35D8–3618	Input 66 and class+0 callback
37CF–37DF	Escape-C returns 18 and marks cancellation

Terminal flow paths

Address range	Function
1392–13A6	Optional printable 04 echo through 05
1476–147A	Output-suppression bit test and conditional 07
34C2–34D8	Clear input inhibition and drain association 8
36DC–3701	Per-partition S pause/resume yield loop
37BC–37CE	Toggle context 0A bit 1 for O
2CC9–2CF1,30E5–30EE	Timer enable/disable and partition-count selection

Sequential EXTEND

Linked overlay 2 address	Function
4303–435D	OPEN mode selection and bit 2 handling
4398–439D	Missing-file EXTEND selects creation
440B–4463	Existing S endpoint read and channel metadata restoration
4464–447C	Backward 1A scan and append pointer/free count
4750–476A	CLOSE current-sector write and zero final-block slack

Disk-error paths

Common address	Function
09BF–0A58	One-sector mode, disk lock, saved request and association validation
0BE0–0C30	Driver result, remaining-count arithmetic and application return
0C31–0C5A	Release disk owner and wait state
0C5B–0C8D	Invalid-association diagnostic and abort
0C8E–0CC5	Optional context 26 callback and automatic attempt counter
0CC6–0D9D	Disk error text, logical coordinates and R/M/I/Q policy
0D9E–0DC8	Driver+3 call, abort 4E and context 26 lookup
0F99–0FCA /1079–10A1	WD read transfer before status check, error translation and pointer rollback
3199–31A5	Register context 26 disk-error callback through 4A

Filename and input translation

Common address	Function
1428–144D	Context+0A bit 6 gates lowercase folding and underscore→Delete
249E–24C6	Sweep 16 caller name/type bytes: retain spaces/$/digits/A–Z, fold a–z, other→$
24C8–24E1	Adjust first byte of each field; belowA except$ adds 11h
24E2 onward	Hash normalized fields and search directory
27CA–2810	Copy/normalize/create directory entry and return success
2811–281E	Failed create returnsFF/carry

Directory retry and label-check map

Common address	Function
2463–2480	FF drive iteration; restores pointers between failures but retains transformed fields
248B–24E1	Repeated caller-field normalization
2510–2513	Call label check when current account equals search account
25C9–25F2	Remember firstFF slot and reread its sector on failure
2600–261D	Public fallback unless context+0C bit 6 inhibits it
33D0–33ED	Read header 1, compare eight mounted label bytes, restoreBC/DE/HL

Separate synchronization mechanisms

Address / region	Function
1C9C–1D1B	Acquire shared/exclusive file-open entry; carry indicates conflict
1D1C–1D79	Remove one matching current-owner file-open entry
1D7A–1DA7	Remove current-owner open entries during 00 cleanup
1DA8–1FAB	Four-byte file-open table,516 bytes
208A–20AB	Byte acquire 54, wait via scheduler bit 4
20AC–20C8	Byte release 55, broadcast bit 4 clearing
262B–2658	Delete wrapper attempts exclusive acquisition but ignores carry
3498–34BF	OPEN admission loop; yield at 34AC on helper carry
Overlay 2:481C	CLOSE call to 1D1C

Address / region	Function
Overlay 3:44A7–44C8	Fixed byte-lock cleanup list

Byte queue paths

Address	Function
3320–3329 /3352–3355	Empty check and carry-set return 47
332B–334F	Read via eight-bit offset, decrement, conditional index update andEI
335E–3364	Cache occupancy, then full comparison without reloading it
3366–3375	Eight-bit occupancy/index addition, modulo and payload offset
337A–3385	Restore callerA, write, increment queue+01,EI and register restoration

Device events and callbacks

Address range	Function
1728–1898	Character-device dispatch and missing-association return
21E4–220D	Indexed owner/address registration 67
22B5–2334	ALP receive/DMA interrupt routing
2335–2351,23F1–23FC	SCC selection and nonzero callback lookup
2379–2385	Owner/address fetch and direct jump 063F
063F–06E9	Enter callback in selected context
2352–2368,23DA–23F0	DMA ready-state merge and owner bit 5 wakeup
06EA–0702	6B: one-partitionEI/HALT or bit 5 scheduler wait

Partition memory transfers

Address range	Function
1FAC–1FEF	68: selected-record bank to caller bank
1FF0–2033	69: caller bank to selected-record bank
2034–2055	Related entry helpers bypassing public record selection
2056–2077	Same-bank comparison, shared-stack selection orLDIR
2078–2089	SavedSP word and apparent 16-byte common stack area

Address range	Function
21DE–21E3	OUT(FD),A and update 046B
initialization 5C3D–5D30	Temporary-record bank search and complement/readback checks

Messaging routines

Address	Interpreted purpose
2141–21BD	Service 58 target checks, seven-bit text conversion, serialized send and release
21BE–21DB	Recipient callback preserving saved main registers
21DC/21DD	Tagged transfer byte / owner lock
063F–06E9	Target-context injection; self-modified entry and saved-frame restore

Resident lookup and timer ownership additions

Address	Function / observed contract
04BC–050B	Eight initialized named-component rows; no enforced numerical lookup limit
20DB–2107	6C exact eight-byte lookup; successAF0044/HLvalue, missAF0101/originalHL
3559–3563	5D eight-byte comparison used by 6C
2EBE–2EF1	35 inputA bit 0 bypasses owner check only for numerically matching head; accesses node in current mapping
2F2B–2F3E	Interior timer removal still requires current owner

Floppy packet and return translation

The private inputs areA=count, B=head, C=logical sector within track, DE=track, HL=buffer andIY=descriptor. Transfer adapter receives common bank byte 0x046B and buffer address. DIS2 programs port 0xC2 with the bank, address high byte and address low byte. The successful driver path advances C and H by count without an established general carry propagation; physical adapter carry across 64 KiB remains unspecified.

Packet offset	Meaning / construction
00	Read 06 or write 05; OR40 whenN=1
01	Saved unit with bit 2 cleared, OR B rotated left twice
02	Track E
03	Head B
04	First physical sector: C+1 forN=1;2*C+1 forN=0
05	Size code N:1 for 256-byte sectors;0 for 128-byte sectors
06	Last physical sector: first+count−1 forN=1;first+2*count−1 forN=0
07	Gap value 0E forN=1;07 forN=0
08	Data-length value 00 forN=1;80 forN=0

Condition	Returned A	Exact common-kernel message
ST0 bit 3	01	`no prep.`
ST1 bit 1	02	`prot. escritura`
ST1 bit 0	03	`no inicializado`
ST2 bit 5	04	`error transferencia`
ST2 bit 4	08	`no existe pista`
ST2 bit 1 or bit 0	03	`no inicializado`
ST1 bit 4, **DIS1 only**	09	`error cabeza`
ST1 bit 5	09	`error cabeza`
ST1 bit 2	07	`no existe sector`
No specified condition	00	Success path

Tape private operations

A selects one of the following operations through driver+0x0F. Targets and controller commands are hexadecimal and relative to the original driver. These are not RST08 services. Strongly inferred and tentative meanings remain distinct from documented physical execution.

A	Target	Controller command(s)	Meaning and notes
80	0165	06 then selector 0–3	selector/state initialization; E determines selector. Physical interpretation of selector Unknown
81	01EA	03	Probable rewind from utility sequences and TENSION help
82	0229	0C	code reads DE bytes at HL usingA1, with special asynchronous path; complete media transfer unspecified
83	032D	0D; retry 0E	code writes DE bytes fromHL throughA1; complete media transfer unspecified
84	0410	09	Probable reverse spacing from read/write retry use; exact physical unit unspecified
85	0462	08	Tentative forward spacing; can returnA05 at composite end condition
86	04A1	12	Probable erase action in retry sequence; extent Unknown
87	0514	0F	Probable write tape mark from label/trailer callers
88	05D9	06 then 80	Probable retension; INITTAPE TENSION invokes it and expectsA05
89	05F8	02 status query	composite status and current selector; selected status combinations specified
8A	04EC	07	Erase-related retry action; exact meaning Unknown
8B	0637	0A	Probable forward search to tape mark across selectors; suspicious final-selector return described below
8C	066A	04 plus reset handshake	reset sequence in code; physical effect unspecified

Tape ports and composite status

Port low byte	Original use	Notes
A0	command write, selector write, status reads	code and specified software path
A1	INIR/OTIR record bytes	code; no native data transfer
A2	control writes and bit 7/6 handshake/status reads	—
A3	FF/F0/5F/40/50/DF control writes	code; chip/pin meanings Unknown

Source	Output bit
second status 0B97 bit 3	A/C bit 5
second status bit 4	A/C bit 3
second status bits 6,1,0 all set	A/C bit 4
controlA2 bit 6 clear	A/C bit 2
constant	A/C bits 7,1,0 set

Appendix C. Operating-system files and language components

System files supply the executive, command environment, programming languages and device support. Other files hold account definitions, messages and temporary work state. This appendix groups related components so that a numbered family can be understood as a whole. Utility COMANDO files are described through their commands in Chapter 31.

The sizes and dates below identify particular component versions; replacement programs and growing work files can have different values. Optional components need not be present on every system.

Reading the file details

Each table below identifies a file in the Component column, followed by its storage format, size information and catalogue timestamp. Read the Format column before interpreting Contents (bytes): the meaning of the size depends on the file's organization.

Format column

The following codes are used in the Format column of this appendix's component tables. They describe how a file is stored or loaded, independently of its name or type suffix.

Format code	Meaning	What Contents (bytes) measures
A	Absolute machine-code program, loaded at a specified address	Stored bytes in whole 256-byte sectors; the total can include padding.
D	Direct file containing fixed-length records	Record capacity: number of records multiplied by bytes per record.
R	Relocatable machine-code program, adjusted to its load address	Load-image length, including code and initialized data but excluding relocation information and sector padding.
S	Sequential file, read as text or serialized binary records	Text tables count CR-inclusive content without terminal padding. Tables labelled “stored binary payload” count all linked payload, including padding.
I	Indexed file with keyed records and an ordered index	Active record data fields; excludes keys, index entries and unused record slots.

For example, R beside SISTEMA.CSI means that it contains relocatable machine code. D beside SISTEMA.EXEC1 means that it provides direct-record work storage, not another copy of the EXEC interpreter. These letters describe file formats; S in this column is unrelated to the `:S` system-volume association in a filename.

Contents, allocated space and dates

Contents (bytes) gives the size defined by the format table above. For S-format text files, the count includes every complete CR-terminated record before the terminal 0x1A padding. Sector links, padding and unused allocation are excluded. This interpretation applies to these text files; it is not a general instruction to truncate every binary S-format file at its first 0x1A byte.

Allocated (bytes) gives the disk space reserved for the file in 1,024-byte blocks. For example, SISTEMA.CSI has a 3,104-byte load image containing code and initialized data, but occupies 4,096 bytes of allocated space, which also accommodates relocation information and padding. SISTEMA.AYUDA contains 37,676 bytes of text and record delimiters within a 38,912-byte allocation. For D files, Contents is record capacity, not a count of occupied records. For the additional binary S-format tables, the

subsection explicitly identifies the count as stored binary payload: each linked sector contributes 254 payload bytes after its two-byte link. Padding is included because the exact program or data endpoint is unspecified. These are not decoded executable lengths.

Read protection is an attribute independent of native format. The compiled BASIC support programs use S storage with read protection; a catalogue display can show SR. They are not R-format relocatable Z80 programs. Indexed MAESTRO.VOC uses yet another measurement: its six active 512-byte data fields total 3,072 bytes, excluding keys and index structure.

Catalogue date and time gives the timestamp stored in the file's catalogue entry. It is file metadata, not necessarily a compilation date or software release date. Work files can change as the system is used, and their dates and allocation sizes can change too.

Where program and record sizes are stored

Program-loading information is held in the 32-byte catalogue entry rather than a universal header at the beginning of every program. For R files, entry offset +0x1E holds the image length, +0x11 the stored-sector count and +0x15 the starting sector. When relocation records are present, they follow the image rounded up to a 256-byte boundary. Some small R modules need no relocation records. For A files, +0x1E instead holds the absolute load address. For D files, +0x11 is the record count and +0x17 the record length. These fields must be interpreted according to format. Appendix A explains loading and language formats; Appendix E gives the byte layouts.

Executive and command environment

SISTEMA.NUCLEO contains the executive, initialization material and kernel overlays. The bootstrap loads it before the command environment. Its absolute load address is 0x0000; the stored component occupies 113 sectors, or 28,928 bytes. GENSIST can save configuration into this component, so it is not a text configuration file to edit by hand.

SISTEMA.STARTUP is executable initialization code that prepares system state, including overlay storage. SISTEMA.CSI is the command processor that reads commands and selects programs. On program completion, the system can reload CSI rather than return to a permanent command-processor stack. SISTEMA.EXECLANG interprets EXEC procedures, including variables, branching and nested execution. It is used by EXEC HELLO from Chapter 8; an EXEC.COMANDO file is not required.

SISTEMA.STARTUP begins by saving AF, DE, BC, HL and IY: its first six bytes are `F5 D5 C5 E5 FD E5`. Its 548-byte load image is followed by sector padding and one relocation sector containing 41 offsets. The complete stored length is 1,024 bytes. There is no opening jump-and-version header.

SISTEMA.IPL is an absolute-format component with load address 0xE000. Its code searches for SISTEMA.NUCLEO, accesses floppy-controller port families and copies 0x7000 bytes into low RAM before transferring through the reset vector. This indicates a kernel-loading role, but its launch requirements and participation in the hard-disk startup path remain unspecified. Its copy length differs from the 0x7100 stored bytes of the NUCLEO component listed here. It is not a documented operator reboot procedure.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
SISTEMA.NUCLEO	A	28,928	29,696	1986-04-15 10:48
SISTEMA.STARTUP	R	548	1,024	1985-02-04 16:07
SISTEMA.CSI	R	3,104	4,096	1981-12-11 07:54
SISTEMA.EXECLANG	R	5,146	7,168	1981-12-11 07:55
SISTEMA.IPL	A	1,792	2,048	1984-01-09 16:59

Account, help and device definitions

SISTEMA.CUENTA is the sequential account database. It holds names, identifiers, passwords where assigned and privileges; its special owner byte is 0xFF. Use the CUENTA procedures for listing accounts, creating or removing an account and changing a password. A listing can reveal stored passwords and should be kept private.

CONECTAR.COMANDO performs account connection and profile processing. It reads and displays SISTEMA.CONECTAR as sequential welcome/banner text during connection. The banner is display data, not executable connection code; its wording can be installation-specific.

SISTEMA.AYUDA holds command-help topics. AYUDA reads those topics, with the fallback limitation described in the command reference. SISTEMA.MENSERR holds 200 fixed records of 64 bytes, giving 12,800 bytes of message capacity. Message-record numbers are separate from BASIC error codes and command completion results; the stored message catalogue provides the lookup.

Account definitions in SISTEMA.CUENTA and device definitions in SISTEMA.NOMBDISP are system configuration, whose contents depend on the installation. SISTEMA.AYUDA and SISTEMA.MENSERR instead provide system reference text. Their shared use by system facilities does not make account names, banner wording or device choices universal defaults.

SISTEMA.NOMBDISP contains sequential device-name definitions. They supply the physical-name vocabulary used for device associations; a definition does not imply that the corresponding hardware is fitted. Chapter 12 explains active associations.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
SISTEMA.CUENTA	S	40	1,024	1990-01-01 00:37
SISTEMA.CONECTAR	S	436	1,024	1984-01-07 20:14
CONECTAR.COMANDO	R	2,141	3,072	1982-09-24 00:15
SISTEMA.AYUDA	S	37,676	38,912	1985-06-19 14:09
SISTEMA.MENSERR	D	12,800	13,312	1983-12-07 16:17
SISTEMA.NOMBDISP	S	783	1,024	1984-01-01 04:29

EXEC work files: SISTEMA.EXEC1–SISTEMA.EXEC4

These four files have the same purpose for different execution partitions. The numeric suffix selects the displayed partition number: EXEC1 belongs to partition 1, EXEC2 to partition 2, and so on. They hold EXEC nesting state and are managed by the interpreter. They are not four versions of EXEC and do not contain user-written command procedures. Saved frames can remain after use; nonzero work bytes alone do not establish that a procedure is still running, and zero-filled work storage can still be required.

Each listed file contains sixteen 128-byte direct records, giving 2,048 bytes of capacity. EXEC uses a 512-byte frame per nesting level, so this allocation holds four frames. The record size and the frame size are different units. Four frames describe this allocation, not a universal maximum nesting depth. Missing or unusable work capacity can prevent automatic account profiles. The frame layout is described in Appendix A.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
SISTEMA.EXEC1	D	2,048	2,048	1983-10-14 08:21
SISTEMA.EXEC2	D	2,048	2,048	1987-01-01 00:04
SISTEMA.EXEC3	D	2,048	2,048	1987-01-01 00:04
SISTEMA.EXEC4	D	2,048	2,048	1987-01-01 00:05

BASIC entry programs and runtime

BASIC.COMANDO selects interactive editing and interpretation when invoked without a source filename, or compilation when given a saved BASICOBJ source file. RUN.COMANDO starts compiled BASICCOM programs through the BASIC runtime. BASIC.LOADFILE supplies the compiled-program runtime, including compiled-header handling, statement and expression evaluation, and loading named BASICUSR extensions. It is an implementation component, not a saved user program.

Follow Chapter 15 to run the same program interactively and after compilation. Keep BASICOBJ source for editing: BASICCOM contains prepared runtime instructions rather than standalone Z80 code.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
BASIC.COMANDO	R	854	2,048	1981-12-11 08:00
BASIC.LOADFILE	R	24,175	27,648	1983-04-11 15:02
RUN.COMANDO	R	1,842	3,072	1981-12-11 07:59

BASIC.OVERLAY1–BASIC.OVERLAY4

These files contain different portions of the BASIC environment. An overlay supplies code when it is required so that all language facilities need not occupy the execution area simultaneously. Their suffixes identify components selected by BASIC, not user numbers, execution partitions or successive versions of one file. BASIC loads them as needed; they are not separate commands to invoke at CSI.

The following responsibilities explain why the family contains several different images. They describe identified facilities, not a complete assignment of every editor command or numbered compiler pass.

Component	Role
BASIC.OVERLAY1	Source parsing and tokenization, editor/language keyword tables, and embedded syntax help.
BASIC.OVERLAY2	Source-reference reporting, including line/label and variable/constant references.
BASIC.OVERLAY3	Compilation and compiled-output construction, including branch emission and compiler statistics.
BASIC.OVERLAY4	Interpreted statement execution, including file operations and named BASICUSR loading.

Each overlay begins with its component number, 0x01 through 0x04, followed by a JP entry slot at file offset 0x0001. BASIC.LOADFILE likewise begins `04 C3 0A 01`. The component byte is data, not an instruction to execute as an ordinary command. The BASIC loader selects the appropriate entry. Preserve the matching family when maintaining the language environment.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
`BASIC.OVERLAY1`	R	22,482	26,624	1981-12-11 07:55
`BASIC.OVERLAY2`	R	7,757	9,216	1981-12-11 07:56
`BASIC.OVERLAY3`	R	9,438	11,264	1982-05-07 17:30
`BASIC.OVERLAY4`	R	28,458	32,768	1983-04-11 14:45

Device drivers: the SISTEMA.DIS family

DIS numbers identify driver components through the physical device definitions. They do not denote a sequence of driver revisions. ASOCIAR selects devices through those definitions; the appropriate driver supplies the input/output routines. Similar terminal drivers can have the same length while serving different terminal ports. Equal lengths do not make them duplicate files: their port and configuration bytes differ.

SISTEMA.DIS1 and SISTEMA.DIS2 begin with four JP entry slots at offsets 0x00, 0x03, 0x06 and 0x09. The terminal, printer and tape drivers listed below have five opening slots through 0x0C. These are driver-specific entry tables; they must not be interpreted as a command program's version header.

Driver	Physical device or function
SISTEMA.DIS1	FLOPPY1
SISTEMA.DIS2	FLOPPY2
SISTEMA.DIS10	TAP tape device
SISTEMA.DIS11	IP printer
SISTEMA.DIS12	CDC printer
SISTEMA.DIS16	PAN9/IS9; SS2 terminal driver
SISTEMA.DIS17	PAN10/IS10; SS2 terminal driver
SISTEMA.DIS18	PAN1; ALP terminal driver
SISTEMA.DIS19	PAN2/IS2
SISTEMA.DIS20	PAN3/IS3
SISTEMA.DIS21	PAN4/IS4
SISTEMA.DIS22	IMP printer
SISTEMA.DIS23	PAN5/IS5
SISTEMA.DIS24	PAN6/IS6
SISTEMA.DIS25	PAN7/IS7
SISTEMA.DIS26	PAN8/IS8

These are relocatable programs. Appendix B gives driver entry offsets and hardware ports; Chapter 12 explains operator access to the devices. A driver file by itself does not supply the required physical equipment.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
SISTEMA.DIS1	R	605	1,024	1983-12-05 16:10
SISTEMA.DIS2	R	575	1,024	1983-12-05 15:53
SISTEMA.DIS10	R	2,980	4,096	1985-07-15 09:13
SISTEMA.DIS11	R	237	1,024	1984-02-04 03:57
SISTEMA.DIS12	R	438	1,024	1983-04-07 01:11
SISTEMA.DIS16	R	650	1,024	1984-01-01 00:40
SISTEMA.DIS17	R	650	1,024	1984-01-01 00:59
SISTEMA.DIS18	R	887	2,048	1984-10-24 17:19
SISTEMA.DIS19	R	887	2,048	1984-10-24 17:23
SISTEMA.DIS20	R	887	2,048	1984-10-24 17:27
SISTEMA.DIS21	R	887	2,048	1984-10-24 17:31
SISTEMA.DIS22	R	314	1,024	1984-08-23 09:54
SISTEMA.DIS23	R	887	2,048	1984-10-25 09:01
SISTEMA.DIS24	R	887	2,048	1984-10-24 17:38
SISTEMA.DIS25	R	887	2,048	1984-10-24 17:43
SISTEMA.DIS26	R	887	2,048	1984-10-24 17:46

Terminal classes: SISTEMA.CLASE1 and SISTEMA.CLASE2

A terminal class translates logical display operations into terminal-specific control sequences. CLASE1 uses ESC G coordinate sequences; CLASE2 uses ANSI-like cursor sequences. The files are executable translation routines, not editable lists of terminal settings. Select the class appropriate to the connected terminal through ASOCIAR, as described in Chapter 12. Repeatedly replacing a class can consume partition memory.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
SISTEMA.CLASE1	R	259	1,024	1984-05-31 19:10
SISTEMA.CLASE2	R	328	1,024	1984-12-04 23:59

BASIC extension modules

A BASICUSR file supplies a relocatable routine callable through BASIC's USR interface. OPTION USR selects a module; the selected routine determines the arguments and result. Sharing this file type does not make two routines interchangeable.

GETPRIV returns the current privilege, GETID returns the account number, and GETUSER fills an eight-byte string with the account name. Their elementary calls follow the table. Other modules have partly understood implementation roles, listed below, while their general argument and side-effect contracts remain incomplete. They are BASIC-loaded extension routines, not automatically members of the executive or interchangeable USR interfaces.

Module	Established operation and limits
CAMPAR	Calls scheduling/yield service 0x4F, then returns. Its complete image is three bytes, `CF 4F C9`; it occupies one stored sector and has no relocation sector.
PUTQUET	Preserves HL, passes HL = 0 to callback-setting service 0x39, restores HL and returns. Its name does not establish a queue operation.
CLAVOUT	Temporarily selects public-account lookup for CLAVE.CUENTA, restores the caller's account byte and reads the first sector into caller storage. If lookup fails, it zeroes the caller's first byte. The general calling contract remains incomplete.
LINPAN	Parses input text and changes device-descriptor fields; it is not a read-only line-count query.
ESPERA	Installs a callback, changes port 0x91 state, waits through service 0x6B and removes the callback. Its general input and device requirements are unspecified.
OCONV, OCONV2 and KEYSET	Extension routines used by application-support facilities; their complete transformation grammars are unspecified.

NOMVOL, TAMDIS and DIREC also remain without a complete documented calling contract. The elementary GETPRIV, GETID and GETUSER examples below should not be adapted to another module by changing only the module name.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
`GETPRIV.BASICUSR`	R	6	1,024	1981-06-01 12:37
`GETID.BASICUSR`	R	6	1,024	1981-06-17 21:20
`GETUSER.BASICUSR`	R	25	1,024	1981-04-16 12:38
`CAMPAR.BASICUSR`	R	3	1,024	1984-11-16 14:26
`PUTQUET.BASICUSR`	R	8	1,024	1983-05-25 14:22
`NOMVOL.BASICUSR`	R	18	1,024	1983-04-27 10:58

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
TAMDIS.BASICUSR	R	41	1,024	1983-06-01 09:25
DIREC.BASICUSR	R	91	1,024	1984-05-18 11:36
CLAVOUT.BASICUSR	R	61	1,024	1983-06-01 08:44
LINPAN.BASICUSR	R	89	1,024	1984-09-13 12:59
ESPERA.BASICUSR	R	65	1,024	1983-06-22 08:09
OCONV.BASICUSR	R	116	1,024	1981-07-28 12:38
OCONV2.BASICUSR	R	119	1,024	1981-09-14 10:21
KEYSET.BASICUSR	R	1,539	2,048	1982-11-17 15:58

CLAVIN, CLAVOUT and CLAVE.CUENTA

CLAVIN.COMANDO is the installed command-side companion of CLAVOUT.BASICUSR. Its header reads `CLAVIN ver 1.0 01JUN83`; its load image is 6,146 bytes with 686 relocation records, and its prologue requires privilege 5. The apparent main body has argument-writing and no-argument-reading paths associated with CLAVE.CUENTA. The pair appears to exchange a counted string or value, whose application meaning remains unspecified.

The command fails during startup with `CR = 254`, before those apparent main-body operations. Its stored code and relocation records are inconsistent: relocation at file offset 0x010D modifies bytes that include an instruction opcode, and another entry at 0x0191 leads to an unintended stack-pointer value on the observed startup path. The fault's origin and an authentic repair remain unknown. Creating CLAVE.CUENTA does not fix this failure. See CLAVIN for the invocation and its limits.

CLAVE.CUENTA is absent from the documented installation and is separate from the account database SISTEMA.CUENTA. Use CUENTA for account and password administration; neither helper provides an established alternative.

Read the current privilege

At the BASIC prompt, select the routine:

```
OPTION USR "GETPRIV"
```

OPTION USR loads the named extension for subsequent calls. Now call it and print its numeric result:

```
PRINT USR(0,0)
```

The number displayed is the current privilege. Control returns to BASIC.

Read the account number

Select the account-number routine:

```
OPTION USR "GETID"
```

This replaces the previously selected extension. Call the new routine:

```
PRINT USR(0,0)
```

The result is the current account number, which is distinct from the execution partition number.

Read the account name

Select the string-returning routine:

```
OPTION USR "GETUSER"
```

Provide an eight-character workspace and print the returned name:

```
PRINT USR$(0,SPACE$(8))
```

SPACE$(8) supplies the eight bytes that GETUSER fills. USR$ retrieves a string result; USR in the preceding examples retrieves a number.

SORT and CONSULTA overlays

SORT.OVERLAY1 and SORT.OVERLAY2 supply different parts of the sorting utility. Both begin with the literal `SORTCB` and a 0xF0-byte (240-byte) shared control/data area. The executable JP entry slots follow at offsets 0xF0, 0xF3 and 0xF6. SORT copies the control area into the selected overlay before calling it and copies it back afterward. OVERLAY2 appears to perform merge/output work; the complete sorting algorithm and phase division remain unspecified.

CONSULTA.COMANDO requests CONSULTA.OVERLAY1, OVERLAY2 and OVERLAY3; OVERLAY3 in turn requests OVERLAY4. These dependencies do not imply that every query loads all four components. A complete functional description of each overlay remains unavailable. In both families, the numbers distinguish code components selected by the utility, rather than independent commands or successive releases.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
`SORT.OVERLAY1`	R	4,183	6,144	1982-02-19 10:17
`SORT.OVERLAY2`	R	4,982	7,168	1982-02-18 09:07
`CONSULTA.OVERLAY1`	R	21,320	24,576	1983-02-07 13:34
`CONSULTA.OVERLAY2`	R	19,547	22,528	1983-01-27 16:15
`CONSULTA.OVERLAY3`	R	20,252	23,552	1982-09-24 00:50
`CONSULTA.OVERLAY4`	R	15,811	18,432	1983-02-07 13:37

CONSULTA.TRAB1–CONSULTA.TRAB4

These are direct work files used by CONSULTA for temporary structured state. Unlike the CONSULTA overlays, they hold data rather than code. Each listed file provides 400 records of 256 bytes, or 102,400 bytes of capacity. Their complete record layout and selection rule are unspecified; the numeric suffix alone is insufficient to promise the same partition mapping as the EXEC work files. CONSULTA maintains their contents, which cannot be replaced with text from EDITAR. Stored work records may remain after a query; their presence alone does not establish current activity.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
`CONSULTA.TRAB1`	D	102,400	102,400	1985-01-01 09:52
`CONSULTA.TRAB2`	D	102,400	102,400	1987-01-01 00:04
`CONSULTA.TRAB3`	D	102,400	102,400	1987-01-01 00:04
`CONSULTA.TRAB4`	D	102,400	102,400	1987-01-01 00:05

Spooler components and generated reports

SPOOLER.OVERLAY1 supplies executable spooler code. SISTEMA.SPOOLCOL is the direct queue file: its 64 records of 16 bytes describe reports, submitting partitions, dates, flags, queues and page counts. Its 1,024-byte capacity is queue metadata, not the combined capacity of the reports.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
SPOOLER.OVERLAY1	R	1,617	3,072	1984-04-27 12:44
SISTEMA.SPOOLCOL	D	1,024	1,024	1981-12-11 07:59

Report contents are stored separately in generated files named SISTEMA.SP$ followed by a report number. Their lengths and dates change with submitted work. The queue links each report to its state and page count. Use the spooler to inspect the current queue:

```
SPOOLER LIST
```

This displays queue information. LIST stops at the queue terminator; nonzero bytes beyond that terminator need not represent active reports. Do not delete report backing files while queue entries still refer to them. Chapter 29 explains report encoding and restart rules.

Large-lettering support data

MAESTRO.TITULO is a direct-data file referenced by TITULO.COMANDO, the utility for printing large lettering. Its compact records appear to provide character-pattern data. The full record interpretation and whether this particular file was distributed unchanged by the manufacturer remain unspecified. Its relationship to the utility supports treating it as probable utility support rather than classifying it from the MAESTRO name alone.

The file provides 100 records of 50 bytes, giving 5,000 bytes of capacity. Its sector-rounded stored length and allocation are both 5,120 bytes. It is data read by a utility, not a program to invoke directly. The TITULO reference explains the command and gives a console example.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
MAESTRO.TITULO	D	5,000	5,120	1983-02-08 00:06

Mailbox and duplication workspace

SISTEMA.BUZON

The sequential mailbox contains recipient/sender headers and message bodies. Its message header occupies 33 bytes; Appendix E documents the fields. This is a generated service file, so a fixed program size or catalogue date would be misleading. It need not exist before stored messaging is used.

To retrieve messages for the current account, enter:

```
BUZON
```

Retrieval removes the messages it displays. This operation is not a repeatable read-only view of the mailbox.

SISTEMA.DUPLICAR

This direct workspace supports floppy duplication. The preparation procedure allocates 4,000 records of 256 bytes, a capacity of 1,024,000 bytes. That is a requested workspace size, not an executable program length. Its catalogue date depends on its creation; no fixed date is specified. Chapter 11 gives the preparation and duplication steps. A workspace alone does not establish that a compatible floppy device is ready.

SISTEMA.BUZON, SISTEMA.DUPLICAR and generated spool report files are conditional service files; they should not be assigned a fixed installed size merely because a program can use them. CONECTAR also refers to SISTEMA.HISTORIA, whose full record contract and availability are unspecified. CLAVE.CUENTA is absent; its relationship to CLAVIN and CLAVOUT is described with the BASIC extension modules above.

Embedded component identification

These literal strings identify individual components. The offset is measured from the beginning of the file before relocation. It is not a universal header location. A date within a marker is reproduced as written and is separate from the catalogue date in the size tables.

Component	File offset	Literal identification text
SISTEMA.CSI	0x0003	`CSI ver 5.5 Copyright by Tim Williams`
SISTEMA.EXECLANG	0x0003	`EXEC ver 5.5 Copyright by T.S.Williams`
BASIC.COMANDO	0x0003	`BASIC VER 1.0 11DIC81`
RUN.COMANDO	0x0003	`RUN VER 1.0 11DIC81`
SORT.COMANDO	0x0003	`SORT ver 1.0A 18FEB82`
SORT.OVERLAY1	0x00FC	`SORTLTAG Ver 2.0 08SEP81`
SORT.OVERLAY2	0x00FC	`SORTMTAG Ver 1.0 02APR81`
CONSULTA.COMANDO	0x0003	`ACCESS ver 1.0D 01OCT82 Copyright de SECOINSA-D.TECNICA`
CONSULTA.OVERLAY1	0x0003	`CONSULTA ver 1.0 01OCT83 Cpyright de SECOINSA-D.TECNICA`
CONSULTA.OVERLAY2	0x0003	`CONSULTA ver 1.0 01OCT82 Cpyright de SECOINSA-D.TECNICA`
CONSULTA.OVERLAY3	0x0003	`CONSULTA ver 1.0 01OCT82 Cpyright de SECOINSA-D.TECNICA`
CONSULTA.OVERLAY4	0x0003	`CONSULTA ver 1.0 01OCT82 Cpyright de SECOINSA-D.TECNICA`

CONSULTA.OVERLAY1 contains the date 01OCT83 even though its catalogue timestamp is 1983-02-07 13:34. OVERLAY3 has a catalogue date of 1982-09-24, preceding its embedded 01OCT82 date. These disagreements are retained; neither field establishes a reliable compilation chronology. ACCESS and Cpyright are literal spellings in the components. Component versions such as CSI 5.5 and BASIC 1.0 are distinct from operating-system Version 6.2.

User-created source and program files

HELLO.BASICOBJ and HELLO.BASICCOM are created by the example in Chapter 15. BASICOBJ holds saved source for the BASIC editor; BASICCOM holds compiled input for the runtime. NOTE.TEXT and HELLO.EXEC are likewise teaching files. They are not bundled operating-system components.

Optional support procedures and compiled tools

The following files extend the BASIC programming and business-file environment. An EXEC procedure supplies prompts, input queues and calls to programs; a BASICCOM file contains compiled instructions consumed by RUN and the BASIC runtime. Identical stems therefore need not denote duplicate programs. For example, ACTUCAP.EXEC prepares arguments for ACTUCAP.BASICCOM. A direct RUN ACTUCAP omits that wrapper's preparation.

These components are optional support software, not resident executive facilities. Their presence alone does not establish that every required dictionary, screen definition, printer or companion program is available. The descriptions below distinguish the callable wrapper from the compiled component and identify modifying operations. Provenance of every optional component as an original manufacturer product has not been established.

Capture definitions and source generation

A capture definition describes the fields and behavior of a data-entry program. The support tools store those definitions in LOCAL.MACROS, maintain vocabulary in CONTROL.VOC, and can generate BASIC source. They do not constitute a general Z80 assembler or linker.

ACTUCAP.EXEC accepts an optional program or screen name, queues it and calls ACTUCAP.BASICCOM. Without the name the compiled program requests it. Its help describes creating or updating the screen, file and modification criteria in LOCAL.MACROS before source expansion. ACTUCAP2.BASICCOM supplies special-process definition support; the complete transition to that program is unspecified. ACTUCAP.PANTALLA and ACTUCAP2.PANTALLA hold supporting screen data.

Begin by requesting the wrapper's help:

```
EXEC ACTUCAP AYUDA
```

This selects its explanatory branch instead of opening the definition-maintenance dialogue. The operational form is illustrated by:

```
EXEC ACTUCAP ENTRY
```

ENTRY is a chosen definition name, not a supplied application. This form can modify LOCAL.MACROS and requires initialized support files. The field definitions must include every key and non-key field the generated program is to access. The complete interactive dialogue is not specified here.

EXPAND.EXEC reads an existing definition and drives EXPAND.BASICCOM, followed by BASIC source loading, indentation and saving. Optional compilation creates BASICCOM output. Its advertised options are PANT, COMPILAR, NOCOM, DISCO=df, NOB and BORRAR. Selected branches delete intermediate BASIC or BASICOBJ files, so expansion is a source-generating maintenance operation, not a read-only preview.

To read the available invocation information first, enter:

```
EXEC EXPAND AYUDA
```

A source-generation syntax illustration is:

```
EXEC EXPAND ENTRY
```

It requires a compatible ENTRY capture definition, the macro files and model, and the BASIC components. It is not a complete example that creates those definitions from scratch. EXPAND.MACMODEL supplies BASIC text with placeholders such as {PNAME}, {READ}, {WRITE}, {DELETE} and {SPECPROC}. USOIMPR.BASICMAC supplies printer-related BASIC source that opens IMPR1 and reconstructs screen lines for output. Neither is an executable command or a relocatable object library.

TESTCAP.EXEC queues an optional screen name and calls TESTCAP.BASICCOM. Its help describes testing a screen with one key: the first data field is the key and the final datum is an end-of-screen label. It states that this tool does not maintain files and that the key cannot be changed in modification mode. To read those restrictions, enter:

```
EXEC TESTCAP AYUDA
```

IMPCAP.EXEC queues a required screen name and calls IMPCAP.BASICCOM to print its definition data. For a screen named ENTRY and an appropriately associated printer, the syntax is:

```
EXEC IMPCAP ENTRY
```

This is a printing request, not screen execution. IMPCAP returns syntax result 8 when the required name is omitted. SCAPTURA.BASICCOM and GENPAN.BASICCOM belong to screen/capture support, but their complete standalone invocation contracts remain unspecified. SCAPTURA includes a named DIREC.BASICUSR call; that dependency does not turn it into a general file-inspection command.

The following EXEC and model entries count CR-inclusive text bytes.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
ACTUCAP.EXEC	S	945	1,024	1981-11-25 11:18
EXPAND.EXEC	S	2,577	3,072	1982-08-06 00:41

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
TESTCAP.EXEC	S	818	1,024	1981-11-25 09:21
IMPCAP.EXEC	S	485	1,024	1981-11-24 16:13
EXPAND.MACMODEL	S	15,898	16,384	1984-06-25 04:59
USOIMPR.BASICMAC	S	661	1,024	1981-11-27 08:58

The following S entries count stored binary payload including padding, not decoded program lengths. The BASICCOM programs require RUN or their wrappers; PANTALLA files are supporting data and are not automatically interchangeable with PANTALLA editor output of type PANT.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
ACTUCAP.BASICCOM	S	4,318	5,120	1981-12-01 17:48
ACTUCAP2.BASICCOM	S	3,556	4,096	1982-08-16 11:38
ACTUCAP.PANTALLA	S	1,270	2,048	1981-11-27 11:40
ACTUCAP2.PANTALLA	S	762	1,024	1982-01-25 08:47
EXPAND.BASICCOM	S	5,588	6,144	1982-09-17 00:32
TESTCAP.BASICCOM	S	3,556	4,096	1982-08-16 11:49
IMPCAP.BASICCOM	S	1,778	2,048	1984-11-30 14:15
SCAPTURA.BASICCOM	S	6,096	6,144	1984-06-19 13:59
GENPAN.BASICCOM	S	6,096	6,144	1984-05-16 18:33

Dictionary creation and vocabulary maintenance

GENFICH.EXEC creates the working files used by capture and query support. It accepts an optional logical disk letter, defaulting to S. First request its help:

```
EXEC GENFICH AYUDA
```

The creation form is:

```
EXEC GENFICH S
```

Use this only when preparing a new support environment with unused working filenames. It creates the indexed files below, copies seed vocabulary through a temporary CONTROL.ANTVOC file, calls COPIVOC.BASICCOM and deletes that temporary file. Existing contents and interrupted creation require separate review; this is not an automatic repair procedure.

Working file	Record bytes	Initial record capacity	Key bytes
LOCAL.MACROS	512	100	8
PROMPT.MACROS	34	200	11
CONTROL.VOC	512	100	30

MAESTRO.VOC supplies the seed vocabulary. Its indexed allocation has room for eleven 512-byte data fields; six are active in the component described here. Each slot also contains the 30-byte key and a two-byte link, making 544 bytes per slot. Index storage and block allocation account for further space. The contents figure reports only the six active data fields.

GENVOC.EXEC creates vocabulary entries from a capture screen. It accepts a screen name and optional target filename; omitting the latter uses the screen name. The NOC option suppresses per-datum questions. Its help restricts creation to files with a single access key. For an existing ENTRY screen, the syntax illustration is:

```
EXEC GENVOC ENTRY
```

This adds definitions to CONTROL.VOC and requires the screen data and dictionary to exist. It does not create an arbitrary application file. To inspect the procedure's full introductory text, use its AYUDA argument.

VOC.EXEC manages relationships between named attributes and stored data. With no function it starts VOC.BASICCOM for maintenance. LIST or PANT lists the dictionary on the screen, IMPR prints it, and BORRAR removes attribute groups through MANTVOC.BASICCOM. FICHERO optionally restricts those operations to a named file. A non-deleting example is:

```
EXEC VOC LIST
```

This invokes CONSULTA with the prepared dictionary-listing query explained in Chapter 35. It requires a usable CONTROL.VOC and the partition's CONSULTA work file. VOC.PANTALLA supports the maintenance interface.

COPIVOC.EXEC enlarges or rebuilds a dictionary. It asks for old and new disk associations and a new record capacity, removes deletion protection, renames the old dictionary to CONTROL.ANTVOC, creates a new CONTROL.VOC, runs COPIVOC.BASICCOM and removes the temporary old file on success. Its error branch attempts restoration. These steps change names, protection and data; they do not make every interrupted migration automatically recoverable. Review the existing dictionary and retain a separate backup before using this interactive procedure.

The following S procedure entries count text bytes; MAESTRO.VOC counts active indexed data fields.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
GENFICH.EXEC	S	943	1,024	1982-07-09 15:52

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
GENVOC.EXEC	S	1,327	2,048	1981-11-25 12:04
VOC.EXEC	S	2,060	3,072	1982-02-18 09:22
COPIVOC.EXEC	S	937	1,024	1983-02-04 08:47
MAESTRO.VOC	I	3,072	7,168	1983-01-01 00:02

The following entries count stored binary payload including padding. COPIVOC.BASICCOM is also called by GENFICH; MANTVOC.BASICCOM implements the vocabulary-removal path. They should not be invoked as substitutes for their wrappers without preparing the expected state.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
GENVOC.BASICCOM	S	1,524	2,048	1983-03-02 01:05
VOC.BASICCOM	S	4,572	5,120	1982-02-18 10:20
MANTVOC.BASICCOM	S	508	1,024	1981-12-01 16:27
COPIVOC.BASICCOM	S	254	1,024	1981-12-01 16:26
VOC.PANTALLA	S	1,524	2,048	1982-02-18 09:51

Report generation and source insertion

INFORME.EXEC coordinates report-program generation. It obtains the current partition through GETPART.BASICCOM, uses the completion result to select that partition's numbered work file (for example, CONSULTA.TRAB1 for partition 1), empties that work file, queues supplied arguments and starts INFORME1.BASICCOM. INFORME1–INFORME5 are members of the report-support family, not five independent versions of one command. A complete assignment of later generation stages to members 2–5 remains unspecified.

Request the wrapper's help before creating a report definition:

```
EXEC INFORME AYUDA
```

This takes the help branch before GETPART and work-file clearing. The documented generation form takes a program name and an optional A or C for an open or closed report. Its syntax is illustrated by:

```
EXEC INFORME REPORT A
```

This can replace the partition's report work state and generates BASIC source from operator-supplied definitions. It needs the corresponding numbered CONSULTA work file, BASIC and the report components. It is not a ready-made report over arbitrary files. INFORME1 uses the OCONV2 and KEYSET BASICUSR modules. GETPART is a compiled helper whose result is a partition selector; it is not an instruction to create a partition.

INSER.EXEC queues supplied arguments, runs INSER1.BASICCOM and asks whether to perform another insertion. INSER1.EXEC is a separate command procedure: it drives BASIC and EDITAR to copy a range of source statements, renumber them and save BASICOBJ output on S. The distinction between INSER1.EXEC and INSER1.BASICCOM matters when choosing an entry path.

INSER1.EXEC takes five fields: destination program, insertion point, source program, first source statement and statement count. An illustrative form is:

```
EXEC INSER1 TARGET 0 SOURCE 100 5
```

Here zero requests insertion at the beginning of TARGET. SOURCE and TARGET must already be compatible saved BASIC/BASICOBJ programs; 100 identifies the first source statement and 5 the number to insert. This operation rewrites saved source and uses temporary INSER$ files. When source and destination are the same, it removes the original statements after moving them to avoid duplication. Save separate copies before using it; default editing positions, malformed ranges and recovery after interruption are not fully specified.

The three procedure entries below count text bytes.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
INFORME.EXEC	S	1,133	2,048	1983-10-01 00:34
INSER.EXEC	S	186	1,024	1982-08-02 18:02
INSER1.EXEC	S	2,042	3,072	1982-08-02 15:03

The compiled entries below count stored binary payload including padding. Use the report or insertion wrapper for their supplied setup.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
GETPART.BASICCOM	S	254	1,024	1983-10-17 16:30
INFORME1.BASICCOM	S	4,826	5,120	1983-10-01 00:39
INFORME2.BASICCOM	S	6,096	6,144	1983-05-19 13:34
INFORME3.BASICCOM	S	6,350	7,168	1983-05-19 13:35
INFORME4.BASICCOM	S	7,112	7,168	1983-05-19 13:54
INFORME5.BASICCOM	S	5,334	6,144	1983-09-23 01:33

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
INSER1.BASICCOM	S	1,524	2,048	1982-12-24 12:54

System setup, copying and version migration

CIPL.EXEC configures accounts, partitions, associations, printer/spooler state and work files. It is administrative setup software, not a prerequisite command for every login. It invokes a further procedure named CIPL1 whose implementation is unavailable, so its complete execution cannot be prescribed as a normal startup procedure.

The following is a syntax illustration only:

```
EXEC CIPL ALP4 A IP
```

ALP4 selects the ALP terminal branch; it does not mean exactly four users. That branch begins with partition 2 and terminal PAN2. NP1 through NP7 reduce the activation limit; the default limit is eight. The A argument requests a public association of logical disk A with physical definition 4, which must already describe appropriate available storage. It does not create or format a volume. IP selects the parallel-printer device in place of the default IMP selection. Use it only with the required hardware and definitions. The procedure allocates or adjusts partitions and initializes spooler/work state, so it is not a harmless status query. Account and partition administration are taught independently in Chapters 10 and 12.

COPSIS.EXEC and COPSIS2.EXEC copy system components. They are alternative procedures, not numbered partitions. COPSIS asks for source and destination and can initialize destination media. COPSIS2 contains an explicit component list that includes optional targets not available in every installation. Neither should replace the planned backup and restoration procedures of Chapter 13 without checking its source, destination and required components.

CAMBVERS.EXEC implements a 6.1/6.1A-to-6.2 migration path. It copies components and invokes O8PRECOM to change disk-precompensation state. It is not required for a system already running 6.2. O8PRECOM performs controller writes; its complete invocation and recovery contract remains unspecified. The command entry therefore provides an inspection example rather than a migration command to try.

These procedure entries count CR-inclusive text bytes.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
CIPL.EXEC	S	1,548	2,048	1984-01-01 04:41
COPSIS.EXEC	S	6,847	7,168	1985-01-18 12:04
COPSIS2.EXEC	S	6,912	7,168	1984-01-02 01:21
CAMBVERS.EXEC	S	1,729	2,048	1986-01-01 19:03

BASIC source routines and other support files

RUTINA1.BASIC through RUTINA4.BASIC are source libraries with different purposes. They are neither separate CSI commands nor revision numbers of one routine. They provide material to incorporate into a BASIC program with the required variables, file state and hardware assumptions.

Source file	Purpose and prerequisites
RUTINA1.BASIC	FNDICO% binary search in a direct file; requires the routine's record/key conventions.
RUTINA2.BASIC	FNVFEC$ due-date calculation using a commercial year; do not assume civil-calendar arithmetic.
RUTINA3.BASIC	Load and display labels from PANT definitions; uses the DIREC file-information interface.
RUTINA4.BASIC	Rectangle drawing described for an 8500 screen; applicability to another terminal is not established.

To inspect one source library before incorporating it, enter:

```
LISTAR RUTINA1.BASIC:S
```

This displays source text and its explanatory comments; it does not execute FNDICO%. Read the routine's required data before copying it into another program. The source insertion tools above can modify programs, so their presence does not remove the need to understand the copied routine.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
RUTINA1.BASIC	S	18,697	19,456	1984-05-18 13:19
RUTINA2.BASIC	S	17,655	18,432	1984-05-16 14:05
RUTINA3.BASIC	S	16,949	17,408	1984-11-12 08:34
RUTINA4.BASIC	S	4,392	5,120	1984-10-05 12:23

The remaining compiled files require individual interpretation. LONCLA.BASICCOM prompts for a filename and contains a key-length result message, but the complete input/error contract is unspecified. It is distinct from CREAR's LONCLA option. DISCO.BASICCOM contains a VERFORM A command string; this establishes a connection with media checking, not a general disk-management interface. RESUME.BASICCOM has no adequately specified standalone purpose and is distinct from BASIC's RESUME statement.

TRAFIC.BASICCOM presents an input/auxiliary/output-file transformation dialogue and can create an EXEC control file. Its prompts describe selecting keys, fields and output format. They do not establish a complete safe transformation recipe. In particular, output selection can write files, and an optional generated control procedure can replay those choices later. No bare RUN example is prescribed for these incompletely specified tools.

These entries count stored binary payload including padding.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
DISCO.BASICCOM	S	254	1,024	1985-01-04 21:39
LONCLA.BASICCOM	S	508	1,024	1984-10-16 14:42
RESUME.BASICCOM	S	254	1,024	1983-06-16 08:42
TRAFIC.BASICCOM	S	5,842	6,144	1983-10-17 13:20

MAESTRO.POLIZA is a retained direct-format template whose consumer and full purpose remain unspecified; its name alone does not identify a supported application. Its Contents count is fixed-record capacity rather than the number of occupied records. MAESTRO.TITULO is described separately above and is not another missing member of this group.

Component	Format	Contents (bytes)	Allocated (bytes)	Catalogue date and time
MAESTRO.POLIZA	D	612	1,024	1982-11-16 15:33

Appendix D. BASIC commands, statements and functions

Use this reference after the first-program example in Chapter 15. Editor commands are entered at the BASIC dash prompt. Numbered statements become stored program lines. Functions produce values inside expressions, often displayed with PRINT. None of these examples is a CSI command unless explicitly labelled as such.

Every documented form has its own elementary example below. A fragment that names a source line, array or file channel requires that object to exist. It is not a complete program by itself. Chapters 15–17 supply complete workflows and programs.

Interface limits. The core tutorials give defined working behavior. Several additional forms have only an incomplete contract. Their entries distinguish their documented argument shape from a probable role or an unknown effect. A syntax illustration is not a promise of the displayed result, device operation or complete editor dialogue. In particular, unfamiliar string, date and device functions must not be assigned another BASIC dialect's semantics.

Angle brackets in syntax identify placeholders. The example replaces them where the argument domain is known. A remaining device placeholder explicitly means that no runnable device example is specified. Source help has unmatched brackets in some forms; those are normalized only where optional punctuation is unambiguous.

Editor and debugger commands

AUTO

Generate source line numbers automatically.

Syntax: `AUTO [<start>[,<incr>]]`

Example

```
AUTO 10,10
```

10 is the first number and 10 the increment. Enter source at the offered numbers. The complete cancellation dialogue is unspecified.

BOTTOM

Select the bottom of the source-editing area.

Syntax: `BOTTOM`

Example

```
BOTTOM
```

This is an editor positioning command, not program execution. The exact screen redraw and subsequent editing interaction are unspecified.

BREAK AT

Set a breakpoint associated with a source line.

Syntax: `BREAK [AT <line> [AFTER <count>]]`

Example

```
BREAK AT 20
```

This requests a stop at line 20 of the loaded program. AFTER adds a count condition; the complete counting and continuation rules are unspecified.

BREAK ON change

Request a variable-related breakpoint.

Syntax: `BREAK ON <var> [CHANGE] [AFTER <count>]`

Example

```
BREAK ON A CHANGE
```

This form names A and requests change monitoring. Use a disposable program; exact watchpoint timing and initial comparison rules are unspecified.

BREAK ON comparison

Request a breakpoint when a variable meets a comparison.

Syntax: `BREAK [ON <var> <relat> <value>]`

Example

```
BREAK ON A = 7
```

The form compares A with 7. The full breakpoint lifecycle is unspecified; it is not a CSI conditional command.

CHANGE

Replace text in source.

Syntax: `CHANGE [/<from>/<to>[/ <range>]]`

Example

```
CHANGE /HELLO/WELCOME/
```

The slash-delimited fields identify old and new text. Save a copy first; the default range and interactive confirmation rules are unspecified.

CONTINUE

Continue a suspended BASIC execution.

Syntax: `CONTINUE`

Example

```
CONTINUE
```

This belongs after a resumable BASIC stop. It does not resume a command terminated by FIN or Escape-Q. Complete resumability rules are unspecified.

DELETE source

Remove source lines.

Syntax: `DELETE [<range>]`

Example

```
DELETE 20
```

This selects source line 20 for deletion. Save the source first. It is distinct from DELETE #channel,key, which affects file records.

INDENT

Control the source indentation setting.

Syntax: `INDENT [<value>]`

Example

```
INDENT 2
```

This supplies an indentation value of 2. The exact application to existing lines versus later entry is unspecified.

LENGTH

Request the BASIC source-length facility.

Syntax: `LENGTH`

Example

```
LENGTH
```

The command has no argument in the documented form. The output units and accounting rules are unspecified; do not equate them with allocated disk blocks.

LIST

Display source lines.

Syntax: `LIST [<range>]`

Example

```
LIST
```

This displays the program in memory. It does not execute it or write it to disk. LIST 10 selects an individual line in the documented range form.

LOAD

Read saved BASIC source into the editor.

Syntax: `LOAD <name>[.<type>][:<drive>]`

Example

```
LOAD HELLO:S
```

This selects HELLO.BASICOBJ on S. Save current work before loading another program.

LOCATE

Search source text.

Syntax: `LOCATE [/string[/<range>]]`

Example

```
LOCATE /HELLO/
```

This supplies HELLO as the search string. Default range, cursor placement and repeat-search behavior are unspecified.

LPnLIST

Direct a source listing to printer n.

Syntax: `LPnLIST`

Example

```
LP1LIST
```

This selects printer 1. The logical printer must be associated with ready hardware; command acceptance does not prove printed output.

LPnXREF

Direct a cross-reference listing to printer n.

Syntax: `LPnXREF`

Example

```
LP1XREF
```

This requests cross-reference output on printer 1. The exact report layout is unspecified.

MODIFY

Enter source modification for a selected range.

Syntax: `MODIFY [<range>]`

Example

```
MODIFY 10
```

This selects line 10. The complete editing keystrokes and completion dialogue are unspecified; re-entering a numbered line is the basic replacement method taught in Chapter 15.

NAME

Set or query the program name used by the editor.

Syntax: `NAME [<name>[.<type>][:<drive>]]`

Example

```
NAME HELLO:S
```

This supplies a name and drive. It is not a substitute for SAVE; persistence and default-name interactions beyond this form are unspecified.

NEW

Begin a new source program.

Syntax: `NEW`

Example

```
NEW
```

Save work before using NEW because it replaces the current editing context. It does not create an on-disk BASICOBJ until you save.

QUIT command

Leave the BASIC environment.

Syntax: `QUIT [<code>|<CSI command>]`

Example

```
QUIT
```

This returns to CSI. Optional result-code and CSI-command forms are documented, but their complete interaction with open files is unspecified. Close files explicitly.

RENUMBER

Renumber source lines.

Syntax: `RENUMBER [<first> [<incr> [<start> [<end>]]]]`

Example

```
RENUMBER 10 10
```

This requests numbering starting at 10 in increments of 10. Back up before using it; handling of every embedded target and optional range is unspecified.

RUN command

Execute BASIC source through the interpreter.

Syntax: `RUN [<name>][ <line>]`

Example

```
RUN
```

This runs the current source. It differs from CSI RUN, which loads BASICCOM. Execution does not save your source.

SAVE

Write saved BASIC source.

Syntax: `SAVE [<name>[.<type>][:<drive>]]`

Example

```
SAVE HELLO:S
```

This creates or updates HELLO.BASICOBJ:S. It saves source rather than producing compiled BASICCOM.

STEP

Request a bounded debugging step.

Syntax: `STEP [<count>]`

Example

```
STEP 1
```

This supplies count 1 to the stepping facility. Exact stepping units and the required suspension state are unspecified.

TOP

Select the top of the source-editing area.

Syntax: `TOP`

Example

```
TOP
```

This is an editor positioning request. It is not the first-line execution command RUN.

TRACE

Enable execution tracing.

Syntax: `TRACE [VARS]`

Example

```
TRACE
```

Tracing concerns execution rather than saving source. VARS requests the documented variable-related variant; exact display and trace granularity are unspecified.

UNBREAK

Remove a breakpoint selection.

Syntax: `UNBREAK [AT <line>|ON <var>]`

Example

```
UNBREAK AT 20
```

This removes the breakpoint associated with line 20. The ON variable form targets a variable watch rather than a line.

UNTRACE

Disable tracing.

Syntax: `UNTRACE`

Example

```
UNTRACE
```

Use this after a tracing session to request ordinary execution display again.

VARS

Display variables.

Syntax: `VARS [<var-list>]`

Example

```
VARS A
```

This selects variable A for inspection. Omitting the list requests the general variable display; exact formatting is unspecified.

XREF

Request source cross-references.

Syntax: `XREF`

Example

```
XREF
```

This is a navigation aid for source identifiers and their uses. It does not compile or change the program by itself.

Line-number display

Display a source line by its number.

Syntax: `nnnn (display line)`

Example

```
10
```

At the BASIC prompt this requests line 10. Distinguish a number alone from 10 PRINT ..., which supplies source text.

Ctrl-Z

Move upward in BASIC source editing.

Syntax: `ctl-z (up line)`

Example

```
Press Ctrl-Z
```

This is a key action, not literal text. The same byte is not a CSI history command.

LF key

Move downward in BASIC source editing.

Syntax: `lf-key (down line)`

Example

```
Press the line-feed key
```

Send LF, character 0x0A. Do not assume that a modern keyboard's Return key sends LF; the usual command terminator is CR.

Return key

Display the current source line in the documented editing mode.

Syntax: `ret-key (display line)`

Example

```
Press Return
```

This sends CR, character 0x0D. Its editor meaning depends on being in BASIC's editing context rather than at CSI.

Program statements

CASE

Introduce a case within structured selection.

Syntax: `CASE <expr>`

Example

```
20 CASE 1
```

This supplies the value 1 for comparison inside a SELECT construct. The full SELECT/CASE matching rules are unspecified; the fragment is not a complete program.

CEND

Terminate a structured selection construct.

Syntax: `CEND`

Example

```
40 CEND
```

CEND belongs with SELECT/CASE. It does not terminate the whole BASIC program like END.

CHAIN

Request transfer to another BASIC program.

Syntax: `CHAIN <str-expr>`

Example

```
10 CHAIN "NEXT:S"
```

The string names the next program. Exact source/compiled selection and variable/option inheritance are unspecified; do not assume another BASIC dialect's CHAIN contract.

CLEAR

Clear selected variable state.

Syntax: `CLEAR [<var-list>]`

Example

```
10 CLEAR A
```

This selects A in the documented variable-list form. Exact treatment of arrays, strings and omitted lists is unspecified.

CLOSE

Close an open file channel.

Syntax: `CLOSE #<channel>`

Example

```
CLOSE #1
```

Use this after finishing channel 1. It completes output handling and releases the open association; it does not mean logout.

COMMON

Declare variables for interprogram sharing.

Syntax: `COMMON <var-list>`

Example

```
10 COMMON A,S$
```

The declaration names numeric A and string S$. Exact CHAIN/LINK lifetime and layout rules are unspecified, so this is a declaration example only.

CSI

Request a command-processor operation from BASIC.

Syntax: `CSI <str-expr>`

Example

```
10 CSI "ESTADO QUIEN"
```

The string supplies a CSI command. Full return, variable retention and open-channel behavior are unspecified; use the direct CSI example for routine identity checks.

DATA

Store literal values for later READ statements.

Syntax: `DATA <lit-list>`

Example

```
10 DATA 11,2.5,"ABC",-4
```

READ consumes these values in order. DATA is not a file-writing operation.

DEF FN

Define a user function.

Syntax: `DEF FN<name>[(<arguments>)][=<expression>]`

Example

```
10 DEF FNT(X)=X+1
```

The function takes X and returns X+1. PRINT FNT(2) uses it after the definition is available. FNEND closes the separate multiline form.

DELETE record

Delete a keyed record through an open channel.

Syntax: `DELETE #<channel>,<key>`

Example

```
DELETE #1,"MMMM"
```

This removes the active reference for key MMMM from a compatible opened indexed/keyed file. It does not delete source line MMMM or the entire file.

DIM

Declare an array.

Syntax: `DIM <dim-var-list>`

Example

```
DIM ARR(2)
```

With OPTION BASE 0, ARR has indices 0–2; with BASE 1, indices 1–2. Set the base before declaring the array.

ELSE

Introduce the alternative action of a conditional.

Syntax: `ELSE [<statement>|<line-ref>]`

Example

```
20 IF A=7 THEN PRINT "YES" ELSE PRINT "NO"
```

ELSE selects PRINT "NO" when the condition fails. It belongs to the IF statement; it is not an independent unconditional action.

END

Finish program execution.

Syntax: `END`

Example

```
20 END
```

Control returns from the program to its execution environment. It does not save source or disconnect the account.

FNEND

End a multiline function definition.

Syntax: `FNEND`

Example

```
100 FNEND
```

This is the terminator within that definition form. The complete multiline return-value contract is unspecified; use the elementary DEF FNT(X)=X+1 form first.

FOR

Start a counted or value-list loop.

Syntax: `FOR <var>=[<expr-list>|<num-expr> TO <num-expr>[ STEP <num-expr>]]`

Example

```
10 FOR I%=1 TO 3
```

NEXT closes the loop. The range uses an implicit step of 1; a list form such as FOR I%=7,9,11 supplies successive values directly.

GET

Read through the language's device-access form.

Syntax: `GET <device> <num-expr>,<var-list>`

Example

```
GET <device> 1,A
```

The device placeholder must be replaced by a supported designator. The valid device vocabulary and transfer contract are unspecified, so this is syntax only, not a runnable hardware-access example.

GOSUB

Call a subroutine at a source line.

Syntax: `GOSUB <line-ref>`

Example

```
10 GOSUB 100
```

Line 100 must exist. RETURN in the subroutine resumes after this GOSUB; see the complete program in Chapter 16.

GOTO

Transfer execution to a source line.

Syntax: `GOTO <line-ref>`

Example

```
10 GOTO 100
```

Execution continues at line 100 without a saved subroutine return. The target must exist.

IF

Choose an action based on a condition.

Syntax: `IF <num-expr>[ THEN <statement>[ ELSE <statement>]]`

Example

```
20 IF A=7 THEN PRINT "SEVEN"
```

The PRINT runs only when A equals 7. ELSE can add an alternative.

IFEND

End a block IF construct.

Syntax: `IFEND`

Example

```
50 IFEND
```

Use it after the block's THEN and optional ELSE sections. It is distinct from END, which finishes the program.

INPUT

Read values into variables.

Syntax: `INPUT [<str-expr>,|#<channel>[,<key>]:]<var-list>`

Example

```
10 INPUT "VALUE",A
```

This is the documented prompted-input form: display the prompt and request a value for A. File INPUT instead uses #channel and a colon. Complete conversion/retry rules are unspecified.

LET

Assign a value.

Syntax: `[LET] <var>[ [<num-expr>:<num-expr>] ]=<expr>`

Example

```
10 LET A=7
```

LET is optional: A=7 is also assignment. It stores the value in A rather than comparing A with 7 in a conditional.

LINK

Request a program link using a string specification.

Syntax: `LINK <str-expr>`

Example

```
10 LINK "NEXT:S"
```

The form supplies a program name. Its return and state-sharing contract is unspecified; it must not be confused with a native object-file linker.

LINPUT

Read line-oriented text into a variable.

Syntax: `LINPUT [[<str-expr>,][USING <str-expr>]|#<channel>[,<key>]:]<var>`

Example

```
10 LINPUT "NAME",A$
```

This supplies a prompt and string destination in the documented form. Exact delimiter and editing rules are unspecified; INPUT and LINPUT need not parse text identically.

MAT copy

Assign one array through a matrix statement.

Syntax: `MAT <var>=<var>`

Example

```
20 MAT A=B
```

Declare compatible arrays first. The form identifies B as source and A as destination; dimension matching and conversion rules are unspecified.

MAT fill

Assign a scalar expression through a matrix statement.

Syntax: `MAT <var>=(<expr>)`

Example

```
20 MAT A=(0)
```

The parentheses distinguish this expression form. Array bounds and exact fill semantics are incompletely specified.

MAT INPUT

Request array input.

Syntax: `MAT INPUT [#<channel>[,<key>]:] <var>`

Example

```
20 MAT INPUT A
```

A must be a declared array. The complete element order, prompts and error behavior are unspecified.

MAT PRINT

Request array output.

Syntax: `MAT PRINT [#<channel>[,<key>]:] <var-list>`

Example

```
20 MAT PRINT A
```

The statement selects A for printing. Exact element layout and separators are unspecified.

MAT READ

Request array values from the READ source.

Syntax: `MAT READ [#<channel>[,<key>]:] <var>`

Example

```
20 MAT READ A
```

Declare the array and supply compatible DATA before using a complete program. Element traversal and conversion details are unspecified.

MAT WRITE

Write array values through a file channel.

Syntax: `MAT WRITE #<channel>[,<key>]:<var>`

Example

```
20 MAT WRITE #1:A
```

Channel 1 must already be open for compatible output. Exact matrix record serialization is unspecified; ordinary WRITE is taught in Chapter 17.

MOUNT

Request media handling through a string specification.

Syntax: `MOUNT <str-expr>`

Example

```
10 MOUNT "A"
```

The string supplies a drive-like specification. The full accepted string grammar and relationship to CSI MONTAR are unspecified, so this is syntax only.

NEXT

Advance and repeat a FOR loop.

Syntax: `NEXT [<var>]`

Example

```
30 NEXT I%
```

This closes the FOR I% loop in Chapter 16. It is not a general jump to the next source line.

ON ERROR

Select an error-handler line.

Syntax: `ON ERROR GOTO <line-ref> | 0`

Example

```
10 ON ERROR GOTO 100
```

Line 100 must contain the handler. ON ERROR GOTO 0 disables the handler path. Nested errors and every RESUME interaction are unspecified.

ON GOTO

Choose a branch from a one-based target list.

Syntax: `ON <num-expr> GOTO <line-ref-list>`

Example

```
20 ON 2 GOTO 100,200,300
```

This selects line 200. Selector zero is out of range; supply all targets in the complete program.

ON GOSUB

Choose a subroutine from a one-based list.

Syntax: `ON <num-expr> GOSUB <line-ref-list>`

Example

```
20 ON 2 GOSUB 100,200,300
```

This calls line 200; RETURN resumes after the entire ON statement. Targets must be valid subroutines.

OPEN

Connect a channel to a file with a mode and organization.

Syntax: `OPEN #<channel>: <str-expr>,<mode><method>[<options>]`

Example

```
OPEN #1: "DIRECT.DATOS:S", UPDATE DIRECT
```

Create the direct file first as in Chapter 17. The colon after #1 is required. OPEN can have immediate side effects, especially sequential replacement.

OPTION

Select a language or runtime option.

Syntax: `OPTION [BASE 0|1]|[PROMPT <str-expr>]|[USR <str>]|[CASE <str>]`

Example

```
OPTION BASE 1
```

This selects one-based array indexing for subsequent declarations. Other options have separate examples below.

OTHERWISE

Introduce the fallback arm of structured selection.

Syntax: `OTHERWISE`

Example

```
30 OTHERWISE
```

This belongs inside SELECT/CASE/CEND. The full matching and fallthrough contract is unspecified; the fragment is not an independent program.

PRINT

Display values or write formatted output to a channel.

Syntax: `PRINT [#<channel>[,<key>]:][USING <str-expr>,][<expr-list>]`

Example

```
PRINT "HELLO"
```

Quotation marks identify literal text. PRINT A instead evaluates variable A. The basic console form does not create a disk file.

PUT

Write through the language's device-access form.

Syntax: `PUT <device><num-expr>,<expr-list>`

Example

```
PUT <device> 1,65
```

The device placeholder and selector require an established device contract. That contract is unspecified here; do not type this as a guessed port write.

QUIT statement

Leave BASIC from program execution, optionally supplying a result.

Syntax: `QUIT [<expr>]`

Example

```
20 QUIT 0
```

This requests successful completion to the surrounding environment. Close file channels beforehand.

RANDOMIZE

Request initialization of random-number generation.

Syntax: `RANDOMIZE`

Example

```
10 RANDOMIZE
```

The form has no argument. Seed selection and reproducibility rules are unspecified; do not rely on a particular sequence.

READ

Consume DATA values or read a selected file record.

Syntax: `READ [#<channel>,<key>:]<var-list>`

Example

```
20 READ A,B,S$,I%
```

With DATA 11,2.5,"ABC",-4, these variables receive the four values in order. The #channel,key form instead reads from a compatible file.

READNEXT

Read the next indexed record and return its key.

Syntax: `READNEXT #<channel>,<var>:<var-list>`

Example

```
READNEXT #1,K$:A$
```

With a compatible indexed channel positioned after key AAAA, this retrieves the next key into K$ and its data into A$. The example file in Chapter 17 next has MMMM/MIDDLE.

REM

Include a source comment.

Syntax: `REM <anything>`

Example

```
10 REM PRINT A GREETING
```

BASIC retains the comment as source text. It is not a PRINT operation.

RESTORE

Reset the DATA reading position.

Syntax: `RESTORE [<line-ref>]`

Example

```
RESTORE
```

The next READ begins again at the first DATA. RESTORE with a line reference selects that DATA line; missing/non-DATA targets are unspecified.

RESUME

Continue from an error-handling context.

Syntax: `RESUME [<line-ref>|0]`

Example

```
100 RESUME 20
```

This documents a specified target line for resumption. Complete retry and nested-error semantics are unspecified; it is not a substitute for fixing the error cause.

RETURN

Return from a GOSUB subroutine.

Syntax: `RETURN [<line-ref>]`

Example

```
110 RETURN
```

Execution resumes after the calling GOSUB. The optional line-reference variant has an incompletely specified contract.

RUN statement

Start execution through the program-level RUN form.

Syntax: `RUN [<str-expr>|<line>]`

Example

```
10 RUN "NEXT:S"
```

The string selects another program. Exact retention and loading rules for this form are unspecified; ordinary interactive RUN is the introductory execution method.

SELECT

Begin structured selection on a value.

Syntax: `SELECT <expr>`

Example

```
10 SELECT A
```

CASE entries supply alternatives and CEND closes the construct. Full coercion and matching rules are unspecified; use IF first for elementary decisions.

SLEEP

Request a delay or suspended interval.

Syntax: `SLEEP <num-expr>`

Example

```
10 SLEEP 1
```

The argument is numeric. Its time unit, rounding and wake conditions are unspecified, so do not interpret 1 as one second without a defined contract.

STOP

Suspend program execution through BASIC.

Syntax: `STOP [<expr>]`

Example

```
20 STOP
```

This differs from END and from CSI PARAR. The complete continuation and open-file behavior is unspecified.

THEN

Introduce the true arm of IF.

Syntax: `THEN [<statement>|<line-ref>]`

Example

```
20 IF A=7 THEN PRINT "YES"
```

THEN is reached when the condition succeeds. It is part of the conditional grammar, not a separate unconditional command.

WAIT

Request the no-argument waiting operation.

Syntax: `WAIT`

Example

```
10 WAIT
```

The wake condition is unspecified. Do not use it as an assumed keyboard prompt or timed delay.

WAIT device

Wait using a device and numeric conditions.

Syntax: `WAIT <device><num-expr>,<num-expr>[,<num-expr>]`

Example

```
WAIT <device> 1,1
```

The supported designator and condition-bit interpretation are unspecified. This is syntax only and must not be used as an invented device-control recipe.

WEND

Close a WHILE loop.

Syntax: `WEND`

Example

```
40 WEND
```

Execution returns to the corresponding WHILE condition. Exact nesting and exceptional-exit rules are unspecified.

WHILE

Start a condition-controlled loop.

Syntax: `WHILE <num-expr>`

Example

```
10 WHILE A<3
```

The intended loop continues while its condition is true and closes with WEND. This is a form illustration; FOR/NEXT is the complete introductory loop example.

WRITE

Serialize values to an open file channel.

Syntax: `WRITE #<channel>[,<key>]:<expr-list>`

Example

```
WRITE #1: "FIRST"
```

Channel 1 must be open for output. CLOSE it afterward. BASIC WRITE stores typed records rather than guaranteeing plain text bytes.

Functions

Entries marked Interface incomplete give a documented argument form and a probable functional interpretation. Their following notes identify the missing contract. Do not assume an expected result where none is stated.

ABS — absolute value

Absolute value. The elementary use below has a defined contract.

Syntax: `ABS(X)`

Example

```
PRINT ABS(-3)
```

The numeric result in this elementary case is 3.

ASC — character-to-number conversion

Interface incomplete.

Syntax: `ASC(A$)`

Example of the documented form

```
PRINT ASC("A")
```

The string is the argument; exact empty-string and multicharacter handling are unspecified.

AT$ — cursor-position string

Interface incomplete.

Syntax: `AT$(X,Y)`

Example of the documented form

```
PRINT AT$(1,1)
```

Two numeric coordinates are supplied. Coordinate order/base and generated terminal bytes are unspecified.

ATN — inverse tangent

Interface incomplete.

Syntax: `ATN(X)`

Example of the documented form

```
PRINT ATN(1)
```

A numeric argument is supplied. Angular units, precision and boundary behavior are unspecified.

BIN — binary-string conversion to a number

Interface incomplete.

Syntax: `BIN(A$)`

Example of the documented form

```
PRINT BIN("101")
```

The argument is text. Accepted alphabet, width and overflow handling are unspecified.

BINOF$ — number conversion to binary text

Interface incomplete.

Syntax: `BINOF$(X)`

Example of the documented form

```
PRINT BINOF$(5)
```

A numeric argument produces a string form; width and signed representation are unspecified.

CHR$ — character from a numeric byte value

Character from a numeric byte value. The elementary use below has a defined contract.

Syntax: `CHR$(X)`

Example

```
PRINT CHR$(65)
```

CHR$(222) supplies the actual 0xDE field delimiter in the EXT$ example. Numeric values are not typed as literal character names.

COS — cosine

Interface incomplete.

Syntax: `COS(X)`

Example of the documented form

```
PRINT COS(0)
```

The elementary form takes one numeric argument. General units, precision and range rules are unspecified.

CRT$ — terminal-control string conversion

Interface incomplete.

Syntax: `CRT$(A$)`

Example of the documented form

```
PRINT CRT$("A")
```

The documented argument is a string; the accepted control vocabulary and resulting bytes are unspecified. This is syntax only.

DATE$ — date string facility

Interface incomplete.

Syntax: `DATE$(X)`

Example of the documented form

```
PRINT DATE$(0)
```

The numeric selector's meaning and output format are unspecified. No current-date result is promised by this illustration.

DAY — date/day conversion facility

Interface incomplete.

Syntax: `DAY(A$)`

Example of the documented form

```
PRINT DAY("01/01/90")
```

The date text is illustrative; accepted format and numeric result meaning are unspecified.

DEL$ — delimited-string deletion family

Interface incomplete.

Syntax of the demonstrated form: `DEL$(A$,X,Y)`

Example of the documented form

```
PRINT DEL$("ABC",1,1)
```

This three-argument example is accepted and prints an empty string, visible as an empty output line. A fourth argument causes FALTAN PARENTESIS and enters source-correction editing. The delimiter, position and boundary rules remain incomplete; this result is not ordinary character deletion producing AC.

DTE$ — date-text conversion facility

Interface incomplete.

Syntax: `DTE$(A$)`

Example of the documented form

```
PRINT DTE$("01/01/90")
```

The accepted input date grammar and output representation are unspecified.

EOF — end condition for a file channel

End condition for a file channel. The elementary use below has a defined contract.

Syntax: `EOF(X)`

Example

```
PRINT EOF(1)
```

Query this after a read on open channel 1. An end condition can be -1; it is separate from a CSI result and from physical chain termination.

ERL — error-location value

Interface incomplete.

Syntax: `ERL`

Example of the documented form

```
PRINT ERL
```

Use in an established error context. The value need not be zero before a deliberate error is raised. Exact value outside a handler and interpreted/compiled correspondence are unspecified.

ERR — error-code value

Interface incomplete.

Syntax: `ERR`

Example of the documented form

```
PRINT ERR
```

The value belongs to BASIC's error state, not necessarily a SISTEMA.MENSERR record number. It need not be zero before a deliberate error is raised. Complete lifetime rules are unspecified.

EXP — exponential function

Interface incomplete.

Syntax: `EXP(X)`

Example of the documented form

```
PRINT EXP(1)
```

A numeric argument is supplied. Numeric limits, rounding and overflow behavior are unspecified.

EXT$ — extract a delimited field or subfield

Extract a delimited field or subfield. The elementary use below has a defined contract.

Syntax: `EXT$(A$,X,Y)`

Example

```
PRINT EXT$("UN"&CHR$(222)&"DOS",2,0)
```

This returns DOS: field 2, whole field selected by subfield 0. Fields use 0xDE and subfields 0xDD.

FIX — numeric conversion/truncation facility

Interface incomplete.

Syntax: `FIX(X)`

Example of the documented form

```
PRINT FIX(3.5)
```

The distinction from INT and FLOAT is not fully specified. This form alone does not promise a rounding result.

FLOAT — conversion to a real numeric value

Interface incomplete.

Syntax: `FLOAT(X)`

Example of the documented form

```
PRINT FLOAT(3)
```

The accepted numeric input and exceptional conversion rules are incompletely specified.

FORMAT$ — numeric formatting into a string

Interface incomplete.

Syntax: `FORMAT$(X,A$)`

Example of the documented form

```
PRINT FORMAT$(7,"###")
```

The second argument is a format string. Its pattern grammar is unspecified; the hashes illustrate a string argument, not a guaranteed supported mask.

HEX — hexadecimal-text conversion to a number

Interface incomplete.

Syntax: `HEX(A$)`

Example of the documented form

```
PRINT HEX("FF")
```

Accepted prefixes, case, width and overflow rules are unspecified.

HEXOF$ — number conversion to hexadecimal text

Interface incomplete.

Syntax: `HEXOF$(X)`

Example of the documented form

```
PRINT HEXOF$(255)
```

The function takes a number; string width and signed representation are unspecified.

INP — input-related numeric function

Interface incomplete.

Syntax: `INP`

Example of the documented form

```
PRINT INP
```

The documented form has no explicit argument. Its source, blocking behavior and result meaning are unknown; this is syntax only.

INS$ — delimited-string insertion family

Interface incomplete.

Syntax: `INS$(A$,X,Y,B$)`

Example of the documented form

```
PRINT INS$("ABC",1,1,"X")
```

Four arguments are supplied. Exact field/subfield positioning and insertion behavior are unspecified.

INT — integer-valued numeric conversion

Interface incomplete.

Syntax: `INT(X)`

Example of the documented form

```
PRINT INT(3.5)
```

Do not infer negative-number rounding from the name. FIX and INT may differ; their complete contracts are unspecified.

LEFT$ — left-hand string selection

Interface incomplete.

Syntax: `LEFT$(A$,X)`

Example of the documented form

```
PRINT LEFT$("ABC",2)
```

The string and numeric length form are documented. Bounds and exceptional-length behavior are unspecified.

LEN — string-length query

Interface incomplete.

Syntax: `LEN(A$)`

Example of the documented form

```
PRINT LEN("ABC")
```

The argument is a string. Treatment of embedded control bytes and length limits is incompletely specified.

LINE — line-state query

Interface incomplete.

Syntax: `LINE(X)`

Example of the documented form

```
PRINT LINE(1)
```

The selector domain and whether it denotes a channel or output device are unspecified. This is syntax only.

With no channel open, this example reports [27] Fich. Cerrado. It is not a complete positive query example: the required open-channel context and useful returned-value contract remain incompletely specified.

LOG — logarithm

Interface incomplete.

Syntax: `LOG(X)`

Example of the documented form

```
PRINT LOG(1)
```

The base, input domain and error handling are not fully specified for this interface.

LPAD$ — left-side string padding

Interface incomplete.

Syntax: `LPAD$(A$,X)`

Example of the documented form

```
PRINT LPAD$("A",4)
```

The numeric argument is a size-like value. Padding character, truncation and width conventions are unspecified.

LRL — left rotation family

Interface incomplete.

Syntax: `LRL(X,Y)`

Example of the documented form

```
PRINT LRL(1,1)
```

Two numeric arguments are documented. Bit width, direction/count interpretation and signed behavior are unspecified.

LRR — right rotation family

Interface incomplete.

Syntax: `LRR(X,Y)`

Example of the documented form

```
PRINT LRR(2,1)
```

Two numeric arguments are documented; exact bit-operation rules are unspecified.

LSL — left shift family

Interface incomplete.

Syntax: `LSL(X,Y)`

Example of the documented form

```
PRINT LSL(1,1)
```

The name suggests shifting, but operand width and count limits are unspecified. No numeric result is asserted.

LSR — right shift family

Interface incomplete.

Syntax: `LSR(X,Y)`

Example of the documented form

```
PRINT LSR(2,1)
```

Logical versus sign-propagating behavior and operand width are unspecified.

LTRIM$ — left-side string trimming

Interface incomplete.

Syntax: `LTRIM$(A$)`

Example of the documented form

```
PRINT LTRIM$("  A")
```

The exact removable character set and empty-string handling are unspecified.

MATCH — string matching

Interface incomplete.

Syntax: `MATCH(A$,B$)`

Example of the documented form

```
PRINT MATCH("ABC","A")
```

The pattern language and numeric result convention are unspecified. Do not assume modern regular-expression syntax.

MAX — larger-value selection

Interface incomplete.

Syntax: `MAX(X,Y)`

Example of the documented form

```
PRINT MAX(2,3)
```

The elementary form has two numeric arguments. Mixed-type and exceptional-value conversion rules are unspecified.

MID$ — interior string selection

Interface incomplete.

Syntax: `MID$(A$,X,Y)`

Example of the documented form

```
PRINT MID$("ABC",2,1)
```

A string and two numeric arguments are supplied. Exact indexing and out-of-range rules are incompletely specified.

MIN — smaller-value selection

Interface incomplete.

Syntax: `MIN(X,Y)`

Example of the documented form

```
PRINT MIN(2,3)
```

Two numeric arguments are supplied; general conversion and boundary rules are unspecified.

MOD — remainder facility

Interface incomplete.

Syntax: `MOD(X,Y)`

Example of the documented form

```
PRINT MOD(7,2)
```

Signed, fractional and zero-divisor behavior is unspecified. Do not infer it from integer division alone.

NBR — numeric query with an unspecified role

Interface incomplete.

Syntax of the demonstrated form: `NBR(A$)`

Example of the documented form

```
PRINT NBR("1")
```

This string-argument example prints -1. A numeric argument produces NECESITA EXPRESION STRING and enters correction editing. The returned value's meaning remains unknown; this example does not establish numeric conversion or any other general interpretation.

OCT — octal-text conversion to a number

Interface incomplete.

Syntax: `OCT(A$)`

Example of the documented form

```
PRINT OCT("10")
```

Accepted input alphabet, prefixes and overflow handling are unspecified.

OCTOF$ — number conversion to octal text

Interface incomplete.

Syntax: `OCTOF$(X)`

Example of the documented form

```
PRINT OCTOF$(8)
```

The output width and representation of negative values are unspecified.

OVR$ — delimited-string overwrite family

Interface incomplete.

Syntax: `OVR$(A$,X,Y,B$)`

Example of the documented form

```
PRINT OVR$("ABC",1,1,"X")
```

The four-argument form is documented; exact field/subfield and replacement rules are unspecified.

PAGE — page-state query

Interface incomplete.

Syntax: `PAGE(X)`

Example of the documented form

```
PRINT PAGE(1)
```

The selector domain, counter units and output-device relationship are unspecified.

With no channel open, this example reports [27] Fich. Cerrado. It is not a complete positive query example: the required open-channel context and useful returned-value contract remain incompletely specified.

PI — pi numeric constant

Interface incomplete.

Syntax: `PI`

Example of the documented form

```
PRINT PI
```

The function-like constant takes no argument. Exact stored precision is unspecified.

POS — position-state query

Interface incomplete.

Syntax: `POS(X)`

Example of the documented form

```
PRINT POS(1)
```

The numeric selector and returned coordinate convention are unspecified.

With no channel open, this example reports [27] Fich. Cerrado. It is not a complete positive query example: the required open-channel context and useful returned-value contract remain incompletely specified.

REP$ — delimited-string replacement family

Interface incomplete.

Syntax: `REP$(A$,X,Y,B$)`

Example of the documented form

```
PRINT REP$("ABC",1,1,"X")
```

Exact replacement range and field/subfield behavior are unspecified; do not substitute it for EXT$.

RIGHT$ — right-hand string selection

Interface incomplete.

Syntax: `RIGHT$(A$,X)`

Example of the documented form

```
PRINT RIGHT$("ABC",2)
```

The documented form accepts a string and number; exact bounds and exceptional lengths are unspecified.

RND — random-number generation

Interface incomplete.

Syntax: `RND`

Example of the documented form

```
PRINT RND
```

The documented form takes no argument. Distribution, range and reproducibility rules are unspecified.

ROUND — numeric rounding

Interface incomplete.

Syntax: `ROUND(X,Y)`

Example of the documented form

```
PRINT ROUND(1.25,1)
```

The second argument's scale and midpoint rule are unspecified. No particular rounded result is promised.

RPAD$ — right-side string padding

Interface incomplete.

Syntax: `RPAD$(A$,X)`

Example of the documented form

```
PRINT RPAD$("A",4)
```

Padding character, truncation and width interpretation are unspecified.

RPT$ — repeat a string

Repeat a string. The elementary use below has a defined contract.

Syntax: `RPT$(X,A$)`

Example

```
PRINT RPT$(3,"X")
```

The count comes first, then the string. This produces three X characters. Reversing the argument types is not accepted.

RTRIM$ — right-side string trimming

Interface incomplete.

Syntax: `RTRIM$(A$)`

Example of the documented form

```
PRINT RTRIM$("A  ")
```

The exact removable character set and bounds are unspecified.

SCH — string searching family

Interface incomplete.

Syntax: `SCH(X,A$,B$)`

Example of the documented form

```
PRINT SCH(1,"ABC","B")
```

The order and interpretation of search operands and the result convention are unspecified; this is a form illustration only.

SEC — time-related string conversion

Interface incomplete.

Syntax: `SEC(A$)`

Example of the documented form

```
PRINT SEC("00:00:01")
```

The accepted time grammar and unit of the numeric result are unspecified.

SGN — sign classification

Interface incomplete.

Syntax: `SGN(X)`

Example of the documented form

```
PRINT SGN(-3)
```

The single-argument form is documented; precise result values and mixed numeric rules are incompletely specified.

SIN — sine

Interface incomplete.

Syntax: `SIN(X)`

Example of the documented form

```
PRINT SIN(0)
```

Angular units, precision and numeric bounds are unspecified.

SPACE$ — create a string of spaces

Create a string of spaces. The elementary use below has a defined contract.

Syntax: `SPACE$(X)`

Example

```
PRINT SPACE$(8)
```

This supplies eight spaces. It is useful as the eight-byte workspace for GETUSER.

SQR — square-root family

Interface incomplete.

Syntax: `SQR(X)`

Example of the documented form

```
PRINT SQR(4)
```

This example displays 1.999999999999, an approximation to 2. Do not use the displayed digits as an exact equality guarantee. The full numeric domain, precision and error behavior remain incompletely specified.

STR$ — convert a numeric value to text

Interface incomplete.

Syntax: `STR$(X)`

Example of the documented form

```
PRINT STR$(7)
```

Sign, spacing and decimal formatting conventions are unspecified.

TAN — tangent

Interface incomplete.

Syntax: `TAN(X)`

Example of the documented form

```
PRINT TAN(0)
```

Angular units, precision and singularity handling are unspecified.

TIME$ — time-string facility

Interface incomplete.

Syntax: `TIME$(X)`

Example of the documented form

```
PRINT TIME$(0)
```

The selector and output format are unspecified; no current-time contract is implied by the example.

TRIM$ — string trimming

Interface incomplete.

Syntax: `TRIM$(A$)`

Example of the documented form

```
PRINT TRIM$(" A ")
```

The exact character set removed and relationship to LTRIM$/RTRIM$ are incompletely specified.

USR — call a numeric basic extension

Call a numeric BASIC extension. The elementary use below has a defined contract.

Syntax: `USR(X,Y)`

Example

```
PRINT USR(0,0)
```

First select GETPRIV or GETID with OPTION USR. Offset 0 and numeric argument 0 follow those getters' contracts; other modules require their own interfaces.

USR$ — call a string basic extension

Call a string BASIC extension. The elementary use below has a defined contract.

Syntax: `USR$(X,A$)`

Example

```
PRINT USR$(0,SPACE$(8))
```

First select GETUSER with OPTION USR. The string supplies an eight-byte workspace; the result contains the account name.

VAL — convert numeric text to a value

Interface incomplete.

Syntax: `VAL(A$)`

Example of the documented form

```
PRINT VAL("7")
```

Accepted whitespace, signs, decimal notation and invalid-tail behavior are unspecified.

Additional controls and build options

OPTION BASE 0

Select zero-based array indices before DIM. DIM ARR(2) then admits 0, 1 and 2.

Example

```
OPTION BASE 0
```

OPTION PROMPT

Supply the prompt string in the documented option form. Exact scope across input statements is unspecified.

Example

```
OPTION PROMPT "? "
```

OPTION CASE

The documented option accepts a string. The supported values and whether U is accepted are unspecified; this is argument-shape illustration only, not an established case-setting recipe.

Example

```
OPTION CASE "U"
```

OPTION USR

Select the account-number extension before PRINT USR(0,0). Selection and invocation are separate operations.

Example

```
OPTION USR "GETID"
```

OPTION PRIV

Require at least privilege 1 when this compiled statement runs. A lower account is rejected; the option does not elevate privilege.

Example

```
OPTION PRIV 1
```

OPTION SERIAL

Illustrate a two-part serial binding. The value must match the intended system; 22-0 is an example, not a universal setting. All-zero binding bypasses the BASIC comparison.

Example

```
OPTION SERIAL 22-0
```

UNLOCK

Release record spans for channel 1. Exclusive OPEN ownership remains until CLOSE.

Example

```
UNLOCK #1
```

OPEN INPUT

Open an existing sequential BASIC data file for reading. It must contain records appropriate to the intended BASIC input operation.

Example

```
OPEN #1: "LINES.DATOS:S", INPUT SEQUENTIAL
```

OPEN OUTPUT

Open sequential output. Choose a new filename or deliberately accept replacement; close the channel after writing.

Example

```
OPEN #1: "LINES.DATOS:S", OUTPUT SEQUENTIAL
```

OPEN UPDATE DIRECT

Open the fixed-record practice file for direct access. This is different from sequential replacement.

Example

```
OPEN #1: "DIRECT.DATOS:S", UPDATE DIRECT
```

OPEN UPDATE SEQUENTIAL

This replaces the existing file through deletion and recreation at OPEN. It is not an in-place text-editing mode.

Example

```
OPEN #1: "LINES.DATOS:S", UPDATE SEQUENTIAL
```

OPEN EXTEND

Append to a correctly closed sequential file. EXTEND is a trailing option; do not use it on unfinished metadata.

Example

```
OPEN #1: "LINES.DATOS:S", OUTPUT SEQUENTIAL, EXTEND
```

OPEN LOCK

Request exclusive open admission. A waiting opener can proceed after CLOSE; UNLOCK alone is insufficient.

Example

```
OPEN #1: "DIRECT.DATOS:S", UPDATE DIRECT, LOCK
```

Compiler LIST

At CSI, compile with a console source listing.

Example

```
BASIC HELLO.BASICOBJ:S (LIST)
```

Compiler NOLIST

At CSI, suppress the compilation listing.

Example

```
BASIC HELLO.BASICOBJ:S (NOLIST)
```

Compiler IMPRn

At CSI, select printer 1 for the listing. The printer must be associated and ready.

Example

```
BASIC HELLO.BASICOBJ:S (IMPR1)
```

Compiler XREF

At CSI, request a compilation cross-reference listing.

Example

```
BASIC HELLO.BASICOBJ:S (XREF)
```

Compiler NOOBJ

NOOBJ suppresses BASICCOM generation; it does not itself request a source listing. Compiler statistics can still appear. To list the source without producing compiled output, combine LIST and NOOBJ at CSI. An older BASICCOM can remain and must not be mistaken for newly generated output.

Example

```
BASIC HELLO.BASICOBJ:S (LIST NOOBJ)
```

LIST requests the source listing; NOOBJ independently suppresses the compiled file. NOOBJ alone does not imply LIST and does not suppress compilation processing.

Compiler NOPANT

At CSI, suppress console output. Avoid this while learning or diagnosing compilation.

Example

```
BASIC HELLO.BASICOBJ:S (NOPANT)
```

Compiler OBJ

At CSI, select A for compiled output. A must be an accessible writable disk association.

Example

```
BASIC HELLO.BASICOBJ:S (OBJ=A)
```

Compiler COMMA

The supplied help advertises COMMA for decimal-comma formatting, but this version of the compiler rejects it. The following is a rejected invocation, not a working build procedure.

Example

```
BASIC HELLO.BASICOBJ:S (COMMA)
```

The result is Opcion no valida, CR = 8. The alternative compiler spelling COMA is also rejected. Use dot-decimal source in the programming examples; a working compiler decimal-comma option has not been established.

Runtime TRACE

The supplied help advertises TRACE for runtime-address tracing. The following invocation runs the compiled greeting or numeric example, but produces no visible runtime-address trace for those programs. It must not be treated as a demonstrated runtime-debugging procedure.

Example

```
RUN HELLO.BASICCOM:S (TRACE)
```

CR = 0 establishes that the program completed, not that this option was recognized. An arbitrary option also completes with the same numeric output. Other internal effects or runtime conditions remain unspecified.

For a compilation trace, use the separate compiler invocation:

```
BASIC HELLO.BASICOBJ:S (TRACE)
```

This displays PASO # 1 through PASO # 8 and source line numbers during compilation. It does not trace execution of the compiled program. The interactive BASIC editor's TRACE command is also a separate interface; see TRACE.

Runtime COMA

The supplied help advertises COMA as a runtime decimal-comma option. Its spelling has one M. The demonstrated numeric program still prints 1.5 with this option, so this is not an established way to change decimal display.

For this example, use HELLO compiled from the following two source lines, following the save-and-compile procedure in Chapter 15:

```
10 PRINT 1.5
20 END
```

This numeric version replaces the greeting used earlier. Compile it before checking decimal display; an older greeting executable cannot demonstrate a numeric-format option.

Example

```
RUN HELLO.BASICCOM:S (COMA)
```

Ordinary RUN and RUN with COMA both display 1.5 and return CR = 0. This does not establish input-conversion behavior or exclude other internal effects. PRINT 1,5 supplies two items to PRINT; it is not a decimal-comma literal. Compiler COMMA and COMA are separately rejected, as described above.

Additional organization, option and expression keywords

The following seventeen names complete the vocabulary beyond the forms above. Their presence in BASIC's keyword table establishes their spelling; it does not establish another BASIC dialect's calling conventions. KEYED and INDEXED have file-opening context, and NOT, AND and OR have identified compiled-expression operations. Several other names still lack a complete programmer-visible contract. They are retained here so that source text can be recognized without suggesting unsupported examples.

INDEXED

INDEXED selects the indexed-file organization in OPEN. For the existing INDEX.DATOS file created in Chapter 17, enter at BASIC:

```
OPEN #1: "INDEX.DATOS:S", UPDATE INDEXED
```

This connects channel 1 to the existing indexed file for update. It does not create its index or define its record and key sizes. Follow Chapter 17 for creation, writing and reading by key, and close the channel before reusing it.

KEYED

KEYED selects keyed access without the I-format ordered index area. A syntax illustration for an existing CLAVE-format file is:

```
OPEN #1: "KEYS.DATOS:S", UPDATE KEYED
```

The illustration requires KEYS.DATOS to have been created with the appropriate keyed organization and record/key dimensions. It is not interchangeable with opening an indexed file. Chapter 17 distinguishes the organizations; the detailed creation and record layout must agree with the file actually opened.

NOT

NOT is the unary logical-negation operation in the compiled expression stream, represented by byte 0x0F. It operates on one expression rather than introducing a statement. Numeric coercion, integer-width effects and the full precedence rules remain unspecified; no particular numeric result is promised here.

AND

AND combines two operands in an expression; compiled byte 0x10 selects its operation. At the conceptual level, conjunction requires both conditions to hold. The full truth-value encoding, coercion and evaluation-order contract remains unspecified. Do not assume that the second operand is skipped when the first is false.

OR

OR combines two operands in an expression; compiled byte 0x11 selects its operation. At the conceptual level, disjunction permits either condition to hold. The full numeric and evaluation-order contract remains unspecified. Its expression marker is distinct from any CSI or CONSULTA keyword. Appendix A locates these operations within the compiled format.

QUOTE

QUOTE is an option keyword. TRAFIC's output dialogue describes using OPTION QUOTE when opening PROCETEX-type output. That is a specific application use, not a complete specification of quoting, escaping or record delimiters. Do not assume that adding QUOTE creates a universally interchangeable text format; its complete syntax and scope remain unspecified.

RECORDSIZE

RECORDSIZE is recognized by BASIC's option/function table. Its permitted context, argument shape, units and result behavior are not yet specified. It must not be substituted for CREAR's REGISTRO option merely because the names describe related concepts.

FILESIZE

FILESIZE is a recognized language keyword whose argument and units remain unspecified. It cannot yet be documented as a byte-count function or a record-capacity setting. The size distinctions in Appendix C explain why those alternatives matter.

KEYSIZE

KEYSIZE is a recognized language keyword. Its complete relationship to an open channel or a file-creation request remains unspecified; use the established CREAR LONCLA syntax when defining a key length at CSI.

XOR

XOR occurs in the expression/operator vocabulary. Its operand types, numeric representation and evaluation behavior remain unspecified. The spelling alone is insufficient to import a bitwise or Boolean example from another BASIC.

IMP

IMP occurs beside the expression-operator names XOR and EQV. Its complete expression contract remains unspecified. This BASIC keyword must be kept separate from the physical printer name IMP and from similarly spelled output options in other utilities.

EQV

EQV belongs to the same expression/operator vocabulary. Its truth representation, operand conversion and precedence remain unspecified. It is not a documented replacement for an equality comparison.

DEVICE

DEVICE is a recognized language keyword. Its argument grammar and relationship to OPEN or device information remain unspecified. Device names used in an OPEN file specification do not establish a DEVICE function signature.

MEMORY

MEMORY is a recognized language keyword whose scope and arguments remain unspecified. It is separate from CSI's PONER MEMORIA partition-management setting. Do not treat it as a documented allocator or a direct-memory-access operation.

PORT

PORT is a recognized language keyword. No complete privilege, argument or hardware-access contract is specified here. Its name alone does not establish a safe BASIC interface to the I/O ports listed in Appendix B.

TAB

TAB is a recognized language keyword. Its permitted expression or output context, argument base and positioning behavior remain unspecified. Its presence does not justify importing the PRINT TAB syntax of another dialect.

UPCASE

UPCASE is a recognized language keyword, but its argument/result type and case-conversion scope remain unspecified. It must be distinguished from EDITAR's CASE U input mode and COPIFI's MAYUSC file-conversion option.

Appendix E. Binary structure atlas

This is the canonical layout reference. Offsets in these tables are hexadecimal unless a decimal quantity is explicitly stated. Multi-byte words are little-endian except the three-byte timestamp, BASIC fields and fields marked BE. A field marked unknown must be preserved rather than zeroed during recovery. The filesystem tables use an example geometry of 151 directory sectors starting at sector 9; actual header values determine another volume's bounds. Context and driver layouts describe the system interface.

Volume geometry and header

The raw physical mapping is defined in Chapter 27. For that geometry, sector 1 begins the header/bitmap region, sectors 9–159 hold 1,208 directory records, and sector 160 begins file allocation. The header's label is not a magic signature. A reader that unconditionally includes sector 160 in the directory manufactures false entries from data.

Offset	Size	Meaning	Notes
00	8	Space-padded volume label	—
08	3	Unresolved, preserve verbatim	Unknown
0B	8	Archive tape label	Probable; ESTADO displays matching bytes
13	4	Archive/date-related metadata	Tentative; exact packing incomplete
17	2	Byte 17 high nibble = logical heads; low nibble followed by byte 18 =12-bit cylinder count	kernel 0AE2–0AF8 and ESTADO09BC–0A37;17 93 =1 head,1939 cylinders
19	1	Sectors per track (20h=32)	loader A0D6
1A	1	Directory sectors (97h=151)	native 1208 entries / eight per sector
1B	2	Unresolved	Unknown
1D	2	Free allocation blocks, little endian	—
1F	1	Low six bits plus 2 gives first directory sector	loader A0DC–A0E7
20 onward	variable	Most-significant-bit-first allocation bitmap	allocation/deletion behavior

Directory record

Each record is 32 bytes, so a 256-byte sector contains exactly eight. Format 00 is an early lookup terminator; FF is a deleted slot. The active format is distinct from the protection bits.

Offset	Size	Meaning	Notes
00	1	Format/attribute bits; FF marks an unused/deleted slot	basic format mapping, other flags incomplete
01	8	Space-padded filename	—
09	8	Space-padded file type	—
11	2	Record count; sector count for A/R; page count for spooler-created SP$ reports	native comparisons and original spool close 3E0C–3E15; see spool subsection
13	2	Allocated 1-KiB blocks	—
15	2	First logical sector	loader, SECTOR and copy
17	2	Maximum counted-write length for specified S output; D record length; A/R length field; packed record/key lengths for I/K	selected native/code cases; S value is not necessarily a complete BASIC record length
19	3	Packed date/time, big-endian bitfield	—
1C	1	Owning account number; recipient account for a sharing alias; special FF namespace	native ownership changes, lookup and sharing controls; see the alias and mailbox layouts below
1D	1	Nonzero target account for a sharing lookup redirect	native COMPART grant/revocation and resolved access; see alias layout below
1E	2	R load-image bytes; S final linked sector; A load address; I/K slot capacity	R/S on specified files; A from loader

Format value	Native letter	Storage interpretation
0x01	R	Relocatable executable
0x02	A	Absolute executable
0x04	S	Linked sequential payload
0x08	D	Direct fixed-length records
0x10	I	Ordered indexed slots
0x18	K	Hashed keyed slots
0x20	Read protection	Attribute, not compression
0x40	Deletion protection	Attribute

Format value	Native letter	Storage interpretation
0x80	Write protection	Attribute; exactly 0x80 can be a sharing alias

For the three-byte big-endian timestamp D: month=`(D>>20)&15`, day=`(D>>15)&31`, year=`1977+((D>>11)&15)`, hour=`(D>>6)&31`, minute=`D&63`. There is no seconds field. Its year range is 1977–1992. Do not extend its range by assuming a modern epoch.

Sharing alias

Offset	Size	Alias-specific meaning
00	1	80h in the specified sharing alias
01	8	Same filename as target
09	8	Same type as target
11–1B	11	Zero in the created alias; no own record count/extent/timestamp
1C	1	Recipient account number
1D	1	Target owner account number
1E	2	Zero in the created alias

Payload structures

Format	Byte-level rule
S sector	+0x00..+0xFD payload 254 bytes; +0xFE LE16 next LBA; zero link terminates
D record n	Starts at byte `(n-1)*R` from file start; full 256-byte sectors; n is one-based
I prefix	N LE16 slot numbers; pad prefix to `B=ceil(2*N/1024)*1024`
I slot s	At B+(s-1)*(K+R+2): key K bytes, data R bytes, next-slot LE16
K slot s	At(s-1)*(K+R): key K bytes followed by data R bytes

For I/K, directory word+0x17 packs `R=(word&0x1FF)+1` and `K=(word>>9)+1`; word+0x1E is slot capacity N. I allocation blocks are `ceil(2*N/1024)+ceil(N*(K+R+2)/1024)`. The first C prefix entries, with C from directory+0x11, identify active slots in sorted key order. K uses the hash/probe algorithm in Chapter 26. A zero first key byte ends an unused probe; FF marks a tombstone.

R relocation records and BASIC object layouts appear in Appendix A. Allocation slack and sequential padding are not application data lengths.

Partition context

The context base is the current pointer at 0x0033 or the pointer in a scheduler row. Its 0x4CC-byte fixed area precedes the program. The following table deliberately includes unknown regions.

Offset	Bytes	Meaning	Notes
0x00	2	Saved SP	—
0x02	2	Context base	—
0x04	2	Memory ceiling	—
0x06	2	Allocated endpoint; zero represents 0x10000	—
0x08	2	Unclassified header bytes	Unknown
0x0A	1	Console policy; bit 1 suppression, bit 6 folding/deletion mode	—
0x0B	1	bit 3 pause; bit 4 deferred interruption; bit 5 event/cancel	—
0x0C	1	Console/date/search policy; bit 5 version, bit 6 inhibit public lookup, bit 7 message inhibition	—
0x0D–0x0E	2	Tracked cursor coordinates	—
0x0F	1	Unclassified	Unknown
0x10	1	bit 1 connected-state eligibility; bit 2 IMPEXEC	—
0x11	1	Privilege	—
0x12–0x19	8	Account name	—
0x1A	1	Account number	—
0x1B	1	EXEC level capacity	—
0x1C	2	EXEC work-file start	—
0x1E–0x25	8	Other execution state	Incompletely classified
0x26	2	Disk-error callback	—
0x28	2	Interruption/control callback	—
0x2A–0x36	13	State preserved by EXEC frames	Opaque subfields
0x37	1	EXEC level	—
0x38	1	EXEC/command continuation flags	—
0x39	3	Inline RST38 hook; C9 disables	—

Offset	Bytes	Meaning	Notes
0x3C	1	bit 5 escape-output state	—
0x3D–0x67	43	Controls/state; escape byte at+0x40	Partially classified
0x68	1	Last command result	—
0x69–0x7C	20	Other context state	Incomplete
0x7D–0x94	24	Association map	—
0x95–0xAB	23	Remaining header	Incomplete
0xAC–0x2CB	544	Seventeen 32-byte channels	—
0x2CC–0x3CB	256	Sector workspace	—
0x3CC–0x4CB	256	Command buffer	—
0x4CC onward	variable	Program	—

Saved execution frame

At the saved SP, words are HL′, DE′, BC′, AF′, IY, IX, HL, DE, BC, AF and continuation PC, in that order, at offsets 0x00, 02, 04, 06, 08, 0A, 0C, 0E, 10, 12, 14. F is the low byte of each AF word. This is 22 bytes; further frames can follow. No I/R/IFF or interrupt-mode field is established here. Callback entry leaves the continuation at SP=savedSP+0x14 for RET.

Caller channel request

Offset	Size	Meaning	Notes
00	1	Channel index, accepted range 0..16	kernel 34D9
01	1	Operation/method flags;38h used by BASIC UPDATE INDEXED	observed value; complete bits partial
02	2	Little-endian pointer to caller-owned sector/index workspace	overlay 1:4692 and I/O callers

Internal indexed channel

Offset	Size	Indexed-file meaning	Notes
00	1	Associated drive index	passed asB to disk services
01	8	Space-padded filename	copied name and live memory
09	8	Space-padded file type	—
11	1	Device/class selector;01 for specified disk file	Full class enumeration unknown
12	1	Channel state flags	Partial bit meanings below
13	2	Packed record/key lengths, as directory+17	overlay 2:461F
15	2	Slot capacityN, as directory+1E	overlay 2:462B
17	2	Index-area starting logical sector	overlay 2:4642
19	2	Record-area starting logical sector	overlay 2:4637–4652
1B	2	Active record count	overlay 2:4610 and deletion 4C05
1D	2	Next slot for sequential traversal	indexed READ/READNEXT
1F	1	Saved original drive selector during OPEN search	store at overlay 2:42FD; full lifetime partial

Sequential channel tail

Offsets below replace the indexed meanings; they are a union, not additional fields.

Offset	Size	Observed meaning
13	2	Pointer to next byte in caller workspace
15	1	Unused payload bytes remaining in current sector
16	2	Current buffered logical sector
18	1	Sectors remaining in current block, including current
19	2	Count of successful counted writes
1B	2	Allocated 1-KiB block count
1D	2	Maximum counted-write length

Synchronization and callback structures

Structure	Fields	Limits and interpretation
Scheduler row	+0 contextLE16; +2 bank/state	16 initialized 3-byte rows; low nibble bank, high wait/stop bits
Record lock	+0 device low 3/channel high 5; +1 firstLBALE16; +3 lastLBALE16; +5 owner	Owner=index+1; bit 7 terminates;64 usable rows by code/layout
File-open row	+0 physical device; +1 firstLBALE16; +3 owner/exclusive	Owner=index+1; bit 7 exclusive; no channel field
Byte lock	owner byte	0 free; index+1 owned; no recursion count
Timer node	+0 deltaLE16; +2 unknown; +3 next owner; +4 next pointer LE16; +6 code	Linked delta list; expiry clears first 6 bytes
Callback registry	+0 owner; +1 pointer LE16	Row at 0x23FF+3*index; indices 0x02–0x20
Named component	+0 key 8; +8 addressLE16	Exact binary key; first-byte 00/space ends table
Memory queue	+0 capacity; +1 occupancy; +2 read index; +3 payload	Eight-bit arithmetic defects; see Chapter 25

Spooler and mailbox

Structure	Offset	Bytes	Meaning
Collector slot	0x00	1	Flags
Collector slot	0x01	1	Free buffer bytes
Collector slot	0x02	2	Buffer pointer
Collector slot	0x04	2	Current LBA
Collector slot	0x06	2	Pages
Collector slot	0x08	2	Blocks
Collector slot	0x0A	2	Queue number
Collector slot	0x0C	1	Queue selection
Collector slot	0x0D	1	Flags/copies
Collector slot	0x0E	1	Line state
Collector slot	0x0F	1	Unknown
Queue record	0x00	8	Report label
Queue record	0x08	1	Submitting partition index+1
Queue record	0x09	3	Packed timestamp
Queue record	0x0C	1	State/retention/copies/printer flags
Queue record	0x0D	1	Queue letter
Queue record	0x0E	2	Page count
Mail header	0x00	8	Exact recipient name
Mail header	0x08	8	Sender name
Mail header	0x10	8	Date text
Mail header	0x18	8	Time text
Mail header	0x20	1	CR

Spooler queue records and collector slots are both 16 bytes but have different layouts. Mail bodies use CR lines and 0x7F boundary padding; their enclosing S sectors still carry 254-byte payloads and two-byte links. Neither framing replaces the filesystem structure.

Physical descriptor unions and driver-private storage

A physical descriptor has 32 bytes and begins at 0x0069+32*index. The following tables describe the identified fields for the stated family; gaps remain unclassified. A disk's geometry/free-space field must not be decoded using a character device's class-length interpretation.

Terminal descriptor

Offset	Size	Documented role
00	2 LE	Loaded DIS driver entry
02/03	1 each	Horizontal wrap threshold/input budget and LF row cap; nativeL79/P23, bytes 4F/17
04	1	Class number,1 or 2
05 bit 7	1 bit	Enable input filtering
05 bit 6	1 bit	Pending Escape prefix
0E	2 LE	Driver loaded length,0377 here
10	2 LE	Class loaded length,0103/0148
16	2 LE	Loaded class entry; input+0, output+3

Printer descriptor

Descriptor offset	Size	Meaning / evidence	Notes
00	2 LE	Loaded driver base	—
02 / 03	1 each	Configured line width / page length; native 84/3C	—
04	1	Terminal/printer class number; zero on the specified paths	—
05 low nibble	4 bits	IMP rate-table selector	—
06 / 07	1 each	CR/LF delay / form-feed delay selectors in 1848–185E	code; zero native
08 bits 7/6	2 bits	Serial parity selection used by DIS22; PO produced 10b, PE11b	code and options
08 bit 5	1 bit	Word-size flag; native W8 sets it	—
08 bits 1/2/3	3 bits	Common page-line counting, automatic-LF policy, page-parity handling	code; combinations unspecified

Descriptor offset	Size	Meaning / evidence	Notes
09	1	Added to configured page length on initialization	—
0A	1	Mutable remaining-page-line counter	code; initial 3C native
0B	1	Last output/control byte tracked by common wrapper	—
0C	1	Initial page length plus 09; bit 7 toggled at page boundaries	code and selected FF changes
0E	2 LE	Loaded driver length	—

ALP input ring

Driver-relative offset	Size	Meaning
0240	1	Capacity 50h, 80 bytes
0241	1	Occupancy
0242	1	Read index
0243–0292	80	Payload

ALP output storage

Driver-relative offset	Size	Meaning
0293–02E5	83	First output buffer
02E6–0338	83	Second output buffer
0339	1	Buffer selector, toggles 00/FF
033A	2 LE	Current append-buffer pointer
033C	2 LE	Physical descriptor pointer
033E	2 LE	Buffered byte count

SS2 initialization fields

Offset	Size	Meaning	Notes
01CC	32	Sixteen LE16 divider values indexed by descriptor 05 low nibble	—
01EC	1	Number of initialization register pairs:0A	—
01ED	20	Register-number/value sequence	—

Offset	Size	Meaning	Notes
01F2 /01F4	1 each	Selected divider low/high bytes in that sequence	—
01F8	1	Register 4 value; parity bits derived from descriptor 08 bits 7/6	—
01FA	1	Register 5 cached value; bit 6 follows descriptor 08 bit 5; receive threshold clears bit 7	—
01FC	1	Register 3 cached value; bit 7 follows descriptor 08 bit 5	—
0201	2 LE	Physical descriptor address remembered for receive callback	—

SS2 ring and saved callbacks

Offset	Size	Meaning
0203	1	Capacity 80h,128 bytes; original constant
0204	1	Occupancy,0..128 in valid state
0205	1	Read index,0..127
0206–0285	128	Payload slots
0286	2 LE	Status path's saved context+28
0288	2 LE	Output path's saved context+28

Floppy private state

DIS1 offset	DIS2 offset	Size	Meaning
024A	022C	2	Saved BC
024C	022E	2	Saved DE
024E	0230	2	Saved HL
0250	0232	3	First three controller result bytes
0253	0235	9	Command packet / mutable parameters
025C	023E	1	OriginalC3; role Unknown

Tape mutable state

Offset	Size	Role / notes
093F	2	Self-patched JP operand to selected driver's+0F; native documented
0A73	2	Record/block progress counter; original code

Offset	Size	Role / notes
0A75	80	Volume/read-label buffer; original code
0AC5	80	First file-label template/buffer
0B15	80	Second file-label template/buffer
0B67	2	Label section progression value; interpretation partial
0B93	2	Retry counter, printed by callback
0B95	1	Mark/retry control count, initialized 2
0B96/0B97	1 each	First/second raw status; selected native values documented
0B98	1	Nonzero completion flag; initialized 1 in original image
0B99	1	Current selector 0–3; native documented
0B9A	2	Record buffer address
0B9C	1	D=0 short/synchronous path flag
0B9D	1	Pending write callback flag
0B9E	1	Read pipeline state
0B9F	1	Pending read callback flag
0BA0	1	Read retry countdown
0BA1/0BA2	1 each	Write/read command-name initialization flags
0BA3	1	Cached leading command-name byte

Glossary

Definitions use the system's own distinctions. Reference numbers identify chapters; letters identify appendices. Chapter 31 defines command names and options, Appendix D defines BASIC commands and statements, and Appendix C explains individual system files. This glossary supplies the shared concepts, abbreviations and device terminology needed to read those references.

A format — Absolute executable format. Its directory entry supplies the load address; it is distinct from drive A. See 18, 30; A.

ABI — Application binary interface: the machine-level agreement on registers, memory layouts, entry points and returns. A driver ABI need not be the same as the system-service ABI. See 19, 24, 27, 28.

ABS / REL / COM — Legacy object or toolchain labels appearing in diagnostic vocabulary. Their presence does not establish a supported object format or a complete assembler/linker procedure. COM must also be distinguished from physical COM1–COM4 device names. See 18, 33; A.

account — Named user identity with account number and privilege. It is distinct from a scheduled partition. See 9, 10, 22.

AF — Z80 register pair combining accumulator A and flags F. A service may return a value in A and a separate condition in a flag. The notation AF′ denotes the alternate pair. See 19, 24, 32; E.

allocation block — Four 256-byte sectors, totaling 1,024 bytes. Allocation is tracked by the volume bitmap. See 23, 26; E.

ALP — Terminal interface represented by SCC channels, DMA controllers and flow-control ports. Four consoles are established in this architecture. See 28; B.

API — Application programming interface: the operations and data conventions exposed to application software. Here, system services, BASIC facilities and device interfaces have their own individual contracts. See 17, 19, 32.

ASCII — American Standard Code for Information Interchange, a character encoding used for letters, digits and control characters. A character value and the terminal action assigned to it are different matters. See 4, 19, 28.

ASI — Label of a firmware diagnostic whose omission is controlled by a configuration bit. Its complete expansion and hardware scope are not established here. See B.

association — Partition-local mapping from a logical device use to a physical descriptor. See 12, 23, 34.

bank — Backing 48-KiB RAM region selected into CPU addresses 4000–FFFF. A scheduler index need not equal a bank number. See 20, 22, 25.

BASIC — The system's programming language and integrated development environment, identified as BASIC S20. It provides source editing, interpretation, compilation and runtime execution. BASICOBJ, BASICCOM and BASICUSR name distinct representations used by that environment. See 15–17; D.

BASICCOM — Compiled BASIC statement/expression stream interpreted by the installed runtime; not an R executable. See 15, 16; A.

BASICOBJ — Tokenized saved BASIC source, despite the object-like type name. See 15; A.

BASICUSR — R-format machine-code extension selected by BASIC OPTION USR. See 19; A.

BC — Z80 register pair composed of B and C. Interfaces use it for counts, selectors and other values; its meaning is specified separately for each call. See 19, 32; E.

BCD — Binary-coded decimal: each nibble represents a decimal digit. Used in clock state and BASIC decimal payloads. See 19; A, B.

BE — Big-endian byte order: the most significant byte of a multi-byte value comes first. Several BASIC fields and the packed directory timestamp use this order, even though ordinary Z80 words are little-endian. See A, E.

bitmap — Allocation map with most-significant-bit-first bits, one per 1-KiB block. See 26; E.

bootstrap — Disk loader read by firmware before the full executive. Physical bootstrap coordinates differ from logical volume addresses. See 3, 21, 27.

BUZON — Mailbox retrieval utility; also the type/name context of the sequential persistent mailbox. Retrieval removes messages. See 9, 29, 31.

callback — Routine entered when an event occurs, sometimes in another partition's saved context. It is not necessarily a new execution thread. See 19, 25, 28.

capture definition — Screen, field and file-access description used by the optional ACTUCAP and EXPAND support tools to prepare a data-entry program. See C.

carry — Z80 flag used by some interfaces for error or emptiness, by others for device readiness. There is no universal OS-wide meaning. See 19, 32.

catalogue — The volume's fixed directory records. Exact-name probing and wildcard enumeration traverse it differently. See 5, 23, 26.

CDC — Physical printer definition served by SISTEMA.DIS12. It is distinct from logical printer names such as IMPR1. See 12, 28, 34; B.

channel — One of seventeen per-partition file/device records, indexed 0–16 at the kernel interface. See 17, 24; E.

CHS — Cylinder, head and sector coordinates of a physical disk. The hard-disk interface uses zero-based sector IDs. See 27.

CLASE — Terminal-class program translating logical console controls. It is separate from the physical DIS driver. See 4, 28.

COM1–COM4 — Physical communication-device definitions. They are different from the partition-local COMU names and from file types. A definition does not establish that the necessary hardware and complete driver behavior are available. See 34; B.

COMANDO — File type used by most CSI utilities; native executable format is independently recorded in the directory. See 7, 30, 31.

command processor — reads command input, resolves commands and starts executable/interpreter paths. See 5, 30 and CSI.

command result — The value reported as CR after command execution. It does not certify media persistence or message delivery. See 13, 33.

common RAM — The low 16-KiB region shared across bank selections after reset alias removal. See 20; B.

COMPART alias — Directory redirect granting a recipient access to another account's private file without transferring ownership. See 10, 22; E.

COMU — Family of logical communication-device names, such as COMU1, resolved through associations. Do not substitute a physical name such as COM1 without establishing its association. See 23, 34.

CONSOLA — Logical console name used in command syntax. Input and output resolve through console associations and need not be treated as a physical terminal model name. See 12, 23, 34.

context — Partition header, channels and workspaces together with pointers to its execution state. See 24, 25; E.

CONTROL.VOC — Indexed attribute dictionary used by query and vocabulary-support software; distinct from EXEC’s CONTROL directive. See 35; C.

counted I/O — Sequential byte transfer through 5E/5F. Counted-write call count is not necessarily an application's record count. See 17, 26, 32.

CPU — Central processing unit: the processor executing instructions. The Z80 has a 64-KiB address window; bank selection changes which memory is visible through part of that window. See 2, 20, 22.

CR — Two different abbreviations occur. In character I/O, CR means carriage return, character 0x0D, sent by Return to submit a command line. In command completion messages, CR means command result, a numeric status. See CR character and command result; 4, 9, 33.

CR character — Carriage return, 0x0D. Distinct from the abbreviation CR for command result. See 4, 33.

CRT — Cathode-ray tube, the display technology of the IS 9502 terminal. CRT also appears as an incompletely specified EXEC word, and CRT$ is a BASIC terminal-control string function whose full control vocabulary is unspecified. See 4, 8; D.

CSI — The command processor: the program that displays the > system prompt, reads command lines, finds commands and starts their execution. “System prompt” is also used informally for this command environment; strictly, the prompt is the visible > character, while CSI is the program behind it. Its system file is SISTEMA.CSI. CSI version 5.5 is separate from operating-system Version 6.2. See 1, 5, 30; C.

D format — Direct fixed-length record storage. Record numbers are one-based. See 17, 26; E.

DE — Z80 register pair composed of D and E. Many services use DE for a pointer or value, but it has no universal argument role. See 19, 32; E.

DEL — Delete character, 0x7F. CSI accepts Delete for removing the preceding input character; other programs can apply different input rules. See 4, 28.

descriptor — A 32-byte physical-device record whose fields depend on device family. See 23, 28; E, B.

DIRAL — ESTADO MEMORIA column showing the current memory ceiling minus one, from context offset 0x04. Loaded drivers or classes can lower this boundary within the allocated region. See 12, 20; E.

DIRBA — ESTADO MEMORIA column showing the partition base address, from context offset 0x02. It is a memory address, not a disk-directory number. See 12; E.

DIRECT — BASIC file organization using fixed-length records selected by record number. It corresponds to native D-format storage. See D format; 17, 26; D, E.

directory entry — A 32-byte record containing filename, type, format, allocation, timestamp and ownership. See 26; E.

DIRMX — ESTADO MEMORIA column showing the final address of the allocated partition region: the word at context offset 0x06 minus one. It can exceed the current usable ceiling shown as DIRAL. See 12; E.

DIS — System driver-file family, for example SISTEMA.DIS10 for tape and SISTEMA.DIS18 for PAN1. The numeric suffix selects a driver definition; it is not a system-call number. See 23, 28, 34; B, C.

DISCO1–DISCO6 — Physical disk definitions. Logical letters S and A–G acquire their physical meaning through associations to definitions such as these. See 11, 23, 34.

DMA — Direct memory access used for established ALP output, with channel-pair bank latches independent of CPU bank selection. See 28; B.

DRQ — Data request: a disk-controller status signal used to coordinate transfers through the data register. Consult the controller map for the relevant status bit and direction. See 27; B.

DTR / REQ — Data-terminal-ready and request signals named in the serial/DMA interface. They concern hardware handshaking, not whether an account is logged in. See 28; B.

EBCDIC — Extended Binary Coded Decimal Interchange Code, the character encoding used by the IBM 3741 interchange utilities. SECIBM converts outgoing sequential text from ASCII; IBMSEC performs the reverse conversion. See 11, 31.

EOF — End-of-file condition. Kernel chain exhaustion, a sequential 1A marker and BASIC's EOF function are different layers. See 17, 26, 33.

EOV — Tape-label prefix used when forming end-of-volume trailer labels; EOF is used in corresponding end-of-file labels. These label prefixes are separate from a program's EOF return condition. See A.

ESC — Escape character, 0x1B, also written Escape in keyboard instructions. It introduces several CSI control actions and terminal-class output sequences. Those uses have different consumers. See 4, 28.

ETX — End-of-text character, 0x03. In the ALP output path it is one of the characters that can trigger a buffered-output flush. Its name does not make it a universal program-termination command. See 28.

exclusive open — Admission policy in the file-open table. It survives UNLOCK and does not reliably prevent native deletion. See 17, 25.

EXEC — Command-file interpreter and sequential file type. Full spelling avoids the established EX/EXE token-transfer defect. See 8, 30.

EXECLANG — Installed executable implementing EXEC; distinct from EXECn work files. See 8, 21.

EXTEND — Sequential append mode. It locates the final sector and trims trailing 1A; a zero endpoint can damage reserved metadata. See 17, 26.

FF — In hexadecimal layouts, FF denotes byte 0xFF: depending on context, a deleted entry, special owner, unassigned association or another sentinel. In terminal-control names, FF means form feed, character 0x0C. Distinguish the character name from the hexadecimal byte value. See 28; E.

FIFO — First in, first out: a queue in which the earliest unread item is consumed first. The term also labels a controller data interface; it does not specify its capacity or error behavior. See 25, 27; B.

file type — The second eight-byte filename field. It does not alone determine storage format. See 5, 23.

FIN — Account disconnection command, not a documented global shutdown. See 3, 9, 14.

FLOPPY1 / FLOPPY2 — Physical floppy-device definitions served by SISTEMA.DIS1 and SISTEMA.DIS2. A logical disk letter such as A must be associated with the intended definition before file access. See 11, 27, 34.

FM-7 — Fujitsu computer also offered under Secoinsa branding during its move into personal and educational computing. It is a separate product from the Secoinsa Serie 20 computer. See Preface.

format byte — Directory field combining native organization with protection attributes. See 23; E.

frame — Saved CPU register/continuation data, or an EXEC level's 512-byte saved interpretation state; specify which. See 8, 25; E, A.

free run — Consecutive unallocated blocks available for contiguous allocation. It can be smaller than total free space. See 11, 26.

Fujitsu — Japanese technology company that was a founding partner of Secoinsa and supplied technology to its product range. Fujitsu España later absorbed Secoinsa. See Preface.

HDD — Hard-disk drive. This abbreviation appears in descriptions of startup diagnostics and hardware configuration. See 21, 27; B.

HL — Z80 register pair composed of H and L, commonly used for pointers, counts and results. HL′ is its alternate pair. Interpret it according to the individual interface. See 19, 32; E.

I format — Indexed storage with a sorted slot-number prefix and next-slot links. See 17, 26; E.

IBM 3741 — The floppy interchange format handled by INICIBM, DIRIBM, SECIBM and IBMSEC. Its headers, records and character conversion require those specialized utilities. See 11, 31.

IFF — Z80 interrupt-enable state, commonly referred to through IFF1 and IFF2. It is separate from flags in AF. The documented scheduler frame does not establish saved IFF fields. See 24, 25; E.

IM — Z80 interrupt mode. See IM 0 for the mode used by the documented interrupt interface; do not infer a different mode from a vector address alone. See 24; B.

IM 0 — Z80 interrupt mode using a supplied opcode; the interrupt interface supplies restart instructions. See 24; B.

IMP — Physical printer definition associated with SISTEMA.DIS22. An application normally selects a logical printer such as IMPR1, which ASOCIAR connects to IMP. See 12, 28, 34.

IMPEXEC — Setting permitting implicit command-file execution. See 8, 31.

IMPR1–IMPR4 — Logical printer names available through partition associations. PRINT1 is an option selecting a printer in commands such as LISTAR; IMP is a physical printer definition. See 12, 23, 34.

INDEXED — BASIC file organization with ordered keys and an index prefix. It corresponds to native I-format storage and differs from KEYED. See I format; 17, 26; D, E.

INI — Instituto Nacional de Industria, Spain's state industrial holding company and a founding partner of Secoinsa. See Preface.

inline service number — Byte immediately after RST08 identifying one of 111 services. See 18, 24, 32.

installation serial — Three-byte software binding checked by selected command/BASIC guards; unrelated to serial-port baud. See 10, 16, 30.

IP — Physical parallel-printer definition associated with SISTEMA.DIS11. Here IP is a device name, not an Internet network address. See 28, 34; B.

IPL — Initial program load/restart path, including Escape-I. Declining the specified request ends the command rather than resuming it. See 14, 21.

IS 9502 — The Secoinsa computer terminal paired with the Secoinsa Serie 20 computer, with a separate keyboard and a green monochrome display of 24 lines by 80 characters. See Preface; 4.

IX — Z80 index register. Its role varies between interfaces; do not assume it always points to the same kind of structure. See 19, 24, 32; E.

IY — Z80 index register. It often points to executive, context or descriptor data in the documented routines, but the required meaning is caller-specific. See 19, 24, 32; E.

K format — Hashed key/data slots without the I-format index prefix or next-link trailer. See 17, 26; E.

kernel — The resident executive and associated kernel overlays in SISTEMA.NUCLEO; native displays separate the spooler extent. See 20, 24.

KEYED — BASIC file organization using hashed keys rather than the ordered index of INDEXED files. It corresponds to native K-format storage. See K format; 17, 26; D, E.

LBA — Logical sector number within a volume. Translate through that volume's direction and replacement table to obtain CHS. See 26, 27.

LE — Little-endian byte order: the least significant byte comes first. LE16 means a two-byte, 16-bit value in that order. Byte order must be read from the specified structure, not assumed from its file type. See A, E.

LE16 — A 16-bit little-endian value occupying two bytes. See LE; A, E.

LF — Line-feed character, 0x0A. Terminal output uses it to advance the line; Return submits CSI input using CR. Device options beginning LF can specify line-feed delay. See 4, 12, 28.

LIFO — Last in, first out: newly queued input is placed ahead of earlier input. Separate from EXEC CONTROL STACK/NOSTACK. See 35.

logical device — A partition-local use such as S, console input or IMPR1, resolved through an association. See 12, 23, 34.

MENSERR — Direct file holding 200 fixed 64-byte message records. See 24, 33.

MM58167 — Real-time-clock device named in the hardware map. Its counters and control registers are separate from the filesystem's compact timestamp representation. See B.

module — Term used in native memory displays and also for loaded code units; it is not automatically a scheduler partition. See 12, 20.

NUCLEO — Kernel file and native memory-region name. The stored boot image includes initialization material beyond the resident body. See 20, 21.

NUL — Null character or zero byte, 0x00. Its role depends on the structure: it may terminate a string or mark another condition. It is not interchangeable with a space, CR or EOF. See 19, 32; A, E.

OASIS — Lineage indicated by original copyrights and tape templates. Exact release/API compatibility remains unknown. See A.

OPEN — Channel setup operation whose mode can replace a sequential file immediately. It is not inherently read-only. See 17, 26, 32.

OS — Operating system: the executive, services and associated command environment described in this document. The identified release is S.O. Multi-usuario Version 6.2. See 1, 20.

overlay — Replaceable code loaded into a shared execution window. Identify its number before interpreting an address. See 20, 24.

owner — Account byte in a file entry, or scheduler-index-derived tag in a lock. The two representations are distinct. See 10, 22, 25.

PAB — Term present in original messages whose complete object/toolchain meaning is not established in this operating system. See 33; A.

PAN — Family of physical terminal definitions, such as PAN1 and PAN2. PAN1–PAN4 use the established ALP console drivers; other numbers can designate optional interfaces. These are device names, not account names. See 12, 28, 34.

partition — The OS's scheduled execution context, with memory, channels, associations and identity. Broadly comparable to a process, but without an assumed modern ABI or protection boundary. See 12, 22, 25.

PC — Program counter, the Z80 register identifying the next instruction. PC in execution-state and memory tables does not mean personal computer. See 12, 24, 25; E.

physical device — Descriptor-selected hardware/driver endpoint; a definition can exist without fitted hardware. See 23, 28, 34.

proceso fantasma — Original help's background-process term. Complete activation syntax and lifecycle remain unspecified. See 7, 31.

prologue — Optional CF41 command entry sequence with preceding privilege/serial data. See 10, 30.

public file — Account-0 file considered by fallback lookup unless inhibited. Special accountFF is different. See 5, 10, 26.

R format — Relocatable machine-code image with directory-defined length and sector-based relocation records. See 18, 30; A.

RAM — Random-access memory used for the executive, partitions, buffers and running programs. The architecture divides it into common memory and bank-selected regions. See 2, 20; B.

record lock — Owned inclusive sector span associated with a channel; not an exclusive file-open entry. See 17, 25; E.

relocation — Adding the selected load base to identified address words in an R image. See 18, 30; A.

resident — Retained in the established memory layout across ordinary command execution; does not imply every byte is common across banks. See 20, 29.

ROM — Read-only memory containing startup firmware. Its diagnostic and bootstrap role is separate from the disk-loaded executive and BASIC environment. See 2, 21; B.

RST — Z80 restart instruction transferring to a fixed vector. RST08 carries OS calls; other vectors have distinct uses. See 18, 24; B.

RTC — Real-time clock, the hardware maintaining date/time information. See MM58167; B.

RTS — Request to send, a serial-interface control signal. The interpretation of the ALP receive-flow-control wiring as RTS remains probable rather than fully specified. See 28; B.

RX / TX — Receive and transmit, respectively, in serial-controller and interrupt maps. Receive input and transmit completion can use different buffers and callbacks. See 28; B.

S format — Sequential linked sectors with 254 payload bytes each. See 17, 23, 26; E.

S.O. Multi-usuario — The operating system's displayed name; this document describes Version 6.2. CSI, EXEC and BASIC carry their own component version numbers. See 1.

S20 — Component designation in the name BASIC S20. It identifies the BASIC environment for the Secoinsa Serie 20 computer; its version number is separate from the operating-system release. See 1, 15.

SCC — Serial communications controller. The ALP serial paths use SCC channels for terminal communication, with separate DMA support for output. See 28; B.

scheduler index — Zero-based execution-record identifier, distinct from displayed partition number and bank selector. See 22, 25.

Secoinsa — Sociedad Española de Comunicaciones e Informática, S.A., the Spanish computer and communications company whose Secoinsa Serie 20 computer is described here. See Preface.

sector — A 256-byte unit in this logical filesystem and hard-disk interface; floppy translation can involve 128-byte physical sectors. See 23, 27.

SEQUENTIAL — BASIC file organization using sequential records over linked S-format sectors. Replacement, append and positioning behavior depend on the selected OPEN mode. See S format and EXTEND; 17, 26; D, E.

sharing — Access through a recipient alias to the owner's file. It differs from ownership transfer and shared OPEN admission. See 10, 17, 25.

SIO / PIO / PSIO / CPSIO — Driver/class labels in physical device definitions. They identify interface behavior and defaults; use the definition and associated driver rather than inferring a complete device contract from the letters alone. See 12, 23, 28, 34; C.

SISTEMA — The administrative account used in the procedures, and separately a common filename stem for operating-system components. SISTEMA as an account, SISTEMA.CSI as a file and S as a logical disk are different identifiers. See 3, 5, 10; C.

slack — Allocated bytes beyond current payload. They are allocated but do not belong to the file's current useful contents. See 13, 26.

SP — Stack pointer, the Z80 register locating the current stack. A saved partition SP points to register words and a continuation address used when execution resumes. See 19, 24, 25; E.

SPOOLCOL — Direct queue file with sixteen-byte records used by the resident spooler. See 29; E.

spooler — Resident deferred-output collector and printer manager. Native collection is established; complete physical printing/restart remains incomplete. See 29, 31.

SS2 — Interface grouping including timer/interrupt and optional serial paths in this architecture; complete physical routing remains uncertain. See 28; B.

ST0 / ST1 / ST2 — Floppy-controller result/status bytes. Interpret each bit through the driver's documented error translation; these are not CSI command-result values. See 27; B.

STARTUP — Executable system-initialization component, SISTEMA.STARTUP. It relocates and initializes system code; it is not an editable EXEC command file. See 2, 21; C.

synonym — Configured alternate command name, optionally with a minimum prefix length. Custom synonyms and actual command files can take precedence over standard abbreviation. See 5, 30, 35.

system volume — Drive S, whose associated filesystem supplies kernel, CSI, messages and other required components. See 11, 21, 27.

TAP — Physical tape-driver definition, associated with SISTEMA.DIS10. It is normally connected to the logical name TAPE1. See 13, 28, 34.

TAPE1 — Logical tape-device name used by the tape utilities and driver initialization. It resolves through an association to TAP; it is not a disk letter or a filesystem directory. See 13, 28, 34.

TC — Terminal count, a DMA completion indication. In the hardware maps, it concerns transfer completion and must not be read as a terminal-user or account count. See 28; B.

TE — Elapsed-time field printed in command completion reports, expressed in minutes. It is separate from CR, the command result, and is not another error code.

Telesincro — Spanish electronics and business-computer company whose Factor product range and engineering experience became part of the Secoinsa group. See Preface.

terminal class — See CLASE. See 28.

tombstone — MarkerFF for a deleted keyed slot or catalogue entry; interpretation depends on the surrounding structure. See 26; E.

UPDATE — BASIC access mode. Sequential UPDATE without EXTEND replaces the file; direct UPDATE uses numbered records. See 17, 26.

VOL1 / HDR1 / HDR2 — Identifiers in the 80-byte tape-label structures. VOL1 is a volume label; HDR1 and HDR2 are file-label records. Familiar label names do not by themselves establish compatibility with another tape format. See 13; A.

volume — Logical filesystem with label, geometry, directory and bitmap; its bounds and mapping are held in volume metadata. See 5, 23, 27.

WD / WD1001 — Names used for the hard-disk controller and its register interface. Driver retries, data-register transfers and status bits must be interpreted according to that interface. See 27; B.

workspace — Caller- or context-owned temporary storage used by a service; ownership and lifetime must be stated. See 17, 24; E.

yield — Voluntary entry to scheduling through 4F. Timer preemption also exists independently. See 22, 25.

Z flag — Z80 zero flag. It can report a zero result or an interface-specific condition; some services return A and flags with different meanings. See 19, 32.

Z80 — The processor family executing this operating system. Native programming examples use its registers, restart instructions and 16-bit address space. See 18–20; B, E.

Index

Links lead directly to the relevant section. Chapter and appendix references remain useful in print.

Concepts

Term	Reference
A format	18, 30; A
account	9, 10, 22
allocation block	23, 26; E
ALP	28; B
association	12, 23, 34
bank	20, 22, 25
BASICCOM	15, 16; A
BASICOBJ	15; A
BASICUSR	Extension calls, 19; A
BCD	19; A, B
bitmap	26; E
bootstrap	3, 21, 27
BUZON	9, 29, 31
callback	19, 25, 28
carry	19, 32
catalogue	5, 23, 26
channel	17, 24; E
CHS	27
CLASE	Terminal setup, 12, 28
COMANDO	7, 30, 31
command processor	5, 30
command result	13, 33
common RAM	20; B
COMPART alias	10, 22; E
context	24, 25; E
counted I/O	17, 26, 32

Term	Reference
CR character	4, 33
CSI	1, 5, 30; Glossary
D format	17, 26; E
descriptor	23, 28; E, B
directory entry	26; E
DMA	28; B
EOF	17, 26, 33
exclusive open	17, 25
EXECLANG	8, 21
EXEC	8, 30
EXTEND	17, 26
FF	E
file type	5, 23
FIN	3, 9, 14
format byte	23; E
frame	8, 25; E, A
free run	11, 26
I format	17, 26; E
IM 0	24; B
IMPEXEC	8, 31
inline service number	18, 24, 32
installation serial	10, 19, 30
IPL	14, 21
IY/IX	19, 24; E, A
K format	17, 26; E
kernel	20, 24
LBA	26, 27
logical device	12, 23, 34
MENSERR	24, 33

Term	Reference
module	12, 20
NUCLEO	20, 21
OASIS	A
OPEN	17, 26, 32
overlay	20, 24
owner	10, 22, 25
PAB	33; A
partition	12, 22, 25
physical device	23, 28, 34
proceso fantasma	7, 31
prologue	10, 30
public file	5, 10, 26
R format	18, 30; A
record lock	17, 25; E
relocation	18, 30; A
resident	20, 29
RST	18, 24; B
S format	17, 23, 26; E
scheduler index	22, 25
sector	23, 27
sharing	10, 17, 25
slack	13, 26
SPOOLCOL	29; E
spooler	29, 31
SS2	28; B
system volume	11, 21, 27
terminal class	28
tombstone	26; E
UPDATE	17, 26

Term	Reference
volume	5, 23, 27
workspace	17, 24; E
yield	22, 25

Administrative procedures

Task	Reference
Account listing; registered accounts and connected users	10
Account creation	10
Account deletion	10
Password change, retention and removal	10
Partition listing	12
Partition creation and activation	12
Partition stopping	12
Partition removal	12
Floppy association and mounting	11
Floppy file copying	11
Floppy formatting	11
Hard-disk formatting and verification	11
Additional volume, conditional preparation	11
Floppy archives and restoration	13
Tape association	13
Tape initialization and tensioning	13
Tape backup	13
Tape restoration	13

Commands

Term	Reference
ACTIVAR	Activate a partition or background command
ARCHDIR	List an archive floppy
ARCHIVAR	Archive files, accounts or volumes
ASIGNAR	Assign channels
ASOCIAR	Associate logical and physical devices
ATRIB	Rename or protect files
AYUDA	Display command help
BACKUP	Copy a complete disk or tape medium
BASIC	Edit, interpret or compile BASIC
BORRAR	Delete files and release storage
BSC	Configure 2780/3780 communications
BUSCAR	Test file existence
BUZON	Read and remove stored messages
CAMBIAR	Change a command's minimum privilege
CAMCU	Change an existing account number
CAMPRO	Transfer private-file ownership
CARGAR	Executable absent; retained CSI table entry
CEDIT	Edit direct, indexed or keyed records
CIFRA	Calculate a file check value
CLASIF	Executable absent; not a SORT alias
CLAVIN	Installed CLAVE.CUENTA helper; startup returns CR 254
COMPART	Grant, revoke or list shared access
CONECTAR	Connect to another account
CONSULTA	Enter the Spanish query language
COPFICH	Copy keyed records between files
COPIFI	Copy files

Term	Reference
CORLUZ	Recount indexed or keyed records
CREAR	Create structured files
CUENTA	Maintain user accounts
DESCARGAR	Executable absent; lookup name DESCARGA
DIRIBM	List an IBM-3741 floppy
DUPLICAR	Duplicate floppies through disk storage
EDITAR	Edit sequential text
ELIMINAR	Remove a damaged catalogue entry
ENVIAR	Send a file synchronously
ENVIARA	Send a file asynchronously
ESPIAR	Display another user's console activity
ESTADO	Inspect system settings and state
EXEC	Execute a command file
FIN	Disconnect the account
FISDEF	Display physical defective tracks
FORZAR	Inject a command into another partition
GENSIST	Save current system configuration
HASH	Inspect key distribution
IBMSEC	Import IBM-3741 records
INICDISC	Initialize or relabel a disk
INICIAR	Initialize a floppy
INICIBM	Write IBM-compatible floppy headers
INITTAPE	Initialize or erase a cartridge
LCOMPART	Inspect or reconstruct sharing/accounts
LISDIR	List catalogue entries
LISTAR	Display or print a file
LOGDEF	Display logical defective tracks
MENSAJE	Send live or stored messages
MODIMPR	Change printer dimensions

Term	Reference
MONTAR	Notify the system of changed media
O8PRECOM	Utility with an unspecified operating interface
ORGDIR	Reorganize the catalogue
PANTALLA	Design data-entry screens
PARAR	Stop a partition
PONER	Change system or session settings
RECARGAR	Restore archived files/accounts/volumes
RECIBIR	Receive synchronous files
RECIBIRA	Receive asynchronous files
REPARAR	Check and reconcile filesystem metadata
REVIVIR	Recover a deleted ASCII catalogue entry
RUN	Run compiled BASIC
SECIBM	Export sequential records to IBM-3741
SECTOR	List a file's occupied sectors
SERIAL	Edit an executable serial binding
SORT	Sort records by keys
SPOOLER	Manage deferred printing
TECLAS	Emit terminal programming sequences
TEDITAR	Edit small files
TERMINAL	Use a console as a remote terminal
TITULO	Print large lettering
UNOSEC	Reconstruct logical sector 1
VERFORM	Check media formatting
VOLCAR	Dump file or logical-sector bytes

BASIC and build controls

Term	Reference
AUTO	Explanation and example
BOTTOM	Explanation and example
BREAK AT	Explanation and example
BREAK ON change	Explanation and example
BREAK ON comparison	Explanation and example
CHANGE	Explanation and example
CONTINUE	Explanation and example
DELETE source	Explanation and example
INDENT	Explanation and example
LENGTH	Explanation and example
LIST	Explanation and example
LOAD	Explanation and example
LOCATE	Explanation and example
LPnLIST	Explanation and example
LPnXREF	Explanation and example
MODIFY	Explanation and example
NAME	Explanation and example
NEW	Explanation and example
QUIT command	Explanation and example
RENUMBER	Explanation and example
RUN command	Explanation and example
SAVE	Explanation and example
STEP	Explanation and example
TOP	Explanation and example
TRACE	Explanation and example
UNBREAK	Explanation and example

Term	Reference
UNTRACE	Explanation and example
VARS	Explanation and example
XREF	Explanation and example
Line-number display	Explanation and example
Ctrl-Z	Explanation and example
LF key	Explanation and example
Return key	Explanation and example
CASE	Explanation and example
CEND	Explanation and example
CHAIN	Explanation and example
CLEAR	Explanation and example
CLOSE	Explanation and example
COMMON	Explanation and example
CSI	Explanation and example
DATA	Explanation and example
DEF FN	Explanation and example
DELETE record	Explanation and example
DIM	Explanation and example
ELSE	Explanation and example
END	Explanation and example
FNEND	Explanation and example
FOR	Explanation and example
GET	Explanation and example
GOSUB	Explanation and example
GOTO	Explanation and example
IF	Explanation and example
IFEND	Explanation and example
INPUT	Explanation and example
LET	Explanation and example

Term	Reference
LINK	Explanation and example
LINPUT	Explanation and example
MAT copy	Explanation and example
MAT fill	Explanation and example
MAT INPUT	Explanation and example
MAT PRINT	Explanation and example
MAT READ	Explanation and example
MAT WRITE	Explanation and example
MOUNT	Explanation and example
NEXT	Explanation and example
ON ERROR	Explanation and example
ON GOTO	Explanation and example
ON GOSUB	Explanation and example
OPEN	Explanation and example
OPTION	Explanation and example
OTHERWISE	Explanation and example
PRINT	Explanation and example
PUT	Explanation and example
QUIT statement	Explanation and example
RANDOMIZE	Explanation and example
READ	Explanation and example
READNEXT	Explanation and example
REM	Explanation and example
RESTORE	Explanation and example
RESUME	Explanation and example
RETURN	Explanation and example
RUN statement	Explanation and example
SELECT	Explanation and example
SLEEP	Explanation and example

Term	Reference
STOP	Explanation and example
THEN	Explanation and example
WAIT	Explanation and example
WAIT device	Explanation and example
WEND	Explanation and example
WHILE	Explanation and example
WRITE	Explanation and example
ABS	Explanation and example
ASC	Explanation and example
AT$	Explanation and example
ATN	Explanation and example
BIN	Explanation and example
BINOF$	Explanation and example
CHR$	Explanation and example
COS	Explanation and example
CRT$	Explanation and example
DATE$	Explanation and example
DAY	Explanation and example
DEL$	Explanation and example
DTE$	Explanation and example
EOF	Explanation and example
ERL	Explanation and example
ERR	Explanation and example
EXP	Explanation and example
EXT$	Explanation and example
FIX	Explanation and example
FLOAT	Explanation and example
FORMAT$	Explanation and example
HEX	Explanation and example

Term	Reference
HEXOF$	Explanation and example
INP	Explanation and example
INS$	Explanation and example
INT	Explanation and example
LEFT$	Explanation and example
LEN	Explanation and example
LINE	Explanation and example
LOG	Explanation and example
LPAD$	Explanation and example
LRL	Explanation and example
LRR	Explanation and example
LSL	Explanation and example
LSR	Explanation and example
LTRIM$	Explanation and example
MATCH	Explanation and example
MAX	Explanation and example
MID$	Explanation and example
MIN	Explanation and example
MOD	Explanation and example
NBR	Explanation and example
OCT	Explanation and example
OCTOF$	Explanation and example
OVR$	Explanation and example
PAGE	Explanation and example
PI	Explanation and example
POS	Explanation and example
REP$	Explanation and example
RIGHT$	Explanation and example
RND	Explanation and example

Term	Reference
ROUND	Explanation and example
RPAD$	Explanation and example
RPT$	Explanation and example
RTRIM$	Explanation and example
SCH	Explanation and example
SEC	Explanation and example
SGN	Explanation and example
SIN	Explanation and example
SPACE$	Explanation and example
SQR	Explanation and example
STR$	Explanation and example
TAN	Explanation and example
TIME$	Explanation and example
TRIM$	Explanation and example
USR	Explanation and example
USR$	Explanation and example
VAL	Explanation and example
OPTION BASE 0	Explanation and example
OPTION PROMPT	Explanation and example
OPTION CASE	Explanation and example
OPTION USR	Explanation and example
OPTION PRIV	Explanation and example
OPTION SERIAL	Explanation and example
UNLOCK	Explanation and example
OPEN INPUT	Explanation and example
OPEN OUTPUT	Explanation and example
OPEN UPDATE DIRECT	Explanation and example
OPEN UPDATE SEQUENTIAL	Explanation and example
OPEN EXTEND	Explanation and example

Term	Reference
OPEN LOCK	Explanation and example
Compiler LIST	Explanation and example
Compiler NOLIST	Explanation and example
Compiler IMPRn	Explanation and example
Compiler XREF	Explanation and example
Compiler NOOBJ	Explanation and example
Compiler NOPANT	Explanation and example
Compiler OBJ	Explanation and example
Compiler COMMA	Explanation and example
Runtime TRACE	Explanation and example
Runtime COMA	Explanation and example

System services

Term	Reference
0x00	End command / restore CSI
0x01	Console line input
0x02	Display string
0x03	Poll console input
0x04	Read console character
0x05	Console character output
0x06	Read console character with direct echo
0x07	Terminal-class character output
0x08	Printer-association character output
0x09	Invalidate cached disk descriptor fields
0x0A	Read logical sector
0x0B	Write logical sector
0x0C	Return initialized kernel signature
0x0D	Create/write directory entry
0x0E	Parse unsigned hexadecimal
0x0F	Parse unsigned decimal
0x10	Format byte as hexadecimal
0x11	Format unsigned word as decimal
0x12	Console newline
0x13	Calibrated busy delay
0x14	Find directory entry
0x15	Get associated device descriptor
0x16	Load named executable without entering it
0x17	Printer-association string output
0x18	Initialize channel name/class
0x19	Convert drive character to association index

Term	Reference
0x1A	Convert disk association to character
0x1B	Allocate exact contiguous block count
0x1C	Free contiguous block range
0x1D	Delete named file
0x1E	Load and execute command
0x1F	Rename named file
0x20	Open channel
0x21	Close channel
0x22	Read record from channel
0x23	Write record to channel
0x24	Format current date
0x25	Format current time
0x26	Unsigned division
0x27	Unsigned multiplication
0x28	Read numbered direct record
0x29	Write numbered direct record
0x2A	Parse decimal or H-suffixed hexadecimal
0x2B	Read indexed/keyed record
0x2C	Read next indexed/keyed record
0x2D	Write indexed/keyed record
0x2E	Pack current date/time
0x2F	Resolve volume label or one-character drive
0x30	Get current partition context
0x31	Pagination pause
0x32	Read multiple logical sectors
0x33	Write multiple logical sectors
0x34	Display string with selected mode flags suppressed
0x35	Schedule, replace or cancel timed callback
0x36	Replace command input and return to CSI

Term	Reference
0x37	Get partition memory ceiling
0x38	Set partition memory ceiling
0x39	Set context word28 callback
0x3A	Resolve association
0x3B	Get associated terminal geometry fields
0x3C	Delete indexed/keyed record
0x3D	Device initialization dispatch
0x3E	Device status dispatch
0x3F	Device input dispatch
0x40	Device output dispatch
0x41	Standard command privilege/serial prologue
0x42	Copy volume label
0x43	Set physical device driver entry
0x44	Terminate/unload associated driver
0x45	Poll and consume console-event flag
0x46	Replace command input with continuation ceiling
0x47	Read byte from memory queue
0x48	Append byte to memory queue
0x49	Change console-mode bits
0x4A	Set disk-error callback
0x4B	Initialize or change system volume
0x4C	Scaled busy delay
0x4D	Get channel-table entry
0x4E	Dispatch terminal control command
0x4F	Yield / schedule
0x50	Get global base
0x51	Read global byte003B
0x52	Get current scheduler index
0x53	Release channel record locks

Term	Reference
0x54	Acquire byte lock
0x55	Release byte lock
0x56	Enter callback in selected partition
0x57	Get association-map base
0x58	Send text to another partition console
0x59	Acquire fixed global lock
0x5A	Release fixed global lock
0x5B	Get sector workspace
0x5C	Get privilege byte
0x5D	Compare memory
0x5E	Read counted sequential bytes
0x5F	Write counted sequential bytes
0x60	Display numbered system message
0x61	Display numbered message and end command
0x62	Load R submodule from compact descriptor
0x63	Allocate a block run with minimum size
0x64	Look up an abbreviable keyword table
0x65	Get account number
0x66	Filter device input for escape controls
0x67	Register indexed callback
0x68	Copy from selected record bank
0x69	Copy to selected record bank
0x6A	Format supplied BCD date
0x6B	Wait for device event or interrupt
0x6C	Find named resident component
0x6D	Write current global time through RTC callback
0x6E	Write current global date through RTC callback

System files

File	Reference
SISTEMA.NUCLEO	Component details
SISTEMA.STARTUP	Component details
SISTEMA.CSI	Component details
SISTEMA.EXECLANG	Component details
SISTEMA.CUENTA	Component details
SISTEMA.CONECTAR	Component details
CONECTAR.COMANDO	Component details
SISTEMA.AYUDA	Component details
SISTEMA.MENSERR	Component details
SISTEMA.NOMBDISP	Component details
SISTEMA.IPL	Component details
SISTEMA.SPOOLCOL	Component details
SISTEMA.BUZON	Component details
SISTEMA.DUPLICAR	Component details
BASIC.COMANDO	Component details
BASIC.LOADFILE	Component details
RUN.COMANDO	Component details
SISTEMA.EXEC1	Component details
SISTEMA.EXEC2	Component details
SISTEMA.EXEC3	Component details
SISTEMA.EXEC4	Component details
BASIC.OVERLAY1	Component details
BASIC.OVERLAY2	Component details
BASIC.OVERLAY3	Component details
BASIC.OVERLAY4	Component details
SISTEMA.DIS1	Component details

File	Reference
SISTEMA.DIS2	Component details
SISTEMA.DIS10	Component details
SISTEMA.DIS11	Component details
SISTEMA.DIS12	Component details
SISTEMA.DIS16	Component details
SISTEMA.DIS17	Component details
SISTEMA.DIS18	Component details
SISTEMA.DIS19	Component details
SISTEMA.DIS20	Component details
SISTEMA.DIS21	Component details
SISTEMA.DIS22	Component details
SISTEMA.DIS23	Component details
SISTEMA.DIS24	Component details
SISTEMA.DIS25	Component details
SISTEMA.DIS26	Component details
SISTEMA.CLASE1	Component details
SISTEMA.CLASE2	Component details
GETPRIV.BASICUSR	Component details
GETID.BASICUSR	Component details
GETUSER.BASICUSR	Component details
CAMPAR.BASICUSR	Component details
PUTQUET.BASICUSR	Component details
NOMVOL.BASICUSR	Component details
TAMDIS.BASICUSR	Component details
DIREC.BASICUSR	Component details
CLAVOUT.BASICUSR	Component details
LINPAN.BASICUSR	Component details
ESPERA.BASICUSR	Component details
OCONV.BASICUSR	Component details

File	Reference
OCONV2.BASICUSR	Component details
KEYSET.BASICUSR	Component details
SORT.OVERLAY1	Component details
SORT.OVERLAY2	Component details
CONSULTA.OVERLAY1	Component details
CONSULTA.OVERLAY2	Component details
CONSULTA.OVERLAY3	Component details
CONSULTA.OVERLAY4	Component details
SPOOLER.OVERLAY1	Component details
CONSULTA.TRAB1	Component details
CONSULTA.TRAB2	Component details
CONSULTA.TRAB3	Component details
CONSULTA.TRAB4	Component details
MAESTRO.TITULO	Component details

Command spelling and component languages

Term	Reference
Abbreviations, full names and synonyms	Command spelling, 5, 30, 35
Option keywords and overlapping prefixes	Utility options, 35
EDITAR internal vocabulary	Editor reference, 7, 35
EXEC directives, FIFO/LIFO and queued input	EXEC reference, 8, 35
CONSULTA query tokens and dictionary listing	Query reference, 35
PANTALLA lowercase controls	Screen controls, 35
BASIC additional option/operator keywords	Additional keywords, D
ACTUCAP, EXPAND, IMPCAP and TESTCAP	Capture support, C
CONTROL.VOC, GENFICH, GENVOC and VOC	Vocabulary support, C
CIPL, COPSIS, INFORME, INSER and RUTINA source libraries	Support components, C

Additional support files

Component	Reference
ACTUCAP.BASICCOM	Component description, C
ACTUCAP.EXEC	Component description, C
ACTUCAP.PANTALLA	Component description, C
ACTUCAP2.BASICCOM	Component description, C
ACTUCAP2.PANTALLA	Component description, C
CAMBVERS.EXEC	Component description, C
CIPL.EXEC	Component description, C
COPIVOC.BASICCOM	Component description, C
COPIVOC.EXEC	Component description, C
COPSIS.EXEC	Component description, C
COPSIS2.EXEC	Component description, C
DISCO.BASICCOM	Component description, C
EXPAND.BASICCOM	Component description, C
EXPAND.EXEC	Component description, C
EXPAND.MACMODEL	Component description, C
GENFICH.EXEC	Component description, C
GENPAN.BASICCOM	Component description, C
GENVOC.BASICCOM	Component description, C
GENVOC.EXEC	Component description, C
GETPART.BASICCOM	Component description, C
IMPCAP.BASICCOM	Component description, C
IMPCAP.EXEC	Component description, C
INFORME.EXEC	Component description, C
INFORME1.BASICCOM	Component description, C
INFORME2.BASICCOM	Component description, C
INFORME3.BASICCOM	Component description, C

Component	Reference
INFORME4.BASICCOM	Component description, C
INFORME5.BASICCOM	Component description, C
INSER.EXEC	Component description, C
INSER1.BASICCOM	Component description, C
INSER1.EXEC	Component description, C
LONCLA.BASICCOM	Component description, C
MAESTRO.POLIZA	Component description, C
MAESTRO.VOC	Component description, C
MANTVOC.BASICCOM	Component description, C
RESUME.BASICCOM	Component description, C
RUTINA1.BASIC	Component description, C
RUTINA2.BASIC	Component description, C
RUTINA3.BASIC	Component description, C
RUTINA4.BASIC	Component description, C
SCAPTURA.BASICCOM	Component description, C
TESTCAP.BASICCOM	Component description, C
TESTCAP.EXEC	Component description, C
TRAFIC.BASICCOM	Component description, C
USOIMPR.BASICMAC	Component description, C
VOC.BASICCOM	Component description, C
VOC.EXEC	Component description, C
VOC.PANTALLA	Component description, C

Emulation

Term	Reference
MAME: startup, menus and shutdown	Emulation in MAME
Hard-disk difference files and backups	Emulation in MAME
Floppy insertion, replacement and formatting	Emulation in MAME
Serial printer capture	Emulation in MAME

Emulation in MAME

Operating the SECOINSA Serie 20 and S.O. Multi-Usuario 6.2 in MAME.

Use a MAME version containing the `secoinsa20` driver and the `somu62` software-list entry (S.O. Multi-Usuario 6.2). Working emulation support begins with MAME 0.290.

Commands beginning with `mame` are entered on the host computer. S.O. Multi-Usuario commands are entered at the emulated system's > prompt.

Installing the images and starting MAME

Place the machine ROM set and system disk in MAME's ROM search path. One suitable arrangement is:

```
roms/
    secoinsa20.zip
    secoinsa20/
        somu62/
            somu62.chd
```

Start the machine with:

```
mame secoinsa20 somu62 -window
```

This selects the pre-installed S.O. Multi-Usuario 6.2 hard disk and the default compatible serial terminal. Supplying only `secoinsa20` does not automatically select the system disk.

Allow the original startup sequence to finish, then log in as:

```
SISTEMA
```

This account has no password. The image contains the system volume, accessible as `:S`.

For example:

```
LISDIR SISTEMA.*:S
AYUDA
```

Switching between the terminal and MAME menus

During normal operation, the keyboard controls the emulated terminal. To operate MAME's menus:

1. Press **Scroll Lock** to enable MAME's UI controls.
2. Press **Tab** to open the menu.
3. After making changes, close the menu and disable UI controls to resume typing into the terminal.

The UI toggle defaults to **Forward Delete** on macOS. It can also be reassigned through MAME's input settings.

The displayed console is an emulated serial terminal. The default terminal understands the SECOINSA control sequences used by the system; no separate terminal application or manual serial configuration is needed.

Saving work and returning to a clean system

When `somu62` is selected through the software list, MAME keeps hard-disk changes in a separate difference file, normally:

```
diff/somu62.dif
```

The original `somu62.chd` remains unchanged. Starting MAME again with the same difference directory continues using the modified disk.

To keep a separate working installation:

```
mame secoinsa20 somu62 -window -diff_directory "work/main/diff"
```

To start from the clean system image for an experiment:

```
mame secoinsa20 somu62 -window -diff_directory "work/experiment1/diff" -noautosave
```

Choose a directory that does not already contain a difference file. Reusing that directory resumes the experiment's disk contents.

Back up the difference file while MAME is closed, together with its matching parent CHD. The parent CHD alone does not contain your subsequent work. MAME saved states are not substitutes for disk backups.

Starting with a floppy inserted

To start with an existing floppy image:

```
mame secoinsa20 somu62 -window -flop1 "media/work.mfi"
```

In the default machine configuration, MAME's `-flop1` device corresponds to **`FLOPPY2` inside S.O. Multi-Usuario**.

After logging in, associate an unused logical disk letter and notify the operating system of the inserted medium:

```
ASOCIAR A FLOPPY2
MONTAR A
LISDIR *.*:A
```

These instructions assume that logical disk letter `A` is available. Enter `ASOCIAR` without arguments to inspect the current associations.

MFI is the tested format for writable floppy images. It preserves the mixed recording densities and sector sizes used by the original formatting utility. An ordinary PC-formatted disk image is not equivalent.

Inserting, removing and replacing floppies while running

To insert a floppy without restarting:

1. Enable MAME's UI controls and open **Tab → File Manager**.
2. Select the floppy drive.
3. Browse to the image and select it.
4. Choose the access mode when prompted.
5. Return to the terminal and use `ASOCIAR` and `MONTAR` as shown above.

The available access modes include:

Mode	Effect
Read-only	Protects the selected image from writes.
Read-write	Saves writes directly into the selected image.
Read this image, write to another image	Uses a separate output image, preserving the selected source.

Floppy writes do not use the hard-disk difference directory. Use read-only access or a separate output image when preserving a floppy master.

Before ejecting, finish applications using the floppy and release its association:

```
ASOCIAR A
```

Then open **File Manager**, select the floppy drive and choose **[empty slot]**. In a multiuser session, first ensure that other users have also finished accessing the disk.

To insert a replacement, select its image through the same menu, then enter:

```
ASOCIAR A FLOPPY2
MONTAR A
```

`MONTAR` tells S.O. Multi-Usuario that the medium has changed. It does not insert an image into MAME or format a blank disk.

Creating and formatting a new floppy

In **File Manager**, select the floppy drive and choose **[create]**. Specify a new filename ending in `.mfi`, select MAME's MFI format, and choose the unformatted 5¼-inch double-sided high-density medium if offered.

The new image represents a blank disk. Return to S.O. Multi-Usuario and enter:

```
ASOCIAR A FLOPPY2
INICIAR A (FORMAT CARA 2)
```

FORMAT requests physical formatting; CARA 2 selects two sides.

Formatting erases the floppy associated with `A`. Confirm that the intended new image is inserted before answering the utility's confirmation prompts.

Supply a disk label when requested and let formatting finish. Then check the result:

```
MONTAR A
VERFORM A
LISDIR *.*:A
```

This uses the tested native format. The emulated drive selection does not establish the manufacturer or exact model of the original physical drive.

Trying another hard-disk image

Close MAME before changing the system hard disk. To select a compatible CHD directly:

```
mame secoinsa20 -harddisk "images/alternative.chd" -window -diff_directory "work/alternative/
diff" -noautosave
```

This replaces the `somu62` selection for that run.

The alternative image must contain a disk layout and boot software compatible with the emulated machine. A `.chd` extension alone does not establish compatibility.

Use a working copy of an unfamiliar image. Directly attached writable images may be modified in place; selecting a difference directory does not force every image to use an overlay. The software-list `somu62` procedure described above provides the defined master-and-difference arrangement.

Capturing serial printer output

Attach the serial printer and select a new output filename:

```
mame secoinsa20 somu62 -window -printerport printer -printout "output.prn"
```

Inside S.O. Multi-Usuario:

```
ASOCIAR IMPR1 IMP
LISTAR SISTEMA.NOMBDISP:S (PRINT1 NOHEAD)
```

The output file contains the printer's received bytes, including control characters. It is not a rendered PDF. Use a new filename to preserve earlier output.

Ending a session

Finish running applications and release removable media before exiting. Enter:

```
FIN
```

`FIN` logs out of S.O. Multi-Usuario; it does not close MAME. After any other users have finished, enable MAME's UI controls and exit normally. This allows pending image writes to be saved.

At the time of writing of this document the following emulation characteristics are pending: Tape operation is not implemented. Floppy booting is not established. The emulated terminal does not yet fully reproduce the original Secoinsa IS 9502 terminal firmware functionality.